408 考纲知识点手册

操作系统

计算机学科专业基础综合　分册

按 2027 年全国硕士研究生招生考试大纲编排

对应大纲	第三部分操作系统
本册章数	5 章
知识点条目	覆盖大纲本部分全部条目
主要素材	语雀「计算机考研 408 笔记」知识库
	王道 2027 系列教材与课件
	工作区历年真题与强化笔记

本分册为依据 2027 年考试大纲整理的「操作系统」知识点汇编，供复习检索使用。

使用说明

本手册按《2027 年全国硕士研究生招生考试计算机学科专业基础考试大纲》的条目顺序编排，覆盖大纲「考查内容」中数据结构、计算机组成原理、操作系统、计算机网络四科的全部条目。

本册范围：本册是《408 考纲知识点手册》的「操作系统」分册，对应 2027 年考试大纲「操作系统」部分的全部考查内容，共 5 章。其余三科另见对应分册；四科合订本见《408 考纲知识点手册》。

一、编排方式

全书分四部分，共 24 章。四部分与大纲的四科一一对应，章与该科下的一级条目一一对应，章内小节按大纲的二级、三级条目顺序展开。每一章的开头用引用块照抄本章对应的大纲原文条目，便于对照检查覆盖情况；每一章末尾固定给出「本章小结表」与「易错清单」两张速查内容。

二、阅读约定

本手册用三种提示框把不同性质的内容区分开：

- 核心结论——必须记住的定义、公式、判定条件，考前反复看的就是这些。
- 易错点——统考中反复出现的陷阱，以及容易与相邻概念混淆的地方。
- 真题考法——该知识点在统考中以什么形式出现；能确认年份与题号的，如实标注。无法确认的不标，不编造。

正文中的公式一律用数学排版；算法与代码用等宽框给出；对比性内容尽量做成表格。凡涉及计算的知识点，都给出公式、符号含义、适用范围，并配一组可以自己动手复核的数字算例。

三、素材来源

本手册的素材来自三处：

- 语雀知识库「计算机考研 408 笔记」（月暗）导出的 182 篇笔记 Markdown 全文；
- 工作区中的王道 2027 系列教材、课件、强化笔记与历年真题汇编；
- 工作区 408 计算机考研目录下的各类复习笔记与答题本。

素材在整理过程中做了三件事：一是按大纲重新组织顺序；二是把原笔记中的口语化叙述、错别字与图片占位改写成规范的知识点条目，公式按上下文重新推导书写；三是逐条核对，凡素材之间口径冲突的，以大纲用词与统考真题为准，并在正文中说明差异。

四、使用建议

第一轮复习时，按章通读，重点看「核心结论」与例题，把「易错点」留到做题后再回看；第二轮复习时只看每章的「本章小结表」，遇到想不起来的再翻回正文；考前冲刺时只看「核心结论」与「易错清单」。

说明：本手册为复习资料汇编，用于知识点检索与自查，不替代教材与真题训练。

目录

使用说明 **i**

第一章 操作系统基础 **1**

本章考纲要求 1

一、 操作系统的基本概念 1

（一） 操作系统的定义与双重角色 1

（二） 操作系统的四大功能 2

（三） 用户接口与程序接口 2

（四） 操作系统的四大特征 3

二、 操作系统发展历程 4

（一） 手工操作阶段 4

（二） 批处理阶段 4

（三） 分时操作系统 5

（四） 实时操作系统 5

（五） 网络操作系统、分布式系统与微机操作系统 5

（六） 各类操作系统对比 6

三、 程序运行环境 6

（一） CPU 运行模式：内核模式与用户模式 6

（二） 中断和异常的处理 8

（三） 系统调用 10

（四） 程序的链接与装入 13

（五） 程序运行时的内存映像与地址空间 15

四、 操作系统结构 16

（一） 分层结构 16

（二） 模块化结构 16

（三） 宏内核 16

（四） 微内核 17

（五） 外核 18

（六） 五种结构对比 18

五、 操作系统引导 19

（一） 引导的完整流程 19

六、 虚拟机 20

（一） 虚拟机与虚拟机管理程序 20

（二） 第一类虚拟机管理程序（裸金属架构） 20

（三） 第二类虚拟机管理程序（寄居架构） . . . 21
（四） 两类虚拟机管理程序对比 . . . 21
本章小结表 . . . 21
易错清单 . . . 23

第二章 进程管理 **25**
本章考纲要求 . . . 25
一、 进程与线程 . . . 26
（一） 进程与线程的基本概念 . . . 26
（二） 进程与线程的状态与转换 . . . 28
（三） 线程的实现 . . . 30
（四） 进程与线程的组织与控制 . . . 31
（五） 进程间通信 . . . 33
二、 CPU 调度与上下文切换 . . . 36
（一） 调度的基本概念 . . . 36
（二） 调度的目标 . . . 36
（三） 调度的实现 . . . 37
（四） CPU 调度算法 . . . 39
（五） 多处理机调度 . . . 44
（六） 上下文及其切换机制 . . . 45
三、 同步与互斥 . . . 46
（一） 同步与互斥的基本概念 . . . 46
（二） 基本的实现方法 . . . 47
（三） 锁 . . . 51
（四） 信号量 . . . 53
（五） 条件变量 . . . 55
（六） 经典同步问题 . . . 56
四、 死锁 . . . 67
（一） 死锁的基本概念 . . . 67
（二） 死锁预防 . . . 68
（三） 死锁避免 . . . 69
（四） 死锁检测和解除 . . . 76
本章小结表 . . . 77
易错清单 . . . 79

第三章 内存管理 **82**
本章考纲要求 . . . 82
一、 内存管理基础 . . . 82

（一） 内存管理的基本概念 . . . 82
（二） 连续分配管理方式 . . . 85
（三） 页式管理 . . . 89
（四） 段式管理 . . . 93
（五） 段页式管理 . . . 96
二、 虚拟内存管理 . . . 97
（一） 虚拟内存的基本概念 . . . 97
（二） 请求页式管理 . . . 99
（三） 页框分配与回收 . . . 101
（四） 页置换算法 . . . 105
（五） 内存映射文件 . . . 111
（六） 虚拟存储器性能的影响因素及改进方法 . . . 113
本章小结表 . . . 116
（七） 三种非连续分配方式总对比 . . . 116
（八） 页面置换算法总对比 . . . 117
（九） 连续分配三种方式总对比 . . . 117
（十） 有效访问时间速查 . . . 117
易错清单 . . . 118

第四章 文件管理 **120**
本章考纲要求 . . . 120
一、 文件的基本概念 . . . 120
（一） 数据项、记录与文件 . . . 120
（二） 文件系统及其功能 . . . 121
（三） 文件的属性与分类 . . . 121
文件元数据和索引节点 . . . 121
（四） 文件控制块 FCB . . . 121
索引节点 inode 与目录项的分离 . . . 122
inode 中的多级索引与最大文件大小计算 . . . 123
（五） inode 位图与数据块位图 . . . 124
二、 文件的操作 . . . 125
（一） 建立文件 . . . 125
（二） 删除文件 . . . 125
（三） 打开文件表与文件描述符 . . . 126
（四） 关闭文件 . . . 127
（五） 读文件与写文件 . . . 127
三、 文件的保护 . . . 128
（一） 可控制的访问类型 . . . 128

（二） 访问控制矩阵与访问控制列表 . . . 128
（三） 传统的权限位 rwx . . . 129
（四） 口令保护与加密保护 . . . 129
四、 文件的逻辑结构 . . . 129
（一） 无结构文件与有结构文件 . . . 130
（二） 顺序文件 . . . 130
索引文件 . . . 130
索引顺序文件 . . . 130
（三） 直接文件或散列文件 . . . 131
（四） 各逻辑结构的查找效率对比 . . . 131
五、 文件的物理结构 . . . 132
（一） 连续分配 . . . 132
（二） 链接分配 . . . 133
索引分配 . . . 134
（三） 三种物理结构的对比 . . . 135
六、 目录的基本概念 . . . 136
七、 树形目录 . . . 136
（一） 目录结构演进 . . . 136
（二） 绝对路径、相对路径与当前工作目录 . . . 137
（三） 路径名解析过程 . . . 137
八、 目录的操作 . . . 138
九、 硬链接与软链接 . . . 138
（一） 硬链接（Hard Link） . . . 138
（二） 软链接（符号链接，Symbolic Link） . . . 139
十、 文件系统的全局结构 . . . 140
（一） 文件系统在外存（磁盘）中的结构 . . . 140
（二） 文件系统在内存中的结构 . . . 141
十一、 外存空闲空间管理方法 . . . 141
（一） 空闲表法 . . . 141
（二） 空闲链表法 . . . 142
（三） 位示图法 . . . 142
（四） 成组链接法 . . . 144
十二、 虚拟文件系统 . . . 145
（一） 为什么需要 VFS . . . 145
（二） VFS 的四大对象 . . . 146
（三） VFS 的工作过程（以 write 为例） . . . 146
十三、 文件系统挂载 . . . 147

（一） 挂载的过程 . . . 147
本章小结表 . . . 148
易错清单 . . . 149

第五章 输入/输出管理 **151**
本章考纲要求 . . . 151
一、 I/O 管理基础 . . . 152
（一） 设备 . . . 152
（二） I/O 控制方式 . . . 154
（三） I/O 软件层次结构 . . . 159
（四） 输入/输出应用程序接口 . . . 161
二、 设备独立软件 . . . 163
（一） 缓冲区管理 . . . 163
（二） 设备分配与回收 . . . 167
（三） 假脱机技术（SPOOLing） . . . 169
（四） 设备驱动程序接口 . . . 171
三、 外存管理 . . . 172
（一） 磁盘 . . . 172
（二） 固态硬盘 . . . 179
本章小结表 . . . 180
易错清单 . . . 182

第一章操作系统基础

本章考纲要求

第三部分操作系统［考查目标］

1. 掌握操作系统的基本概念、方法和原理，了解操作系统的结构、功能和服务，理解操作系统所采用的策略、算法和机制。
2. 能够从计算机系统的角度理解并描述应用程序、操作系统内核和计算机硬件协作完成任务的过程。
3. 能够运用操作系统的原理，分析并解决计算机系统中与操作系统相关的问题。

一、操作系统基础（一）操作系统的基本概念（二）操作系统发展历程（三）程序运行环境

1. CPU 运行模式：内核模式，用户模式。
2. 中断和异常的处理
3. 系统调用
4. 程序的链接与装入
5. 程序运行时的内存映像与地址空间

（四）操作系统结构分层，模块化，宏内核，微内核，外核。（五）操作系统引导（六）虚拟机

本章是操作系统部分的开篇，也是整份卷子里「文字辨析题」密度最高的一章：知识点本身不难，但要求精确到词——「内核态只能执行特权指令」错在哪、「中断发生时 CPU 处于什么态」，考的都是对概念边界的把握。

高频点集中在四处：处理机运行模式（用户态 / 内核态）、系统调用、批处理阶段（多道程序设计）、中断和异常；此外操作系统结构（宏内核 / 微内核）、操作系统引导、虚拟机也反复出现。近年还考过「内核态转为用户态的时机」「中断向量表采用的数据结构」「进程虚拟地址空间中各区域的存放内容」等更细的角度。

学习本章的正确姿势是把「用户态 / 内核态」当成一条主线：中断、异常、系统调用、引导过程、虚拟机，本质上都是「谁在什么特权级下、以什么方式获得 CPU 控制权」的不同回答。

一、操作系统的基本概念

（一）操作系统的定义与双重角色

操作系统（**Operating System**，**OS**）是控制和管理整个计算机系统的硬件与软件资源、合理调度任务与分配资源、并为用户和其他软件提供统一接口与运行环境的最基本的系统软件。

从使用层次看，计算机系统自下而上分四层：硬件 → 操作系统 → 应用程序 → 用户。硬件提供基本计算资源，操作系统把资源管起来并向上提供运行环境、充当桥梁，应用程序利用这些资源解决用户问题。这是「人机交互视角」的分层，与计算机组成原理中 CPU 内部结构的分层不是一回事。

操作系统的角色可概括为两重身份、外加一个效果：

1. 作为计算机系统资源的管理者。把用户看作「雇主」、计算机看作「机器」，操作系统就是那位「工人」：协调并控制各个部件工作。
2. 作为用户与计算机硬件系统之间的接口。工人接收雇主命令并执行任务——这对应操作系统提供的用户接口与程序接口；用户使用硬件必须经过操作系统这一必经媒介。
3. 实现对计算机资源的抽象与扩充。只有硬件、没有任何软件支撑的计算机称为裸机；操作系统在裸机之上把底层硬件抽象成更好的逻辑资源，于是呈现给用户的是一台功能更强、更易用的扩充机器（有些教材也叫「虚拟机」，这里指的是操作系统提供的逻辑计算机环境，不是 VMware 那种虚拟化软件造出来的虚拟机）。

（二）操作系统的四大功能

作为资源管理者，操作系统的功能按管理对象分为四类，这四类恰好对应本手册后面四章的内容。

处理机管理：多道程序环境下，处理机的分配和运行都以进程（或线程）为基本单位，所以处理机管理本质上就是对进程的管理——提到「进程管理」时指的也是这块。主要内容包括进程控制、进程同步、进程通信、死锁处理以及处理机调度。

存储器管理：为多道程序提供良好的内存运行环境，提高内存利用率。主要内容包括内存分配与回收、地址映射、内存保护与共享、内存扩充（虚拟存储）。

文件管理：用户可见的持久化信息通常以文件形式组织，负责这部分的部分称为文件系统，功能包括文件存储空间的管理、目录管理、文件读 / 写操作及访问保护。注意操作系统管理的是文件的物理结构、逻辑结构、目录组织，而不是文件的内容——所以「操作系统管理源程序的内容」是错的。

设备管理：处理用户的 I/O 请求，屏蔽设备差异，方便用户使用各类外设并提高设备利用率。主要包括缓冲管理、设备分配、设备驱动处理和虚拟设备。

（三）用户接口与程序接口

操作系统向上提供的接口分两大类，这一组概念经常被拿来设陷阱。

用户接口面向普通终端用户，供其直接向操作系统发出请求：

- 命令接口：用户通过输入命令组织和控制作业的执行。
  - 联机命令接口（交互式命令接口）：适用于分时系统或实时系统，特点是「说一句，做一句」——用户输入一条命令，系统立即解释并执行。
  - 脱机命令接口（批处理命令接口 / 作业控制语言）：适用于批处理系统，特点是「列好清单，再逐项完成」——用户预先提交一组作业控制命令，系统调度到该作业时自动解释执行，无须实时干预。
- 图形用户接口（**GUI**）：通过窗口、图标、菜单、按钮等图形元素操作计算机，底层仍通过系统调用与内核交互。GUI 程序本身不算操作系统的一部分，但它完成的功能都是通过系统调用实现的。

程序接口面向应用程序，本质上就是一组系统调用。它是应用程序在运行时请求操作系统服务、访问系统资源的唯一合法途径。

用户接口与程序接口不能互换

「操作系统提供给应用程序的接口」是系统调用（程序接口），不是库函数、不是中断、不是原语。库函数由编程语言的函数库提供，属于用户程序的一部分，位于系统调用的上层；中断是硬件与操作系统之间的通信手段，对应用程序透明；原语是操作系统内部不可中断的底层程序，不直接暴露给应用层。

（四）操作系统的四大特征

操作系统之所以区别于其他系统软件，在于它具备四个基本特征：并发、共享、虚拟、异步。

并发（**Concurrency**）：两个或多个事件在同一时间间隔内发生。多道程序环境下，内存中同时驻留若干道程序，操作系统通过调度让它们交替执行，宏观上都在运行、微观上轮流占用 CPU。

必须区分并行（**Parallel**）：两个或多个事件在同一时刻真正同时发生。并行需要硬件支持（多流水线或多处理机 / 多核）。单处理机系统中任一时刻只有一个程序占用处理机，多道程序之间只能并发而不能并行；但处理机与设备、处理机与通道、设备与设备之间相互独立，是可以并行的。多核计算机上，多道程序之间既并发也并行。

共享（**Sharing**）：系统中的资源可供内存中多个并发执行的进程共同使用，有两种方式：

- 互斥共享方式：某些资源虽可供多个进程使用，但一个时间段内只允许一个进程访问。典型例子是打印机、磁带机等物理设备，以及软件中的栈、变量、表格等临界资源。
- 同时共享（同时访问）方式：某些资源允许一个时间段内由多个进程「同时」访问，这里的「同时」往往是宏观上的，微观上各进程交替访问（即分时共享）。典型例子是磁盘设备、用可重入代码编写的文件。

判断方法：若若干进程交替使用某设备对结果没有影响，就允许同时访问（磁盘）；若会产生影响，就必须互斥访问（打印机）。

虚拟（**Virtual**）：把一个物理实体变为若干个逻辑上的对应物。物理实体实际存在，逻辑对应物是用户感受到的。三种典型实现：虚拟处理器（多道程序分时使用一个处理器，时分复用）、虚拟存储器（利用覆盖、交换、请求分页 / 分段 / 段页等技术逻辑扩充容量，空分复用）、虚拟设备（SPOOLing 把一台物理 I/O 设备虚拟为多台逻辑设备）。

异步（**Asynchronism**）：多道程序并发执行、争抢有限的系统资源，因此进程的执行不是一贯到底的，而是走走停停、以不可预知的速度向前推进。可以通过同步机制约束执行顺序、限制最终结果。

并发与共享互为存在条件

这是本章第一条必背结论：

1. 资源共享以程序并发为条件——若系统不允许程序并发执行，同一时间段内只有一个程序在跑，自然不存在资源共享问题；
2. 若系统不能对资源共享实施有效管理，则必将影响程序的并发执行，甚至根本无法并发执行。

另外两条从属关系也要记住：没有并发性就谈不上虚拟性（一个时间段内只运行一道程序，虚拟就没有意义）；只有系统拥有并发性，才有可能导致异步性。所以四个特征中，并发和共享是最基本的两个。

真题考法：并发与并行、多道程序系统

- **2009-23**：单处理机系统中可并行的是（处理机与设备、处理机与通道、设备与设备），进程与进程不能并行。
- **2018-23**：多任务操作系统「具有并发和并行的特点」「需要实现对共享资源的保护」正确；「需要运行在多 CPU 平台上」错误——多任务与硬件是否多核无关。
- **2022-23**：多道程序系统「不必支持虚拟存储管理」正确（早期多道批处理直接把所有进程的数据全部装入内存），「进程数越多 CPU 利用率越高」错误——进程过多会因争抢 CPU 而降低利用率。

二、操作系统发展历程

（一）手工操作阶段

早期用户把程序和数据穿孔在纸带或卡片上，装入输入设备后手动控制读入内存并启动运行；程序的装入、执行、结果输出全过程都依赖人工。这一阶段没有操作系统。两个突出缺点：① 用户独占全机，无资源竞争但资源利用率极低；② **CPU** 长时间空闲，等待人工完成 I/O，计算能力没有发挥出来。

（二）批处理阶段

为解决人机矛盾以及 CPU 与 I/O 设备速度不匹配的矛盾，出现了脱机输入 / 输出技术：引入一台外围机，在 CPU 不参与的情况下预先完成 I/O 准备——输入时把纸带 / 卡片上的数据读入磁带供 CPU 后续调入，输出时 CPU 先把结果写入磁带再由外围机输出。由于 I/O 完全脱离主机控制，故称脱机 I/O；由主机直接管理 I/O 的方式则称联机 I/O。脱机 I/O 显著减少了 CPU 空闲时间。

在脱机 I/O 的基础上，为实现作业的自动连续处理，出现了配备监督程序（**Monitor**）的批处理系统，这是操作系统开始出现的标志。

单道批处理系统：作业成批提交，但内存中始终保持只有一道作业。三个特征：

- 自动性：磁带上的作业可自动连续执行，无须人工干预，缓解了一定程度的人机速度矛盾。
- 顺序性：作业按磁带顺序依次装入内存，先装入者先完成。
- 单道性：内存中仅允许一道用户程序运行，待其完成或异常终止后才调入下一道。

缺点是：当正在运行的作业发起 I/O 请求时，高速 CPU 不得不空闲等待低速 I/O 结束，资源利用率依然很低。

多道批处理系统：每次往内存中读入多道程序，作业成批运行。作业先存放在外存的后备队列中，作业调度程序按一定算法选取若干作业调入内存，使其并发执行、共享系统资源。当某道程序因 **I/O** 请求而暂停运行时，借助中断技术 **CPU** 立即转去运行另一道程序。

多道程序设计技术的三个特点（必背）：

1. 多道：计算机内存中同时存放多道相互独立的程序。
2. 宏观上并行：同时进入系统的多道程序都处于运行过程中——它们先后开始运行，但都还没有运行完毕。
3. 微观上串行：内存中的多道程序轮流占有 CPU，交替执行。

主要优点：多道程序并发执行、共享资源，资源利用率大幅提升，CPU 和其他资源更能保持「忙碌」状态，系统吞吐量增大。主要缺点：用户响应时间长，没有人机交互功能——提交作业后只能等待处理完成，中间不能控制自己的作业（无法调试程序、无法在运行过程中输入参数）。

多道程序设计需要解决：① 处理器的分配策略；② 多道程序的内存分配与保护；③ I/O 设备的分配与调度；④ 大量程序与数据的组织、存储方式及其安全性与一致性。此外，引入多道技术后系统失去了封闭性和顺序性（这是异步性的来源），并且由于需要频繁中断，系统开销比单道批处理更大。

多道 ≠ 并行，也不依赖虚拟存储

- 多道批处理系统的「并行」是宏观并行、微观串行。单 CPU 上多道程序之间没有真正的并行，真正的并行要求多处理机 / 多核。
- 多道程序设计技术与硬件是否多核无关，单核也可以是多道批处理系统。

- 多道程序系统不必支持虚拟存储管理。早期多道批处理把需要执行的所有进程的数据一次性全部引入内存。所以「多道 → 必须有虚拟内存」是错的。
- 多道程序设计并不提高单个作业的处理速度，它提高的是资源利用率和系统吞吐量，代价是牺牲了交互性。

（三）分时操作系统

分时技术把处理器时间划分为很短的时间片，轮流分配给各用户程序使用；若某程序在其时间片内未完成计算，则暂停运行、释放处理器给其他程序，待下一轮再继续。由于计算机运行极快、程序轮转迅速，每个用户都获得「独占计算机」的体验。

分时操作系统允许多个用户通过终端同时连接一台主机并与系统交互，彼此互不干扰。它基于多道程序设计，但与多道批处理系统有本质区别：批处理追求高吞吐量和资源利用率，无须人工干预；分时以人机交互为核心目标。四个主要特征：

1. 同时性（多路性）：允许多个终端用户同时使用同一台计算机。
2. 交互性：用户可通过终端以人机对话方式直接控制程序运行并与之交互。
3. 独立性：各用户的操作相互隔离，任一用户都感觉系统为其独占。
4. 及时性：用户请求能在较短时间内获得响应。

主要缺点：不能优先处理一些紧急任务——系统对各个用户 / 作业完全公平，循环服务一个时间片，不区分任务紧急性。

（四）实时操作系统

实时操作系统是为在严格的时间限制内完成关键任务而设计的系统，它摒弃分时的时间片轮转机制，改用基于任务紧迫性或截止时间的调度策略，确保高优先级任务及时获得处理器；主要特点是及时性和可靠性（确定性、高可靠性、强时效性）。

- 硬实时系统：必须保证某个特定动作在规定时间内完成，否则可能引发灾难性后果（飞机的控制系统、导弹制导系统）。
- 软实时系统：允许偶尔违反时间规定，只要不引起永久性损害即可（飞机订票系统、银行管理系统）。

（五）网络操作系统、分布式系统与微机操作系统

网络操作系统在单机操作系统基础上扩展网络功能，支持计算机之间的通信、数据传输与资源共享（如文件、打印机共享）。各计算机保持独立运行，用户需显式指定远程资源的位置才能访问，系统不提供统一的全局视图。

分布式系统由多台计算机组成，节点地位对等、无固定主从关系，资源可被所有用户透明共享，可扩展性与容错能力强，能把一个任务分解为多个子任务分布到不同节点并行执行、协同完成。管理这类系统的操作系统称分布式操作系统，核心特点是分布性、并行性和透明性，其中透明性最关键。

二者区别：网络操作系统仅实现资源共享与通信，用户需主动干预远程操作；分布式操作系统通过协同机制共同完成同一任务，向用户提供单一系统映像。

微机操作系统按用户数与任务并发能力分三类：单用户单任务（早期的 MS-DOS）、单用户多任务（Windows XP 及后续版本、macOS、现代 Linux 桌面版）、多用户多任务（UNIX、Linux 服务器版、Windows Server 系列）。

（六）各类操作系统对比

阶段	核心思想	主要优点	主要缺点	是否支持交互
手工操作	人工装入、人工控制	无资源竞争	用户独占全机，CPU 利用率极低	有人工干预，无系统交互
单道批处理	脱机 I/O + 监督程序自动逐个运行作业	自动性、顺序性，缓解人机速度矛盾	单道性，I/O 时 CPU 空闲，利用率低	无
多道批处理	内存驻留多道程序，I/O 时切换	资源利用率高，吞吐量大	响应时间长，无交互能力，开销大	无
分时系统	时间片轮转，多用户共享一台主机	及时响应，解决人机交互，各用户独立	对紧急任务不能优先处理	有（核心目标）
实时系统	按紧迫性 / 截止时间调度，抢占式	及时性、可靠性强，可优先响应紧急任务	交互流畅性不如分时，开销大	有，但面向特定任务
网络操作系统	互连各机，实现通信与资源共享	扩大资源共享范围	需显式指定远程资源，无全局视图	有
分布式系统	节点对等，协同完成同一任务	分布性、并行性、透明性、可扩展	实现复杂，一致性维护难	有

各操作系统对比的四条排序

资源利用率：多道批处理 > 分时 > 实时；交互性：分时 > 实时 > 批处理；及时性：实时 > 分时 > 批处理；可靠性：实时系统最强。

真题考法：多道批处理系统的特点

- **2016-23**：「批处理系统允许多个用户与计算机直接交互」错误（批处理没有交互能力）；「批处理系统分为单道和多道」「中断技术使多道批处理系统的 I/O 设备可与 CPU 并行工作」正确。
- **2017-28**：多道程序系统的优点是 CPU 利用率高、系统吞吐量大、I/O 设备利用率高；系统开销小是错的——组织作业和切换作业都要额外开销。
- **2022-23**：见前文「并发与共享」处的考法说明。

三、程序运行环境

（一）CPU 运行模式：内核模式与用户模式

*1.*内核与内核态

为确保操作系统正确运行，必须严格区分操作系统代码与用户程序的执行环境。绝大多数计算机系统通过硬件支持实现这一区分：为处理机设置至少两种独立的运行模式——用户态（目态）和内核态（管态、系统态、核心态）。

硬件通过程序状态字寄存器（**PSW**）中的一个模式位标识当前运行模式。不同教材约定可能相反，统考按「**0** 表示内核态，**1** 表示用户态」理解（也有资料写 1 表示内核态，题目若给出约定以题目为准）。系统引导时硬件从内核态开始工作，完成操作系统加载后才切换到用户态执行用户程序。

与两种模式对应，内存空间也分两块：内核空间是操作系统程序和数据占用的内存空间，只有内核态下才可以访问；用户空间是用户代码占用的内存空间，用户态下可以访问。CPU 访问内存时会

进行特权级检查，只有当前特权级高于被访问数据的特权级时才能访问。

操作系统内核是紧靠硬件、常驻内存的核心软件，把与硬件紧密相关的模块（中断处理程序、设备驱动程序）、运行频率较高的模块（时钟管理、进程调度）以及各模块共用的基础操作集中放置，以保护核心模块免受应用程序破坏并提升运行效率。内核功能分两类：

1. 支撑功能：时钟管理（提供系统时间、产生定时中断，用于时间片轮转、实时任务的截止时间控制、批处理作业进度衡量）、中断机制（保存与恢复现场、转移控制权到处理程序）、原语（由若干条指令组成、完成特定功能、具有原子性即不可被中断的操作，如进程切换、进程同步、资源分配，常通过关中断保证原子性）。
2. 系统资源管理功能：进程管理、存储器管理、设备管理，以及支持它们的内核数据结构（PCB、设备控制块、内存分配表、缓冲区、各类链表和消息队列等）及基本操作。

*2.*特权指令与非特权指令

双重模式机制通过指令分类提供保护：

- 特权指令：仅能在内核态执行，可访问全部地址空间（包括用户空间和内核空间），执行关键操作。包括：I/O 指令（输入 / 输出指令）、开 / 关中断指令、置时钟指令、设置中断屏蔽字、修改 PSW（模式位）、存取用于内存保护的寄存器（如页表基址寄存器）、修改中断向量表、停机 / 重启指令，以及由内核态转为用户态的操作本身。
- 非特权指令：可在任意模式下执行，主要用于一般计算和数据处理，其内存访问被限制在用户地址空间，无法直接操作硬件或系统核心资源。例如：算术逻辑运算指令（加法、取非）、数据传送指令、跳转 / 分支指令、压栈指令、设置断点指令、陷入指令（**trap**）。

上述限制由硬件强制实施：若用户程序试图执行特权指令，硬件将产生非法指令异常，操作系统捕获该异常后终止该程序并重新调度。

关于模式的四条高频错误说法

1.「CPU 处于内核态时只能执行特权指令」——错。内核态既能执行特权指令，也能执行非特权指令。
2.「CPU 处于用户态时只能执行非特权指令」——对（2022-27 的答案）。
3.「从用户态转到内核态是靠一条指令修改 PSW 的模式位完成的」——错。修改 PSW 的指令本身是特权指令，用户态根本执行不了。用户态到内核态是在响应中断或异常时由硬件自动完成的。
4.「中断处理程序负责把 CPU 从用户态转为内核态」——错。模式切换发生在中断响应阶段（硬件完成），中断处理程序开始执行时 CPU 已经处于内核态了。

*3.*状态切换的时机与方式

用户态 → 内核态（由硬件在中断 / 异常响应阶段自动完成，共四条途径）：

1. 用户程序请求操作系统服务，执行系统调用（主动）。
2. 发生外中断（如 I/O 完成、时钟信号到来）。
3. 用户程序执行过程中产生异常（如除零、缺页）。
4. 用户程序试图执行特权指令，触发非法指令异常。

内核态 → 用户态：由内核通过特定的返回指令（如 x86 的 `IRET`）完成，该操作在内核态执行。

两条「发生」与「执行」的区分

- 2012-23：问「不可能在用户态发生的事件」，答案是进程切换。系统调用在用户态发生（用户程序发起）、

外部中断和缺页也都可能在 CPU 执行用户程序时发生；只有进程切换必须由内核完成，用户态不可能发生。

- 2024-23：问「中断或异常发生时 CPU 处于内核态」，这是错的——事件发生的那一瞬间 CPU 可能还在用户态，只有响应中断的过程中硬件才切换到内核态。
- 因此做题时务必分清「事件发生时的模式」与「处理程序执行时的模式」，以及「在用户态发生 / 执行」两种不同问法。

内核栈与用户栈：每个进程都有用户栈和内核栈。用户栈运行在用户态，存放用户程序的函数局部变量、参数、返回地址，仅在用户态下被访问。内核栈运行在内核态，当进程通过系统调用、中断或异常进入内核态时，硬件自动切换到内核栈，它存放内核函数的调用链、临时变量及内核执行上下文。

每个进程都需要独立的内核栈：所有进程运行时都可能通过系统调用陷入内核态，若第一个进程陷入内核后主动让出 CPU，第二个进程再陷入内核时若共享内核栈，就会破坏第一个进程的栈数据。

普通函数调用不需要保存 **PSW**，因为全程发生在用户态、不涉及特权级切换；而允许多重中断时 **PSW** 必须保存，因为中断可能在用户态也可能在内核态响应，需要记录被中断时的模式以便正确返回。

真题考法：用户态 / 内核态

- **2011-24**：在用户态执行的是命令解释程序（shell）；进程调度程序、缺页处理程序、时钟中断处理程序都在内核态。
- **2013-28**：会导致用户态切到内核态的是整数除以零（异常）和 **read** 系统调用；`sin()` 函数调用不会（纯计算库函数全程用户态）。
- **2014-25**：不能在用户态执行的是关中断指令；trap 指令、跳转指令、压栈指令都可以在用户态执行。
- **2015-24**：不可能导致用户态转内核态的是 `NOT R0`（寄存器取非，不访存、不触发调用）；`DIV`（可能除零）、`INT n`（软中断）、`MOV`（访存可能缺页或越界）都可能。
- **2021-23**：只能在内核态执行的是 **I/O** 指令；trap、数据传送、设置断点指令都可在用户态。
- **2022-27**：CPU 处于用户态时只能执行非特权指令。
- **2023-26**：会导致 CPU 从内核态转为用户态的是执行系统调用（服务例程执行完毕返回时）；阻塞进程、执行 CPU 调度、唤醒进程完成时仍在内核态。

（二）中断和异常的处理

引入内核态与用户态后，操作系统必须解决两者之间的安全切换问题：系统在内核中设立若干称为「门」的受控入口，而用户程序进入内核态的唯一合法途径就是中断或异常。当 CPU 运行用户程序时一旦发生中断或异常，硬件会自动把处理器切换到内核态并跳转到对应处理程序，这一机制由硬件强制保障。

中断机制对现代操作系统至关重要：当某程序因等待 I/O 等事件暂时无法继续执行时，系统需及时将其挂起、转而调度其他就绪程序。在抢占式多任务系统中，这一调度主要依赖中断（尤其是时钟中断）触发。中断机制是多道程序设计得以实现的前提。

*1.*中断与异常的定义

- 中断（**Interrupt**），也称外中断：来自 CPU 执行指令外部的事件，通常用于信息输入 / 输出。例如设备发出的 I/O 结束中断（表示 I/O 操作已完成）、时钟中断（表示一个固定时间片已到，用于计时、进程调度或启动定时任务）。外中断与当前执行指令无关，只发生在指令执行结束后。
- 异常（**Exception**），也称内中断：来自 CPU 执行指令内部的事件，如程序的非法操作码、地址越界、运算溢出、虚存系统的缺页，以及专门的陷入指令（系统调用）等引起的事件。异常与

当前执行指令相关，一旦发生就必须立即处理。

*2.*中断与异常的分类

外中断（硬件中断）分为两类：

- 可屏蔽中断：通过 **INTR** 线发出的中断请求，通过改变屏蔽字可以实现多重中断，使中断处理更灵活（如时钟中断、I/O 中断）。
- 不可屏蔽中断：通过 **NMI** 线发出的中断请求，通常是紧急的硬件故障，如电源掉电。

异常（内中断）分为三类：

- 故障（**Fault**）：由指令执行引起的异常，如非法操作码、缺页故障、除数为 0、运算溢出、存储保护错。故障处理后返回第 **i** 条指令重新执行（例如缺页处理完成后重访原内存地址）。
- 自陷（**Trap**，也称陷入）：一种事先安排的「异常」事件，用于在用户态下调用操作系统内核程序，如条件陷阱指令、系统调用指令。它返回下一条指令（第 i+1 条）。
- 终止（**Abort**）：出现了使 CPU 无法继续执行的硬件故障，如控制器出错、存储器校验错。这类错误一般无法修复，处理程序只能强制退出程序。

从软硬件归属看：故障和自陷属于软件中断（程序性异常），终止和外中断属于硬件中断。

几个容易混淆的归类

- 时钟中断属于外中断（来自 CPU 外部的事件），不是内中断。
- 缺页属于内中断中的故障（**Fault**），不是外中断；缺页处理（调页、页面置换）只在核心态完成。
- 系统调用（访管指令 / **trap** 指令引发）属于内中断中的自陷，也叫访管中断。所以「系统调用属于内中断」是正确的说法。
- 异常不能被屏蔽——包括故障、自陷、终止，以及外中断里的不可屏蔽中断。
- 中断判优时，异常和不可屏蔽中断不受中断屏蔽字影响。

*3.*中断处理流程

以 CPU 正在执行用户程序的第 i 条指令为例，大致过程如下：

1. 检测与判优：CPU 在指令执行过程中发现异常，或在指令执行结束后检测到中断请求信号，并对同时到达的多个请求做优先级裁决。
2. 检查是否可响应：检查 EFLAGS 寄存器的 IF 标志，判断能否响应可屏蔽中断。
3. 查中断向量表：硬件根据中断类型号找到对应的中断向量，其中保存该处理程序的入口地址。
4. 特权级校验与切栈：若当前是用户态，硬件自动改模式位、转入内核态，并把栈指针从用户栈切到内核栈。
5. 保护现场：硬件在中断隐指令阶段自动保存断点（**PC**）和程序状态字（**PSW**）压入内核栈；操作系统软件保存通用寄存器与通用地址寄存器的内容以及中断屏蔽字（每位对应一种中断类型，用于屏蔽同级或低级中断、防止同类型中断嵌套，是按类型屏蔽而非全局关中断）。
6. 执行中断处理程序。
7. 恢复现场并返回：从内核栈弹出软件保存的寄存器与用户栈指针，执行 `IRET` 指令，由硬件自动弹出 EIP（PC）、CS、EFLAGS，恢复用户态执行流。

关于断点：异常在第 i 条指令执行过程中被发现，处理后返回第 i 条指令重新执行（如缺页处理后重访原地址）；中断在指令执行结束后被发现，处理后返回第 $i+1$ 条指令继续执行。若判定为不可恢复的致命错误，则终止该用户程序。

中断向量表：本质是一个数组，索引是中断类型号（连续整数编号），表项是中断服务程序的入口地址，由操作系统初始化。地址关系为：中断向量基址寄存器 + 中断信号偏移量 = 中断向量的地址。用数组实现是因为它需要随机访问且编号连续，能按索引在 $O(1)$ 时间内定位。

真题考法：中断处理

- **2009-22**：能引起外部中断的事件是键盘输入；除数为 0、浮点运算下溢、访存缺页都属于内部异常。
- **2012-24**：中断处理一定会保存而子程序调用不需要保存的是程序状态字寄存器。
- **2015-23**：处理外部中断时应该由操作系统保存的是通用寄存器的内容；PC 和 PSW 由硬件自动保存，TLB 和 Cache 由硬件透明管理。
- **2018-29**：时钟中断服务程序要更新「内核中时钟变量的值」「当前进程占用 CPU 的时间」「当前进程在时间片内的剩余执行时间」三项。
- **2020-27**：由操作系统完成的是「提供中断服务」「初始化中断向量表」「保存中断屏蔽字」；「保存被中断程序的断点」由硬件完成。
- **2022-31**：由操作系统完成的是「保存通用寄存器的内容」和「执行系统调用服务程序」；「保存断点和 PSW」「将 CPU 模式改为内核态」由硬件完成。
- **2023-24**：中断向量表适合采用的数据结构是数组。

*4.*中断处理与子程序调用的区别

比较项	中断处理	子程序调用
程序间关系	中断处理程序与被中断程序相互独立，无固定关联	子程序与主程序属于同一程序，是主从关系
触发方式	通常随机发生	由 CALL 指令显式触发，程序员事先安排
实现基础	需要硬件电路支持才能完成	完全由软件实现
入口地址来源	由中断向量表提供	由 CALL 指令的地址码指定
现场保护	中断隐指令（硬件）自动保存 PC、PSW；OS 保存通用寄存器	CALL 指令把当前 PC 压栈；不保存 PSW
优先级裁决	需对同时到达的多个请求判优	无此操作
执行时的特权级	被调用过程（系统调用）一定在内核态	可能用户态也可能内核态

（三）系统调用

*1.*系统调用的定义与功能

系统调用（**System Call**）是操作系统提供给应用程序（程序员）使用的接口，可视为一种特殊的公共子程序调用，是应用程序主动向操作系统发出的服务请求。

系统中各种共享资源均由操作系统内核统一管理，因此凡是与共享资源有关的操作（存储分配、**I/O** 传输、文件管理等）都必须通过系统调用向内核提出服务请求，由内核代为完成，再把结果返回应用程序——目的是保证系统的稳定性和安全性，防止用户程序进行非法操作。

一个操作系统提供的系统调用有几十条乃至上百条，每个系统调用都有唯一的系统调用号。按功能大致分为六类：

功能类别	具体内容
设备管理	设备的请求或释放、设备启动等
文件管理	文件的读、写、创建及删除等
进程控制	进程的创建、撤销、阻塞及唤醒等
进程通信	进程之间的消息传递或信号传递等
内存管理	内存的分配、回收，获取作业占用内存区的大小和起始地址等
系统控制	时钟设置、系统状态查询等

判断某个操作是否「通过系统调用完成」的诀窍是站在程序员的角度想：这件事是不是我（应用程序）主动请求的?

哪些操作必须通过系统调用，哪些不必

判断诀窍是站在程序员角度想：这件事是不是我（应用程序）主动请求的?

- 需要：创建新进程（`fork`）、打开 / 读写文件（`open`、`read`、`write`）、申请动态内存（`malloc` 空间不够时可能调用 `brk` / `mmap`）、`sleep` 睡眠、设备启动。
- 不需要：页置换（由缺页异常触发，是操作系统内部机制）、进程调度（由时钟中断或阻塞 / 唤醒事件触发）、生成随机整数（一般由用户态库函数 `rand()` 算得）、纯计算类库函数（`strlen`、`sqrt`、`sin`）。
- 陷阱：`printf` 最终会通过 `write` 输出数据，`malloc` 在小块分配时可能完全在用户态完成，所以「库函数一定不含系统调用」也是错的。

2. 系统调用与库函数的区别

比较项	系统调用	库函数
提供者	操作系统内核	编程语言的函数库
运行特权级	内核态（调用者发起时在用户态）	用户态
是否切换特权级	必须切换（用户态 → 内核态 → 用户态）	一般不切换
与共享资源的关系	凡涉及共享资源的操作必须使用	有些封装了系统调用，有些完全不含系统调用
是否可移植	不同操作系统接口不同（Windows 的 Win32 API 与 UNIX/Linux 的 POSIX 标准不同）	依赖于具体语言的运行库

因此：「不同的操作系统为应用程序提供了统一的系统调用接口」是错的——不同操作系统的内核架构不同，系统调用编号、语义和参数形式都可能不同，较统一的只是上层 API 标准。操作系统真正提供给应用程序的接口本质上仍是系统调用，库函数只是一层封装。

系统调用与普通过程调用的四点差异：

1. 运行状态不同：普通过程调用的调用者与被调用者运行在同一特权级；系统调用的调用者在用户态、被调用者在内核态。
2. 状态转换不同：普通过程调用无须特权级切换、可直接跳转；系统调用必须通过陷入指令（**Trap**）触发，先切至内核态再由内核分派。
3. 返回机制不同：系统调用执行完毕后，内核可能根据调度策略决定是否切换进程；普通过程调用总是直接返回原调用点。

4. 嵌套调用限制不同：系统调用的嵌套深度受内核栈空间限制；普通过程调用仅受用户栈大小约束。

3. 系统调用的执行过程

系统调用的四个阶段（必背）

① 传递系统调用参数 → ② 执行陷入指令（**trap**）并切换至内核态 → ③ 定位并执行相应的服务程序 → ④ 恢复现场并返回用户态

对应 2017-24 的答案是 ③→②→④→①。

1. 传递参数：用户程序把系统调用号及参数压入堆栈或放入指定的通用寄存器。
2. 执行陷入指令：该指令在用户态执行，执行后立即引发一个内中断（访管中断），硬件把 CPU 从用户态切到内核态；硬件把 PC、PSW 压入内核栈，内核（软件）保存通用寄存器内容。
3. 定位并执行服务程序：内核按系统调用号查询系统调用表（数组，表项为各服务例程的入口地址），执行服务逻辑。
4. 恢复现场并返回：内核恢复被中断进程的 CPU 现场，把模式切回用户态，控制权交还用户程序，从断点继续执行。

几个必须记住的细节：

- 陷入指令 = **trap** 指令 = 访管指令 = 自陷指令 = 陷阱指令，它是非特权指令，在用户态执行。
- 发出系统调用请求在用户态，对系统调用的响应处理在核心态。
- 系统调用号标识系统调用的类型；中断类型号（中断向量号）标识中断或异常的类型，两者不能混。
- 自陷指令触发异常时保存的断点是下一条指令的地址（若保存的还是该指令的地址，就会陷入死循环）。
- 由硬件完成：保存断点和 **PSW**、把 **CPU** 模式改为内核态；由操作系统完成：保存通用寄存器内容、执行系统调用服务程序。

```
/* 用户态发起系统调用的直观过程（x86 Linux 早期实现示意） */
/* 1. 传递参数：系统调用号放 eax，参数依次放 ebx、ecx、edx */
/* 2. 执行陷入指令（用户态执行），引发访管中断，CPU 转内核态 */
/* 3. 内核查系统调用表，执行对应的服务例程 */
/* 4. 返回用户态，从断点继续执行，返回值在 eax 中 */

int fd = open("data.txt", 0); /* 库函数封装，内部最终是一条 int 0x80 / syscall */
```

真题考法：系统调用

- **2010-23**：操作系统提供给应用程序的接口是系统调用。
- **2017-24**：执行系统调用过程的正确顺序是传递参数 → 陷入指令 → 执行服务程序 → 返回用户态。
- **2019-25**：「执行系统调用服务程序时 CPU 处于内核态」「操作系统通过提供系统调用避免用户程序直接访问外设」「系统调用是内核为应用程序提供服务的接口」正确；「不同操作系统提供统一的系统调用接口」错误。
- **2021-32**：通过系统调用完成的操作是创建新进程；页置换、进程调度不是应用程序发起的，生成随机整数一般无需系统调用。
- **2022-31**：见前文（由硬件完成的 vs 由操作系统完成的）。
- **2024-23**：每个系统调用都有对应的内核服务例程，系统添加新类型设备时需注册相应的中断服务例程，均正确。

（四）程序的链接与装入

*1.*从源程序到可执行程序的三步

把一个用户源程序变成可在内存中执行的程序，通常要经过三步：

1. 编译：由编译程序把用户源代码翻译成若干目标模块。
2. 链接：由链接程序把这些目标模块及其所需的库函数合并，形成一个完整的装入模块（可执行文件）。
3. 装入（加载）：由装入程序把装入模块载入内存，并启动其执行。

*2.*逻辑地址与物理地址

编译后的每个目标模块通常从 0 号单元开始编址，这种地址称为该模块的相对地址（逻辑地址）。链接程序把多个目标模块合并为完整的可执行程序时，依次把各模块的相对地址拼接成一个统一的、从 0 开始编址的逻辑地址空间（虚拟地址空间）。以 32 位系统为例，该逻辑地址空间的范围为 $0 \sim 2^{32}-1$。

进程运行时使用的地址都是逻辑地址：用户程序和程序员只需关注逻辑地址，底层的内存管理机制对其完全透明。不同进程可以拥有相同的逻辑地址，因为它们各自拥有独立的逻辑地址空间，这些地址会被映射到主存的不同位置。

物理地址空间是主存中所有物理存储单元的集合，是地址转换的最终目标——无论执行指令还是访问数据，最终都必须通过物理地址从主存读写。早期系统在程序装入内存时把逻辑地址转换为物理地址，这一过程称地址重定位；现代支持虚拟内存的系统则由内存管理单元（**MMU**）在硬件层面于运行时自动完成：操作系统为每个进程维护一张页表记录逻辑页到物理页框的映射，MMU 依页表动态换算，对用户程序完全透明。

*3.*三种链接方式

静态链接：在程序运行之前，把各目标模块及其所需的库函数链接成一个完整的装入模块，此后不再拆分。链接过程中需完成两项关键操作：

- 地址调整：各目标模块编译后均以 0 为起始地址，链接时需根据其在装入模块中的实际位置统一加上偏移量。例如模块 A 长度为 L、模块 B 紧邻其后存放，则原本从 0 编址的模块 B 在装入模块中从地址 L 处开始，其内部所有地址都要加上 L。
- 外部符号解析：把模块间引用的外部符号（如函数名）替换为确定的地址。若模块 A 中含语句 `CALL B`（B 是模块 B 入口的外部符号），链接程序会把它解析为模块 B 在装入模块中的起始地址 L。

装入时动态链接：编译后生成的一组目标模块在装入内存时采用边装入边链接的方式。装入某个模块时若遇到对外部模块的调用，装入程序立即查找并装入相应的外部目标模块，并替换其中的外部符号。优点是便于修改和更新（只需替换被修改的模块，无须重建整个程序）、便于实现对目标模块的共享。

运行时动态链接：对装入时动态链接的改进。在程序执行过程中，仅当需要调用某个尚未装入内存的目标模块时，操作系统才动态加载并链接该模块；运行中未使用的模块既不会被装入内存，也不会进行链接。优点是既能加快程序装入过程，又能节省内存空间。

链接方式	链接时机	优点	缺点
静态链接	程序运行之前	装入后运行无额外开销	模块修改需重新生成整个装入模块；无法共享目标模块
装入时动态链接	装入内存时边装入边链接	便于修改更新；便于共享目标模块	仍要把用到的模块全部装入，未用到的模块也占空间
运行时动态链接	执行过程中调用到时才链接	加快装入；节省内存，未用模块不装入	实现复杂，运行时才有链接开销

*4.*三种装入方式

绝对装入：仅适用于单道程序环境。内存中始终只有一个程序，其驻留位置在编译前即可确定，编译程序直接生成使用绝对地址的目标代码，装入程序按这些地址把程序和数据载入内存；此时逻辑地址与物理地址完全一致，无须任何修改。程序中的符号地址由编译器或汇编器在编译 / 汇编阶段转换为绝对地址；绝对地址也可由程序员直接指定，但缺乏灵活性。

可重定位装入（静态重定位）：多道程序环境下编译程序无法预知模块在内存中的位置，因此装入模块通常从地址 0 开始编址，程序中的地址都是相对于模块起始位置的逻辑地址。装入时系统根据内存空闲情况为它分配一块连续的内存区域，把整个模块载入，随后一次性把所有逻辑地址改为对应的物理地址。转换在装入时完成且运行期间不再改变，故称静态重定位。缺点是：进程必须在装入前一次性获得所需的全部内存空间，内存不足则无法装入；一旦装入，进程运行期间既不能移动位置，也不能动态申请额外内存。

动态运行时装入（动态重定位）：装入程序把模块载入内存后并不立即转换地址，而是把转换推迟到指令实际执行时进行：系统用重定位寄存器存放该模块在内存中的起始物理地址，程序中保留逻辑地址，物理地址由硬件在运行时计算（逻辑地址 + 重定位寄存器内容）。优点是支持程序在内存中移动，便于操作系统进行紧凑（合并分散的空闲块）、有效减少外部碎片，并允许在运行期间动态调整进程的内存布局。

装入方式	地址转换时机	适用环境	能否在内存中移动	典型配合
绝对装入	编译（汇编）时	单道程序环境	否	无
可重定位装入（静态重定位）	装入时一次性完成	多道程序环境	否	固定分区、单一连续分配
动态运行时装入（动态重定位）	指令执行时由硬件完成	多道程序环境	是	页式、段式等需移动进程的方案

重定位三问

1. 动态重定位依赖硬件支持：必须有重定位寄存器（基址寄存器）和地址变换机构，所以「动态重定位依赖目标程序本身」是错的。
2. 静态重定位不能移动进程：它把地址一次性改死，进程一旦装入就固定，所以「页式存储管理可以采用静态重定位」是错的（页式需要移动和换出换入）。
3. 固定分区可以采用静态重定位：作业装入后位置不变且占连续空间，正适合静态重定位。

（五）程序运行时的内存映像与地址空间

程序被加载运行时，操作系统为它建立一个虚拟地址空间，该空间的逻辑布局称为进程的内存映像，反映运行时代码、数据、堆、栈等部分的组织方式。与磁盘上的静态可执行文件不同，内存映像是动态的：虚拟内存机制下进程的整个地址空间无须全部驻留物理内存，只有在实际访问某一部分时才通过缺页中断把它从磁盘调入。

典型的进程虚拟地址空间从低地址到高地址依次为：

1. 只读代码段：含.init（启动时自动调用的初始化代码）、.text（主程序及自定义函数编译后的机器指令）、.rodata（只读数据，如字符串常量）。代码段只读，防止运行时意外修改指令，同时允许多个进程共享同一份代码以节省内存。
2. 读 / 写数据段：存放全局变量和静态变量。.data 存放已初始化的（如 int x = 5;）；.bss 存放未初始化的（如 int y;）。
3. 堆：运行时动态申请内存的区域，用 malloc() 申请、free() 释放。堆从低地址向高地址增长，大小不固定。
4. 共享库的存储映射区：提供进程依赖的公共函数代码（如 printf()），运行时动态映射到进程地址空间，只读且可被多个进程共享。
5. 用户栈：支持函数调用，存放局部变量、函数参数、返回地址等临时信息，由系统自动管理，遵循后进先出（**LIFO**）。栈从高地址向低地址扩展。

32 位 Linux 的典型布局中，0xFFFFFFFF 附近是操作系统内核区（对用户进程不可见），往下依次是用户栈、共享库映射区、堆、读 / 写数据段、只读代码段，最低处是未使用区。注意代码段和数据段的大小在程序加载时就已确定，而堆和栈可以在运行过程中动态扩展和收缩。

内存映像的三条必背结论

1. 增长方向：堆从低地址向高地址增长，栈从高地址向低地址增长，两者相向而生，中间是共享库映射区。
2. .bss 段存放未初始化的全局 / 静态变量，它在可执行文件中不占实际磁盘空间（只需记录大小），但在内存映像中占位。
3. **PCB** 不属于进程的虚拟地址空间：虽然 PCB 是管理进程的核心数据结构，但它由操作系统内核维护、存放在系统内核区中，对用户进程不可见。

```
/* 各区域存放内容对照示例 */
int g_init = 5;       /* .data : 已初始化的全局变量 */
int g_uninit;         /* .bss  : 未初始化的全局变量 */
const char *s = "hi";/* .rodata: 字符串常量（只读数据段） */

int main(void) {
    static int s_init = 1;  /* .data: 已初始化的静态变量 */
    int local = 3;          /* 栈    : 局部变量 */
    int *p = malloc(4);     /* 堆    : 动态申请，返回的地址交给指针 p */
    free(p);
    return 0;
}
```

真题考法

- 进程虚拟地址空间中各区域的存放内容（含.text、.data、.bss、堆、栈的归属判断）是近年新出现且反复出现的角度，要求看到一段 C 代码能立刻指出每个变量落在哪个区。
- 「代码段和数据段大小在加载时确定、堆栈可动态伸缩」也是常见判断点。

四、操作系统结构

随着操作系统功能不断扩展、代码规模持续增长，合理的结构设计对降低系统复杂度、提升安全性和可靠性至关重要。考纲要求掌握五种：分层、模块化、宏内核、微内核、外核。

（一）分层结构

分层法把操作系统划分为若干层，最底层（层 **0**）为硬件，最高层（层 **N**）为用户接口，每层只能调用紧邻它的下层所提供的功能和服务（单向依赖）。

优点：① 便于调试与验证，系统可自底向上逐层开发与测试——第 1 层完成且验证无误后再开发第 2 层，因为它只依赖已验证的下层；若某层出错，可把错误定位在该层内部，因为它的所有下层都已通过验证。② 易于扩充与维护，在保持层间接口不变的前提下，可以独立修改、替换或新增某一层的实现而不影响其他层。

问题：① 层次划分困难，合理界定各层的功能边界较复杂，且一旦依赖关系固化，系统架构可能缺乏灵活性。② 运行效率较低，执行一个高层功能通常需自上而下穿越多个中间层，每层之间的参数传递与控制转移都会引入额外开销。

分层的两条秒杀结论

看到「分层式架构」就要立刻想到：单向依赖（各层只能调用紧邻下层）与便于调试与验证 / 结构清晰。反过来，「具有非常灵活的依赖关系」必定是错误选项——分层法恰恰是牺牲灵活性换取清晰性。

（二）模块化结构

模块化把操作系统按功能划分为若干具有一定独立性的模块，每个模块实现特定的管理功能，并明确定义与其他模块之间的接口，通过接口通信；还可以进一步把模块细分为子模块，同样规定子模块之间的接口。这种方法称模块-接口法。

模块划分的粒度需要权衡：模块过小虽然降低了单个模块的复杂性，但会导致模块间交互频繁、增加系统整体复杂度；模块过大则模块内部结构复杂、难以维护。

模块的独立性至关重要——模块独立性越高、相互依赖越少，系统结构越清晰。衡量模块独立性的两个标准：

- 内聚性：模块内部各组成部分之间联系的紧密程度。内聚性越高，模块独立性越好。
- 耦合度：模块之间相互依赖和影响的程度。耦合度越低，模块独立性越好。

优点：① 提高了操作系统设计的正确性、可理解性和可维护性；② 增强了系统的可适应性，便于功能扩展；③ 支持并行开发，加速操作系统研制进程。

缺点：① 模块间的接口规定很难满足对接口的实际需求；② 各模块设计者齐头并进，每个决定无法建立在上一已验证的正确决定的基础上，因此无法找到一个可靠的决定顺序（即更难调试和验证）。

（三）宏内核

宏内核（也称单内核、大内核）把系统的主要功能模块集成在一个紧密耦合的整体中，统一运行于内核态，从而为用户程序提供高性能的系统服务。目前主流操作系统如 Linux、Windows、Android、macOS、iOS 都是宏内核或混合内核架构。

优点：性能高。各模块共享地址空间，可直接调用彼此的函数，减少了上下文切换和消息传递的开销，信息共享和相互调用都很方便。

缺点：随着体系结构和应用需求发展，需要操作系统提供的服务日益复杂、代码规模急剧膨胀，导致内核庞大、功能复杂、结构混乱、难以维护；更严重的是，大内核中某个功能模块出错就可能导致整个系统崩溃（例如文件系统运行在内核态，它一出问题系统就直接崩溃），可靠性下降，面临典型的「软件危机」。

需要说明的是，现在的宏内核早已不是当年纯粹的形式，而是广泛吸取微内核架构的优点后揉合而成的混合内核。例如 macOS 和 iOS 采用的 XNU 内核即融合了 Mach 微内核与 BSD 宏内核的特性；谷歌的 Fuchsia 和华为的 HarmonyOS NEXT 则采用了微内核架构。这正说明宏内核与微内核并非对立，而是相互借鉴、融合发展。

（四）微内核

微内核架构把操作系统中最基本的功能保留在内核，其他非核心功能移到用户态执行，以降低内核的设计复杂性。移出的功能组织为若干独立的服务程序（服务器），彼此之间以及与客户进程之间的交互都通过微内核提供的消息传递机制完成。所以微内核结构把操作系统划分为两大部分：微内核 + 多个服务器。

微内核是精心设计的小型内核，仅实现最核心的机制，通常包含：① 与硬件紧密相关的部分；② 基本的资源管理机制；③ 客户与服务器之间的通信支持。它遵循「机制与策略分离」原则——与硬件紧密相关的机制放入微内核，策略交由用户态服务器实现：

微内核中的机制部分	用户态服务器中的策略部分
进程 / 线程的创建、切换、调度队列管理、多处理机同步	进程如何分类、优先级如何确认
低级存储器管理：页表管理、逻辑地址到物理地址的映射等地址变换机制	采用何种页面置换算法、如何进行内存分配与回收
中断和陷入的捕获与分发（识别事件类型后转交服务器）	各类中断 / 异常事件的具体处理逻辑

于是进程管理、存储器管理和 I/O 管理都被一分为二：属于机制的很小一部分在内核，绝大部分在微内核外的各种服务器中实现。大多数服务器都要比微内核大，因此微内核能保持极小规模。

优点（2023 统考真题的四个正确项）：

1. 扩展性和灵活性：新增或修改功能只需调整相应服务器，无须改动内核代码。
2. 可靠性和安全性：只有微内核运行在内核态，其余模块都运行在用户态，一个模块中的错误只会使这个模块崩溃，而不会使整个系统崩溃（文件服务出问题只需强行停止再重启，系统不会崩）；同时内核代码量和特权代码量少，攻击面更小。
3. 可移植性：与 CPU 和 I/O 硬件有关的代码均放在内核中，其他各种服务器与硬件平台无关，因而移植到另一平台所需修改较小。
4. 分布式计算：客户和服务器之间、服务器和服务器之间采用消息传递机制，使微内核系统能很好地支持分布式系统和网络系统。

缺点（主要是性能差）：每次服务请求都需要频繁地在核心态和用户态之间进行切换，操作系统的执行开销偏大；用户态下的各功能模块不能直接相互调用，只能通过内核中的「消息传递」间接通信，性能因此降低。为改善效率，可以把频繁使用的系统服务移回内核，但这又会使微内核的容量明显增大。

尽管宏内核在桌面操作系统中取得了绝对胜利，微内核因其高可靠性和可验证性，在实时、工业控制、航空航天及军事等关键任务领域特别流行。

（五）外核

外核（**exokernel**）的设计思想截然不同：尽可能减少抽象层次，把物理资源直接暴露给用户级系统，自身仅负责资源的安全分配与隔离。

外核是运行于内核态的极小程序，核心职责是把底层硬件资源（磁盘块、内存页、网络包缓冲区等）划分为互不重叠的子集分配给不同的用户级系统（例如一个系统分到磁盘 0 ~ 1023 块、另一个分到 1024 ~ 2047 块），并检查各系统只能访问被授权的资源。应用请求资源时外核只检查是否空闲，空闲就直接交给应用，怎么访问由应用自己决定。外核分离了对硬件的使用与保护，使应用能直接使用未经内核抽象的硬件。其上可在用户空间构建库操作系统（**library OS**）自行实现文件系统、进程调度、虚拟内存管理等传统功能，由于直接操作物理资源、无须经过强制抽象层，避免了不必要的数据拷贝、地址转换与上下文切换开销。

优点：

1. 减少抽象开销（减少了映像层）：传统操作系统强制引入虚拟内存、虚拟文件系统等映射层，每个虚拟机都以为自己的磁盘块号从 0 到最大编号，虚拟机监控程序因而必须维护重映像表；有了外核，这个重映像处理就不需要了，外核只需记录已分配给各虚拟机的资源。
2. 提升灵活性与性能：用户级系统可按应用特性优化资源管理策略（例如数据库直接管理磁盘块），充分发挥硬件潜力。
3. 清晰分离保护与实现：外核仅负责资源保护，功能实现完全置于用户空间，结构简洁且易于验证；它把多道程序（在外核内）与用户操作系统代码（在用户空间内）加以分离，负载不重，因为外核所做的只是保持多个虚拟机彼此不发生冲突。

（六）五种结构对比

结构	核心思想	优点	缺点	代表
分层	分若干层，层 0 是硬件、层 N 是用户接口，各层只能调用紧邻下层（单向依赖）	便于调试验证、结构清晰；易扩充维护	层次划分困难、依赖关系固定不灵活；穿越多层效率低	早期的 THE、Multics
模块化	按功能划分模块，规定接口通信，可再分子模块（模块-接口法）	正确性 / 可理解性 / 可维护性提高；可适应性强；支持并行开发	接口难以设计；各模块决定无先后依赖，难以找到可靠的决定顺序，更难调试验证	多数现代系统采用模块化思想
宏内核	主要功能模块紧密耦合，全部运行在内核态	性能高，模块间可直接调用、信息共享方便	内核庞大复杂、难维护；一个模块出错可能导致整个系统崩溃	Linux、Windows、Android
微内核	只保留最基本功能在内核，其余作为用户态服务器，靠消息传递通信	扩展灵活、可靠安全、可移植、支持分布式	频繁特权级切换与消息传递导致性能差	Minix、Fuchsia、HarmonyOS NEXT
外核	内核只负责资源分配与保护，把物理资源直接交给用户级系统，由库操作系统实现功能	减少抽象与重映像开销、灵活高效、保护与实现分离	用户级系统实现负担重，缺乏统一抽象，应用兼容性差	Exokernel 实验系统

五、操作系统引导

操作系统（如 Windows、Linux 等）以程序形式存储在硬盘等外部存储设备中。操作系统引导是指计算机通过执行一系列引导程序，依次定位启动设备、确定可引导分区、加载其中的操作系统内核，最终完成系统启动的链式过程。

（一）引导的完整流程

1. **CPU** 激活与 **BIOS** 初始化。通电后 CPU 的 `RESET` 引脚产生逻辑值复位 CPU；CPU 唤醒后在 `0xFFFFFFF0` 处执行一条长跳转指令，跳到固化在 ROM 中的启动代码处（即 **BIOS**，基本输入 / 输出系统），把它加载到 RAM 执行。BIOS 中主要存放自诊断程序、**CMOS** 设置程序、系统自动装载程序与主要 I/O 驱动和中断服务；它还在内存最开始处建立临时的中断向量表供引导过程使用。BIOS 针对具体主板设计，与所安装的操作系统无关。
2. 硬件自检（**POST**，**Power-On Self-Test**）与启动设备选择。BIOS 检查硬件能否满足运行的基本条件；有问题则蜂鸣报警、启动中止，无问题则显示 CPU、内存、硬盘等信息。随后 BIOS 依据 **CMOS** 设置（主板上由电池供电、保存启动顺序与系统时间等参数的配置存储器）或 **UEFI** 固件确定启动顺序（**Boot Sequence**）——排在前面的设备优先获得控制权。
3. 加载主引导记录 **MBR**。BIOS 按启动顺序把控制权转交给排在第一位的存储设备，即读取该设备的第一个扇区（最前面的 **512** 字节）并放入内存指定位置（BIOS 的最后一个任务）。若这 512 字节的最后两个字节是 `0x55` 和 `0xAA`（主引导记录签名，标识该磁盘是否为引导盘），该设备就可用于启动；否则控制权转交给启动顺序中的下一个设备。这 512 字节就是主引导记录（**MBR**），作用是告诉计算机到硬盘的哪个位置去找操作系统，由三部分组成：

字节范围	内容
第 1 ~ 446 字节	调用操作系统的机器码（引导代码）
第 447 ~ 510 字节	分区表（Partition Table），共 64 字节，分 4 项、每项 16 字节
第 511 ~ 512 字节	主引导记录签名 `0x55` 和 `0xAA`

每个主分区的 16 个字节中，第 **1** 个字节若为 `0x80` 就表示该主分区是激活分区（活动分区），控制权要转交给它；四个主分区里面只能有一个是激活的。其余字节记录分区的起止物理位置、分区类型、第一个扇区的逻辑地址和扇区总数。扇区总数只有 4 字节、最多表示 2^{32} 个扇区，若每扇区 512 字节则单个分区最大不超过 **2TB**；要使用更大硬盘只能提高每扇区字节数或增加扇区总数（GPT 方案解决的正是这个问题）。

1. 定位活动分区并加载分区引导记录 **PBR**。MBR 中的引导代码扫描分区表找到标记为「活动」的条目，读取其第一个扇区——分区引导记录（**PBR**），有的资料称卷引导记录 **VBR**——加载到内存。PBR 包含该分区特定的引导代码：识别并加载该分区的文件系统驱动（如 FAT32 / NTFS 驱动）、文件系统元数据（簇大小、根目录位置等），以及指向启动管理器文件的指针。
2. 加载启动管理器。按分区引导记录找到并加载活动分区内的启动管理器（`boot loader`），如 Linux 的 **GRUB**、Windows 的 **Bootmgr**，它提供多操作系统或内核版本的选择界面。
3. 加载操作系统内核。启动管理器把用户选定的内核映象加载到内存并跳转执行。内核完全接管系统控制权，重建自己的中断处理机制（替换 BIOS 的实模式中断向量，建立页故障、系统调用、外设中断等机制），不再依赖 **BIOS** 提供的服务，同时启用分页机制、建立内核页表。
4. 内核初始化与用户环境启动。内核初始化内存管理（内核页表、进程地址空间框架）、进程调度（PCB 链表、就绪队列与阻塞队列）和设备驱动子系统，然后启动第一个用户态进程（如 Linux

的 `init`，PID 为 1），由它加载系统各模块直至出现登录界面，启动过程完成。

引导链一句话记忆

CPU 复位 → **ROM** 中的 **BIOS/UEFI** → 硬件自检 **POST** → 按 **CMOS** 启动顺序 → **MBR**（**446** 引导码 + **64** 分区表 + **0x55AA** 签名）→ 活动分区 → **PBR** → 启动管理器（**GRUB/Bootmgr**）→ 加载内核到内存 → 内核接管、重建中断机制、初始化 → 启动 **init**（**PID=1**）

引导过程的易错点

1. **MBR** 只有 **512** 字节，位于磁盘第 **1** 个扇区（**0** 号磁头 **0** 道 **1** 扇区），签名 `0x55` 和 `0xAA` 在最后两个字节。
2. 分区表只有 **64** 字节，只能存 **4** 个主分区项；四个主分区里有且只有一个可以标记为激活。
3. 「计算机开机后操作系统所处的位置」答案是「以上都不对」——先执行 ROM 中的引导程序，把 MBR 中的程序放入内存执行，再由它把操作系统装入内存并移交 CPU 控制权；操作系统本身既不在 ROM 也不在 MBR。
4. 操作系统最终被加载到 **RAM**（不是 BIOS、ROM、EPROM），因为内核需要被 CPU 频繁读写并建立运行时数据结构。
5. 中断向量表的两次建立：BIOS 阶段在内存最低端建立临时中断向量表供引导使用，操作系统初始化阶段重建属于自己的中断向量表，所以「中断向量表需要在操作系统初始化过程中创建」正确；而硬盘分区表在磁盘分区时创建、文件系统根目录和索引节点表在格式化文件系统时创建（2022-24 的干扰项）。
6. 实地址模式：开机后系统首先工作在实地址模式下（Intel 为 80286 及其后处理器提供的 8086 兼容模式，20 位地址空间即 1MB，不支持分页存储管理）；一旦进入保护模式就不再使用 BIOS。BIOS 通过 `INT` 指令调用中断服务程序把内核从磁盘装入内存，例如 `int 0x19` 把启动盘 0 号磁头对应盘面 0 磁道 1 扇区中的引导程序装入内存。

真题考法：操作系统引导

- **2013-29**：计算机开机后操作系统最终被加载到 **RAM**。
- **2022-24**：需要在操作系统初始化过程中创建的是中断向量表。
- **2021**、**2022** 统考均考查过引导的阶段性任务划分（谁把什么加载到哪、哪一步依赖中断机制），要求能把上面的链条按顺序说清。

六、虚拟机

（一）虚拟机与虚拟机管理程序

虚拟机（**Virtual Machine**，**VM**）是指通过虚拟化技术，把一台物理计算机抽象为多台逻辑上独立的计算环境，每台虚拟机都表现为完整硬件系统的精确复制品。虚拟化技术的核心在于隐藏底层物理硬件的细节，向上层提供统一且隔离的执行平台，每台虚拟机都可独立运行一个操作系统。

提供虚拟化功能的程序称虚拟机管理程序，同义术语还有虚拟机监控程序、**Virtual Machine Monitor**（**VMM**）、**Hypervisor**。注意提供虚拟机管理程序并不属于操作系统的任务——第二类虚拟机管理程序本身就是应用程序。两个必须区分的主体：运行在底层硬件上的操作系统称宿主操作系统（**Host OS**），运行在虚拟机中的操作系统称客户操作系统（**Guest OS**）。

（二）第一类虚拟机管理程序（裸金属架构）

第一类虚拟机管理程序直接运行在物理硬件之上，从技术上讲它就像一个操作系统，因为它是系统中唯一处于最高特权级的软件。它具备多道程序功能，向上层提供若干台虚拟机；每台虚拟机都是裸机的精确复制品，因此可以在不同虚拟机上运行任何不同的操作系统。

虚拟机作为用户态的一个进程运行，不允许执行敏感指令；然而虚拟机上的操作系统认为自己运行

在内核态（实际上不是），这种状态称为虚拟内核态。当客户操作系统执行一条仅允许在真实内核态下执行的敏感（特权）指令时，会触发异常并陷入虚拟机管理程序：在支持硬件虚拟化的 CPU（Intel VT-x、AMD-V）上，管理程序能判断指令来自客户操作系统还是用户程序，前者代为完成、后者模拟真实硬件的异常响应；在早期不支持硬件虚拟化的 CPU 上，所有敏感指令被透明重定向到管理程序，由它用软件模拟。

（三）第二类虚拟机管理程序（寄居架构）

第二类虚拟机管理程序以应用程序的形式运行在宿主操作系统（如 Windows 或 Linux）之上，依赖宿主系统进行资源分配与调度，本质上是一个用户态进程；尽管如此，它仍向客户操作系统伪装成完整的硬件平台。**VMware Workstation 是 x86 平台上首个广泛使用的第二类虚拟机管理程序。**

首次启动时，第二类虚拟机管理程序模拟一台新计算机的行为，期望找到的驱动器可以是虚拟设备；用户把客户操作系统安装到虚拟磁盘上（实际上只是宿主操作系统中的一个文件），安装完成后即可正常启动和运行。

虚拟化技术在 Web 主机领域应用广泛：若无虚拟化，服务商只能提供共享托管（用户无法控制软件环境）或独占托管（成本高昂）；借助虚拟化，单台物理服务器可同时运行多个虚拟机，每台虚拟机对用户而言都是一台完整的服务器，支持自定义操作系统与软件，且成本显著降低——这正是当前主流云服务器服务的基础。

（四）两类虚拟机管理程序对比

比较项	第一类（裸金属架构）	第二类（寄居架构）
运行位置	直接运行在硬件之上，唯一处于最高特权级	运行在宿主操作系统之上，本质是用户态进程
资源分配方式	直接控制和分配硬件资源，分配方式类似外核	依赖宿主操作系统分配和调度资源
性能	更好	较差，需要宿主操作系统中介
可支持的虚拟机数量	更多	较少，宿主操作系统自身也消耗资源
运行模式	运行在最高特权级，可执行最高特权指令	部分运行在用户态、部分在内核态，部分特权指令会被 VMM 截获
可迁移性	较差	导出镜像文件即可迁移
典型代表	VMware ESXi、Xen、KVM	VMware Workstation、VirtualBox

真题考法：虚拟机

近年统考考查过虚拟化技术的原理与特点（如两类 VMM 的位置判断、「客户操作系统认为自己处于内核态」这一虚拟内核态的说法是否正确）。答题要点是抓住：第一类 **VMM** 独占最高特权级、第二类 **VMM** 寄居在宿主 **OS** 之上，以及客户操作系统的「内核态」是虚拟的。

本章小结表

考纲条目	核心内容	必须记住的结论
操作系统的概念	控制和管理软硬件资源、合理调度任务与分配资源、为上层提供统一接口与运行环境的最基本系统软件	四层结构：硬件 → 操作系统 → 应用程序 → 用户；裸机 + OS = 扩充机器
双重角色	系统资源的管理者；用户与硬件之间的接口；实现对资源的抽象与扩充	用户接口（命令接口 + GUI）与程序接口（系统调用）不能混淆
四大功能	处理机管理、存储器管理、文件管理、设备管理	处理机管理本质上是进程管理；OS 管文件的组织方式而非内容
四大特征	并发、共享、虚拟、异步	并发与共享互为存在条件；没有并发就谈不上虚拟；并发才能导致异步
并发与并行	并发是同一时间间隔内、宏观同时微观交替；并行是同一时刻真正同时	并行需要多处理机 / 多流水线等硬件支持；单处理机上进程只能并发
手工操作阶段	用户独占全机，人工控制全过程	此阶段没有操作系统；CPU 利用率极低
单道批处理	内存中始终保持一道作业，脱机 I/O + 监督程序	特征为自动性、顺序性、单道性；I/O 时 CPU 空闲
多道批处理	内存驻留多道程序，I/O 时借助中断切换	多道、宏观并行、微观串行；优点利用率高吞吐大，缺点无交互响应长
分时系统	时间片轮转，多用户共用一台主机	同时性、交互性、独立性、及时性；首次实现人机交互
实时系统	按紧迫性 / 截止时间调度，抢占式	及时性、可靠性；分硬实时与软实时
各类系统排序	——	资源利用率：批处理 > 分时 > 实时；交互性：分时 > 实时 > 批处理；及时性：实时 > 分时 > 批处理
CPU 运行模式	内核态（管态）与用户态（目态），由 PSW 的模式位标识	内核态可执行全部指令；用户态只能执行非特权指令
特权指令	I/O、开关中断、置时钟、修改 PSW 与内存保护寄存器、改中断向量表、内核态转用户态	用户程序执行特权指令会触发非法指令异常，由硬件强制实施
状态切换的途径	系统调用、外中断、异常、试图执行特权指令	用户态 → 内核态由硬件在中断响应时自动完成；内核态 → 用户态由 `IRET` 完成
中断与异常	中断是外中断（外设、时钟）；异常是内中断（故障、自陷、终止）	外中断分可屏蔽（INTR）与不可屏蔽（NMI）；异常不可屏蔽
中断处理流程	判优 → 查中断向量表 → 特权级校验与切栈 → 保护现场 → 执行处理程序 → 恢复现场	硬件保存 PC 和 PSW；操作系统保存通用寄存器和中断屏蔽字
中断向量表	索引为中断类型号，表项为处理程序入口地址	本质是数组，由操作系统初始化，支持随机访问
系统调用	应用程序主动请求内核服务的唯一合法接口	与库函数不同：内核提供、必须切换特权级；不同 OS 接口不统一
系统调用过程	传参 → 陷入指令（用户态）→ 内核态执行服务程序 → 返回用户态	陷入指令 = trap = 访管指令，在用户态执行；硬件保存断点与 PSW

考纲条目	核心内容	必须记住的结论
链接方式	静态链接、装入时动态链接、运行时动态链接	越晚链接越省空间、越灵活，但运行时开销与实现复杂度更高
装入方式	绝对装入、可重定位装入（静态重定位）、动态运行时装入（动态重定位）	静态重定位装入后不能移动；动态重定位需要重定位寄存器等硬件支持
内存映像	由低到高：代码段、数据段、堆、共享库映射区、栈	堆向高地址增长、栈向低地址增长；PCB 不在进程虚拟地址空间内
分层结构	层 0 是硬件、层 N 是用户接口，单向依赖	便于调试验证、易扩充维护；划分困难、效率较低
模块化结构	按功能划模块、定接口，模块-接口法	内聚性越高、耦合度越低，模块独立性越好
宏内核	主要功能模块紧密耦合，全部运行在内核态	性能高；内核臃肿难维护，一个模块出错可能全系统崩溃
微内核	只保留最基本机制在内核，其余为用户态服务器，靠消息传递	扩展灵活、可靠安全、可移植、支持分布式；性能差
外核	内核只管资源分配与保护，资源直接交给用户级系统	减少映像层与抽象开销；用库操作系统实现传统功能
操作系统引导	BIOS/UEFI → POST → MBR → PBR → 启动管理器 → 内核 → init	MBR 共 512 字节（446 + 64 + 2），签名 `0x55AA`，只能有 4 个主分区项
虚拟机	把一台物理机虚拟为多台逻辑独立、硬件完整的虚拟机	第一类为裸金属架构（VMM 在最高特权级）；第二类为寄居架构（VMM 是应用）

易错清单

1. 「内核态只能执行特权指令」错（内核态能执行所有指令）；「用户态只能执行非特权指令」对——2022-27 的直接考点。
2. 修改 **PSW** 模式位的指令本身是特权指令，所以用户态 → 内核态不可能靠执行指令完成，一定是中断 / 异常响应时由硬件自动切换。
3. 「中断处理程序负责切换模式」错：中断处理程序开始执行时 CPU 已经在内核态了；「中断或异常发生时 **CPU** 处于内核态」也错：事件发生那一瞬间 CPU 可能仍在用户态。要分清「发生」与「处理」，2012-23 与 2024-23 都考这个区分。
4. 进程切换不可能在用户态发生（2012-23），但系统调用、外部中断、缺页都可能在 CPU 执行用户程序时发生。
5. **trap**、跳转、压栈、数据传送、设置断点指令都是非特权指令，可在用户态执行；只有 **I/O** 指令、开 / 关中断指令这类才必须在内核态（2014-25、2021-23）。
6. `NOT RO` 不会引起模式切换（不访存、不触发调用）；而 `DIV`（可能除零）、`INT n`（软中断）、`MOV`（访存可能缺页或越界）都可能（2015-24）。
7. 中断与异常的归属容易搞反：时钟中断是外中断；缺页、除零、非法操作码、系统调用都是内中断（异常），其中缺页属故障、系统调用属自陷。
8. 异常不能被屏蔽，不可屏蔽中断也不能被屏蔽；中断屏蔽字是按类型屏蔽而非全局关中断。
9. 硬件保存断点（**PC**）和 **PSW**，操作系统保存通用寄存器和中断屏蔽字（2015-23、2020-27、2022-31 反复正反问）。

10. 普通函数调用不保存 **PSW**（全程同特权级）；允许多重中断时才必须保存，因为可能在用户态也可能在内核态响应中断。
11. 中断向量表用数组实现，因为需要按下标随机访问（2023-24）；不要选队列或链表。
12. 「不同的操作系统为应用程序提供统一的系统调用接口」是错的；「操作系统提供给应用程序的接口」指的是系统调用，不是库函数、中断或原语（2010-23、2019-25）。
13. 库函数不等于系统调用：`sin()`、`strlen()`、`rand()` 完全在用户态；`printf` 底层会调用 `write`，`malloc` 在大块分配时可能调用 `brk` / `mmap`。
14. 创建新进程需要系统调用，页置换和进程调度不需要（前者由缺页异常自动触发，后者由时钟中断等隐式触发）——2021-32 的考点。
15. 系统调用顺序必须是「先传参、再陷入、后执行服务程序、最后返回用户态」，不要把返回用户态排到中间（2017-24）。
16. 多道批处理系统不必支持虚拟存储管理（2022-23）；多道程序系统的系统开销并不更小（2017-28 的干扰项）；它也不提高单个作业的运行速度，提升的是资源利用率与吞吐量。
17. 进程数越多 **CPU** 利用率不一定越高，进程过多会因争抢 CPU 反而降低利用率。
18. 静态重定位的进程装入后不能移动、也不能动态申请内存；需要移动进程（页式、紧凑操作）的场景必须用动态重定位。
19. `.bss` 存放未初始化的全局 / 静态变量，已初始化的放 `.data`；代码段只读；堆向高地址增长、栈向低地址增长，方向相反不能记反。
20. **PCB** 不属于进程的虚拟地址空间，它由内核维护、对用户进程不可见。
21. **MBR** 只能描述 **4** 个主分区，四个里只有一个可以是激活分区；签名 `0x55AA` 位于第 511 ~ 512 字节；开机时操作系统既不在 **ROM** 也不在 **MBR** 中，最终落在 RAM。
22. 中断向量表会被建立两次：BIOS 阶段建立临时表供引导使用，操作系统初始化时重建属于自己的中断向量表；分区表、文件系统根目录、索引节点表都不是操作系统初始化阶段创建的（2022-24）。
23. 宏内核性能高但可靠性差，微内核可靠性高但性能差；「微内核具有较好的性能」是错误选项（2023-23 中 Ⅰ 项为错）。
24. 第二类虚拟机管理程序本身是应用程序，「提供虚拟机管理程序属于操作系统的任务」是错的；客户操作系统的「内核态」是虚拟内核态。

第二章进程管理

本章考纲要求

（一）进程与线程

1. 进程与线程的基本概念
2. 进程/线程的状态与转换
3. 线程的实现：内核支持的线程，线程库支持的线程
4. 进程与线程的组织与控制
5. 进程间通信：共享内存，消息传递，管道，信号

（二）CPU 调度与上下文切换

1. 调度的基本概念
2. 调度的目标
3. 调度的实现：调度器/调度程序（scheduler），调度的时机与调度方式（抢占式/非抢占式），闲逛进程，内核级线程与用户级线程调度
4. CPU 调度算法
5. 多处理机调度
6. 上下文及其切换机制

（三）同步与互斥

1. 同步与互斥的基本概念
2. 基本的实现方法：软件方法，硬件方法
3. 锁
4. 信号量
5. 条件变量
6. 经典同步问题：生产者-消费者问题，读者-写者问题，哲学家进餐问题等

（四）死锁

1. 死锁的基本概念
2. 死锁预防
3. 死锁避免
4. 死锁检测和解除

本章是操作系统分值与难度最高的一章。历年统考中，本章在选择题中稳定占 3～4 分，在综合应用题中几乎每年出现（PV 操作题 7～9 分、银行家算法或调度算法计算题 6～8 分）。本章是必须拿到满分的大章。

一、进程与线程

（一）进程与线程的基本概念

*1.*进程的定义与进程映像

程序是存放在磁盘上的静态指令和数据集合；要让程序运行，必须把它的指令和数据装入内存，并为它分配 CPU、栈等资源。进程（**Process**）就是程序在一个数据集合上的一次动态执行过程，是系统进行资源分配和调度的独立单位。

进程在某一时刻的全部状态信息称为进程映像（也称进程实体），由三部分构成：

组成部分	内容	特点
PCB（进程控制块）	进程描述信息、控制管理信息、资源清单、处理机上下文	常驻内存，进程存在的唯一标志
程序段	二进制机器码、只读常量（字符串字面量、`const` 全局常量）	静态、只读、可共享
数据段	全局/静态变量（`.data`、`.bss`）、堆（动态分配）、栈（函数调用）	动态、可读可写、进程私有

数据段的细分值得记牢：`.data` 存放已初始化的全局变量和静态变量；`.bss` 存放未初始化（或初始化为 0）的全局变量和静态变量，可执行文件中只记录” 需要多大空间”，装入时由操作系统清零；堆由程序员手动申请释放（`malloc/free`），从低地址向高地址增长，忘记释放即” 内存泄漏”；栈由编译器自动管理，保存局部变量、函数参数、返回地址，从高地址向低地址增长，递归过深会栈溢出。

进程与程序的区别：

- 程序是静态的永久存在，进程是动态的、有生命周期的；
- 进程 = 程序 + 执行状态，即” 进程是程序的一次执行”；
- 同一个程序可以对应多个进程（例如同时打开多个浏览器窗口），它们代码段相同，但数据段、堆、栈各不相同；
- 一个进程也可以执行多个程序。

创建与撤销的本质

创建进程实质上是创建进程实体中的 **PCB**；撤销进程实质上是撤销进程的 **PCB**。

真题考法

2023 年曾考” 进程虚拟空间”、2021 年曾考” 进程组织之进程控制块（PCB 内容）”。选择题常问” 下列哪一项不属于 PCB 的内容” 或” 进程切换时必须更新哪些寄存器”。

*2.*线程的基本概念

引入进程是为了让多道程序并发执行、提高资源利用率和吞吐量；但进程作为资源拥有单位，创建、撤销、切换的系统开销都很大。于是引入线程（**Thread**）——” 轻量级进程”。

线程是程序执行流的最小单位，是处理机独立调度和分派的基本单位；进程是资源分配的基本单位。

线程自己私有的部分（即线程上下文）：

- 线程标识符 TID；
- 程序计数器 PC；
- 一组通用寄存器（线程现场）；
- 栈（每个线程一个独立的栈，保存局部变量、函数调用上下文）。

同一进程内的所有线程共享：代码段、数据段（全局变量、静态变量）、堆、打开的文件描述符、网络套接字、信号处理程序、进程 ID 与父进程 ID、地址空间（页表）。

一句话记忆

线程私有：**TID**、**PC**、寄存器、栈。其余全共享。

因为每个线程有自己的栈，线程的局部变量是线程私有的。如果需要线程间共享数据，值必须放在堆或全局可访问位置。

易错

”同一进程的线程还可以修改别的线程栈里的数据吗？”从地址空间角度看可以（因为共享地址空间），但同一进程的线程属于同一用户，编程时会主动避免这种非法访问。统考按”栈是线程私有的”理解即可。

*3.*进程与线程的比较

比较项	进程	线程
定义	资源分配的基本单位，程序在一个数据集上的一次动态执行	处理机调度的基本单位，进程中的一个执行实体
调度	由操作系统调度	可由用户空间线程库调度，也可由操作系统调度
资源	拥有独立的内存空间和系统资源	基本不拥有资源，共享所属进程的全部资源
地址空间	进程间相互独立（内存保护）	同进程内线程共享地址空间
并发性	进程间可并发	同进程内线程可并发；线程切换开销小，系统并发度更高
系统开销	创建、撤销、切换开销大	创建、撤销、切换开销小
通信	必须借助 IPC 机制	直接读写进程内共享变量即可
独立性	一个进程崩溃不影响其他进程	一个线程崩溃可能导致整个进程崩溃
多处理机	单线程进程只能在一个处理机上运行	多线程进程的不同线程可分配到不同处理机上并行执行
相似之处	PCB、PID、就绪/运行/阻塞三态、父进程创建子进程、不同进程可执行同一程序	TCB、TID、就绪/运行/阻塞三态、父线程创建子线程、不同线程可执行同一程序

线程相比进程减少开销的原因（高频考点）：

1. 线程创建比进程创建快——不必维护内存管理信息、文件管理信息，而是共享它们；
2. 线程终止比进程终止快——释放的资源少得多；
3. 同进程内线程切换比进程切换快——它们共用同一页表，切换时不用切换页表（页表切换在上下文切换开销中占比很大）；
4. 同进程线程间数据传递不经过内核，交互效率更高。

易错

- 页表基址寄存器（页目录基址寄存器）在同一进程的不同线程间切换时不需要修改，因为共用同一个虚拟地址空间；同时原有 TLB 不会失效。切换时要更新的只有程序计数器、通用寄存器组、栈基址寄存器。
- ”操作系统为每个用户级线程建立一个线程控制块”是错误说法（2019 年真题）。内核级线程才有内核维护的 TCB，用户级线程的 TCB 由用户态线程库维护。

（二）进程与线程的状态与转换

*1.*三态模型与五态模型

三种基本状态：

状态	含义
运行态（**Running**）	占有 CPU 正在执行。单处理机下任一时刻至多一个进程处于运行态，多处理机下可以有多个
就绪态（**Ready**）	已获得除处理机以外的一切所需资源，一旦得到处理机便可立即执行
阻塞态（等待态，**Waiting/Blocked**）	正在等待某资源可用（不包括处理机）或等待 I/O 完成，即使处理机空闲也不能运行

五种状态（加上创建态、终止态）：

状态	含义
创建态（**New**）	进程正在被创建，已分配 PCB，正在分配除 CPU 外的其他资源
就绪态	见上
运行态	见上
阻塞态	见上
终止态（**Exit**）	进程运行终止后，操作系统正在回收其占用的资源和 PCB

*2.*状态转换的合法与非法情况

合法转换只有五条：

1. 创建态 → 就绪态：资源分配完毕，PCB 初始化完成，插入就绪队列。
2. 就绪态 → 运行态：被调度程序选中，分配到处理机。
3. 运行态 → 就绪态：时间片用完，或被更高优先级的进程抢占（被动行为，由调度引起）。
4. 运行态 → 阻塞态：进程主动请求某种资源或等待某事件（主动行为，通过系统调用发出）。
5. 运行态 → 终止态：进程正常结束或异常终止。

非法转换（必考）：

- 阻塞态 → 运行态：不允许。阻塞的进程必须先被唤醒变为就绪态，再经调度才能运行。
- 就绪态 → 阻塞态：不允许。进入阻塞态是进程在运行中主动发出的请求，就绪态没有 CPU，无法发出请求。
- 阻塞态 → 就绪态是合法的，且是被动行为——必须由其他进程（如释放 I/O 设备的进程）调用唤醒原语完成。

- 创建态 → 运行态不允许；创建完成后只能先进入就绪态。

判断口诀

”运行”往下走：要么回就绪（被抢），要么去阻塞（自己等），要么结束。”阻塞”只能回到就绪，”就绪”只能去运行。

*3.*引起状态转换的典型事件

事件	转换
时间片用完	运行 → 就绪
被更高优先级进程抢占	运行 → 就绪
主动出让 CPU（`yield`）	运行 → 就绪
请求临界资源 / 申请外设	运行 → 阻塞
启动 I/O 设备、读文件（等待 I/O）	运行 → 阻塞
对值 ≤ 0 的信号量执行 P（wait）操作	运行 → 阻塞
对管程条件变量执行 `wait()`	运行 → 阻塞
申请内存失败 / 创建进程资源不足	运行 → 阻塞
I/O 完成（中断处理）	阻塞 → 就绪
某进程退出临界区并发出唤醒	阻塞 → 就绪
等待的资源被释放、事件到达	阻塞 → 就绪

三个高频陷阱

1. ”键盘输入”不是”运行 → 就绪”。当前进程等待键盘输入，是从运行态转为阻塞态（等待 I/O 事件）。
2. ”缺页异常”不是”运行 → 就绪”。缺页后进程等待页面调入，进入阻塞态。
3. ”执行 **P(wait)** 操作”是”运行 → 阻塞”，只有”主动出让 CPU”才是”运行 → 就绪”（2023 年真题答案即为此）。

真题考法

2015-25（下列哪个事件导致执行态变就绪态）、2018-27（哪些事件导致当前进程阻塞）、2019-24（哪些事件可能唤醒进程）、2022-28（哪些操作导致执行态变阻塞态）、2023-27（哪个操作让线程从执行态变就绪态）。几乎每年一道，属于必得分题。

*4.*挂起态与七状态模型

内存空间有限，操作系统把短期内不执行的进程换出到外存，称为挂起（**Suspend**）；把更可能执行的进程从外存调回内存，称为激活（**Activate**）。加上两个挂起态就得到七状态模型：

- 就绪挂起态：进程在外存，但只要调入内存即可立即运行；
- 阻塞挂起态：进程在外存并且仍在等待某事件。

关键转换：

- 运行 → 就绪挂起：程序被直接挂起；
- 阻塞 → 阻塞挂起：内存紧张时，优先换出阻塞态进程（它占着内存却不能运行）；
- 阻塞挂起 → 就绪挂起：所等待的事件到来（如 I/O 完成）；
- 就绪挂起 → 就绪：内存中没有其他就绪进程，或该进程优先级高于内存中所有就绪进程；

- 就绪 → 就绪挂起：只有在挂起阻塞态进程仍不足以释放足够内存时才这样做。

核心结论

换出/换入（Swapping）由中级调度负责。被挂起的进程即使优先级很高也不能运行，因为它不在内存中。

（三）线程的实现

*1.*内核支持的线程与线程库支持的用户级线程

用户级线程（**ULT**，**User-Level Thread**）：由用户程序自己管理和调度，创建、同步、调度全部由运行在用户空间的线程库（如 POSIX Pthreads、Java 线程库）完成，内核完全不知道其存在。

内核级线程（**KLT**，**Kernel-Level Thread**）：由操作系统内核管理和调度，内核为每个内核级线程建立 TCB，创建、同步、调度通过系统调用完成。

对比表：

对比项	用户级线程（**ULT**）	内核级线程（**KLT**）
管理者	用户空间的线程库，每个进程维护私有的 TCB 列表	操作系统内核，内核为每个线程建立 TCB
内核是否感知	不感知（可用于不支持线程的操作系统）	感知并能控制
切换开销	小，完全在用户空间完成，不需要陷入内核	大，需要通过系统调用，存在用户态/内核态切换
阻塞影响	一个线程阻塞则整个进程被阻塞（内核只看到进程）	一个线程阻塞不影响同进程其他线程继续执行
并行能力	不能利用多处理机，同一个进程的多个线程只能在一个 CPU 上分时复用	可以分配到多个处理器上真正并行执行
CPU 时间分配单位	以进程为单位分配时间片，多线程进程中的每个线程分到的时间更少	以线程为单位分配，多线程进程可获得更多 CPU 时间
调度算法	允许每个进程有自己的线程调度策略，程序员可自定义	由内核统一调度，可利用内核的优先级管理等机制
是否需系统调用	创建、撤销、切换都不需要	创建、撤销、切换都需要

组合方式：内核支持多个内核级线程的建立、调度和管理，同时允许用户程序建立、调度和管理用户级线程；若干个用户级线程通过时分多路复用地映射到若干个内核级线程上。它结合了 KLT 与 ULT 的优点：同一进程的多个线程可同时在多处理机上并行运行，且阻塞一个线程时不需要阻塞整个进程。

*2.*多线程模型

模型	映射关系	优点	缺点
多对一	多个用户级线程映射到一个内核级线程	线程管理开销小、效率高，可在不支持线程的内核上实现	一个线程阻塞则整个进程阻塞；不能利用多处理机

模型	映射关系	优点	缺点
一对一	每个用户级线程对应一个内核级线程	阻塞系统调用不影响其他线程；允许多线程在多处理机上并行	创建一个用户线程就要创建一个内核线程，内核线程数量受限，开销大
多对多	n 个用户级线程映射到 m 个内核级线程（n > m）	兼有前两者的优点，可并行又可控制开销	实现复杂

真题考法

2012-31（进程与线程的基本叙述）、2019-23（线程的描述中错误的——答案是” 操作系统为每个用户级线程建立一个线程控制块”）、2011-25（线程不能共享的是栈指针）。这类题目本质都在考” 哪些共享、哪些私有” 和” 谁负责调度”。

（四）进程与线程的组织与控制

1.PCB 的作用与内容

PCB 是进程存在的唯一标志，进程创建后该结构常驻内存，进程结束时删除。操作系统通过 PCB 管理和控制进程，因此管理控制所需的一切参数一定放在 **PCB** 中。

类别	具体内容
进程描述信息	进程标识符 PID（系统内唯一，PID 是操作系统分配的有限资源，即可分配的 PID 个数固定，对应系统允许同时存在的最大进程数）、用户标识符 UID
进程控制和管理信息	进程状态、进程优先级、代码运行入口地址、程序外存地址、进入内存时间、处理机占用时间、信号量使用情况、CPU 调度信息（调度队列指针等）
资源分配清单	内存/虚存地址、代码段指针、数据段指针、堆栈段指针、打开文件表、I/O 设备信息
处理机相关信息（处理机上下文）	程序计数器 PC、通用寄存器值、地址寄存器值、控制寄存器值、标志寄存器值、状态字、累加器、索引寄存器、其他条件码寄存器，以及基址/界限寄存器的值等内存管理信息

*2.*进程控制块的组织方式

组织形式	做法	特点
线性表	所有 PCB 组织在一张线性表中，表起始地址存放在内存特定位置或专用寄存器中	简单，但查找慢
链接队列	不同状态对应不同的队列：就绪队列、各类阻塞队列	插入删除灵活；就绪队列只有一个，但等待不同事件的阻塞队列有多个
索引表	不同状态对应不同的索引表（就绪索引表、阻塞索引表）	查找快，需要额外索引空间

为什么阻塞队列有多个而就绪队列只有一个

进程处于就绪态的原因只有一个——还没分配到 CPU；而阻塞原因多种多样（等 I/O、等打印机、等信号量、等消息……），因此每一类阻塞原因单独建立一个 **PCB** 等待队列。

*3.*进程控制的原语操作

进程控制的主要功能是对系统中所有进程实施有效管理，包括创建新进程、撤销已有进程、实现状态切换。这些功能都由原语实现。原语是不可被中断的原子操作，执行期间不允许中断，因此原语的执行过程中不能进行进程调度与切换。

创建原语：

1. 为新进程分配唯一的 PID，并申请一个空白 PCB（PCB 有限，申请失败则创建失败）；
2. 为进程分配运行所需资源（内存、文件、I/O 设备、CPU 时间等），资源不足时不是创建失败，而是停留在创建态等待；
3. 初始化 PCB：初始化标志信息、处理机状态信息、处理机控制信息，设置优先级等；
4. 若就绪队列能接纳新进程，将新进程插入就绪队列，等待调度。

引起进程创建的事件：终端用户登录、作业调度、系统提供服务、用户程序的应用请求。

撤销原语：

1. 找到被撤销进程的 PCB；
2. 若进程正在运行，立即剥夺其处理机；
3. 若进程还有子进程，通常应同时撤销所有子进程（统考按此理解；实际 Linux 中父进程结束时子进程由 init 接管成为孤儿进程，Windows 中同一进程组会一起结束）；
4. 归还进程占用的全部资源给父进程或系统；
5. 将 PCB 从所在队列中移出。

阻塞原语（**Block**）：

1. 找到被阻塞进程的 PCB；
2. 若该进程为运行态，则保护现场（保存上下文），将其状态转为阻塞态，停止运行；
3. 把 PCB 插入相应事件的等待队列，将处理机调度给其他就绪进程。

唤醒原语（**Wakeup**）：

1. 在该事件的等待队列中找到相应进程的 PCB；
2. 将其从等待队列中移出，置其状态为就绪态；
3. 把 PCB 插入就绪队列，等待调度。

易错

- 阻塞是进程自身的主动行为，只有运行态的进程（占有 CPU）才可能转为阻塞态；唤醒是被动行为，必须由其他进程调用唤醒原语完成。
- 阻塞原语与唤醒原语必须成对使用，否则进程将永远阻塞。
- 阻止进程调度的情况：中断处理过程中、访问内核临界区时、禁用中断期间、原语执行过程中。注意用户级临界区中允许调度：例如某进程已获得打印机（处于临界区）后主动等待 I/O，仍会阻塞自己并触发调度。

真题考法

2010、2019、2020、2021、2022、2024 年多次考查。典型问法：” 创建进程时必须完成的操作包括哪些”” 撤销进程时不一定执行的是”（2024 年答案为” 终止子进程”）、” 操作系统为新建进程完成的工作按顺序是”。

（五）进程间通信

每个进程的用户空间相互独立、不能直接互访，因此需要借助内核空间（所有进程共享同一份内核空间）来实现通信。

*1.*共享内存

共享内存机制是拿出一块物理内存，映射到多个进程各自的虚拟地址空间中，各进程通过自己的页表把虚拟地址映射到同一物理页框。这样进程 A 写入的内容进程 B 立刻可见，不需要任何数据拷贝，是最快的 **IPC** 方式。

工作过程：

1. 创建共享内存：`shmget`（System V）或 `shm_open`（POSIX），内核在物理内存中分配区域并指定访问权限和大小；
2. 附加到进程地址空间：`shmat` 或 `mmap`，内核修改该进程的页表，把虚拟地址范围映射到共享内存的物理页框；
3. 读写共享内存：进程像访问普通内存一样直接读写，无需系统调用，零拷贝；
4. 同步机制：共享内存本身不提供任何同步，必须配合信号量、互斥锁等使用，否则会竞态；
5. 分离：`shmdt` 或 `munmap`，内核移除该进程页表中的映射，但物理内存仍保留；
6. 销毁：`shmctl` 或 `shm_unlink` 标记删除，所有附加进程都分离且无引用时内核才回收物理内存。

共享内存的三个必须记牢的结论

1. 共享内存位于进程的用户地址空间，不在内核空间；
2. 建立共享内存的操作（修改页表）在内核态；访问共享数据的操作在用户态；
3. 共享的物理页框唯一（f1 = f2 必然相等），但各进程中的逻辑页号可以不同（p1 与 p2 不一定相等）。

分页与分段共享的统一原则

物理地址唯一（必须同源），逻辑地址独立（可以不同）。因此 2023-30 题答案是” 页号 p1、p2 不一定相等，页框号 f1、f2 一定相等”；2019-28 题中” 段 S 在 P1 和 P2 中应该具有相同的段号” 是错误叙述。

补充：进程虚拟地址空间中的内核空间部分映射到相同的物理内存区域（操作系统的代码和全局数据），因此内核空间天然是共享的；但每个进程在内核态执行时使用自己独立的内核栈，内核栈指针保存在该进程的 PCB 中。

*2.*消息传递

直接通信：通信进程之间直接建立链路，一条链路对应一对进程，`send(P1, message)` 发送给进程 P1，`receive(P0, message)` 接收进程 P0 发来的消息。

间接通信：通过操作系统维护的消息队列（或信箱）完成。每个消息队列有唯一标识，只有共享了相同消息队列的进程才能通信。链路属性：只有共享同一消息队列的进程才建立连接；连接可以是单向或双向；一个消息队列可以与多个进程关联；每对进程可以共享多个消息队列。

同步方式：

- 阻塞（同步）模式：发送者在消息被接收者收到前阻塞；接收者在没有消息时阻塞。
- 非阻塞（异步）模式：发送者发出消息后立即继续；接收者没有消息时也立即返回。

消息队列的缺点：不适合大数据传输（Linux 中有 `MSGMAX` 限制单条消息最大长度、`MSGMNB` 限制队

列总长度）；每次读写都要经过用户态与内核态之间的数据拷贝。

*3.*管道

匿名管道：位于内核缓冲区（不在用户空间，也不在磁盘上）的 IPC 机制，主要用于具有亲缘关系的进程（父子、兄弟）之间的单向数据流通信。

工作流程：

1. 创建管道：`pipe(int pipefd[2])`，返回两个文件描述符，`pipefd[0]` 读端、`pipefd[1]` 写端；内核为每个管道分配一个环形缓冲区（循环队列）和两个文件描述符；
2. 进程间共享：父进程 `pipe()` 后 `fork()`，子进程继承父进程的文件描述符表，从而共享同一管道的读写端；
3. 数据读写：写入时数据从用户空间拷贝到内核缓冲区；读取时从内核缓冲区拷贝到用户空间；
4. 生命周期：随进程终止而销毁；所有引用管道的文件描述符关闭后，内核回收缓冲区资源（不会持久化到磁盘）。

pipe 的读写规则与阻塞条件（必背）：

情形	行为
写进程写满的管道	阻塞，直到有空间
读进程读空的管道	阻塞，直到有数据
读进程读一个写端全部关闭的管道	`read` 返回 0（EOF），不阻塞
写进程写一个读端全部关闭的管道	内核向写进程发送 `SIGPIPE` 信号（即”Broken pipe”）
单次写入不超过 `PIPE_BUF`（通常 4096 字节）	保证原子性，多个进程同时写时数据不会交叉混合
管道是否支持定位	不支持 `lseek` 之类的文件定位操作，只能队尾写入、队头读出

其他必须记牢的结论：

- 同一时刻，管道只允许一个读进程执行读操作或一个写进程执行写操作（读写都要互斥进行）；
- 但一个管道允许存在多个读进程或多个写进程（它们都有该管道的文件描述符，只是操作时要抢管道的内核互斥锁）；
- 管道自身通过互斥锁 + 两个等待队列（`rd_wait`、`wr_wait`）+ 读写指针实现了同步与互斥，用户不需要额外写信号量；
- 管道是半双工的，某一时刻只能单向传输；要实现全双工需创建两个管道；
- 匿名管道只能用于有亲缘关系的进程；命名管道（**FIFO**）存在于文件系统中，可以用于任意进程之间通信。
- 管道以字节流为单位（无格式、无消息边界），不是以消息为单位。

十二个自测判断（全部为” 错误” 的典型说法）

1. ” 管道位于进程的用户空间，可直接在用户态读”——错，管道在内核区，读写要通过系统调用在内核态完成；
2. ” 管道与共享内存这两种 IPC 都自己实现了同步/互斥”——错，共享内存本身没有同步互斥机制；
3. ” 管道是存放在磁盘中的一种特殊文件”——错，匿名管道在内存中；
4. ” 管道支持定位到某个具体位置读写”——错，只能队尾写、队头读；

5. ”所有引用管道的文件描述符都被关闭后，管道内未读完的数据会被持久化到磁盘”——错，内核直接回收缓冲区；
6. ”管道通信是以消息为单位进行写入和读出的”——错，是字节流；
7. ”对管道的写操作不需要互斥”——错，读写都需要互斥。

真题考法

2014-30 曾考管道的特性（答案选”进程对管道进行读操作和写操作都可能被阻塞”）。常考选项包括”一个管道仅支持半双工通信”” 管道位于内存的内核区域”” 一个管道在同一时刻只能有一个读或一个写进程操作”。

*4.*信号

信号（**Signal**）是一种异步的、非阻塞的通信机制，也叫”用户态中断”或”软中断”，用于通知接收进程某个事件已经发生。每个信号有一个唯一编号和对应的处理动作。信号通常只用于简单的事件通知，不适合复杂的数据传输。

信号的来源：

- 硬件来源：硬件异常（浮点除零触发 SIGFPE）、内存访问越界（段错误 SIGSEGV）、非法指令；
- 软件来源：系统调用（kill()、raise()、alarm()、setitimer()）、用户输入（Ctrl+C 触发 SIGINT）、系统事件（子进程退出时父进程收到 SIGCHLD）、定时器到期（SIGALRM）。

常见信号：SIGINT（终端中断）、SIGKILL（强制终止，不可捕获、不可忽略）、SIGTERM（默认终止信号，可捕获）、SIGCHLD（子进程终止）、SIGALRM（定时器）。

传递与响应流程：信号产生 → 信号传递（内核把信号递给目标进程）→ 信号处理 → 执行处理函数 → 信号返回。

进程对信号有三种响应方式：

1. 执行默认操作：多数信号的默认行为是终止进程、生成核心转储文件或暂停进程；
2. 捕捉信号：执行用户自定义的信号处理函数；
3. 忽略信号：不做任何处理。但 SIGKILL 和 SIGSTOP 不能被忽略或捕捉。

核心结论

信号传递不支持排队：短时间内多次传递同一信号，进程只会处理一次。

*5.*四种 *IPC* 机制对比

特性	共享内存	消息传递（消息队列）	（匿名）管道	信号
数据形式	原始字节流，直接内存访问	结构化消息（带类型/优先级）	字节流（无结构）	简单整数信号（几乎无数据）
通信方向	双向	双向	单向（半双工）	单向（发送 → 接收）
同步需求	需要（信号量、互斥锁）	部分需要（阻塞接收）	内核自动同步	无（异步通知）
性能	极高（零拷贝）	中（数据拷贝开销）	中（内核缓冲区拷贝）	极高（无数据传输）
复杂度	高	中	低	低

特性	共享内存	消息传递（消息队列）	（匿名）管道	信号
进程关系	任意进程	任意进程	仅亲缘进程	任意进程（需知道 PID）
持久性	分离后需重新映射	内核维护	随进程终止销毁	瞬时触发
典型应用	高频数据交换（视频处理）	微内核	Shell 管道	事件通知

二、CPU 调度与上下文切换

（一）调度的基本概念

调度分为三个层次：

层次	要做什么	调度发生在	发生频率	对进程状态的影响
高级调度（作业调度）	按某种规则从后备队列中选择合适的作业调入内存，为其创建进程	外存 → 内存（面向作业）	最低	无 → 创建态 → 就绪态
中级调度（内存调度）	按某种规则从挂起队列中选择合适的进程将其数据调回内存	外存 → 内存（面向进程）	中等	挂起态 → 就绪态等
低级调度（进程调度 / CPU 调度）	按规则从就绪队列中选择一个进程为其分配处理机	内存 → CPU	最高	就绪态 → 运行态

- 高级调度主要用于多道批处理系统；每个作业只调入一次、调出一次；作业调入时建立 PCB，调出时才撤销 PCB。
- 中级调度即换入/换出（Swapping），目的是提高内存利用率和系统吞吐量。
- 低级调度是最基本的调度，所有操作系统都必须有。

（二）调度的目标

指标	定义	说明
CPU 利用率	$CPU利用率 = \frac{CPU有效工作时间}{总时间}$	希望 CPU 尽可能忙碌
系统吞吐量	$系统吞吐量 = \frac{总共完成的作业数}{总共花费时间}$	短作业能提升吞吐量，长作业会降低吞吐量
周转时间	周转时间 = 作业完成时间 − 作业提交（到达）时间	用户更关心单个作业的周转时间
平均周转时间	$平均周转时间 = \frac{各作业周转时间之和}{作业数}$	操作系统更关心整体表现
带权周转时间	$带权周转时间 = \frac{作业周转时间}{作业实际运行时间}$	必然 ≥ 1，越小越好

指标	定义	说明
平均带权周转时间	平均带权周转时间 $=\frac{\text{各作业带权周转时间之和}}{\text{作业数}}$	综合指标
等待时间	进程处于就绪态的时间之和	CPU 调度算法只影响等待时间，不影响运行时间和 I/O 时间
响应时间	用户提交请求到系统第一次产生响应的时间	交互式系统衡量调度算法好坏的主要标准

等待时间的口径

等待时间只统计进程在就绪队列中等待 **CPU** 的时间之和，不包括在后备队列中等待进入内存的时间（那属于另一指标），也不包括进程运行和 I/O 的时间。

带权周转时间为什么必然 ≥ 1

因为 周转时间 $=$ 等待时间 $+$ 运行时间 $\geq$ 运行时间，故 $\frac{\text{周转时间}}{\text{运行时间}} \geq 1$。两个作业周转时间相同时，实际运行时间更长的那个带权周转时间更小，用户满意度更高。

（三）调度的实现

*1.*调度器 / 调度程序

调度程序（Scheduler）由三部分组成：

组成部分	职责
排队器（**Queuing**）	按进程优先级、到达时间等因素把就绪进程排序并插入相应队列
分派器（**Dispatcher**）	根据调度程序选出的进程，从就绪队列中取出，把处理机分配给它
上下文切换器（**Context Switcher**）	把当前进程的内核上下文存入其 PCB，再从新进程 PCB 取出上下文装入 CPU

调度是一种决策行为，切换是一种执行行为。两者不一定同时发生（例如选出的下一个进程仍是当前进程时就不需要切换）。

*2.*调度的时机

会引起调度的事件：

1. 当前进程正常结束或异常终止（运行 → 终止）；
2. 当前进程主动阻塞（如发起 I/O、P 操作阻塞自己，运行 → 阻塞）；
3. 当前进程时间片用完（运行 → 就绪）；
4. 有更高优先级的进程进入就绪队列（创建新进程、I/O 完成唤醒阻塞进程等），当前进程被抢占（运行 → 就绪）；
5. 中断/异常/系统调用处理结束返回用户空间时，内核检查” 需要重新调度” 标志，若有效则触发调度；

6. 就绪队列为空时刚来的第一个进程直接上处理机，这也算一次 **CPU** 调度。

不能进行调度与切换的情况（必背）：

1. 当前进程处于中断处理过程中——中断处理涉及核心功能和关键资源，逻辑上不属于某一进程，不应被剥夺处理机；
2. 进程正在访问操作系统内核临界区——如调度器访问就绪队列。若此时被切换，可能导致内核数据结构不一致；
3. 禁用中断（屏蔽中断）过程中；
4. 原语执行过程中——加锁、解锁、中断现场保护与恢复等原子操作不可中断，连中断都要屏蔽，更不能进行调度与切换。

内核临界区 vs 用户级临界区

只有内核临界区才禁止调度。用户级临界区中允许调度：例如当前进程已请求到打印机（处于临界区），但它因等待 I/O 主动阻塞，此时完全可以进行调度。2012-30 题的答案就是” 在进程处于临界区时不能进行处理机调度” 这一说法错误（缺少” 内核” 二字）。

内核中常通过 PCB 里的一个标志位（如 Linux 的 `TIF_NEED_RESCHED`）来延迟抢占：

- 设置标志位的时机：时间片耗尽时；有更高优先级进程进入就绪态时（如 `fork()` 出优先级更高的子进程、I/O 完成后唤醒的进程优先级更高）；
- 响应标志位的时机：中断/异常/系统调用处理结束返回用户空间时，内核检查该标志，有效则调用调度程序。

*3.*抢占式与非抢占式

方式	定义	调度时机	特点与适用场景
非抢占式（非剥夺）	进程除非运行结束或自己主动放弃 CPU，否则不会被其他进程抢占	进程运行结束；进程主动阻塞	调度不频繁，系统开销小，吞吐量较大，CPU 利用率一般更高；实时性差，一般用于批处理系统，不适用于分时系统和大部分实时系统
抢占式（剥夺）	进程在执行中可能因时间片用完或有更高优先级进程进入就绪态而被抢占 CPU	主动放弃 CPU；时间片用完；更高优先级进程就绪	调度频繁，实时性强，可防止进程长期独占处理器；等待时间和响应时间更短，适用于分时系统和实时系统

核心结论

若不考虑 `yield` 等主动释放处理机的系统调用，则非抢占式调度下只会出现” 运行 → 终止” 和” 运行 → 阻塞”，不可能出现” 运行 → 就绪”。

*4.*闲逛进程

闲逛进程（**Idle Process**），又称空闲进程或空闲任务，在没有其他进程需要运行时占据 CPU，保证 CPU 始终有任务可执行。

特点：

1. 优先级最低，一旦有其他进程需要 CPU 就立即让出；
2. 持续运行，多数情况下只执行一个空循环，不执行任何实际操作；
3. 只运行在内核态下；

4. 占用资源最小，不需要 CPU 以外的资源，也不会阻塞。

*5.*内核级线程与用户级线程的调度

- 用户级线程的调度由用户态线程库完成：线程切换不需要陷入内核，开销小；但内核以进程为单位分配时间片，一个线程阻塞会导致整个进程阻塞。
- 内核级线程的调度由内核完成：线程切换需要上下文切换、修改内存映像、使高速缓存（Cache）失效，开销大；但可以真正做到多处理机并行，且一个线程阻塞不影响其他线程。
- 在采用一对一或多对多模型的系统中，调度的实际对象是线程，进程只提供虚拟内存、全局变量等资源。

（四）CPU 调度算法

本节的算例统一使用下面这组进程（到达时刻、服务时间、优先级三列，优先级数值越大优先级越高）：

进程	到达时间	服务时间	优先级
P1	0	5	2
P2	1	4	4
P3	2	2	1

*1.*先来先服务（*FCFS*）

算法步骤：按进程到达的先后顺序依次分配 CPU，直到进程执行完毕或被阻塞（例如请求 I/O），非抢占式。既可用于作业调度，也可用于进程调度。

算例（**t=0** 起开始调度）：

执行顺序：P1(0→5) → P2(5→9) → P3(9→11)。

进程	到达	服务	开始	完成	周转时间	带权周转时间
P1	0	5	0	5	5	1.00
P2	1	4	5	9	8	2.00
P3	2	2	9	11	9	4.50

平均周转时间 $= (5+8+9)/3 = 7.33$；平均带权周转时间 $= (1+2+4.5)/3 = 2.50$。

优点：简单、公平、实现容易，不会导致饥饿，对长作业（CPU 繁忙型）有利。缺点：排在长作业后面的短作业等待时间过长，带权周转时间很大，对短作业不利；I/O 繁忙型进程在等待 I/O 期间不能让出 CPU（严格先来后到），使 CPU 和设备利用率都低。

*2.*短作业优先（*SJF*）/ 短进程优先（*SPF*）

算法步骤（非抢占式）：每次调度时，在就绪队列中选择估计运行时间最短的作业/进程，把处理机分配给它，直到它完成或因某事件阻塞才释放处理机。

算例（非抢占式）：

t=0 只有 P1 到达，先运行 P1；P1 在 t=5 结束后，P2、P3 均已到达，选服务时间短的 P3(2) → P2(4)。

执行顺序：P1(0→5) → P3(5→7) → P2(7→11)。

进程	到达	服务	开始	完成	周转时间	带权周转时间
P1	0	5	0	5	5	1.00
P3	2	2	5	7	5	2.50
P2	1	4	7	11	10	2.50

平均周转时间 $= (5+5+10)/3 = 6.67$；平均带权周转时间 $= (1+2.5+2.5)/3 = 2.00$。

抢占式版本：最短剩余时间优先（**SRTN**）

算法步骤：每当有新进程进入就绪队列（或发生其他改变就绪队列的事件）就检查一次；比较新进程的剩余执行时间与当前运行进程的剩余执行时间，若新进程更短则立即抢占；若相等则按 FCFS 决定。

算例（抢占式 **SRTN**）：

t=0：只有 P1，P1 运行，剩余 5。t=1：P2 到达（剩余 4）。P1 剩余 4，与 P2 剩余 4 相等，不抢占（按规定相等时按 FCFS，保持 P1）。t=2：P3 到达（剩余 2）。P1 剩余 3 > 2，**P3** 抢占。t=4：P3 完成。就绪队列中 P1 剩余 3、P2 剩余 4，选 P1。t=7：P1 完成。运行 P2（剩余 4），到 t=11 完成。

执行顺序：P1(0→1) → P1(1→2) → P3(2→4) → P1(4→7) → P2(7→11)。

进程	到达	服务	完成	周转时间	带权周转时间
P1	0	5	7	7	1.40
P2	1	4	11	10	2.50
P3	2	2	4	2	1.00

平均周转时间 $= (7+10+2)/3 = 6.33$；平均带权周转时间 $= (1.4+2.5+1.0)/3 = 1.63$。**SRTN** 的平均等待时间和平均周转时间优于非抢占式 **SJF**。

408 考试必须记住的细节

1. 题目未特殊说明时，”短作业/进程优先”默认是非抢占式的；
2. 在有前提条件”所有进程可同时运行”或”所有进程几乎同时到达”时，才能说 SJF 的平均等待时间、平均周转时间最少；若无此前提，应说抢占式的 **SRTN** 平均等待时间、平均周转时间最少；
3. 严格来说，该算法应叫”最短下次 **CPU** 执行（**shortest-next-CPU-burst**）”，因为调度取决于进程下一次 CPU 执行的长度，而不是其总长度。

优点：平均等待时间、平均周转时间最短，可提高系统吞吐量。缺点：需要预先知道运行时间（实际不可靠，由用户提供）；对长作业不利，可能造成长作业饥饿；完全不考虑作业的紧迫程度。

*3.*高响应比优先（*HRRN*）

响应比：

$$\text{响应比} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}}$$

算法步骤：非抢占式。每次都只在当前运行的进程主动放弃处理机（正常/异常完成或主动阻塞）时才需要调度，此时计算就绪队列中所有进程的响应比，选择响应比最高者。

算例：

t=0：只有 P1，直接运行 P1 到 t=5。t=5：就绪队列中有 P2（等待 4，服务 4）与 P3（等待 3，服务 2）。$P2$响应比 $=\frac{4+4}{4}=2.00$；$P3$响应比 $=\frac{3+2}{2}=2.50$。选 P3。t=7：P3 完成。P2 等待 6，响应比 $=\frac{6+4}{4}=2.50$，运行 P2 到 t=11。

执行顺序：P1(0→5) → P3(5→7) → P2(7→11)。平均周转时间 $=(5+5+10)/3=6.67$，与本题的非抢占式 SJF 结果相同（巧合，因为 t=5 时 P3 响应比也最高）。

优点：兼顾等待时间与服务时间——等待时间相同时服务时间短的优先（SJF 的优点）；服务时间相同时等待时间长的优先（FCFS 的优点）；长作业随着等待时间增加响应比不断提高，不会饥饿。
缺点：需要知道服务时间；每次调度都要计算响应比，有一定开销。现实中不可实现，属于”理想型”算法。

*4.*时间片轮转（*RR*，*Round-Robin*）

算法步骤：就绪进程按 FCFS 排队；每次把 CPU 分配给队首进程，让它执行一个时间片（典型 20 ms ~ 50 ms）；时间片用完时由时钟中断通知 CPU，把该进程放到就绪队列队尾，再取队首进程执行；若进程在时间片内完成则直接撤离，并立即调度。

时间片轮转是绝对抢占式算法，进程未在时间片内运行完就会被强行剥夺处理机。计时依靠时钟中断：例如设时间片 10 ms、时钟中断周期 5 ms，则每次中断把该进程的剩余时间减 5 ms，减到 0 时内核就知道时间片用完、需要切换进程。时间片越短，进程切换次数越多，系统开销越大，真正用于运行用户进程的时间越少；影响时间片大小的主要因素是系统响应时间、就绪队列中的进程数目、系统的处理能力。做题时若要精确到”发生了几次调度”，还要把被抢占进程放回队尾这一动作单独计入。

算例（时间片 = **2 ms**）：

时间区间	运行进程	说明
0–2	P1	t=1 P2 到达入队尾；t=2 P3 到达入队尾；P1 剩余 3，入队尾
2–4	P2	P2 剩余 2，入队尾
4–6	P3	P3 剩余 0，完成，完成时间 **6**
6–8	P1	P1 剩余 1，入队尾
8–10	P2	P2 剩余 0，完成，完成时间 **10**
10–11	P1	P1 完成，完成时间 **11**

进程	到达	服务	完成	周转时间	带权周转时间
P1	0	5	11	11	2.20
P2	1	4	10	9	2.25
P3	2	2	6	4	2.00

平均周转时间 $=(11+9+4)/3=8.00$；平均带权周转时间 $=(2.2+2.25+2.0)/3=2.15$。

RR 的两个做题要点

1. 若某一时刻进程 **Pi** 下处理机，同一时刻进程 **Pj** 到达，默认新到达的进程 **Pj** 先进入就绪队列（即排在被剥夺的 Pi 之前）；

2. 时间片大小的影响：时间片足够大（所有进程都能在一个时间片内完成）时，RR 退化为 **FCFS**；时间片太小会导致进程切换过于频繁，CPU 开销增大，真正用于用户进程的时间减少。时间片长短通常由系统响应时间、就绪队列中的进程数目、系统的处理能力决定。

优点：公平，响应快，不会导致饥饿，适用于分时操作系统，可实现人机交互。缺点：频繁的进程切换带来不小开销；不区分任务的紧急程度。

真题考法

2024-30：10 个进程同时就绪，时间片 5 ms，队尾进程 P 只需 25 ms，求 P 的周转时间。P 需要 5 个时间片，每轮 10×5=50 ms，P 在队尾故每轮最后一个执行，共 5 轮，周转时间 = 5×50 = 250 ms（选 C）。

*5.*优先级调度

算法步骤：每个进程/作业都有优先级，调度时选择优先级最高的进程。可用于作业调度、进程调度、I/O 调度。

- 非抢占式优先级调度：只在进程主动释放处理机时调度；执行过程中若有更高优先级进程到来，只需将其插入就绪队列队头即可。
- 抢占式优先级调度：就绪队列发生变化时就检查是否发起抢占，若有更高优先级进程进入则立即暂停当前进程，把处理机分配给它。
- 静态优先级：创建进程时确定，之后一直不变（依据进程类型、资源要求、用户要求）。
- 动态优先级：创建时给定初值，之后动态调整（依据占用 CPU 与就绪等待时间长短）。

算例（非抢占式，数值越大优先级越高）：

t=0：只有 P1，运行 P1。t=5：P1 结束，P2（优先级 4）、P3（优先级 1）就绪，选 P2。t=9：P2 结束，运行 P3 到 t=11。

进程	到达	服务	优先级	完成	周转时间	带权周转时间
P1	0	5	2	5	5	1.00
P2	1	4	4	9	8	2.00
P3	2	2	1	11	9	4.50

优先级设置原则（必背）：

- 系统进程 > 用户进程；
- 交互型（前台）进程 > 非交互型（后台）进程；
- **I/O** 繁忙型进程 > **CPU** 繁忙型（计算型）进程——I/O 设备与 CPU 可并行工作，且 I/O 设备速度远比 CPU 慢，让 I/O 繁忙型进程优先可让设备尽早投入工作，提升资源利用率和吞吐量。

动态优先级的调整时机：某进程在就绪队列中等待时间过长 → 提高其优先级；某进程占用处理机运行时间过长 → 降低其优先级；某进程频繁进行 I/O → 提高其优先级。

降低优先级的合理时机

只有在进程时间片用完时才适合降低其优先级（2010 年真题答案）。刚完成 I/O 进入就绪队列时不应降低（它此前并未占用处理机）；长期处于就绪队列的进程应提高优先级以免饥饿；从就绪态转为运行态时不应降低。

优点：能区分紧急程度、重要程度，适用于实时操作系统，可灵活调整偏好。缺点：若源源不断有

高优先级进程到来，可能导致饥饿。

*6.*多级队列调度

把就绪队列按进程类型分成多个队列（例如前台/交互进程队列与后台/批处理进程队列），每个队列内部可以使用不同的调度算法，队列之间可以有固定的优先级（如只有高优先级队列为空时才调度低优先级队列），也可以按时间比例分配 CPU。

缺点：进程一旦进入某个队列就不能在队列间移动，若队列间采用固定优先级，低优先级队列中的进程仍可能饥饿。多处理机中这种单一调度策略的缺点更突出。

*7.*多级反馈队列（*MLFQ*）

算法步骤：

1. 设置多个就绪队列，优先级从高到低（队列 1 最高）；各队列的时间片从小到大（例如队列 1 时间片 8 ms，队列 2 时间片 16 ms，队列 3 按 FCFS）；
2. 新进程一律进入队列 **1** 队尾，按 FCFS 等待调度；
3. 队列 1 中的进程若在一个时间片内完成则撤离；若未完成则转入队列 **2** 队尾；
4. 仅当第 1 级队列为空时才调度第 2 级队列；仅当第 1、2 级都为空时才调度第 3 级；
5. 若正在执行第 **k** 级队列的进程时，有更高优先级队列（**1**～**k-1** 级）的新进程到达，则被抢占，原进程放回第 **k** 级队列队尾；
6. 处于最低优先级队列的进程按时间片轮转（或 FCFS）反复执行，不再降级；通过在低优先级队列中等待过久的进程提升到高优先级队列（”老化”）来防止饥饿。

算例（**2019-27** 原型）：二级反馈队列，Q1 时间片 10 ms 用 RR，Q2 用短进程优先；Q1 优先，新进程先进 Q1，Q1 中执行完一个时间片未结束则转入 Q2。当前 Q1、Q2 为空，依次创建 P1（需 30 ms）、P2（需 20 ms）后立即开始调度。

- 0–10：P1 在 Q1 运行 10 ms，剩余 20 ms，转入 Q2 队尾；
- 10–20：P2 在 Q1 运行 10 ms，剩余 10 ms，转入 Q2 队尾；
- 此时 Q1 为空，调度 Q2（短进程优先）：P2 剩余 10 ms 先执行（20–30），再执行 P1 剩余 20 ms（30–50）。

P1 在 0–10 运行、10–30 就绪等待（20 ms）、30–50 运行，故 P1 等待时间为 20 ms；P2 在 0–10 就绪等待（10 ms）、10–20 运行，故 P2 等待时间为 10 ms。平均等待时间 = **(20+10)/2 = 15 ms**。

2019 年这道题的踩坑点

算 P1 的等待时间时极易漏掉**”P2** 在 **Q1** 执行的 **10 ms** 期间 **P1** 在等待**”** 这一段。必须把每个时刻谁在运行、谁在就绪逐段列出，再累加就绪时间。

优点：可动态调整进程优先级和时间片大小，兼顾多方面系统目标；能区分对待 CPU 密集型和 I/O 密集型进程（I/O 阻塞后回到原队列可保持较高优先级）；短作业周转时间短，一定程度上避免饥饿。缺点：算法设计复杂，极端情况下仍可能饥饿。

设计 MLFQ 需要考虑的因素（2020 年真题答案选” 全部四项”）：就绪队列的数量、就绪队列的优先级、各就绪队列的调度算法、进程在就绪队列间的迁移条件。

*8.*同一组进程用不同算法的完整对照表

仍用上面那组进程（P1：到达 0、服务 5；P2：到达 1、服务 4；P3：到达 2、服务 2），非抢占式算法中优先级的数值越大优先级越高：

算法	执行顺序	**P1** 周转	**P2** 周转	**P3** 周转	平均周转	平均带权周转	是否可能饥饿
FCFS	P1 → P2 → P3	5	8	9	7.33	2.50	否
SJF（非抢占）	P1 → P3 → P2	5	10	5	6.67	2.00	是
SRTN（抢占）	P1 → P3 → P1 → P2	7	10	2	**6.33**	**1.63**	是
HRRN（非抢占）	P1 → P3 → P2	5	10	5	6.67	2.00	否
RR（时间片 2）	P1 → P2 → P3 → P1 → P2 → P1	11	9	4	8.00	2.15	否
优先级（非抢占）	P1 → P2 → P3	5	8	9	7.33	2.50	是

从对照表能读出的三条规律

1. 平均周转时间最短的是抢占式 **SRTN**，其次是非抢占式 **SJF/HRRN**——只要题目问” 平均等待时间/平均周转时间最少”，先考虑 SRTN（有” 同时到达” 前提时可答 SJF）；
2. **RR** 的平均周转时间通常较大，但它的优势在于响应时间短、公平、不饥饿；
3. **FCFS** 与优先级调度的本算例结果相同只是巧合（恰好优先级顺序与到达顺序一致），不要据此认为两者等价。

（五）多处理机调度

*1.*与单处理机调度的区别

多处理机调度要额外解决两个问题：亲和性与负载均衡。

1. 亲和性（**Affinity**）：让一个进程尽量总在同一个 CPU 上运行，以保持 Cache 命中率，避免频繁到内存取数据。软亲和指尽量让进程在同一 CPU 上运行但不强制；硬亲和可写入 PCB，表示该进程只能由某一个 CPU 运行。
2. 负载均衡：各 CPU 的繁忙程度应大致相等，不能一个 CPU 长期空闲而另一个长期繁忙。

*2.*两种就绪队列组织方式

方式	做法	优点	缺点
公共就绪队列	所有进程共享一个就绪队列	队列需要上锁互斥（防止多个 CPU 同时抢同一个进程），天然实现负载均衡	亲和性差（进程可能被不同 CPU 调度）
私有就绪队列	每个 CPU 拥有自己的就绪队列	天然解决亲和性问题	需要额外的迁移机制实现负载均衡

负载均衡通过迁移实现：推迁移（**push migration**）是系统程序把负载高的 CPU 上的进程推到负载低的 CPU 上；拉迁移（**pull migration**）是空闲 CPU 从超载 CPU 的就绪队列中” 拉” 一些进程过来

运行。推迁移和拉迁移可以并行执行。

（六）上下文及其切换机制

*1.*上下文的组成

上下文（**Context**）是进程/线程在某一时刻的处理器状态，主要包含：

- 程序计数器 PC（下一条要执行的指令地址）；
- 通用寄存器组的值；
- 栈指针 SP、栈基址寄存器的值；
- 程序状态字 PSW、标志寄存器；
- 地址空间相关寄存器（页基址寄存器 / 页目录基址寄存器 PDBR）；
- 其他控制寄存器、条件码寄存器。

上下文的保存位置是 **PCB**，不是栈。（进程的内核栈只保存函数调用信息与中断现场；PCB 中保存的才是切换时恢复的上下文。）

*2.*进程切换的完整过程

1. 挂起当前进程，把它的处理机上下文保存到它的 PCB 中；
2. 把该 PCB 移入相应队列（就绪队列或阻塞队列）；
3. 由调度程序选择另一个进程，更新它的 **PCB**：把状态从就绪改为运行，记录本次开始运行的时间（用于统计 CPU 占用率、响应时间），重置或更新与调度策略相关的字段（如把时间片重置为系统规定的初始值）；
4. 从新进程的 PCB 中取出上下文并装入 CPU 各寄存器；
5. 跳转到新进程的程序计数器所指向的位置继续执行。

Linux 的实现顺序正是如此：`__schedule()` 先 `pick_next_task()` 选出下一个进程，若 `prev != next` 则调用 `context_switch()`：先 `switch_mm_irqs_off()` 切换地址空间（页表），再 `switch_to()` 切换寄存器上下文。

> **为什么”进程切换”必须切换地址空间，而”中断处理”不用**
>
> 中断处理前后仍是同一个进程、同一个地址空间，只是从用户态陷入内核态（用户栈切到内核栈），不涉及地址空间切换，因此中断处理程序的执行不算进程切换。进程切换必须切换页表，所以涉及地址空间切换。

*3.*切换的开销

- 上下文切换是纯开销，切换期间 CPU 不做任何有用工作；
- 页表切换（地址空间切换）在开销中占比很大，还要加上 TLB/Cache 失效的代价；
- 保存和恢复寄存器需要执行多条 `load/store` 指令，比较耗时；
- 优化手段：设置两组寄存器组（一组供内核、一组供用户），切换时只需改指针，无须搬移寄存器内容。

*4.*中断响应、模式切换与进程切换的区别

对比项	中断响应	进程切换
上下文保存/恢复的时机	分开进行，分别位于中断响应和中断返回中	连续进行（两个不同进程之间）
保存的上下文	中断前进程执行的上下文（可能是用户上下文）	一定是内核态下的上下文
是否切换地址空间	不涉及	涉及（切换页表）
特权级	可能发生特权级切换	全程在内核态下进行

模式切换 **vs** 进程切换：

- 模式切换发生在用户态响应中断/异常（用户态 → 核心态）和处理完中断异常后返回用户态（核心态 → 用户态）两种情况；它是同一进程在自身用户态和内核态之间的切换，二者共用地址空间，涉及特权级切换与用户栈/内核栈切换，只需保存和恢复用户态上下文（通用寄存器、PC、PSW、SP 等断点和现场信息）；
- 进程切换发生在内核态，是从一个进程的内核态切换到另一个进程的内核态。PCB 中保存的是进程在内核态被切换时的完整状态；用户态的恢复依赖中断返回机制，不是进程切换的直接结果；
- 因此：任何进程的切换都依靠内核完成；进程的调度和切换均发生在内核态下（注意：用户级线程之间的切换可以由线程库在用户态完成，这是特例）。

典型错误认知

错误：”进程切换是直接在用户态之间切换，PCB 中保存的是用户态上下文。”正确：

1. 切换永远发生在两个进程的内核态之间；
2. PCB 保存的是进程在内核态被切换时的完整状态；
3. 用户态恢复依赖中断返回机制，不是进程切换的直接结果。

真题考法

2024-25：支持页式存储管理的系统中，进程切换时要执行的操作包括”更新程序计数器的值”“更新栈基址寄存器的值”“更新页基地址寄存器的值”——三项都要做。2018 年曾考”同一进程不同线程切换时页基址寄存器是否需要改动”（不需要，因为共用同一虚拟地址空间，原有 TLB 不会失效）。

三、同步与互斥

（一）同步与互斥的基本概念

*1.*临界资源与临界区

临界资源（**Critical Resource**）：一个时间段内只允许一个进程使用的资源。物理设备（摄像头、打印机）是临界资源；共享变量、数据、内存缓冲区同样是临界资源。

临界区（**Critical Section**）：进程中访问临界资源的那段代码。注意一个临界资源的相关临界区是由不同进程各自的临界区共同构成的——各进程对同一临界资源的操作不同，代码段自然也不同。

对临界资源的互斥访问在逻辑上分为四个部分：

部分	作用
进入区	检查能否进入临界区，并设置正在访问临界区的标志
临界区	访问临界资源的代码
退出区	将正在访问临界区的标志清除
剩余区	代码的其余部分

可重入代码

可重入代码是可以被重复调用执行、中断后重新执行也不会出错的代码，可同时被多个进程调用，不属于临界资源；可重入代码不允许被任何进程修改。进程映像中的程序段若被共享，必须是可重入代码。

*2.*同步与互斥

概念	核心思想	定义	等待原因
互斥（**Mutual Exclusion**）	竞争	多个进程因争夺同一个、临时的、不可共享的资源而产生的制约关系（间接制约）	资源正忙
同步（**Synchronization**）	协作	多个进程为共同完成任务，在执行次序上需要遵循某种先后顺序而产生的制约关系（直接制约）	前驱任务未完成，条件不满足

*3.*四条原则

原则	含义
空闲让进	临界区空闲时，应允许一个请求进入临界区的进程立即进入
忙则等待	已有进程进入临界区时，其他试图进入的进程必须等待（即互斥）
有限等待	对请求访问的进程，应保证它能在有限时间内进入临界区（不会饥饿）
让权等待	进程不能进入临界区时，应立即释放处理机（主动阻塞），防止忙等

核心结论

前三条是必须实现的；”让权等待”是可选原则——Peterson 算法、TSL/Swap 指令、自旋锁都不满足让权等待，但它们实现互斥是正确的。2020-32 题的答案就是”仅Ⅰ、Ⅱ、Ⅲ”。

真题考法

2020-32 直接考四条原则；2018-32 考”下列同步机制中可以实现让权等待的是”（答案：信号量方法）；2016-27 考 TSL 实现互斥的性质（答案：等待进入临界区的进程不会主动放弃 CPU）。

（二）基本的实现方法

*1.*软件方法

判断某一软件方法是否满足四条原则的统一方法：

- 判断空闲让进：观察是否存在”表达式 1 与表达式 2 同时成立”的可能；再看没有对方进程时，本进程能否通过循环进入临界区；

- 判断忙则等待：看本进程通过了 while 循环但还没进入临界区时切换到对方，对方能否也通过 while 循环；
- 判断有限等待：本进程卡在进入区时，新来的同类进程能否不断抢先进入；
- 所有软件方法都不满足让权等待：进入区的 while(表达式)；循环体为空语句，一直空转，这就是忙等。

（1）单标志法

```
// 共享整型变量
int turn = 0;   // turn = i 表示允许进程 Pi 进入临界区

/* 进程 P0 */
while (TRUE) {
    while (turn != 0);   // 若 turn 不等于本进程编号，则循环等待
    临界区;
    turn = 1;            // 访问完毕，把权限让给对方
    剩余区;
}
```

```
/* 进程 P1 */
while (TRUE) {
    while (turn != 1);
    临界区;
    turn = 0;
    剩余区;
}
```

- 满足忙则等待；
- 两个进程必须交替进入临界区。若其中一个进程不再进入临界区，另一个进程因得不到授权也永远进不去，违背" 空闲让进"；
- 使用 while 判断作为进入区，违背" 让权等待"。

（2）双标志先检查

```
bool flag[2];           // flag[i] = TRUE 表示进程 Pi 想进入临界区
flag[0] = flag[1] = FALSE;

/* 进程 P0 */
while (TRUE) {
    while (flag[1] == TRUE);   // 先检查对方是否想进
    flag[0] = TRUE;            // 对方不想进，自己上锁
    临界区;
    flag[0] = FALSE;
    剩余区;
}
```

- 解决了单标志法" 必须交替进入" 的缺点，可以连续使用；
- 缺点：若 P0 通过了 while 检查但尚未执行 flag[0] = TRUE 时发生进程切换，P1 也能通过检查并上锁，两个进程可能同时进入临界区，违背" 忙则等待"。根源在于" 检查" 与" 上锁" 不是一气呵成的，并发程序不能保证 **load** 和 **store** 的原子性。

（3）双标志后检查

```
/* 进程 P0 */
while (TRUE) {
    flag[0] = TRUE;             // 先上锁
    while (flag[1] == TRUE);    // 再检查对方
    临界区;
    flag[0] = FALSE;
    剩余区;
}
```

- 解决了先检查法可能同时进入的问题；
- 缺点：若 P0 置 `flag[0] = TRUE` 后发生切换，P1 也置 `flag[1] = TRUE`，则双方都在 while 中无限轮询，谁也进不去，违背" 空闲让进" 和" 有限等待"。

（4）Peterson 算法（正确的软件方案）

```
int turn;             // 表示该让谁进入临界区
bool flag[2];         // 表示进程是否希望进入临界区
flag[0] = flag[1] = FALSE;

/* 进程 P0 */
while (TRUE) {
    flag[0] = TRUE;                       // 1. 表示本进程希望进入
    turn = 1;                             // 2. 礼让 P1
    while (flag[1] == TRUE && turn == 1); // P1 想进且轮到 P1，则等待
    临界区;
    flag[0] = FALSE;                      // 3. 退出临界区
    剩余区;
}
```

```
/* 进程 P1 */
while (TRUE) {
    flag[1] = TRUE;
    turn = 0;
    while (flag[0] == TRUE && turn == 0);
    临界区;
    flag[1] = FALSE;
    剩余区;
}
```

逐条验证四条原则：

- 空闲让进：若 P1 不打算进入临界区，则 `flag[1] = FALSE`，P0 的 while 条件为假，可以立即进入，不会像单标志法那样" 临界区空闲却进不去"；
- 忙则等待：双方都希望进入时由 `turn` 决定，双方互相礼让，后礼让的一方等待，保证同一时刻只有一个进程在临界区；
- 有限等待：P0 进入临界区后，P1 等待；P0 出来时会置 `flag[0] = FALSE`，P1 随即可以进入，不会饥饿；
- 不满足让权等待：进不去时一直执行 while 空循环，直到时间片用完。

软件方法小结：

方法	思想	违背的原则
单标志法	引入共享变量 turn，循环检查 turn 是否为本进程编号	空闲让进（和让权等待）
双标志先检查	引入 flag 数组，先检查对方再上锁	忙则等待
双标志后检查	先设置自身 flag，再检查对方	空闲让进、有限等待
Peterson 算法	引入 turn 与 flag，检查两个条件	让权等待（且只适用于两个进程）

*2.*硬件方法

（1）中断屏蔽方法

```
...
关中断;
临界区;
开中断;
...
```

思想：利用” 开/关中断指令” 实现，与原语的实现思想相同——某进程开始访问临界区到结束访问为止都不允许被中断，也就不可能发生进程切换，因此不可能出现两个进程同时访问临界区。由于进程切换通常由中断或系统调用触发，对于单处理机环境，只需在进入临界区时禁止中断，就能保证当前指令流有序执行、不被抢占。非抢占式内核常采用此方法。

优点：简单、高效。缺点：

- 要求所有调用线程都能执行特权操作，因此只适用于操作系统内核进程，不适用于用户进程（开/关中断是特权指令，2021 年真题考点）；
- 只能用于单处理机环境。多处理机下，即使关闭本 CPU 的中断，其他 CPU 上的线程依然可以进入临界区；
- 频繁关中断会影响系统时钟的更新与时间片计时，因此只能在很短的时间内禁用中断。

（2）TestAndSet 指令（**TS**，也称 **TSL**）

TSL 指令由硬件实现，执行是原子的、不可被中断，即使两条 TSL 指令在不同 CPU 上同时执行，也会按任意次序顺序执行。功能是” 测试并设置”：

```
// 硬件原子指令（伪代码）：返回 lock 的旧值，同时把 lock 置为 TRUE
bool TestAndSet(bool *lock) {
    bool old = *lock;
    *lock = TRUE;
    return old;
}
```

1. 从内存单元读取值；
2. 测试该值是否为 TRUE，返回真或假；
3. 把内存单元的值设置为 TRUE。

```
do {
    ...
```

```
    while (TSL(&lock));   // 上锁失败则一直轮询
    critical section;      // 临界区
    lock = FALSE;          // 退出区解锁
    ...
} while (TRUE);
```

分析：若开始时 lock 为 FALSE，TSL 返回 FALSE 并把 lock 置为 TRUE，while 条件不满足，进程进入临界区；若开始时 lock 为 TRUE，TSL 返回 TRUE，while 条件满足，一直轮询，直到临界区中的进程在退出区解锁或自己时间片耗尽下处理机。

相比软件方法，TSL 把”上锁”和”检查”用硬件变成了一气呵成的原子操作，避免了双标志先检查法的错误。

（3）Swap 指令（EXchange / XCHG）

```
bool key = TRUE;
do {
    key = TRUE;
    do {
        Swap(&key, &lock);   // 原子地交换 key 与 lock
    } while (key == TRUE);
    critical section;
    lock = FALSE;
    ...
} while (TRUE);
```

思想：先用 key 记录之前是否被上锁，然后自己主动上锁（lock = TRUE），再检查 key 是否是 TRUE；若 key 为 FALSE，说明自己是第一个上锁的，可以进入临界区。

逻辑上 Swap 与 TSL 并无太大区别。

两种硬件指令的优缺点：

- 优点：实现简单，容易验证正确性；适用于任意数目的进程，单处理机和多处理机都适用；支持系统中有多个临界区，只需为每个临界区设立一个布尔型变量；
- 缺点：不满足”让权等待”，无法进入临界区的进程会占用 CPU 循环执行 TSL/Swap，导致忙等；从等待进程中随机选择进入临界区，有的进程可能一直选不上，导致饥饿。

TSL 的三个关键结论（2016 年真题）

1. 等待进入临界区的进程不会主动放弃 **CPU**（所以它处于就绪态/运行态，不是阻塞态）；
2. TSL 机制下不存在阻塞态的进程，会一直轮询；
3. while (TSL(&lock)) 应在开中断状态下执行——TSL 本身是一条原子指令，不需要关中断；若整段程序都关中断，则可能很久都不响应中断。

（三）锁

1.互斥锁

用互斥锁保护临界区：进程在进入临界区时 acquire() 获取锁，退出时 release() 释放锁。每个锁有一个 locked 变量表示锁是否可用。

互斥锁必须用硬件机制实现，因为 acquire() 中”检查锁是否可用”和”上锁”之间若发生进程切换，可能违背忙则等待，让多个进程进入临界区。实现方式有两种：获取锁之前关中断（只对单处理机有效）；或者采用原子操作指令（TSL / Swap）。

*2.*自旋锁与忙等待

自旋锁：申请锁失败后一直循环申请，直到成功。

```
// 使用 TSL 指令实现自旋锁
struct Lock { int locked; };   // 0 表示未被占用，1 表示被占用

acquire(Lock *lock) {
    while (TestAndSet(&lock->locked)) {
        // CPU 在此" 忙等待" 或" 自旋"
    }
}

release(Lock *lock) {
    lock->locked = 0;
}
```

自旋锁申请失败时进程处于什么状态

就绪态或运行态。若时间片没用完，则处于运行态一直尝试获取锁；若时间片用完被剥夺处理机，则处于就绪态。注意：单处理器上自旋锁需要抢占式调度器才能使用，否则自旋的线程永远不会放弃处理机。

自旋锁特性总结：

- 具有随机性，不能保证按先来后到顺序使用 CPU 资源；
- 需要忙等，时间片用完才下处理机，违反”让权等待”；
- 等待锁时没有上下文切换（上下文切换开销很大），因此在多处理器系统中若上锁时间很短，等待代价很低；
- 常用于多处理机系统，一个核忙等，其他核照常工作并能快速释放临界区；
- 不适用于单处理机系统（忙等过程中不可能解锁），即使有抢占式调度器性能开销也很大；
- 不提供任何公平性保证，自旋的线程在竞争条件下可能永远自旋，可能导致饿死。

无忙等待锁：申请锁失败后主动阻塞自己，加入该锁的等待队列并让出处理机；其他进程释放锁后从等待队列中唤醒一个进程加入就绪队列。

```
struct Lock {
    int locked;
    waitqueue q;          // 该锁对应的等待队列
};

acquire(Lock *lock) {
    while (TestAndSet(&lock->locked)) {
        block the process into q;   // 主动阻塞
        schedule();                 // 发起调度
    }
}

release(Lock *lock) {
    lock->locked = 0;
    wake a process from q;          // 从等待队列唤醒一个阻塞进程
}
```

无忙等待锁特性：由于有等待队列，可以实现申请锁进程的先来后到，较为公平；未申请到锁时不需要忙等，但要阻塞自己并切换其他进程，存在上下文切换的代价。

忙等待的代价

忙等的本质是”用 CPU 时间换取不发生上下文切换”。因此：

- 临界区很短、多处理器 → 自旋锁划算；
- 临界区很长、单处理器 → 自旋锁极其浪费，应使用阻塞式锁。

（四）信号量

*1.*整型信号量

用一个整数型变量作为信号量，表示系统中某种资源的数量。与普通整型变量的区别是：对信号量的操作只有三种——初始化、**P** 操作、**V** 操作，不能进行值判断。

```
int S = 1;   // 整型信号量，初值表示资源数量

void wait(int S) {          // P 操作，相当于进入区
    while (S <= 0);         // 资源不够就一直轮询（忙等）
    S = S - 1;              // 申请到一个资源
}

void signal(int S) {        // V 操作，相当于退出区
    S = S + 1;              // 释放一个资源
}
```

缺点：`wait` 中用 `while (S <= 0)` 检测资源是否可用，若资源不够则一直轮询，违反”让权等待”。

*2.*记录型信号量

用一个记录型数据结构表示信号量，不存在忙等现象，遵循让权等待。

```
typedef struct {
    int value;          // 剩余资源数
    struct PCB *L;      // 等待队列（PCB 链表，通常用 FIFO）
} semaphore;

void wait(semaphore S) {          // P 操作
    S.value--;                    // 先减 1
    if (S.value < 0) {            // 资源已不够分配
        block(S.L);               // 自我阻塞，由运行态 -> 阻塞态
    }                             // 并插入该信号量的等待队列
}

void signal(semaphore S) {        // V 操作
    S.value++;                    // 先加 1
    if (S.value <= 0) {           // 仍有进程在等待该资源
        wakeup(S.L);              // 唤醒等待队列中的第一个进程
    }                             // 被唤醒进程由阻塞态 -> 就绪态
}
```

物理意义（必背）：

- `S.value > 0`：表示当前还可用的资源数量；
- `S.value = 0`：表示资源刚好用完，没有进程在等待（注意：不是”没有资源”）；
- `S.value < 0`：表示资源已分配完，且有 **|S.value|** 个进程正在等待该资源。

出现负值的原因是 `wait()` 中先递减再判断，互换了递减和测试的顺序；而在整型信号量中信号量的值不能为负。

举例说明 `signal()` 中 `S.value <= 0` 的含义：

设 `sem = 1`：

1. 第一个线程 P 操作后，`sem = 0`（刚好拿走最后一个资源）；
2. 第二个线程 P 操作后，`sem = -1`（资源不够，该线程阻塞在等待队列，等待队列长度 1）；
3. 第三个线程 P 操作后，`sem = -2`（第三个线程也阻塞，等待队列长度 2）；
4. 第一个线程执行 V 操作后，`sem = -1`，因为 `sem <= 0`，说明仍有进程在等待，于是唤醒第二个线程；第二个线程执行完后释放，`sem = 0`，此时仍有第三个线程在等待，再唤醒它；最后第三个线程释放后 `sem = 1`，回到初始状态。

三个必须记住的结论

1. **P(S)**、**V(S)** 都是原语，执行必须一气呵成、不可被打断。对信号量的访问必须是互斥的，因为信号量 S 是多个进程共享的变量，多个进程都可对它读写——这本身就是一个临界区问题。单处理机上可通过关中断实现；多处理机上需要用自旋锁或 `compare_and_swap` 保证原子性。
2. 考试中出现 **P(S)**、**V(S)** 操作，除非特殊说明，否则默认 **S** 为记录型信号量。
3. 信号量是结构体，只能进行 **P/V** 操作，不能进行值判断。写 `if (bowl >= 1)` 是极其错误的。

*3.*用信号量实现互斥

步骤：

1. 分析并发进程的关键活动，划定临界区；
2. 设置互斥信号量 **`mutex`**，初值为 **1**；
3. 在进入区 `P(mutex)`——申请资源；
4. 在退出区 `V(mutex)`——释放资源。

```
semaphore mutex = 1;    // 互斥信号量

P1() {
    ...
    P(mutex);           // 进入区：申请锁
    临界区;
    V(mutex);           // 退出区：释放锁
    ...
}
```

注意：对不同的临界资源要设置不同的互斥信号量；**P**、**V** 操作必须成对出现——缺少 `P(mutex)` 不能保证互斥，缺少 `V(mutex)` 会导致资源永不释放、等待进程永不被唤醒；P、V 必须夹紧使用互斥资源的行为，中间不能有其他冗余代码。

*4.*用信号量实现同步

步骤：

1. 分析什么地方需要实现同步关系，即必须保证” 一前一后” 执行的两个操作；
2. 设置同步信号量 **S**，初值为 **0**；
3. 在**”** 前操作**”** 之后执行 **V(S)**；

4. 在”后操作”之前执行 **P(S)**。

```
semaphore S = 0;    // 同步信号量

P1() {
    ...
    语句 x;          // 前操作
    V(S);
    ...
}

P2() {
    ...
    P(S);
    语句 y;          // 后操作，要使用 x 的运行结果
    ...
}
```

同步信号量的初值不一定是 0，也可以是正整数，取决于”所期待的条件是否已经发生”。例如初值为 2 时，前驱进程释放两次后资源数达到 2，后进程才能继续。

*5.*用信号量实现前驱关系

每一对前驱关系就是一个同步问题，因此：

1. 为每一对前驱关系各设置一个同步信号量，初值均为 **0**；
2. 在”前操作”之后对相应的同步信号量执行 **V** 操作；
3. 在”后操作”之前对相应的同步信号量执行 **P** 操作。

例如前驱图 $S_1 \to S_2$、$S_1 \to S_3$、$S_2 \to S_4$、$S_3 \to S_4$，则需 4 个信号量 a, b, c, d（初值全 0），S_1 末尾写 `V(a); V(b);`，S_2 开头写 `P(a);`、末尾写 `V(c);`，S_3 开头写 `P(b);`、末尾写 `V(d);`，S_4 开头写 `P(c); P(d);`。

三句话总结 P/V 模板

互斥：`P(mutex)` …临界区…`V(mutex)`，`mutex` 初值 1，夹紧临界区。同步：前操作之后 `V(S)`，后操作之前 `P(S)`，`S` 初值 0。前驱：每对前驱关系一个信号量，初值 0，前驱末尾 V，后继开头 P。

（五）条件变量

*1.*为什么要引入条件变量

用信号量解决同步问题虽然强大，但同步问题越复杂、条件越多，需要的信号量就越多，必须非常谨慎地处理信号量之间的顺序，否则很容易造成死锁。为此产生了管程（**Monitor**）：管程把共享数据结构和对它的操作封装起来，并保证每次只有一个进程使用管程，天然实现互斥，程序员无需自己实现互斥；管程还提供条件变量，让程序员灵活实现同步。

管程的组成：管程名称、局部于管程的共享数据结构说明、对该数据结构进行操作的一组过程（函数）、对局部于管程的共享数据设置初始值的语句。

管程的特性：

- 把操作函数封装在管程内（类似 Java 的类），外部进程只能通过调用操作函数来获取和释放资源；

- 每次只允许一个进程使用管程，天然实现互斥；
- 管程内的数据只能被管程内的操作函数修改；
- 管程是被进程调用的，进程通过调用管程中定义的操作来访问共享资源；
- 管程是语法范围（语言级别的抽象），由编程语言的语法定义；
- 无法创建和撤销，生命周期与程序一致。

*2.*条件变量的用法

条件变量的本质是" 进程进入管程后因某原因被阻塞，把该原因定义为一个条件变量"。不同进程被阻塞的原因可能不同，所以可以有多个条件变量；每个条件变量保存一个等待队列。

```
// 在管程内部
condition x;                    // 声明一个条件变量

// 资源不够时（注意：先判断资源是否够，再决定是否 wait）
while (资源不够) {
    x.wait();                   // 当前进程阻塞在该条件变量上，并释放管程
}
// 资源可用时
x.signal();                     // 唤醒等待在该条件变量上的一个进程
```

对条件变量使用 `while` 循环而不是 `if`，可以解决假唤醒（**spurious wakeup**）问题。

*3.*条件变量与信号量的区别

对比项	信号量	条件变量
是否有值	有值，代表资源数量	没有值，只实现排队等待和唤醒功能（资源数由管程内的共享数据结构记录）
操作	`wait(S)` / `signal(S)`（P/V）	`x.wait()` / `x.signal()`
释放操作的影响	**V** 操作一定会改变信号量的值	`signal` 操作可能不产生任何影响（当没有进程因该条件变量阻塞时）
是否需要额外互斥	需要自己保证 P/V 原子性	管程天然实现互斥，程序员无需自己处理

核心结论

管程中的 `signal` 操作和信号量的 `V` 操作只是类似，而不是相同——这是高频易错点。

（六）经典同步问题

*1.*生产者-消费者问题

问题描述：一组生产者进程和一组消费者进程共享一个初始为空、大小为 **n** 的缓冲区。生产者每次生产一个产品放入缓冲区，消费者每次从缓冲区取出一个产品。

三条约束：

1. 只有缓冲区没满时，生产者才能放入产品，否则必须等待（同步关系）；
2. 只有缓冲区不空时，消费者才能取出产品，否则必须等待（同步关系）；
3. 缓冲区是临界资源，各进程必须互斥访问（互斥关系）。

信号量设置：

```
semaphore mutex = 1;     // 互斥访问缓冲区
semaphore empty = n;     // 空闲缓冲区的数量（同步）
semaphore full  = 0;     // 已有产品的数量（同步）
```

完整伪代码：

```
semaphore mutex = 1;
semaphore empty = n;
semaphore full  = 0;

producer() {
    while (1) {
        生产一个产品;
        P(empty);              // 1. 申请一个空闲缓冲区（同步 P）
        P(mutex);              // 2. 申请缓冲区互斥锁（互斥 P）
        把产品放入缓冲区;
        V(mutex);              // 3. 释放缓冲区互斥锁
        V(full);               // 4. 产品数加 1，唤醒可能的消费者
    }
}

consumer() {
    while (1) {
        P(full);               // 1. 申请一个产品（同步 P）
        P(mutex);              // 2. 申请缓冲区互斥锁
        从缓冲区取出一个产品;
        V(mutex);              // 3. 释放缓冲区互斥锁
        V(empty);              // 4. 空闲缓冲区加 1，唤醒可能的生产者
        消费产品;
    }
}
```

为什么这样设计（关键分析）：

1. 必须先对同步信号量 **P**，再对互斥信号量 **P**（即 P(empty) 在 P(mutex) 之前）。若颠倒顺序：假设缓冲区已满，生产者先 P(mutex) 拿到锁，再 P(empty) 时发现 empty = 0 而被阻塞；此时消费者想进入临界区消费，却因为 mutex = 0 而被阻塞。双方互相等待对方唤醒自己 → 死锁。
2. **V** 操作的顺序可以交换，因为 V 操作不会引起阻塞（V(mutex) 与 V(full) 谁先谁后都不影响正确性）。
3. ”生产一个产品”和”消费产品”这两个动作最好放在临界区之外。虽然从逻辑上也可以放进临界区，但那样会延长临界区的长度，降低并发度、降低系统效率。
4. 注意唤醒关系：消费者可以唤醒生产者（缓冲区空时），生产者也可以唤醒生产者（缓冲区满时第二个生产者阻塞，第一个生产者完成后将它唤醒）。

单缓冲与多缓冲的区别：当 $n=1$（单缓冲区）时，empty 与 full 的取值只能是 0 或 1，此时 empty 或 full 这两个一般信号量已经天然充当了互斥信号量，可以省去 mutex。但当缓冲区数量 ≥ 2 时，必须用 **mutex** 实现互斥访问。

生产者-消费者问题的四个易错点

1. 顺序不能颠倒：同步 P 必须在互斥 P 之前；
2. **P**、**V** 必须成对：漏掉 V(mutex) 会让缓冲区永不释放；
3. 单缓冲区可省 **mutex**，多缓冲区不可省；
4. 题目中的”产品”/”数据”就是普通资源，不要与后面的信号混淆。

多生产者多消费者（放水果问题）：

爸爸专放苹果，妈妈专放橘子，儿子专吃橘子，女儿专吃苹果，盘子只能放一个水果。

```
semaphore plate  = 1;    // 盘子空位只有 1 个，可充当互斥信号量
semaphore apple  = 0;
semaphore orange = 0;

爸爸 () { while (1) { P(plate); 往盘子里放苹果;   V(apple);  } }
妈妈 () { while (1) { P(plate); 往盘子里放橘子;   V(orange); } }
儿子 () { while (1) { P(orange); 从盘子取出橘子; V(plate); 吃橘子; } }
女儿 () { while (1) { P(apple);  从盘子取出苹果; V(plate); 吃苹果; } }
```

为什么不设 mutex 也能互斥

因为盘子空位只有 1，且 apple、orange、plate 在任何时刻最多只有一个为 **1**，四个进程执行 P 操作时最多只有一个人不会被阻塞，因此天然实现了互斥。缓冲区数量为 **1** 时可能天然互斥，但缓冲区数量为 **2** 时必须用 **mutex**。

共享缓冲区导致的隐含死锁（必考难点）：

两个生产者 A、B 共用一个大小为 N 的缓冲区，消费者需要”一个 A 和一个 B”才能消费。若 A 推进过快把缓冲区全部占满 A，消费者已持有 A 却还需要 B 而 B 放不进去（缓冲区满）→ 死锁。

解决思路：把问题转化为双缓冲区问题，构造两个虚拟缓冲区，各留出配额：

```
semaphore empty    = N;      // 箱子内空位数量
semaphore frame    = 0;      // 箱子内车架数量
semaphore wheel    = 0;      // 箱子内车轮数量
semaphore mutex    = 1;      // 互斥访问箱子
semaphore frameMax = N - 2;  // 最多还能放几个车架（保证至少给车轮留 2 个空位）
semaphore wheelMax = N - 1;  // 最多还能放几个车轮（保证至少给车架留 1 个空位）

工人 1() {   // 生产车架
    while (1) {
        加工一个车架;
        P(frameMax);
        P(empty);
        P(mutex);
        车架放入箱中;
        V(mutex);
        V(frame);
    }
}

工人 2() {   // 生产车轮
    while (1) {
        加工一个车轮;
        P(wheelMax);
        P(empty);
        P(mutex);
        车轮放入箱中;
```

```
        V(mutex);
        V(wheel);
    }
}

工人 3() {   // 组装
    while (1) {
        P(frame); P(mutex); 箱中取出一个车架; V(mutex); V(empty); V(frameMax);
        P(wheel); P(wheel); P(mutex); 箱中取出二个车轮; V(mutex);
        V(empty); V(empty); V(wheelMax); V(wheelMax);
        组装为一台车;
    }
}
```

*2.*读者-写者问题

问题描述：有读者和写者两组并发进程共享一个文件。要求：

- 允许多个读进程同时对文件执行读操作（读者读数据后不改变数据，与消费者不同）；
- 同一时刻只允许一个写者写；
- 任一写者在完成写操作之前不允许其他任何进程工作；
- 写者执行写操作之前，应让已有的读者、写者全部退出。

这是典型的分类互斥（**categorical mutual exclusion**）：同类进程不互斥，异类进程互斥。

通用解题模板（**”** 三段式**”**）：本质是” 连续多个同类进程访问同一个临界资源”——第一个进程开始访问前对资源加互斥锁，后续同类进程不再加锁；最后一个进程访问完后解锁。就像第一个人进房间时开门，后面的人不用开门，最后一个人离开时关门。

（**1**）读者优先

```
semaphore rw    = 1;    // 用于读者与写者、写者与写者之间的互斥
semaphore mutex = 1;    // 用于对 count 变量的互斥访问
int count = 0;          // 正在读的读者数量

writer() {
    while (1) {
        P(rw);              // 写之前加锁
        写文件;
        V(rw);              // 写完之后解锁
    }
}

reader() {
    while (1) {
        P(mutex);           // 各读者互斥地访问 count
        if (count == 0)     // 我是第一个读者
            P(rw);          // 由我负责对文件加锁
        count++;            // 读者数加 1
        V(mutex);
        读文件;
        P(mutex);
        count--;            // 读者数减 1
        if (count == 0)     // 我是最后一个读者
            V(rw);          // 由我负责对文件解锁
        V(mutex);
```

```
    }
}
```

为什么必须给 **count** 再设一个 **mutex**?

假设没有 mutex：进程 1 判断 count == 0 成立（认为自己是第一个读者），此时发生进程切换；进程 2 也判断 count == 0 成立（也认为自己是第一个读者），也去 P(rw) 加锁；结果第一个读者加锁成功、第二个读者加锁失败被永久阻塞。根源在于**"** 判断 **count"** 和**"** 加锁**"** 无法一气呵成。

缺点：只要有一个读者处于活动状态，后来的读者都会被接纳。若读者源源不断到达，写者将始终阻塞，可能饿死。

（**2**）写者优先

```
semaphore rw    = 1;    // 读者与写者、写者与写者互斥
semaphore mutex = 1;    // 互斥访问 rcount
semaphore read  = 1;    // 用来禁止所有读进程（写者到来后封锁后续读者）
semaphore wmutex= 1;    // 互斥访问 wcount
int rcount = 0;         // 读者数量
int wcount = 0;         // 写者数量

reader() {
    while (1) {
        P(read);            // 若有写者在排队/执行，后来的读者阻塞在这里
        P(mutex);
        if (rcount == 0) P(rw);
        rcount++;
        V(mutex);
        V(read);
        读文件;
        P(mutex);
        rcount--;
        if (rcount == 0) V(rw);
        V(mutex);
    }
}

writer() {
    while (1) {
        P(wmutex);
        if (wcount == 0) P(read);   // 第一个写者封锁后续读者
        wcount++;
        V(wmutex);
        P(rw);                       // 等现有读者全部退出
        写文件;
        V(rw);
        P(wmutex);
        wcount--;
        if (wcount == 0) V(read);    // 最后一个写者解除对读者的封锁
        V(wmutex);
    }
}
```

read 的作用分析：开始有若干读者在读，此时来了一个写者，执行 P(read) 之后，后续的读者由于阻塞在 **read** 上都不能再进入读者队列，而写者可以进入写者队列，因此保证了写者优先。同时，第一个写者执行了 P(read) 之后也不能马上开始写，必须等到所有已在读者队列中的读者都读完、最后一个读者通过 V(rw) 唤醒写者。

(3) 读写公平法

```
semaphore rw    = 1;    // 读者与写者、写者与写者互斥
semaphore mutex = 1;    // 互斥访问 count
semaphore queue = 1;    // 让读者和写者排队，实现读写公平
int count = 0;

writer() {
    while (1) {
        P(queue);           // 先排队
        P(rw);
        写文件;
        V(rw);
        V(queue);
    }
}

reader() {
    while (1) {
        P(queue);           // 先排队
        P(mutex);
        if (count == 0) P(rw);
        count++;
        V(mutex);
        V(queue);           // 排队结束，允许下一个进程排队
        读文件;
        P(mutex);
        count--;
        if (count == 0) V(rw);
        V(mutex);
    }
}
```

为什么加一个 **queue** 就实现了公平？对比读者优先方案，读者优先中只要有读者到达就能进入读者队列；而 queue 阻止了读者的这种” 特殊权限”——当写者执行了 P(queue) 后，后续到达的读者都阻塞在 queue 上，读者队列逐渐为空（count 减为 0），写者最终能拿到 rw 进入临界区。因此 queue 实现了读写进程优先级平等。

把三种方案放在一起比较：

方案	核心机制	结果
读者优先	第一个读者加 rw 锁，最后一个读者解锁	写者可能饿死
写者优先	第一个写者执行 P(read) 封锁后续读者	读者可能饿死
读写公平	加 queue 信号量让读写统一排队	按到达顺序访问，都不饿死

读者-写者类问题的识别与迁移：该类问题的特点是分类互斥——同类进程不互斥、异类进程互斥。凡是” 同一类可以多人同时进行、不同类必须独占” 的场景，都可套用此模板，关键点是设置一个计数器，第一个同类进程负责上锁，最后一个负责解锁。

*3.*哲学家进餐问题

问题描述：一张圆桌上坐着 5 名哲学家，每两名哲学家之间放一根筷子（共 5 根）。哲学家只做两件事：思考和进餐。饥饿时必须同时拿起左右两根筷子才能进餐，进餐完放下筷子继续思考。

该类题目的特点：只有一类进程，但要占用多种资源才能运行。

最初的错误解法：

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};

philosopher(int i) {
    while (1) {
        P(chopstick[i]);              // 拿左筷子
        P(chopstick[(i+1) % 5]);      // 拿右筷子
        进餐;
        V(chopstick[i]);
        V(chopstick[(i+1) % 5]);
        思考;
    }
}
```

死锁原因：若 5 名哲学家同时拿起各自的左筷子，则所有筷子的信号量都为 0，谁也无法拿起右筷子，5 个进程全部阻塞，形成循环等待 → 死锁。

三种正确解法：

解法一：限制并发进程数（最多允许 **4** 人同时拿筷子）

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
semaphore limit = 4;      // 最多允许 4 名哲学家同时尝试拿筷子

philosopher(int i) {
    while (1) {
        P(limit);
        P(chopstick[i]);
        P(chopstick[(i+1) % 5]);
        进餐;
        V(chopstick[i]);
        V(chopstick[(i+1) % 5]);
        V(limit);
        思考;
    }
}
```

原理：至少有一名哲学家能同时拿到两根筷子，打破循环等待。破坏了”循环等待”条件。缺点：并发度不高。

解法二：限制申请资源的顺序（奇数号先拿左、偶数号先拿右）

```
philosopher(int i) {
    while (1) {
        if (i % 2 == 1) {                    // 奇数号：先左后右
            P(chopstick[i]);
            P(chopstick[(i+1) % 5]);
        } else {                             // 偶数号：先右后左
            P(chopstick[(i+1) % 5]);
            P(chopstick[i]);
        }
        进餐;
        V(chopstick[i]);
```

```
        V(chopstick[(i+1) % 5]);
        思考;
    }
}
```

原理：相邻两名哲学家拿筷子的顺序相反，不可能形成” 每人持有一根、都在等下一根” 的环。破坏了**”** 循环等待**”** 条件。缺点是不通用，只适用于本题这种对称结构。

解法三：拿筷子前先检测，且拿筷子动作互斥（**AND** 型信号量思想）

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
semaphore mutex = 1;      // 保证" 拿起两根筷子" 是一个原子操作

philosopher(int i) {
    while (1) {
        P(mutex);                          // 先申请拿筷子的权利
        P(chopstick[i]);                   // 拿左筷子
        P(chopstick[(i+1) % 5]);           // 拿右筷子
        V(mutex);                          // 两根都拿到后才释放权利
        进餐;
        V(chopstick[i]);
        V(chopstick[(i+1) % 5]);
        思考;
    }
}
```

原理：一名哲学家在” 拿两根筷子” 的整个过程中持有 mutex，其他哲学家无法同时尝试拿筷子。因此不存在” 每人拿一根、都在等第二根” 的局面，破坏了**”** 请求并保持**”** 条件（要么一次拿到两根，要么一根都不拿）。

关键辨析：这种方案不会导致饥饿——因为用完餐的哲学家总会放下两根筷子并且不再拿起，所以处于等待中的哲学家一定会等到” 两根筷子都空闲” 的时刻。有人误以为” 旁边的筷子可能被多次拿起”，这是错的，因为 mutex 保证了拿筷子的动作是串行的、完整的。

三种解法小结：

解法	原理	破坏的条件	通用性
限制并发数（n-1 人尝试）	至少一人能拿到两根筷子	循环等待	通用
奇偶不同顺序拿筷子	相邻哲学家拿筷顺序相反，无法成环	循环等待	不通用（对称结构可用）
拿筷子动作整体互斥	要么一次拿到两根，要么一根不拿	请求并保持	通用（推荐）

真题考法

2019 年曾考” 哲学家进餐问题” 的 PV 操作大题，要求写出完整的过程并说明信号量的含义及初值。答题时必须写出信号量定义与初值，并用 P、V 操作（或 wait()、signal()）完整描述每个进程。

*4.*理发师问题

核心思路转换：理发师问题与生产者-消费者问题的思路是一样的——顾客可以视为” 生产者”（生产顾客资源），服务人员可以视为” 消费者”（消费顾客资源）；反过来服务人员叫号也可视为生产了一个” 服务人员资源”。取号和叫号类似于往缓冲区放东西，需要一个互斥锁。

万能模板 **1**（无等待上限，服务人员可休息）

```
int waiting = 0;                // 正在等待的顾客数
semaphore customers = 0;        // 等待服务的顾客数
semaphore barbers   = 0;        // 空闲的服务人员数
semaphore mutex     = 1;        // 对 waiting 的互斥访问

barber() {                      // 服务人员进程（有 N 个）
    while (1) {
        P(customers);           // 有顾客才工作，否则阻塞休息
        P(mutex);
        waiting--;              // 少一名等待的顾客
        V(mutex);
        V(barbers);             // 叫号：生产一个" 空闲服务人员"
        为顾客服务;
    }
}

customer() {                    // 顾客进程
    P(mutex);                   // 取号（互斥）
    if (waiting < N) {
        waiting++;
        V(mutex);
        V(customers);           // 叫醒一名休息的服务人员
        P(barbers);             // 等待被叫号
        接受服务;
    } else {
        V(mutex);               // 没位置了，离店
    }
}
```

万能模板 **2**（有等待上限，服务人员可休息）：在上面的基础上把 `waiting` 作为判断条件，顾客到店时若 `waiting` 已达上限则直接离店（模板 1 中就是这种写法）。

万能模板 **3**（有等待上限，服务人员忙等）：服务人员在没有顾客时也不可以休息，必须不断询问有没有顾客。因此在服务人员进程中加入判断顾客数量的语句——对顾客资源的 **P** 操作就不需要了，因为 P 操作会引发阻塞或休息。

万能模板 **4**（座位已满后顾客仍继续等待，服务人员可休息）：把 `waiting` 变量变为信号量 **`empty`**（表示店里空位数），顾客先 `P(empty)` 阻塞等待空位，再取号等待叫号。

理发师问题的做题关键

1. 只需把模板 **1** 的思路记下来，其余都是从它延伸来的。做题时先默写模板 1，再按题意修补；
2. 服务人员的数量在模板中通常是冗余条件（N 个服务人员对应 N 个进程，或用一个信号量记录空闲服务人员数）；
3. 上述模板都没有满足先来先服务（因为 `V(server)` 后任意一个顾客进程都可能继续执行）。统考一般不要求先来先服务。

*5.*吸烟者问题

供应者每次把两种材料放在桌上，拥有第三种材料的吸烟者才能卷烟抽。设三种材料对应三个信号量 `offer1`、`offer2`、`offer3`，再加一个 `finish` 信号量让供应者知道上一轮已被取走。

```
semaphore offer1 = 0, offer2 = 0, offer3 = 0;
semaphore finish = 0;
int i = 0;                          // 供应者轮流提供的组合
```

```
provider() {
    while (1) {
        if (i == 0)      { 将烟草和纸放到桌上; V(offer1); }
        else if (i == 1) { 将烟草和胶水放到桌上; V(offer2); }
        else             { 将纸和胶水放到桌上; V(offer3); }
        i = (i + 1) % 3;
        P(finish);                // 等吸烟者抽完再放下一组
    }
}

smoker1() {   // 有胶水的人
    while (1) { P(offer1); 卷烟抽; V(finish); }
}

smoker2() {   // 有纸的人
    while (1) { P(offer2); 卷烟抽; V(finish); }
}

smoker3() {   // 有烟草的人
    while (1) { P(offer3); 卷烟抽; V(finish); }
}
```

易错

吸烟者问题也是”单缓冲区”结构（桌上同时只有一组材料），但由于供应者每次只放一组、且吸烟者抽完后才放下一组，不需要额外的 **mutex**。判断是否需要 mutex 的方法是：看是否存在”多个进程可能同时进入缓冲区”的风险。

*6.*独木桥问题

问题描述：桥不允许两车交汇，但允许同方向多辆车依次通过（桥上可以有多个同方向的车）。用 P、V 操作实现交通管理防止堵塞。

识别：这是同类不互斥、异类互斥的分类互斥问题，与读者-写者问题同构（同方向的车相当于”读者”，对向的车相当于”写者”）。

```
semaphore mutex = 1;      // 对两个计数器的互斥访问
semaphore bridge = 1;     // 桥的互斥访问（相当于读者-写者中的 rw）
int countA = 0;           // A 方向在桥上的车数
int countB = 0;           // B 方向在桥上的车数

carA() {
    while (1) {
        P(mutex);
        if (countA == 0) P(bridge);   // 第一辆 A 方向的车负责上锁
        countA++;
        V(mutex);
        过桥;
        P(mutex);
        countA--;
        if (countA == 0) V(bridge);   // 最后一辆 A 方向的车负责解锁
        V(mutex);
    }
}
```

carB() 完全对称。

判断”是否类似读者-写者”的方法

同类进程之间不互斥、异类进程之间互斥 ⇒ 套读者-写者模板。关键是设一个计数器记录同类进程数量，第一个进程上锁、最后一个进程解锁。独木桥、更衣室、录像厅、寿司店等题目都是这个套路。

7.流水线与前驱关系问题

问题描述：一条流水线上有多个工序，后一工序必须等前一工序完成后才能开始；相邻工序之间有一个容量为 1 的缓冲区。

识别：这是前驱关系 + 单缓冲区问题，每个工序既是前一工序的消费者，又是后一工序的生产者。

```
semaphore empty1 = 1, empty2 = 1;   // 各缓冲区空位
semaphore full1  = 0, full2  = 0;   // 各缓冲区产品

工序 1() {
    while (1) {
        加工零件;
        P(empty1);
        把零件放到缓冲区 1;
        V(full1);
    }
}

工序 2() {
    while (1) {
        P(full1);
        从缓冲区 1 取出零件;
        V(empty1);
        加工零件;
        P(empty2);
        把零件放到缓冲区 2;
        V(full2);
    }
}

工序 3() {
    while (1) {
        P(full2);
        从缓冲区 2 取出零件;
        V(empty2);
        加工零件;
    }
}
```

流水线问题的通用套路

对每一段缓冲区设一对信号量：`empty_i`（初值 = 缓冲区容量，容量为 1 时初值 1）和 `full_i`（初值 0）。前一工序放入产品时”先 `P(empty_i)` 再 `V(full_i)`”，后一工序取出产品时”先 `P(full_i)` 再 `V(empty_i)`”。这实际上就是把每个工序都当作一个小型的生产者-消费者。

PV 操作大题的答题规范

1. 必须写出信号量定义并赋初值，并说明每个信号量的含义；
2. 必须写出每个进程的完整循环体，用 P/V（或 `wait/signal`）操作描述；
3. 用中文描述动作（如”把产品放入缓冲区”），不要遗漏；
4. 检查是否存在死锁：连续的 P 操作是否可能造成循环等待；多个进程共享同一缓冲区时是否存在某一方把缓冲区占满的隐患；如果有，加入 `mutex` 让连续 P 操作一气呵成，或转化为双缓冲区。

四、死锁

（一）死锁的基本概念

*1.*定义

死锁（**Deadlock**）：在并发环境下，各进程互相等待对方手里的资源，导致各进程都阻塞、无法向前推进的现象。死锁的定义包含两点：互相循环等待 + 各个进程都被阻塞。

*2.*死锁、饥饿、死循环的区别

对比项	死锁	饥饿	死循环
定义	各进程互相等待对方手里的资源，都阻塞无法推进	由于长期得不到想要的资源，某进程无法向前推进	进程在不满足终止条件时无限循环
原因	循环等待对方手里的资源	优先级倒置、资源分配不公平或错误	编程错误、逻辑错误
例子	银行家算法问题中的死锁状态	低优先级进程长期得不到 CPU	空循环、忘记更新循环条件
进程状态	阻塞态	阻塞态（得不到 I/O 设备）或就绪态（上不了处理机）	运行态
涉及进程数	两个或两个以上	可能只有一个，也可能多于一个	大于等于一个
性质	操作系统（管理者）的问题	操作系统（管理者）的问题	被管理者（程序员）的问题

*3.*产生死锁的原因

1. 对系统资源的竞争：各进程竞争不可剥夺资源，若资源数目满足不了多个进程的需要，就会因争夺资源陷入僵局。可剥夺资源（CPU、主存）不会引发死锁。
2. 进程推进顺序不当：P1 占有 R1 又申请 R2，P2 占有 R2 又申请 R1，互相等待。
3. 信号量使用不当：例如生产者-消费者问题中把实现互斥的 P 操作放在实现同步的 P 操作之前。
4. 由可消耗性资源引起：多个进程通信时 `send/receive` 顺序处理不当，都在等一条永远不会发出的消息。

*4.*四个必要条件

条件	含义
互斥条件	任何时刻只能有一个进程使用一个资源实例。只有对必须互斥使用的资源（临界资源、锁）的争夺才会导致死锁；内存、CPU、扬声器这类可同时共享的资源不会导致死锁
不可剥夺（不可抢占）条件	进程所获得的资源在未使用完之前不能被其他进程强行夺走，只能主动释放
请求并保持（请求与保持）条件	进程已经保持了至少一个资源，但又提出新的资源请求，而该资源被其他进程占有，此时请求进程被阻塞，但对自己已有的资源保持不放
循环等待条件	存在一种进程-资源的循环等待链，链中每一个进程已获得的资源同时被链中下一个进程所请求

四个条件必须”同时满足”才可能死锁，只要破坏其中一个，死锁就不会发生。

循环等待 $\neq$ 死锁

发生死锁时一定有循环等待，但发生循环等待时未必死锁。若同类资源数大于 1，即使存在循环等待，等待链中的某个进程仍可能从链外的进程那里获得资源，从而打破循环。只有当系统中每类资源都只有一个时，循环等待才是死锁的充要条件。

*5.*两个常用结论

设系统有 n 个进程，每个进程最多需要 y 个同类资源：

- 系统必然不会发生死锁的最少资源数为 $n \times (y-1)+1$；
- 反之，若系统有 x 个资源、每个进程最多需要 y 个资源，则可能发生死锁的最小进程数为 $\left\lceil \dfrac{x}{y-1} \right\rceil$。

推导思路：最坏情况是每个进程都只拥有 $y-1$ 个资源并都在申请第 y 个。若资源总数恰好为 $n(y-1)$，资源已用完，必然死锁；再多加一个资源，令其中一个进程获得所需的最大资源，死锁即被解除。

用例题验证：

- 系统有 8 台打印机，K 个进程竞争，每个进程最多需要 3 台。要保证不死锁需 $8 \geq K(3-1)+1 = 2K+1$，即 $K \leq 3.5$，故 $K=4$ 时可能发生死锁，答案是 4。
- 三个进程分别需要 3、4、5 台设备：最坏情况各拿到 2、3、4 台，共 9 台，只要再能额外提供 1 台就能让某个进程完成，故确保不发生死锁的最小设备数 $n=9+1=10$。
- n 个进程每个需要 2 个临界资源：最坏情况各拿到 1 个，共 n 个，再加 1 个即可打破僵局，故至少需要 $n+1$ 个资源。

*6.*死锁处理策略

策略	思想	优缺点
死锁预防（**Prevention**）	设计系统时破坏四个必要条件之一，从根本上排除死锁可能	一定能保证不死锁，但资源利用率低、并发度差
死锁避免（**Avoidance**）	不破坏必要条件，在资源动态分配中防止系统进入不安全状态（银行家算法）	需要预先知道进程运行所需的资源总量，并发性次于检测法
死锁检测与解除（**Detection & Recovery**）	允许死锁发生，定期检测并在发现后解除	并发性最大，但检测有开销，解除可能造成工作损失

按并发性从大到小排序：检测死锁 > 银行家算法 > 资源预分配。

（二）死锁预防

*1.*破坏互斥条件

把互斥资源改造成共享资源。例如用 **SPOOLing** 技术把打印机这种独占设备在逻辑上改造成共享设备；再如通过强大的硬件指令把需要互斥访问的代码构造成不需要锁的数据结构。

缺点：有的资源本身就是非共享的（一个互斥锁不能同时被多个进程共享），有些地方还必须保护

这种互斥性。因此这种方法通常不太可行。

*2.*破坏不可剥夺条件

- 方案一：当进程请求新资源得不到满足时，立即释放它保持的所有资源；
- 方案二：当进程需要的资源被其他进程占有时，由操作系统协助强行剥夺。

缺点：实现复杂、代价大；释放已获得的资源可能造成前一阶段工作失效，使进程前后两次运行的信息不连续；反复申请和释放资源会增加系统开销、降低吞吐量；可能导致进程饥饿。因此只适用于状态容易保存和恢复的资源（如 CPU 寄存器、内存），一般不适用于互斥锁、信号量、打印机、CD 刻录机。

*3.*破坏请求并保持条件

要求" 进程在请求资源时不能持有不可剥夺资源"，有两种协议：

- 协议一（静态分配）：进程在运行前一次申请完它所需要的全部资源，资源未满足就不让它投入运行；
- 协议二（改进版）：允许进程只获得运行初期所需的资源便开始运行，运行过程中逐步释放已用完的资源，然后再请求新的资源。

缺点：需要准确预知需要哪些资源并提前获得；有些资源只在运行初期或快结束时短暂使用，若整个运行期一直保持就会严重降低并发和资源利用率；协议一实现简单但资源浪费严重，且可能导致饥饿；协议二改进了协议一的缺点，能更快完成任务、提高设备利用率、减少饥饿概率。

*4.*破坏循环等待条件

最实用、最经常采用的预防技术：采用顺序资源分配法（资源有序分配算法）。给系统中所有资源编号，规定每个进程必须按编号递增的顺序请求资源，同类资源一次申请完。也就是说，进程一旦申请了资源 R_i，以后的申请就只能申请编号大于 R_i 的资源。

优点：从机制上杜绝了循环等待链。

缺点：

- 不方便增加新设备，可能需要重新分配所有编号；
- 进程实际使用资源的顺序可能与编号递增顺序不一致，导致资源浪费（例如 1 号资源是最后需要的，却被最先持有，使其他只需 1 号资源的进程被长时间阻塞）；
- 必须按规定次序申请资源，用户编程麻烦。

常考对比

死锁预防靠破坏四个必要条件；死锁避免靠银行家算法保证系统始终处于安全状态。二者常被放在一起考（如 2015-26、2019-30、2013-32）。另外，限制用户申请资源的顺序属于死锁预防（破坏循环等待），不属于死锁避免。

（三）死锁避免

*1.*安全状态与安全序列

- 安全序列：指如果系统按这种序列为进程分配资源，则每个进程都能顺利完成。只要能找出一个安全序列，系统就处于安全状态。安全序列可能有多个。
- 不安全状态：如果分配资源后系统中找不出任何一个安全序列，则系统进入不安全状态。

- 关键结论：
  1. 处于安全状态，一定不会发生死锁；
  2. 处于不安全状态，可能发生死锁，但不一定（例如进程提前终止释放了资源，或实际所需资源小于声明的最大需求）；
  3. 发生死锁，一定处于不安全状态；
  4. 若进程提前归还部分资源，系统也有可能从不安全状态重新回到安全状态，但在分配资源以前总要考虑最坏情况；
  5. 单个进程阻塞不是死锁，必须形成阻塞环才是死锁。

死锁避免与死锁检测的区别：

对比项	死锁避免	死锁检测
需要的资源信息	需要进程整个生命周期所需的资源总量（最大需求）	只需要当前时刻的资源请求与分配信息
是否限制资源申请顺序	不会（限制顺序属于死锁预防）	不会
是否会给可能导致死锁的进程分配资源	不会	会（允许死锁发生，之后再解除）
典型算法	银行家算法	资源分配图化简法（死锁定理）

*2.*银行家算法的数据结构

设有 n 个进程、m 类资源：

数据结构	类型	含义
Available	长度为 m 的向量	`Available[j]` 表示第 j 类资源的当前可用数量
Max	$n \times m$ 矩阵	`Max[i][j]` 表示进程 P_i 对第 j 类资源的最大需求
Allocation	$n \times m$ 矩阵	`Allocation[i][j]` 表示已分配给 P_i 的第 j 类资源数量
Need	$n \times m$ 矩阵	`Need[i][j]` 表示 P_i 还需要第 j 类资源的数量

三者关系（必须自洽）：

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

且满足 $\sum_i Allocation[i][j] + Available[j] =$ 系统中第j类资源总数。

*3.*安全性算法（完整步骤）

1. 令 **Work = Available**，**Finish[i] = FALSE**（$i = 0, 1, \ldots, n-1$）；
2. 寻找满足下列两个条件的进程 P_i：
   - `Finish[i] == FALSE`；
   - `Need[i][j]` $\leq$ `Work[j]`（对所有 j 成立）；若找到，转步骤 3；若找不到，转步骤 4；
3. 该进程可以顺利执行完毕并释放它的全部资源：

$$Work[j] = Work[j] + Allocation[i][j] \quad (\text{对所有} j)$$

`Finish[i] = TRUE`，把 P_i 加入安全序列，返回步骤 **2**；

1. 若所有 `Finish[i] == TRUE`，则系统处于安全状态（步骤 3 中得到的进程顺序就是一个安全序列）；否则系统处于不安全状态。

一句话概括：检查当前剩余可用资源能否满足某个进程的最大需求，可以就把它加入安全序列并回收它持有的全部资源，不断重复，看最终能否让所有进程都加入安全序列。

*4.*资源请求算法

当进程 P_i 发出资源请求向量 $Request_i$ 时：

1. 检查是否超过声明的最大需求：若 `Request_i[j] > Need[i][j]`，说明请求量超过它事先声明的最大需求，报错；
2. 检查系统是否有足够资源：若 `Request_i[j] > Available[j]`，说明资源暂时不够，P_i 必须等待；
3. 试探性分配（修改数据结构）：

$$Available[j] = Available[j] - Request_i[j]$$

$$Allocation[i][j] = Allocation[i][j] + Request_i[j]$$

$$Need[i][j] = Need[i][j] - Request_i[j]$$

1. 执行安全性算法检查此次分配后系统是否安全：
   - 若安全，则正式分配；
   - 若不安全，则撤销本次试探性分配（恢复原先的 Available、Allocation、Need），让 P_i 等待。

*5.*完整算例一：*T0* 时刻的安全性检测

设系统有 5 个进程 P0 ~ P4、3 类资源 A、B、C，各类资源总数分别为 **A = 10**、**B = 5**、**C = 7**。T0 时刻状态如下：

进程	**Allocation (A,B,C)**	**Max (A,B,C)**	**Need (A,B,C)**
P0	0, 1, 0	7, 5, 3	7, 4, 3
P1	2, 0, 0	3, 2, 2	1, 2, 2
P2	3, 0, 2	9, 0, 2	6, 0, 0
P3	2, 1, 1	2, 2, 2	0, 1, 1
P4	0, 0, 2	4, 3, 3	4, 3, 1

第一步：求 **Available**

已分配总量：$\mathrm{A} = 0+2+3+2+0 = 7$；$\mathrm{B} = 1+0+0+1+0 = 2$；$\mathrm{C} = 0+0+2+1+2 = 5$。

$$Available = (10-7,\ 5-2,\ 7-5) = (3,\ 3,\ 2)$$

第二步：逐个查找满足 $Need \le Work$ 的进程

第 1 轮，Work = (3, 3, 2)：

进程	**Need**	是否 $\leq$ **Work**	判定
P0	(7,4,3)	7 > 3	否
P1	(1,2,2)	全部满足	是
P2	(6,0,0)	6 > 3	否
P3	(0,1,1)	全部满足	是（但按编号先取 P1）
P4	(4,3,1)	4 > 3	否

选 **P1**。P1 执行完毕，释放 Allocation(P1) = (2,0,0)：

$$Work = (3,3,2) + (2,0,0) = (5,\ 3,\ 2)$$

安全序列当前为：**P1**

第 2 轮，Work = (5, 3, 2)：

进程	**Need**	是否 $\leq$ **Work**
P0	(7,4,3)	7 > 5，否
P2	(6,0,0)	6 > 5，否
P3	(0,1,1)	是
P4	(4,3,1)	是（按编号先取 P3）

选 **P3**。释放 Allocation(P3) = (2,1,1)：

$$Work = (5,3,2) + (2,1,1) = (7,\ 4,\ 3)$$

安全序列：P1 $\rightarrow$ **P3**

第 3 轮，Work = (7, 4, 3)：

进程	**Need**	是否 $\leq$ **Work**
P0	(7,4,3)	是（按编号先取 P0）
P2	(6,0,0)	是
P4	(4,3,1)	是

选 **P0**（按编号顺序）。释放 Allocation(P0) = (0,1,0)：

$$Work = (7,4,3) + (0,1,0) = (7,\ 5,\ 3)$$

安全序列：P1 $\rightarrow$ P3 $\rightarrow$ **P0**

第 4 轮，Work = (7, 5, 3)：

进程	**Need**	是否 $\leq$ **Work**
P2	(6,0,0)	是
P4	(4,3,1)	是

选 **P2**。释放 Allocation(P2) = (3,0,2)：

$$Work = (7,5,3) + (3,0,2) = (10,\ 5,\ 5)$$

安全序列：P1 → P3 → P0 → **P2**

第 5 轮，Work = (10, 5, 5)：P4 的 Need = (4,3,1) 满足，选 **P4**。

$$Work = (10,5,5) + (0,0,2) = (10,\ 5,\ 7)$$

安全序列：**P1** → **P3** → **P0** → **P2** → **P4**，所有进程 Finish 均为 TRUE，系统处于安全状态。

结果校验

安全序列执行完毕后，Work = (10, 5, 7)，恰好等于系统资源总数 (A=10, B=5, C=7)。这是检查算例是否算错的快捷方法：安全序列跑完后的 **Work** 必须等于资源总数。

安全序列的唯一性

本题中第一步只有 **P1** 能执行（P0、P2、P4 的 Need 都超过 Available），因此所有安全序列都必须以 **P1** 开头。之后有多个分支，因此安全序列不唯一（例如 P1 → P3 → P4 → P2 → P0 也是安全序列）。做选择题时要逐个验证选项中的序列是否真的每一步都可行。

*6.*完整算例二：资源请求的处理

仍用上面的系统，当前 $Available = (3,3,2)$。

情形 **1**：**P0** 请求 **Request = (2, 1, 0)**

1. 检查最大需求：`Request(2,1,0)` $\leq$ `Need(P0)=(7,4,3)`，通过；
2. 检查可用资源：`Request(2,1,0)` $\leq$ `Available(3,3,2)`，通过；
3. 试探性分配：

进程	**Allocation**	**Need**	**Available**
P0	(2, 2, 0)	(5, 3, 3)	(1, 2, 2)
P1	(2, 0, 0)	(1, 2, 2)	
P2	(3, 0, 2)	(6, 0, 0)	
P3	(2, 1, 1)	(0, 1, 1)	
P4	(0, 0, 2)	(4, 3, 1)	

1. 安全性检查，Work = (1, 2, 2)：
   - P1 的 Need = (1,2,2) $\leq$ (1,2,2)，选 P1；Work = (1,2,2)+(2,0,0) = **(3, 2, 2)**；
   - P3 的 Need = (0,1,1) $\leq$ (3,2,2)，选 P3；Work = (3,2,2)+(2,1,1) = **(5, 3, 3)**；
   - P4 的 Need = (4,3,1) $\leq$ (5,3,3)，选 P4；Work = (5,3,3)+(0,0,2) = **(5, 3, 5)**；

- P0 的 Need = (5,3,3) ≤ (5,3,5)，选 P0；Work = (5,3,5)+(2,2,0) = **(7, 5, 5)**；
- P2 的 Need = (6,0,0) ≤ (7,5,5)，选 P2；Work = (7,5,5)+(3,0,2) = **(10, 5, 7)**。得到安全序列 **P1 → P3 → P4 → P0 → P2**，系统安全。因此可以正式把资源分配给 **P0**。

情形 **2**：**P4** 请求 **Request = (3, 3, 0)** ——拒绝

1. `Request(3,3,0)` ≤ `Need(P4) = (4,3,1)`（3 ≤ 4 且 3 ≤ 3），通过；
2. `Request(3,3,0)` ≤ `Available = (3,3,2)`，通过；
3. 试探性分配：`Allocation(P4) = (3, 3, 2)`，`Need(P4) = (1, 0, 1)`，`Available = (0, 0, 2)`；
4. 安全性检查，Work = (0, 0, 2)：逐个检查所有进程的 Need，没有一个能被满足：
   - P0 需 (7,4,3)：A 不足；
   - P1 需 (1,2,2)：A 不足（需 1，只有 0）；
   - P2 需 (6,0,0)：A 不足；
   - P3 需 (0,1,1)：B 不足（需 1，只有 0）；
   - P4 需 (1,0,1)：A 不足。一步都推不动，系统进入不安全状态，因此必须拒绝该请求。

情形 **3**：**P1** 请求 **Request = (0, 2, 0)** ——拒绝（这是最典型的" 看似可行实则不安全" 的例子）

1. `Request(0,2,0)` ≤ `Need(P1)=(1,2,2)`，通过；
2. `Request(0,2,0)` ≤ `Available(3,3,2)`，通过；
3. 试探性分配：

进程	**Allocation**	**Need**	**Available**
P0	(0, 1, 0)	(7, 4, 3)	**(1, 1, 2)**
P1	(2, 2, 0)	**(1, 0, 2)**	
P2	(3, 0, 2)	(6, 0, 0)	
P3	(2, 1, 1)	(0, 1, 1)	
P4	(0, 0, 2)	(4, 3, 1)	

1. 安全性检查，Work = (1, 1, 2)：逐个检查
   - P0 需 (7,4,3)：A 分量 7 > 1，不满足；
   - P1 需 (1,0,2)：(1,0,2) ≤ (1,1,2)，满足。选 P1！Work = (1,1,2) + (2,2,0) = **(3, 3, 2)**；
   - P3 需 (0,1,1) ≤ (3,3,2)，选 P3；Work = (3,3,2)+(2,1,1) = **(5, 4, 3)**；
   - P4 需 (4,3,1) ≤ (5,4,3)，选 P4；Work = (5,4,3)+(0,0,2) = **(5, 4, 5)**；
   - P0 需 (7,4,3)：A 分量 7 > 5，不满足；
   - P2 需 (6,0,0)：A 分量 6 > 5，不满足。死循环，找不到更多可执行进程，系统进入不安全状态。

结论：该请求必须被拒绝，让 P1 等待。这个算例是最经典的考点——请求量本身没有超过 **Need** 也没有超过 **Available**，但分配后系统不安全，所以仍要拒绝。

*7.*经典真题算例

（**1**）**2011-27**（答案：**D**，不存在安全序列）

进程	已分配 **R1 R2 R3**	尚需 **R1 R2 R3**
P1	2 0 0	0 0 1
P2	1 2 0	1 3 2
P3	0 1 1	1 3 1
P4	0 0 1	2 0 0

可用资源 = (0, 2, 1)。P1 的 Need (0,0,1) ≤ (0,2,1)，可执行；释放后 Work = (2, 2, 1)。此后 P2 需 (1,3,2) 的 R2 分量 3 > 2，P3 需 (1,3,1) 的 R2 分量 3 > 2，P4 需 (2,0,0) 可满足但执行后 Work = (2,2,2)，R2 仍只有 2，P2、P3 依然无法执行。最终无法让所有进程加入安全序列，故不存在安全序列。

（**2**）**2018-26**（答案：**A**，系统处于不安全状态）

系统有 4 个同类资源，P1、P2、P3 分别需要 4、3、1 个，已申请到 2、1、0 个。

- 可用资源 = 4 − (2+1+0) = **1**；
- Need：P1 = 4−2 = 2，P2 = 3−1 = 2，P3 = 1−0 = **1**；
- 只有 P3 能满足，P3 执行完毕释放 0 个资源，Work 仍为 **1**；
- 此时 P1、P2 都需 2 个而只有 1 个，谁也无法执行。

结论：不存在安全序列，系统处于不安全状态。

（**3**）**2022-26**（答案：**B**，安全序列个数为 **2**）

进程	已分配 **A B C**	尚需 **A B C**
P0	2 0 1	0 2 1
P1	0 2 0	1 2 3
P2	1 0 1	0 1 3

可用资源 = (1, 3, 2)。P0 的 Need (0,2,1) ≤ (1,3,2) 满足；P1 的 Need 需要 C 资源 3 > 2，不满足；P2 的 Need 需要 C 资源 3 > 2，不满足。第一步只能运行 **P0**。

P0 执行完毕释放 (2,0,1)，Work = (1,3,2) + (2,0,1) = **(3, 3, 3)**。此时 P1 需 (1,2,3) ≤ (3,3,3) 满足，P2 需 (0,1,3) ≤ (3,3,3) 也满足。

- 序列 P0 → P1 → P2；
- 序列 P0 → P2 → P1。

共 **2** 个安全序列。

银行家算法速算技巧

1. 先把 **Available** 算对：Available = 资源总数 − 各进程已分配之和。题目不给总数时，往往给出” 可用资源” 行，直接抄；
2. **Need = Max − Allocation**，逐个算，不要漏；
3. 每选出一个进程就立刻更新 **Work** 并重新从第一个进程开始扫描（题目若规定了扫描顺序，就按规定的顺序取）；
4. 用**”Work** 最终是否等于资源总数**”** 校验；
5. 资源请求题按” 三步检查 + 试探分配 + 安全性算法” 作答，写清” 拒绝” 或” 可以分配” 并给出理由。

（四）死锁检测和解除

*1.*死锁检测的时机

死锁检测算法会按系统真实情况设置合适的时间间隔，每隔一段时间执行一次来检测系统是否存在死锁。若检测过于频繁会浪费系统资源；若间隔太长又没有意义。

检测的触发时机：每隔一段时间，或当 **CPU** 利用率显著降低时。（不能以”死锁发生时”为触发点——只有运行了检测算法之后系统才知道是否死锁；也不能在每次进程申请资源时都检测，开销过大。）

若系统既不采取预防措施也不采取避免措施，就必须提供两个算法：死锁检测算法（确定系统是否已发生死锁）和死锁解除算法（从死锁状态恢复）。

*2.*资源分配图

需要的两种信息：用某种数据结构保存资源的请求和分配信息；提供一种算法利用这些信息检测是否进入死锁状态。

资源分配图的构成：

元素	含义
进程结点（圆圈）	对应一个进程
资源结点（方框）	对应一类资源，方框中的小圆圈表示该类资源有几个实例
请求边（进程结点 → 资源结点）	表示进程想申请几个资源，每条边代表一个
分配边（资源结点 → 进程结点）	表示已经为进程分配了几个资源，每条边代表一个

简化资源分配图的步骤：

1. 在图中找出既不阻塞又不是孤点的进程 P_i——即存在一条与它相连的有向边，且该边对应的资源申请数量小于等于系统中已有的空闲资源数量。若所有连接该进程的边都满足条件，则该进程能继续运行直至完成；
2. 消去它所有的请求边和分配边，使之成为孤立的结点（相当于该进程释放了所占有的全部资源）；
3. 它释放的资源可以唤醒某些因等待这些资源而阻塞的进程，使原来的阻塞进程变为非阻塞进程，于是对这些新解放的进程重复上述操作。

死锁定理：如果某时刻系统的资源分配图是不可完全简化的，那么此时系统发生死锁。

- 若能消去图中所有的边，则称该图是可完全简化的，此时一定没有发生死锁（相当于能找到一个安全序列）；
- 若最终不能消除所有边，则发生了死锁；
- 注意：并不是系统中所有进程都是死锁状态。用死锁检测算法化简资源分配图后，还连着边的那些进程才是死锁进程。

死锁检测不关心”资源总量”

死锁检测只检测系统中是否发生死锁（是否存在循环等待的环路），一旦发现就要采取解除措施，因此不需要知道各进程运行所需的资源总量信息，只需要知道当前时刻的资源请求与分配情况。这正是 2015-26 题中”Ⅱ：S1 需要进程运行所需资源总量信息，而 S2 不需要”正确的理由。

真题考法

2015-26（死锁避免与死锁检测的对比）、2016-25（4 个进程共享 3 个临界资源，问死锁时处于死锁状态的进程数至少是几个——答案 3，因为 p4 只需 R2，无法单独成环，至少需要 p1、p2、p3 三个进程构成环）。

*3.*死锁的解除

方法	做法	特点
资源剥夺法	挂起（暂时放到外存）某些死锁进程，抢占它的资源并分配给其他死锁进程	应防止被挂起的进程长时间得不到资源而饥饿
撤销进程法（终止进程法）	强制撤销部分甚至全部死锁进程，剥夺它们的资源	实现简单，但代价可能很大（有些进程已运行很久、接近结束，被终止则功亏一篑）
进程回退法	让一个或多个死锁进程回退到足以避免死锁的地步	要求系统记录进程的历史信息、设置还原点；过程中进程主动释放资源而非被剥夺

选择”牺牲者”（撤销哪个进程、剥夺谁的资源）时可以考虑的因素：

- 进程的优先级；
- 进程已执行了多长时间；
- 进程还要多久才能完成；
- 进程已经使用了多少资源；
- 进程是交互式还是批处理式。

如何避免解除死锁时造成饥饿？如果一个系统基于”代价”来选择牺牲进程，那么同一进程可能总是被选为牺牲者，结果永远不能完成指定任务而饿死。因此可以在代价的考虑因素中加入回滚次数。

死锁与本节的整体脉络

死锁预防：破坏四个必要条件之一（互斥、不可剥夺、请求并保持、循环等待）。死锁避免：银行家算法，保证系统不进入不安全状态。死锁检测：资源分配图化简，看是否可完全简化（死锁定理）。死锁解除：资源剥夺、撤销进程、进程回退。

本章小结表

知识点	核心结论
进程映像组成	PCB + 程序段 + 数据段；创建进程 = 创建 PCB，撤销进程 = 撤销 PCB
进程 vs 线程	进程是资源分配的基本单位，线程是处理机调度的基本单位
线程私有部分	TID、PC、通用寄存器组、栈；其余全部共享
三态模型	就绪、运行、阻塞（单处理机下运行态至多一个）
五态模型	新建、就绪、运行、阻塞、终止
非法转换	阻塞 → 运行（不允许）；就绪 → 阻塞（不允许）
运行 → 就绪原因	时间片用完、被更高优先级进程抢占、主动 `yield`
运行 → 阻塞原因	请求 I/O、申请临界资源、P 操作阻塞、等管程条件变量

知识点	核心结论
阻塞 → 就绪原因	I/O 完成、资源被释放、事件到达（被动行为）
用户级线程	线程库管理、切换快、一个阻塞则全进程阻塞、不能利用多处理机
内核级线程	内核管理、切换慢、一阻塞不影响其他线程、可多处理机并行
多线程模型	多对一、一对一、多对多（n > m）
PCB 组织	线性表、链接队列、索引表；就绪队列 1 个，阻塞队列多个
进程通信	共享内存（最快、需同步）、消息传递、管道（半双工、亲缘进程）、信号（异步通知）
管道阻塞条件	写满阻塞、读空阻塞；读端全关返回 EOF，写端全关收到 SIGPIPE
三级调度	高级（作业，频率最低）、中级（内存/挂起）、低级（进程，频率最高）
周转时间	完成时间 − 到达时间
带权周转时间	周转时间 / 运行时间（必然 ≥ 1）
等待时间	进程处于就绪态的时间之和
不能调度的情况	中断处理中、内核临界区中、禁用中断时、原语执行中
非抢占式特点	不出现” 运行 → 就绪”；调度不频繁、开销小、实时性差
抢占式特点	出现” 运行 → 就绪”；响应快、适用于分时与实时系统
FCFS	非抢占；公平、不饥饿；对短作业不利、不利于 I/O 繁忙型
SJF（默认非抢占）	平均周转/等待时间最短；需预知运行时间；长作业饥饿
SRTN（抢占式 SJF）	平均周转时间最优；同样长作业饥饿
HRRN	响应比 =（等待时间 + 要求服务时间）/ 要求服务时间；不饥饿
RR	抢占式、公平、响应快、不饥饿；时间片过大退化为 FCFS
优先级调度	抢占/非抢占均可；静态/动态；可能饥饿
MLFQ	队列优先级从高到低、时间片从小到大；I/O 型回到高优先级；设计要考虑 4 项因素
多处理机调度	亲和性（软/硬）、负载均衡（推迁移/拉迁移）；公共队列亲和性差、私有队列需迁移
上下文切换	保存/恢复寄存器与地址空间；PCB 保存的是内核态上下文
模式切换 vs 进程切换	模式切换是同一进程的用户态 ↔ 内核态；进程切换必在内核态、前后不同进程
临界区四原则	空闲让进、忙则等待、有限等待（必做）；让权等待（可选）
单标志法	违背空闲让进（和让权等待）
双标志先检查	违背忙则等待
双标志后检查	违背空闲让进和有限等待
Peterson	满足空闲让进、忙则等待、有限等待；不满足让权等待；只适用于两个进程
中断屏蔽	只适用于内核进程、只适用于单处理机
TSL / Swap	原子操作、适用多处理机；不满足让权等待，可能饥饿
自旋锁	忙等、无上下文切换开销、多处理机短临界区适用、单处理机需抢占式调度器
整型信号量	`while (S <= 0);` ——违反让权等待
记录型信号量	`S.value--` 后判断 `< 0` 则阻塞；`S.value++` 后判断 `<= 0` 则唤醒；遵循让权等待

知识点	核心结论
S.value 含义	>0 表示可用资源数；=0 表示资源刚好用完且无等待者；<0 时其绝对值等于正在等待的进程数
互斥模板	P(mutex) …临界区…V(mutex)，mutex = 1
同步模板	前操作后 V(S)、后操作前 P(S)，S = 0
前驱模板	每对前驱一个信号量（初值 0），前驱末尾 V、后继开头 P
条件变量 vs 信号量	条件变量没有值；signal 可能不产生任何影响，V 一定会改变信号量
生产者-消费者	P(empty) 必须在 P(mutex) 之前，否则死锁；V 操作顺序可交换
读者-写者	分类互斥；第一个读者上锁、最后一个读者解锁；count 需要 mutex 保护
哲学家进餐	三种解法：限并发数、奇偶不同顺序、拿筷整体互斥
死锁四条件	互斥、不可剥夺、请求并保持、循环等待（必须同时满足）
循环等待 vs 死锁	死锁一定有循环等待；循环等待不一定死锁（同类资源 > 1 时可被链外资源打破）
不死锁最少资源数	$x \times (y-1)+1$（x 个进程、每个最多需 y 个同类资源）
可能死锁最小进程数	$\lceil x/(y-1) \rceil$
预防方法	破坏互斥（SPOOLing）、破坏不可剥夺（释放/剥夺）、破坏请求并保持（静态分配）、破坏循环等待（顺序资源分配）
安全状态性质	安全 → 一定不死锁；不安全 → 可能死锁；死锁 → 一定不安全
银行家算法	Need = Max − Allocation；安全性算法反复找 Need ≤ Work 的进程并回收资源
资源请求处理	检查 ≤ Need → 检查 ≤ Available → 试探分配 → 安全性算法 → 安全则分配，否则撤销并等待
死锁检测	资源分配图；死锁定理：图不可完全简化则死锁；还连着边的进程才是死锁进程
死锁解除	资源剥夺、撤销进程、进程回退
并发性排序	检测死锁 > 银行家算法 > 资源预分配

易错清单

1. 进程状态转换的死记法：阻塞态不能直接到运行态，就绪态不能直接到阻塞态。只有运行中的进程才能主动请求阻塞。
2. ”键盘输入”“缺页异常”导致的是”运行 → 阻塞”，不是”运行 → 就绪”；只有”时间片用完”“被抢占”“主动 yield”才是”运行 → 就绪”。
3. **PCB** 是进程存在的唯一标志，进程切换保存的上下文放在 **PCB** 中而不是栈中；每个进程有独立的内核栈。
4. 同一进程的线程切换不需要修改页基址寄存器、不需要切换页表，原有 TLB 不失效；需要更新的是 PC、通用寄存器、栈基址寄存器。
5. ”操作系统为每个用户级线程建立一个线程控制块”是错误的——用户级线程的 TCB 由用户态线程库维护。
6. 用户级线程的一个线程阻塞会导致整个进程阻塞；内核级线程不会。用户级线程无法利用多处理机并行。
7. 管道位于内核缓冲区，不在用户空间也不在磁盘上；读写都需要互斥；不支持 **lseek** 定位；是

字节流而不是消息；所有文件描述符关闭后缓冲区被回收而不持久化。

8. 共享内存本身没有任何同步机制，必须配合信号量；共享物理页框一定相同，但各进程中的逻辑页号可以不同。
9. 等待时间只统计进程处于就绪态的时间，不要把后备队列等待时间和 I/O 时间算进去。
10. 带权周转时间必然 $\geq$ **1**；周转时间相同时，运行时间长的进程带权周转时间更小。
11. 题目未说明时，**"** 短作业优先**"** 默认为非抢占式；只有带" 所有进程同时到达" 前提时才说 SJF 平均周转时间最少，否则应说 SRTN 最少。
12. **RR** 中同一时刻进程下处理机、另一进程到达时，新到达的进程先进入就绪队列。
13. 时间片过大 **RR** 退化为 **FCFS**；时间片过小则切换开销剧增。
14. **2019-27** 多级反馈队列题的坑：算 P1 等待时间时极易漏掉"P2 在 Q1 执行的那 10 ms 期间 P1 在就绪队列等待" 这一段。
15. **2010** 年考过**"** 降低优先级的合理时机**"** 是**"** 进程时间片用完**"**，不是" 完成 I/O 进入就绪队列" 或" 从就绪态转为运行态"。
16. **"** 进程处于临界区时不能进行调度**"** 是错的——只有处于内核临界区时才不能调度，用户级临界区中可以调度。
17. 中断处理过程中、内核临界区中、禁用中断期间、原语执行期间不能进行进程调度与切换。
18. 模式切换不一定是进程切换：中断处理完成返回用户态时，前后可能是同一个进程。进程切换一定发生在内核态、前后一定是不同进程。
19. 四条原则中只有前三条必须实现，让权等待是可选的（Peterson、TSL、Swap、自旋锁都不满足）。
20. 所有软件互斥方法都不满足让权等待，因为进入区是空循环忙等。
21. **TSL/Swap** 机制下不存在阻塞态的进程，等待者一直是就绪态或运行态；`while (TSL(&lock))` 应在开中断状态下执行。
22. 关中断是特权指令，用户进程不能使用；中断屏蔽只能用于单处理机。
23. 自旋锁在单处理机上必须配合抢占式调度器，否则永远自旋无法解锁。
24. 记录型信号量中 `S.value = 0` 表示资源刚好用完、无进程等待，而不是" 没有资源"；`S.value < 0` 时其绝对值是等待进程数。
25. 信号量只能进行 **P/V** 操作，不能做值判断。`if (bowl >= 1)` 是错误写法。
26. 信号量的 **P(S)**、**V(S)** 本身必须原子执行，因为 S 是共享变量；单处理机可关中断，多处理机需要自旋锁或 CAS。
27. 生产者-消费者中 `P(empty)` 必须在 `P(mutex)` 之前，颠倒就会死锁；**V** 操作顺序可以交换。
28. 缓冲区数量为 **1** 时可以省去 `mutex`（一般信号量天然互斥），但数量 ≥ 2 时必须用 `mutex`。
29. 读-写问题中 `count` 必须用 `mutex` 保护，否则多个读者可能都认为自己是第一个读者，造成第二个读者永久阻塞。
30. 读者优先会让写者饿死，写者优先会让读者饿死，加入 `queue` 信号量才是读写公平。
31. 哲学家进餐的死锁根源是循环等待，三种解法分别破坏" 循环等待"（限并发数、奇偶顺序）和" 请求并保持"（拿筷整体互斥）。
32. 循环等待不是死锁的充分条件——同类资源数大于 1 时，等待链外的资源可以打破环。
33. 死锁进程至少 **2** 个；单个进程阻塞不是死锁；发生死锁时一定有循环等待。
34. 死锁和饥饿是操作系统的问题，死循环是程序员的问题；死锁进程处于阻塞态，饥饿进程可能处于就绪态或阻塞态。
35. **"** 银行家算法可以预防死锁**"** 是错的（它属于死锁避免，没有破坏任何必要条件）；**"** 银行家算

法可以判断系统是否处于死锁状态**”** 也是错的（判断死锁要用死锁检测）。

36. 不安全状态不一定发生死锁；但发生死锁时一定处于不安全状态。
37. 银行家算法的请求检查有三步：不超过 Need、不超过 Available、试探分配后系统仍安全；三者缺一不可，任何一步失败都必须拒绝或让进程等待。
38. 死锁检测不需要进程的资源总量信息，只需要当前时刻的请求与分配情况；而死锁避免需要整个生命周期的最大需求。
39. 化简资源分配图后，还连着边的进程才是死锁进程，并不是系统中所有进程都死锁。
40. 死锁检测的合理触发时机是**”** 每隔一段时间或 **CPU** 利用率显著降低时**”**，不是” 死锁发生时”（逻辑悖论）也不是” 每次申请资源时”（开销过大）。

第三章内存管理

本章考纲要求

三、内存管理（一）内存管理基础

1. 内存管理的基本概念：逻辑地址空间与物理地址空间，地址变换，内存共享，内存保护，内存分配与回收。
2. 连续分配管理方式
3. 页式管理
4. 段式管理
5. 段页式管理

（二）虚拟内存管理

1. 虚拟内存的基本概念
2. 请求页式管理
3. 页框分配与回收
4. 页置换算法
5. 内存映射文件（memory-mapped files）
6. 虚拟存储器性能的影响因素及改进方法

内存管理是操作系统部分分值最重、计算题最密集的一章，在统考中以三种形态出现：年年必有的 **2** 分选择题（地址变换、页表规模、置换算法判定、抖动与工作集）、计算大题（置换算法填表、EAT 计算、页表项与页表占用空间计算）、以及概念辨析题（分页与分段的区别、装入方式与地址变换时机）。特点是「不背公式算不出」，但「背了公式不看条件也一定算错」——地址变换的每一步都要清楚是谁在做、访问了几次内存。本章按考纲顺序把五大块全部写透，并为每个可计算的知识点配完整算例。

一、内存管理基础

（一）内存管理的基本概念

*1.*逻辑地址空间与物理地址空间

程序从源代码到在 CPU 上运行，要经历编译、链接、装入三个步骤，每一步产生的「地址」含义都不同，这是本章第一组必须分清的概念。

- 编译：编译程序把用户源代码编译成若干目标模块。此时每个目标模块都从 0 号单元开始编址，这个 0 号地址只是相对于该模块的。
- 链接：链接程序把这一组目标模块以及所需的库函数链接在一起，形成一个完整的装入模块（可执行文件）。链接完成后，各模块的相对地址被拼接成一个统一的、仍然从 0 号单元开始编址的地址空间，这就是逻辑地址空间（虚拟地址空间）。
- 装入：装入程序把装入模块装入内存运行，此时才真正涉及物理位置。

由此得到两个核心定义：

- 逻辑地址（**Logical Address**）：又称相对地址、虚拟地址。是程序视角看到的地址，由编译链接产生，从 0 开始编址。逻辑地址的集合构成逻辑地址空间。
- 物理地址（**Physical Address**）：又称绝对地址。是内存单元的真实地址，由硬件（内存管理部件 **MMU**）在访存时使用。物理地址的集合构成物理地址空间。

一句话区分两个地址空间

逻辑地址空间是「程序以为自己住在哪里」，物理地址空间是「程序实际住在哪里」。两者可以完全不同：逻辑地址从 0 开始连续编号，而物理地址可以离散、可以从任意位置开始。把逻辑地址翻译成物理地址的过程叫地址重定位，由 **MMU** 完成。

真题考法：逻辑地址在哪一步形成

【2011-30】形成该逻辑地址的阶段是（ ）。A. 编辑 B. 编译 C. 链接 D. 装载答案 **C**。编辑不产生地址；编译后每个目标模块各自从 0 开始编址，还不是完整逻辑地址；链接才把各模块拼成统一从 0 开始编址的完整逻辑地址空间；装载做的是把它送进内存并（或运行时）变换成物理地址。

*2.*地址变换与装入方式

地址变换（地址重定位）就是把逻辑地址转换成物理地址。按照「物理地址在什么时刻生成」，有三种装入方式，而这三种方式正好决定了地址变换的时机——这正是考纲把「装入方式」与「地址变换」并列的用意。

装入方式	物理地址生成时机	是否可移动 / 可申请新空间	硬件支持	典型搭配
绝对装入	编译（汇编）时	不可移动	不需要	单一连续分配
可重定位装入（静态重定位）	装入时一次性生成	装入后不可移动、不能再申请空间	只需一个加法器	固定分区分配
动态运行时装入（动态重定位）	执行时才生成	运行中可移动、可动态申请	必须要有重定位寄存器 + **MMU**	动态分区、分页、分段、段页式

- 绝对装入：编译时已知程序将驻留在哪里，编译程序直接产生绝对地址目标代码，装入时不需修改地址。只能用于单道环境。
- 可重定位装入：编译时起始地址未知，编译器产生可重定位代码；装入时把模块搬到空闲区，并将模块内所有逻辑地址一次性加上该区起始地址。要求装入时一次分配所需的全部内存空间，装入后既不能移动，也不能再申请新空间。
- 动态运行时装入：装入后程序中仍是相对地址，直到真正执行到该条指令时才由重定位寄存器配合 MMU 完成变换。优点是：可分配到不连续的存储区；便于程序段共享；可只装入部分代码即投入运行、运行中动态申请；允许程序在内存中移动（因此进程被多次换出换入到不同位置也能正确运行）。

静态重定位与动态重定位的三个易错点

1. 静态重定位 ≠ 静态链接。前者是装入方式（决定物理地址何时生成），后者是链接方式（决定目标模块何时拼接、是否生成完整逻辑地址），两者是不同阶段的事。
2. 页式管理不适合静态重定位。因为页式下进程的页面离散存放，且运行中页面可能被换出换入到不同页框，物理位置会变化，必须用动态重定位。
3. 单一连续分配与固定分区分配不需要 **MMU** 做地址变换，因为其逻辑地址与物理地址的关系只是简单地「加上一个固定偏移」，甚至可以直接等价；而动态分区、分页、分段都必须有地址变换硬件。

程序的链接同样有三种方式，常在选择题中与装入方式一起出现：静态链接（运行前链接成一个完整装入模块）、装入时动态链接（装入内存时边装入边链接）、运行时动态链接（执行中用到某模块时才链接并调入）。

*3.*内存保护

内存保护的目标是保证各进程在各自的存储空间内运行、互不干扰。考纲点名的两种做法都属于存储区域保护（判断是否「地址越界」）：

- 方法一：上下限寄存器。在 CPU 中设置一对上、下限寄存器存放进程所占内存区间的上、下限地址，每次访存时比较目标地址，超出区间即产生越界异常。
- 方法二：重定位寄存器 + 界地址寄存器，也叫基址寄存器 + 限长寄存器。
  - 重定位寄存器（基址寄存器）：存放进程所占内存的起始物理地址。
  - 界地址寄存器（限长寄存器）：存放进程的最大逻辑地址（即进程长度上限）。
  - 访存顺序：先拿逻辑地址与界地址寄存器比较，不超过才允许，然后与重定位寄存器保存的基址相加得到物理地址。

两个寄存器：一个「比」，一个「加」

界地址寄存器用来比（判断越界），重定位寄存器用来加（完成变换）。比较必须在加法之前完成，否则越界的地址已经被变换成物理地址，保护就失效了。这两个寄存器只能由操作系统内核通过特权指令加载，用户程序不能修改——这正是「页表必须存放在内核区」的同一个道理。

除了「地址越界」，还有一类保护叫访问方式保护（存取控制保护）：判断是否发生了「访问越权」。即使逻辑地址没有越界，若实际访问操作与该页/段的权限不符，也会触发存储保护错。它通过在页表项或段表项中设置访问权限位来实现，通常规定：

- 各程序对本程序所在的存储区可读可写；
- 对共享区或已获授权的其他用户信息可读不可写；
- 对未获授权的信息（如操作系统内核、页表本身）不可访问；
- 数据段可指定为可读可写或只读，程序段（代码段）可指定为只可执行或只读。

真题考法

【2009-26】分区分配内存管理方式的主要保护措施是（ ）。A. 界地址保护 B. 程序代码保护 C. 数据保护 D. 栈保护答案 **A**。分区分配（连续分配）中进程占据一段连续内存，防止越界的核心手段就是界地址（基址 + 限长）保护。【2023-28】涉及「进程对数据段和代码段可以有不同的访问权限」——正确，这正是访问方式保护的体现。

*4.*内存共享

内存共享指允许多个进程访问内存的同一部分。

- 只有只读的区域才可以共享。可写区域一旦共享，一个进程的修改会破坏其他进程的数据，产生不确定性。
- 可重入代码（纯代码，**reentrant code / pure code**）：允许多个进程同时访问但不允许被任何进程修改的代码，执行中不依赖任何会改变的全局状态，因此可以共享。为了让「共享代码 + 私有数据」并存，实际执行时为每个进程配一个局部数据区，把执行中可能改变的部分复制到该数据区，程序只修改自己的私有数据区，绝不去改共享代码。可重入代码不属于临界资源。

为什么分段比分页更容易实现共享

假设共享内容占 N 个页框大小：分页系统要在每个进程的页表中都建立 N 个页表项，分别指向被共享的 N 个页框；分段系统不管共享段多大，都只需要一个段表项指向共享段始址。所以「段的共享非常简单易行」。例如 40 个用户共享 160KB 代码区（页大小 4KB）：分页要每个进程建 40 个页表项，分段只需 1 个段表项。

*5.*内存分配与回收

内存分配与回收是内存管理最基本的功能，操作系统必须解决三个问题：如何记录哪些区域已被划分出去、哪些还空闲；进程结束时如何回收；进程应该放在内存的哪个位置。

分配方式的演化主线是：

$$\text{单一连续分配} \rightarrow \text{固定分区分配} \rightarrow \text{动态分区分配} \rightarrow \text{离散分配（页 / 段 / 段页式）}$$

前三种统称连续分配管理方式（为一个程序分配一段连续的内存空间），最后一种为非连续分配管理方式。连续分配的固有缺陷是：一个 1GB 的进程必须要找到连续的 1GB 空间，即使内存总空闲量足够、但不连续，也无能为力。非连续分配正是为解决这个矛盾而生。

（二）连续分配管理方式

*1.*单一连续分配

把内存划分为系统区和用户区两部分：系统区位于低地址，供操作系统使用；用户区位于高地址，整个用户区只装入一道用户程序，由该程序独占。

- 优点：实现最简单；可以采用覆盖技术扩充内存；因为是单道，不一定需要内存保护（无其他程序可干扰）。
- 缺点：只能用于单道批处理系统；有内部碎片（进程用不满整个用户区）；存储器利用率低。

单一连续分配的碎片类型

单一连续分配只有内部碎片，没有外部碎片。理由是「分配单元」就是整个用户区，用户区之外不存在可分配的空闲块，因此不存在「分配单元之间」的碎片。

*2.*固定分区分配

把整个用户空间预先划分为若干个固定大小的分区，每个分区只装入一道作业，进程与分区一一对应。这是最早出现的、可运行多道程序的内存管理方式。

分区大小的划分有两种做法：

- 分区大小相等：实现简单，但缺乏灵活性。程序太小浪费分区（内部碎片大），程序太大放不进去。
- 分区大小不等：把分区划分为多个较小分区、适量中等分区、少量大分区，以兼顾大小作业。可以按作业的常见大小分布来设计，提高利用率。

操作系统用分区说明表来管理这些分区。表中每个表项对应一个分区，通常按分区大小排列，记录：分区号、分区大小、分区起始地址、状态（已分配 / 未分配）。分配时由操作系统内核检索该表，找到一个能满足程序大小且未分配的分区，分配给该程序并把状态改为已分配；回收时把状态改回未分配。

- 优点：实现简单，无外部碎片；可以采用静态重定位；适合嵌入式微型设备（是代价最小的多

道存储管理方式，不需要特殊数据结构和硬件支持）。

- 缺点：会产生内部碎片，内存利用率低；当用户程序太大时可能所有分区都装不下，只能改用覆盖技术，而覆盖又会降低性能。

*3.*动态分区分配

动态分区分配不预先划分分区，而是在进程装入内存时，根据进程的大小动态地建立分区，分区的大小和数目都是可变的。

内存碎片分为两类，必须分清：

- 内部碎片（**Internal Fragmentation**）：分配单元内部未被使用的内存。产生原因是「分配单元大小 > 进程实际所需」，即「过量分配」。
- 外部碎片（**External Fragmentation**）：分配单元之间未被使用的内存。产生原因是「按需分配」后，相邻已分配区之间留下了小的、无法被任何进程利用的空隙。

三种连续分配的碎片对照

分配方式	内部碎片	外部碎片
单一连续分配	有	无
固定分区分配	有	无
动态分区分配	无	有

「外部碎片是动态分区分配才有的概念」（准确地说，是「按需分配」类方式的通病）。解决外部碎片的方法是紧凑（**Compaction**，拼凑）技术：移动已分配给进程的内存分区，把它们向低地址端挤压合并，从而在高地址端腾出大片连续空闲空间。紧凑的前提是所有程序都支持动态重定位（否则程序移动后内部地址全部失效），代价是移动耗时、开销大。

（1）记录空闲分区的数据结构

数据结构	组织方式	说明
空闲分区表	数组 / 顺序表	每个空闲分区对应一个表项，表项包含分区号、分区大小、分区起始地址等
空闲分区链	双向链表	每个分区的起始部分和末尾部分分别设置前向指针和后向指针，起始部分还可记录分区大小等信息

（2）四种基于顺序搜索的动态分区分配算法

这是考纲明确点名的四个算法，必须能说清楚「空闲分区按什么顺序链接」「从哪里开始找」以及优缺点。

算法	空闲分区链的排列	检索起点	选择规则
首次适应（First Fit）	按地址递增	每次都从链首开始	找到第一个能满足要求的空闲分区

算法	空闲分区链的排列	检索起点	选择规则
邻近适应（Next Fit，循环首次适应）	按地址递增（可排成循环链表）	从上次查找结束的位置开始	找到第一个能满足要求的空闲分区
最佳适应（Best Fit）	按容量递增	从链首开始	找到第一个能满足要求的最小空闲分区
最坏适应（Worst Fit）	按容量递减	从链首开始	找到第一个能满足要求的最大空闲分区

四个算法的优缺点：

- 首次适应：综合性能最好，开销小（回收后一般不需重新排序），能在高地址保留大内存块；缺点是每次从链头查起，易在低地址积累大量小空闲区，且每次分配都要经过它们，增大查找开销。
- 邻近适应：开销小，不必每次从低地址的小分区查起；但低地址与高地址被使用的概率相同，它会把高地址的大分区也逐步切碎，最终无大分区可用，隐含了最佳适应的缺点，通常比首次适应更差。
- 最佳适应：大分区被更多地保留下来，但每次挑最小分区会留下越来越多很小的、难以利用的外部碎片，且开销大（回收后可能需重新排序）。它最容易产生内存碎片。
- 最坏适应：可减少难以利用的小碎片，中等大小分配较多时效果最好；但大分区很快被用完、不利于大进程，且开销大。

最佳适应 $\neq$ 性能最好

「最佳适应」这个名字极具误导性。它只是让本次分配的剩余空间最小（局部最优），但从全局看，它把大分区反复切碎、留下大量小碎片，反而使后续大作业无处可放。所以统考的标准答案是：最容易产生内存碎片的是最佳适应算法；而综合性能最好的是首次适应算法。

（3）同一组作业请求下四种算法的完整分配过程对照

设定场景：内存中初始有 5 个空闲分区，按地址递增依次为

$$100\text{KB},\quad 500\text{KB},\quad 200\text{KB},\quad 300\text{KB},\quad 600\text{KB}$$

依次请求：$A=15\text{KB}$，$B=30\text{KB}$，$C=50\text{KB}$，$D=70\text{KB}$，$E=100\text{KB}$。

各算法的最终结果先列总表：

算法	A=15KB	B=30KB	C=50KB	D=70KB	E=100KB	分配完成后剩余空闲分区（按地址序）
首次适应	100K 区，剩 85K	500K 区，剩 470K	200K 区，剩 150K	300K 区，剩 230K	失败（无 $\geq$100K 的空闲区）	85K、470K、150K、230K、600K
邻近适应	100K 区，剩 85K	500K 区，剩 470K	200K 区，剩 150K	300K 区，剩 230K	失败（同首次适应）	85K、470K、150K、230K、600K
最佳适应	100K 区，剩 85K	200K 区，剩 170K	300K 区，剩 250K	85K 区，恰好用尽	失败（无 $\geq$100K 的空闲区）	170K、250K、500K、600K

算法	**A=15KB**	**B=30KB**	**C=50KB**	**D=70KB**	**E=100KB**	分配完成后剩余空闲分区（按地址序）
最坏适应	600K 区，剩 585K	585K 区，剩 555K	555K 区，剩 505K	505K 区，剩 435K	500K 区，剩 400K	435K、400K、100K、200K、300K

下面给出逐步分配过程，这是考场上真正要写出来的东西。

首次适应（**First Fit**）——空闲区始终按地址递增排列，每次都从链首开始：

步骤	请求	检索过程	选中分区（起始）	分配后该分区剩余
1	A = 15K	从链首起，100K 是第一个 ≥15K 的	100K @ 低地址	85K
2	B = 30K	从链首起，85K 已不够？85K ≥ 30K，就在 **85K** 里分配	85K	55K
3	C = 50K	链首 55K ≥ 50K，仍在最前面的小碎片里分配	55K	5K
4	D = 70K	链首 5K 不够，下一个 500K ≥ 70K	500K	430K
5	E = 100K	链首 5K、430K 中 5K 不够，430K ≥ 100K	430K	330K

可见首次适应确实会在低地址端不断切出越来越小的碎片（本例中切出了 5K 的碎片），这正是它「低地址积累小碎片、增加查找开销」这一缺点的直接体现。

最佳适应（**Best Fit**）——空闲区按容量递增排列，每次选最小的够用分区：

步骤	请求	空闲区容量序	选中的最小够用分区	分配后
1	A = 15K	100, 200, 300, 500, 600	100K	剩 85K
2	B = 30K	85, 200, 300, 500, 600	85K（不是 **200K**）	剩 55K
3	C = 50K	55, 200, 300, 500, 600	55K	剩 5K
4	D = 70K	5, 200, 300, 500, 600	200K	剩 130K
5	E = 100K	5, 130, 300, 500, 600	130K	剩 30K

最佳适应把两个大分区留了下来（500K、600K 未被触碰），确实「保留了大分区」，但代价是造出了 **5K**、**30K** 这种几乎永远用不上的小碎片——这就是它「最容易产生外部碎片」的原因。

最坏适应（**Worst Fit**）——空闲区按容量递减排列，每次选最大的分区：

步骤	请求	当前最大分区	分配后剩余
1	A = 15K	600K	585K
2	B = 30K	585K（切完仍最大）	555K
3	C = 50K	555K	505K
4	D = 70K	505K	435K

步骤	请求	当前最大分区	分配后剩余
5	E = 100K	500K（此时 500K 已大于 435K）	400K

最坏适应的特点是剩余分区大小趋于均匀，本例中剩下的全是三四百 KB 的中等块，确实没有小碎片；但 **600K** 的大块被反复切削、彻底消失，一旦后续来一个 500K 的大作业就无处可放。

邻近适应（**Next Fit**）——排列方式与首次适应相同，但检索从上次结束位置继续：

步骤	请求	指针起点	检索过程	选中分区	剩余
1	A = 15K	链首 100K	100K 够	100K	85K
2	B = 30K	85K 处	85K 够	85K	55K
3	C = 50K	55K 处	55K 够	55K	5K
4	D = 70K	5K 处	5K 不够 → 500K 够	500K	430K
5	E = 100K	430K 处	430K 够	430K	330K

前三次分配与首次适应完全一致（因为指针一直在低地址端），差别要在回收之后才会显现：回收一个低地址分区后，首次适应下次分配会立刻用上它，而邻近适应的指针已经走到高地址，低地址的空闲区会被「跳过」很久，于是低地址碎片越积越多，同时高地址大分区也被逐步切碎——这正是「邻近适应隐含最佳适应缺点」的含义。

真题考法

【2010-28】主存 55MB（初始空闲），最佳适应算法，顺序为：分配 15MB、分配 30MB、释放 15MB、分配 8MB、分配 6MB，此时最大空闲分区大小是（　）。答案 **B**（**9MB**）。推演：55MB 分 15MB 后剩 40MB；再分 30MB 后剩 10MB；释放 15MB 后有 15MB 与 10MB 两个空闲区；分配 8MB 时最佳适应选 10MB → 剩 2MB；分配 6MB 时只剩 15MB 够用 → 剩 9MB。最大空闲分区 9MB。【2017-25】按字节编址、最佳适应、每次分配回收后都重新排序。当前空闲分区为 20K 处 40KB、500K 处 80KB、1000K 处 100KB、200K 处 200KB。回收起始 60K、大小 140KB 的分区后，空闲分区个数、链首分区的起始地址和大小是（　）。答案 **B**（**3**、**500K**、**80KB**）。回收的 60K~200K 与紧邻的 20K 处 40KB、200K 处 200KB 上下同时合并，得 380KB；空闲区只剩 3 个。最佳适应按容量递增排，链首是最小的 80KB，起始地址 500K。【2019-32】最容易产生内存碎片的是（　）。答案 **C**，最佳适应算法。【2024-27】每次回收分区时仅合并大小相等的空闲分区的是（　）。答案 **A**，伙伴算法（伙伴系统只把大小相等、地址相邻的两个伙伴块合并）。

*4.*真题补充：伙伴系统

四种顺序搜索算法之外，伙伴系统（**Buddy System**）在真题中出现过，只需掌握其判定特征：规定无论已分配还是空闲分区，大小均为 2 的整数次幂；需要大小 s 且 $2^{i-1} < s \le 2^i$ 时把 2^i 的块一分为二，重复划分直到合适为止。回收时只合并「大小相同」且地址相邻的伙伴块（2024-27 的直接考点），合并后大小为原来两倍。它既产生内部碎片，也产生外部碎片。

（三）页式管理

连续分配的根本困境是「要求连续」。非连续分配允许把一个进程分散地装入到许多不相邻的分区中，内存利用率因此大幅提升。基本分页存储管理是最基本的一种。

*1.*页面、页框与页表

- 页框（页帧、物理页面、**Frame**）：把物理地址空间划分为大小相同的固定基本分配单位，编号叫页框号，从 0 开始。

- 页面（页、逻辑页面、**Page**）：把逻辑地址空间也划分为大小相同的单位，编号叫页号，也从 0 开始。
- 页框大小与页面大小必须相等（这样页内偏移才能原样保留）。
- 操作系统以页框为单位为进程分配内存：进程的每个页面分别放入一个页框中，二者一一对应，各页面不必连续存放，也不必遵循先后顺序。
- 页表（**Page Table**）：为记录每个页面被放进哪个页框，操作系统为每个进程建立一张页表：一个进程对应一张页表，进程的每个页面对应一个页表项（**PTE**），页表项记录页号 → 页框号的映射（页表项等长且顺序连续存放，故页号可隐含，类比数组下标）；页表项中还通常包含有效位、访问位、修改位、存取权限位等控制信息。

页表寄存器（**PTBR / PTR**）：系统中只设置一个，存放当前运行进程页表在内存的起始地址 F 和页表长度 M。进程未执行时这两个值存放在该进程的 **PCB** 中，被调度时才装入页表寄存器。

页式管理的两个「一定」

1. 页表一定存放在内核区。页表负责地址映射与权限管理，若放在用户区，用户进程就能篡改它，从而非法访问任意内存或使系统崩溃。所以页表由内核维护，受硬件特权级保护。
2. 页式管理的地址是一维的。只要确定了页面大小，逻辑地址结构就确定了：给出一个逻辑地址，硬件可以自动把它切成「页号 + 页内偏移」两部分，不需要程序显式告诉系统偏移量占多少位。

*2.*地址结构与地址变换

逻辑地址结构：分页的逻辑地址被划分为两部分

$$\underbrace{\text{页号}P}_{m\text{ 位}}\ \Big|\ \underbrace{\text{页内偏移量}W}_{k\text{ 位}}$$

- 若用 k 位表示页内偏移量，则该系统一个页面的大小为 2^k 个内存单元（字节）；
- 若用 m 位表示页号，则该系统中一个进程最多允许有 2^m 个页面，即进程的逻辑地址空间最大为 2^{m+k}。

为了便于计算，页面大小一般取 **2** 的整数次幂。此时「切分」就是纯粹的位运算：若页面大小为 2^k B，则逻辑地址末尾 k 位就是页内偏移量，其余高位就是页号。

基本地址变换机构（无快表）的完整过程，设页面大小为 L，页表起始地址为 F，页表长度为 M（表项数），逻辑地址为 A：

1. 进程被调度时，内核把 PCB 中的 F 和 M 装入页表寄存器。
2. 计算页号 $P = A/L$，页内偏移量 $W = A \bmod L$（页面大小为 2 的幂时即取高位 / 低位）。
3. 越界检查：若 $P \geq M$（页号 ≥ 页表长度），产生越界中断；否则继续。
4. 查页表：页号 P 对应的页表项地址为

$$\text{PTE 地址} = F + P \times (\text{页表项长度})$$

从该地址取出页表项内容 b，即该页所对应的页框号。（注意：页表里记录的是内存块号，不是内存块的起始地址！b 号页框的起始物理地址 $= b \times L$。）

1. 形成物理地址并访存：

$$E = b \times L + W$$

物理地址计算公式（必背）

$$\boxed{\text{物理地址} = \text{页框号} \times \text{页面大小} + \text{页内偏移量}}$$

等价写法：先由页框号得到页框起始物理地址 $b \times L$，再与页内偏移量 W 拼接（因为页面大小是 2 的幂，b 与 W 在二进制上刚好拼在一起，不会进位冲突）。

无快表时，访问一个逻辑地址需要两次访存：第一次访存查内存中的页表，第二次访存才是访问目标数据。

快表（**TLB**，**Translation Lookaside Buffer**）：由于局部性原理，一段时间内连续查到的往往是同一个页表项。于是在 CPU 中设置一个相联存储器（**associative memory**）实现的、速度远快于内存的高速缓冲器，用来存放当前访问的若干页表项，以加速地址变换。TLB 对应的、内存中的页表被称为慢表。

引入快表后的地址变换过程：

1. CPU 给出逻辑地址，硬件算得页号 P 与页内偏移量 W，把 P 与快表中所有页号同时比较。
2. 命中：直接取出该页对应的页框号，与 W 拼成物理地址然后访存。此时只需一次访存。
3. 未命中：访问内存中的页表，找到页表项得到页框号，再拼成物理地址访存。此时需要两次访存；同时基于局部性原理，应把刚查到的这个页表项存入快表，以便后面再用。若快表已满，需要按一定算法替换掉一个旧页表项。

TLB 的三个细节

1. TLB 中的页号是显式保存的（不像内存页表那样可以隐含），因为 TLB 里存放的是任意若干个页表项，彼此不连续。
2. TLB 位于 **CPU** 内部，由相联存储器实现，具备快速（并行）查找能力。
3. 快表中有的页面一定在内存中；若某页被换出外存，必须同时删除快表中的相应表项，否则会访问到错误的页面。进程切换时，TLB 要刷新（或按 ASID 区分）。

*3.*地址变换算例

算例 **1**（逻辑地址 $\rightarrow$ 物理地址）

某分页系统按字节编址，页面大小为 4KB，逻辑地址为 32 位。已知进程页表起始物理地址为 0020 0000H，页表项长度为 4B。现给出逻辑地址 $A =$ 0000 3020H，求物理地址。

1. 页面大小 $4\text{KB} = 2^{12}\text{B}$，故低 **12** 位为页内偏移量，高 **20** 位为页号。
2. 拆分 $A =$ 0000 3020H：低 12 位 $=$ 020H，即 W；高 20 位 $=$ 3，即 $P = 3$。
3. 页号 3 的页表项物理地址 $= F + P \times 4 =$ 0020 0000H $+ 3 \times 4 =$ 0020 000CH。
4. 设从该地址读出页表项内容为 $b = 10$（即页号 3 映射到 10 号页框）。
5. 物理地址 $E = b \times L + W = 10 \times 4096 + 32 = 40960 + 32 = 40992$；用十六进制拼接更直接：页框号 $10 =$ 00AH 与 $W =$ 020H 拼成 $E =$ 0000 A020H。

校验：$E = 10 \times 2^{12} + 2^5 = 40960 + 32 = 40992$，与 A020H $= 10 \times 4096 + 32$ 一致。

算例 **2**（页表项大小与页表占用空间）

设系统物理内存为 4GB，页面大小为 4KB，问每个页表项至少需要多少字节？整个页表至少占多少字节？

1. 内存块大小 = 页面大小 = $4\text{KB} = 2^{12}\text{B}$。
2. 4GB 内存被划分成 $2^{32}/2^{12} = 2^{20}$ 个内存块。

3. 内存块号范围是 $0 \sim 2^{20}-1$，至少需要 **20 bit** 表示；取整字节数，至少 **3B**（$3\times 8=24$ bit）。
4. 页号隐含不占空间，故一个页表项占 3B。一个进程逻辑地址空间对应 2^{20} 个页面，即 2^{20} 个页表项，页表至少占 $3\times 2^{20}\text{B}=3\text{MB}$。

进一步的细节：若页表项为 3B、页表起始地址为 X，则 M 号页对应的页表项地址为 $X+3M$。一个 4KB 页框可存放 $\lfloor 4096/3\rfloor=1365$ 个页表项，剩余 $4096 \bmod 3=1$B 的页内碎片；因此 1365 号页表项并不在下一个页框的开头，而要额外加 1B，地址为 $X+3\times 1365+1$。

为什么页表项常常做成 4B 而不是 3B

理论上 3B 就够表示 4GB 内存的块号，但 4B 恰好使得一个 **4KB** 页框能整存 **1024** 个页表项（$4096/4=1024$），不产生页内碎片，查表时用「页框号 + 偏移」也更规整。这就是「为了方便页表的查询，常让一个页表项占更多字节，使每个页面恰好装得下整数个页表项」的含义。统考做题时以题目给出的页表项大小为准。

*4.*多级页表

单级页表的缺陷是空间上的：每个进程都有独立的虚拟地址空间，因而都有自己的页表，页表必须覆盖全部虚拟地址空间。32 位环境下虚拟地址空间 4GB、页大小 4KB，需要约 2^{20}（100 万）个页表项；每个页表项 4B，则每个进程的页表就要 **4MB**，100 个进程就是 400MB——纯属浪费，因为大多数进程实际用到的空间远不到 4GB。

多级页表的原理：把长长的单级页表再分页，形成多张页表，同时为这些离散的页表再建立一张索引表（页目录表，又称外层页表 / 顶层页表）。所以两级页表实质上就是「页表的页表」。

32 位、4KB 页、4B 页表项下的经典划分：规定页目录表最多占一个页面，故页目录表可容纳 $4096/4=1024=2^{10}$ 个页表项。于是 32 位逻辑地址结构变为

$$\underbrace{\text{页目录号}}_{10\text{ 位}}\ \big|\ \underbrace{\text{页表索引（二级页号）}}_{10\text{ 位}}\ \big|\ \underbrace{\text{页内偏移量}}_{12\text{ 位}}$$

为什么这样能省空间？关键在于顶级页表覆盖全部虚拟地址空间，而二级页表按需创建：如果某个一级页表项没有被用到，就不需要创建它对应的二级页表。假设只有 20% 的一级页表项被用到，页表占用就是

$$4\text{KB}+20\%\times 4\text{MB}=0.804\text{MB}$$

对比单级页表的 4MB，是巨大的节约。多级页表的代价是地址变换需要多次访存，变换速度变慢。

多级页表的核心结论

1. 多级页表的最大优点是「减少页表所占的连续内存空间」，而不是加快地址变换速度（实际上更慢）、也不是减少页表项所占字节数（反而更多）、更不直接减少缺页中断次数。
2. 不管是几级页表，必须满足各级页表的大小不能超过一个页面。
3. **CPU** 页表基址寄存器中存放的是当前进程「一级页表的起始物理地址」（不是虚拟地址）。
4. 出现「页目录号」字样，就说明一定是多级页表存储管理方式。

多级页表算例 **1**（二级页表求页目录项数）

某计算机采用二级页表分页存储管理，按字节编址，页大小为 2^{10}B，页表项大小为 2B，逻辑地址空间大小为 2^{16} 页。则页目录表中包含的表项个数至少是（　）。

1. 一个页面可存放 $2^{10}/2=2^9=512$ 个页表项。

2. 逻辑地址空间共 2^{16} 页，即需要 2^{16} 个二级页表项。
3. 这些页表项需占 $2^{16}/2^9 = 2^7 = 128$ 个页面，即需要 128 个二级页表。
4. 每个二级页表在页目录表中占一个表项，故页目录表至少 **128** 个表项。答案 B。

多级页表算例 **2**（求页表级数）

某页式系统逻辑地址空间 256TB，页表项大小 8B，页面大小 4KB。该系统应该采用几级页表?

$$256\text{TB} = 2^{48}\text{B}, \quad \text{页数} = 2^{48}/2^{12} = 2^{36}$$

一个页面可存 $2^{12}/2^3 = 2^9 = 512$ 个页表项，故级数

$$\left\lceil \frac{36}{9} \right\rceil = 4 \text{ 级}$$

多级页表算例 **3**（页表总占用）

32 位系统、页大小 4KB、页表项 4B，二级页表。若一个进程的 4GB 虚拟空间中只有 2 个 4MB 的有效区域，且不共享页表，问页表占多少空间? 页目录表恒为 1 页（4KB）；每个 4MB 区域占 $4\text{MB}/4\text{KB} = 1024$ 个页面，恰好由一个二级页表（4KB）覆盖，故合计 $4\text{KB} + 2 \times 4\text{KB} = 12\text{KB}$。若用单级页表，无论实际用了多少都要为全部 2^{20} 个页表项付出 4MB——这就是多级页表的价值。

真题考法

【2010-31】二级页表求页目录项数，答案 128（见上文算例 1）。【2014-32】属于多级页表优点的是：**D.** 减少页表所占的连续内存空间。【2019-31】地址结构「页目录号 10 位 | 页表号 10 位 | 页内偏移 12 位」，虚拟地址 2050 1225H 对应的页目录号、页号分别是（ ）。答案 **A**（**081H**、**101H**）：把十六进制展开成 32 位二进制后按 10/10/12 切分即可。【2021-29】二级页表中 CPU 页表基址寄存器中存放的是：**B.** 当前进程的一级页表的起始物理地址。【2023-30】进程 R 与 S 共享数据 data，页号分别为 p_1, p_2，页框号分别为 f_1, f_2，则（ ）。答案 **C**：p_1 与 p_2 不一定相等（页号相对于各自的虚拟地址空间），f_1 与 f_2 一定相等（同一份物理数据只有唯一的页框）。【2024-25】进程切换时要执行：Ⅰ更新程序计数器 Ⅱ更新栈基址寄存器 Ⅲ更新页基地址寄存器 → **D**（Ⅰ、Ⅱ、Ⅲ）。

*5.*页面大小对性能的影响

页面大小是分页系统的关键设计参数，影响是双向的：页面较小时页内碎片小、内存利用率高，但每个进程需要的页面数更多，页表过长、占用内存多，且缺页率更高（换入换出更频繁）；页面较大时页表短、缺页率较低（局部性原理），但页内碎片增大，一次换入换出的 I/O 数据量也大。因此页面大小必须折中选择（现实中常见 4KB）。记住：页内碎片 $\leq$ 页面大小 **−1**。

（四）段式管理

*1.*段的基本概念与段表

一个进程的地址空间天然由多个逻辑模块组成，如代码段、数据段、栈段、堆段，各有不同属性（可执行、可读写、可增长），分段就是按这种用户自身的逻辑关系来划分。

- 段（**Segment**）：访问方式和存储数据等属性相同的一段逻辑地址空间。若干段组成进程的逻辑地址空间，段的大小不等。
- 基本思想：按用户进程自身的逻辑关系划分为若干段，每段从 0 开始编址；以段为单位分配，段内连续，段间可以不连续。
- 逻辑地址由二元组 $(s, addr)$ 表示，s 是段号，$addr$ 是段内偏移量。

- 段表：每个进程一张，进程的每一段对应一个段表项。由于各段长度不同，段表项必须含段长 C 与段基址 b，此外还常有状态位、存取权限位、外存始址等。段表项等长且连续存放，故段号可隐含。

分段存储管理的五个特点：方便编程（汇编程序天然按逻辑关系分段）；信息共享（段是有内容共性的逻辑单位体，不像页面可能把一个完整逻辑功能切到不同页）；信息保护（可对特定函数、数据方便地设定权限）；动态增长（段长可变，容易应对数据段增长）；动态链接（以独立逻辑单位为链接单元，恰合分段原则）。

引入分段的目的 $\neq$ 实现离散分配

引入分页的目的是实现离散分配、提高内存利用率（系统行为）；引入分段的目的是满足用户需求——方便编程、分段共享、分段保护、动态链接与增长（用户行为）。这是分页与分段对比题的第一层含义，也是最常被混淆的一层。

*2.*段的地址变换

逻辑地址结构：

$$\underbrace{\text{段号}S}_{n\text{ 位}} \mid \underbrace{\text{段内偏移量}W}_{k\text{ 位}}$$

进程未执行时，段表的起始地址 F 和段表长度 M 存放在 PCB 中；当进程被调度时，系统把它们装入段表寄存器。

完整变换步骤：

1. 由逻辑地址取出段号 S 和段内偏移量 W。
2. 第一次越界检查：若 $S \geq M$（段号 $\geq$ 段表长度），产生越界中断；否则继续。
3. 查段表，找到对应段表项（段表项地址 $= F + S\times$ 段表项长度），取出段长 C 与段基址 b。
4. 第二次越界检查：若 $W \geq C$（段内偏移量 $\geq$ 段长），产生越界中断；否则继续。
5. 计算物理地址：

$$E = b + W$$

1. 用 E 访存。

分页与分段越界检查的次数不同

- 分页只做一次检查：$P \geq M$ 则越界。页内偏移量不需要检查，因为页内偏移量的位数由硬件固定，其最大值恰好等于页面大小，不可能越界。
- 分段必须做两次检查：先是 $S \geq M$，再是 $W \geq C$。因为段的长度是变长的，段内偏移量的位数固定不代表该段真的有这么长。

算例：某分段系统中，段表如下：

段号	段长	段基址（起始物理地址）
0	1000	8000
1	500	3000
2	300	2000

段号	段长	段基址（起始物理地址）
3	1500	6000

问：(1) 访问逻辑地址 (1, 200) 得到的物理地址是多少？(2) 访问 (2, 400) 会怎样？

(1) 段号 1 < 段表长度 4，通过；段内偏移 200 < 段长 500，通过；物理地址 = 3000 + 200 = **3200**。

(2) 段号 2 < 4，通过；段内偏移 400 ≥ 段长 300，越界，产生越界异常（注意不是「段缺失异常」，也不是「越权异常」——那是访问方式保护的事）。

*3.*段的共享与保护

段的保护主要有两种：界地址保护与存取控制保护。

- 界地址保护：通过逻辑地址与段（页）表信息比较来防止越界，即上面说的两次检查。分页方式只检查页号，分段方式要同时检查段号与段内偏移量。
- 存取控制保护：对进程的段（页）设置只读、读/写、不可访问等限制，访问时进行权限检查。若实际访问操作与权限不符，则发生存储保护错（访问越权）。

段的共享：除每个进程自身的段表外，系统还设置一个供相关进程共同使用的共享段表，其表项除记录共享段的物理地址等信息外，还设一个 **count** 字段记录共享该段的进程数。当 P1 与 P2 共享段 S 时，段 **S** 在内存中只有一份；P1、P2 的段表与共享段表中各有一项记录段 S 的物理地址。段 S 的物理地址唯一，但它在 **P1** 与 **P2** 中的逻辑段号不一定相同。某进程释放该段时，系统并不立即调出，只把共享段表中的 **count** 减 **1**；**count** 减至 **0** 时才真正调出该段。

共享段表的两个易错点

1. 「进程 P1 和 P2 共享段 S，则段 S 在 P1 和 P2 中应具有相同的段号」——错误。段号由各进程自己管理，可以不同。
2. 「P1 和 P2 都通过共享段表中同一个段表项」——正确（2019 年真题考的正是汤小丹旧版教材的「共享段表」做法）。新版教材的做法是在每个进程的段表中各设一个段表项、指向被共享的同一个物理段。统考按「共享段表」理解即可，两者不矛盾：共享段只有一份物理内容，这是根本点。

*4.*分段与分页的对比

对比项	分页	分段
信息的单位性质	页是物理单位，是系统行为	段是逻辑单位，是用户行为
划分目的	实现离散分配，提高内存利用率	满足用户需求：方便编程、共享、保护、动态链接与增长
大小	页大小固定，由系统决定	段长不固定，由用户程序的逻辑决定
对用户是否可见	不可见（透明）	可见（不透明），编程时需显式给出段名/段号
地址空间维度	一维	二维
碎片	有少量页内碎片（内部碎片），无外部碎片	会产生外部碎片，无内部碎片
地址变换	查页表，无快表时 2 次访存	查段表，无快表时 2 次访存

对比项	分页	分段
越界检查	1 次（页号 vs 页表长度）	2 次（段号 vs 段表长度；段内偏移 vs 段长）
共享与保护	实现复杂（需 N 个页表项）	实现简单（只需 1 个段表项）

为什么两种地址结构都是「两段」，维度却不同

形式上页式（页号 + 页内偏移）与段式（段号 + 段内偏移）都是两个量组合，像是二维空间。但分页的页号与页内偏移不是两个独立变量：只要给出「数据在逻辑地址空间中的相对位置」这一个数，配合已知的页面大小就能唯一算出页号和页内偏移量，所以地址是一维的。而段大小不等，段号与段内偏移量是两个独立变量，必须显式给出两个值，所以地址是二维的。典型例子：8086 CPU 的程序计数器由 **CS:IP** 两个寄存器给出，这就是「二维」的直接体现。

（五）段页式管理

*1.*基本思想与地址结构

分页与分段各有优缺点：

	优点	缺点
分页管理	空间利用率高，仅有少量页内碎片，不产生外部碎片	不利于实现信息的共享与保护
分段管理	利于实现逻辑模块间信息的共享与保护	段长过大，分配连续空间不方便，产生外部碎片

段页式管理正是二者的结合：

- 基本思想：先将进程按逻辑模块分段，再把各段分页。对内存空间的管理与分页管理相同，划分为若干与页面大小相同的存储块，以存储块为单位分配。
- 一句话概括：采用分段方法分配和管理用户地址空间，采用分页方法管理物理存储空间。它兼有两者的优点，但开销比纯段式或纯页式都大。
- 逻辑地址结构由三部分组成：

$$\underbrace{\text{段号}S}_{n\text{ 位}} \mid \underbrace{\text{页号}P}_{m\text{ 位}} \mid \underbrace{\text{页内偏移量}W}_{k\text{ 位}}$$

- k 位页内偏移量 $\rightarrow$ 页面大小为 2^k 个内存单元；
- m 位页号 $\rightarrow$ 每个段最大有 2^m 个页面；
- n 位段号 $\rightarrow$ 每个进程最多可分为 2^n 个段。

- 段表与页表：段页式管理中，先由段表获得该段对应的页表在内存中的存放位置，再由页表获得每一页对应的页框号。
  - 每个进程只有一张段表；进程有多个段，每个段各有一张页表，所以一个进程对应多张页表。
  - 每个段对应一个段表项，由段号、页表长度、页表存放的块号（页表起始地址）组成；段表项等长，故段号可隐含。
  - 每个页面对应一个页表项，由页号、块号组成；页表项等长，故页号可隐含。

段页式的碎片类型

段页式会产生内部碎片（页内碎片）。因为对内存的管理方式与分页完全相同，分配单位是页框，进程的最后一个页面通常装不满。例如一个段是 5KB，页大小是 4KB，则该段要占 2 个页面，第二个页面里就有 3KB 的页内碎片。段页式不产生外部碎片。

*2.*段页式的地址变换完整流程

进程未执行时，段表的起始地址 F 和段表长度 M 存放在 PCB 中；当进程被调度时，系统把它们装入段表寄存器。

1. 由逻辑地址取出段号 S、页号 P、页内偏移量 W。
2. 越界检查一：若 $S \geq M$（段号 $\geq$ 段表长度），产生越界中断；否则继续。
3. 查段表，找到对应段表项（段表项地址 $= F + S\times$ 段表项长度），取出该段的页表长度与页表存放的块号。
4. 越界检查二：若 $P \geq$ 页表长度，产生越界中断；否则继续。
5. 依据「页表存放的块号 + 页号」查页表，找到对应页表项，取出内存块号（页框号）。
6. 由内存块号与页内偏移量形成物理地址：$E =$ 页框号 $\times$ 页面大小 $+ W$。
7. 用 E 访存。

访存次数：未引入快表时需要三次访存（查段表、查页表、访问目标单元）；引入快表后以段号和页号为关键字查询，命中时仅需一次访存。

段页式也可以实现内存共享：让共享段指向相同的页表，从而实现进程间的段共享。

真题考法（段页式共享）

在段页式存储管理系统中，进程 R 和 S 共享数据段 data，若 x 在 R 和 S 的段表中的段号分别为 p_1, p_2，x 所在页对应的页框号分别为 f_1, f_2，则（　）。答案 **C**：p_1 和 p_2 不一定相等，f_1 和 f_2 一定相等。理由同分页共享：段号是各进程自己地址空间内的编号，物理内容只有一份、页框号唯一。

算例（段页式地址变换）：某系统按字节编址，段表项 4B，页表项 4B，页面大小 4KB。逻辑地址结构为「段号 8 位 | 页号 12 位 | 页内偏移 12 位」。进程的段表起始地址为 `0000 1000H`，段表长度 5。现访问逻辑地址 `0000 0001 0000 0010 0000 0101 0000 1010 B`（即段号 1，页号 2，页内偏移 `00AH`）。

1. 拆分：段号 $S = 1$，页号 $P = 2$，页内偏移 $W =$ `00AH`。
2. 段号 $1 < 5$，通过。
3. 段表项地址 $=$ `1000H` $+ 1 \times 4 =$ `1004H`。设读出：该段页表长度为 16，页表存放块号 **= 5**。
4. 页号 $2 < 16$，通过。
5. 页表起始物理地址 $= 5 \times 4096 = 20480 =$ `5000H`。页表项地址 $=$ `5000H` $+ 2 \times 4 =$ `5008H`。设读出页框号 **= 9**。
6. 物理地址 $E = 9 \times 4096 + 10 = 36864 + 10 = 36874 =$ `900AH`。

二、虚拟内存管理

（一）虚拟内存的基本概念

*1.*传统存储管理方式的弊端与局部性原理

前面讲的所有分配方式（无论连续还是非连续）都有两个共同特征，也就是两个弊端：

- 一次性：作业必须一次性全部装入内存后才能开始运行。后果有两个：作业很大而装不进内存时，作业根本无法运行；大量作业要求运行时内存不够，只能让少数作业先运行，降低了多道程序的并发度。
- 驻留性：作业装入内存后就一直驻留，任何部分都不会被换出，直至作业运行结束。事实上一个时间段内只需访问作业的一小部分数据，这就导致内存中驻留大量暂时用不到的数据，浪费宝贵的内存资源；运行中的进程还会因等待 I/O 而长期阻塞却仍占着内存。

突破这两点的理论依据是局部性原理（**Principle of Locality**）：

- 时间局部性：一条指令或一个数据被访问后，不久之后很可能再次被访问（典型来源是循环）。
- 空间局部性：一个存储单元被访问后，其邻近的存储单元很可能很快被访问（典型来源是数组、顺序执行的指令）。

局部性原理是虚拟存储的基石

局部性越好，虚拟存储系统越能更好地发挥作用。反过来，如果一个程序的访存完全随机、毫无局部性，虚拟存储不但不能提升性能，反而会因为频繁缺页而性能崩溃（抖动）。

*2.*虚拟内存的定义与特征

定义：基于局部性原理，操作系统可以这样控制程序：

- 程序装入时：只把程序中很快会用到的部分先装入内存，暂时用不到的留在外存；
- 程序执行时：如果所访问的信息不在内存，由操作系统负责把它从外存调入内存；
- 发现内存不够时：由操作系统负责把内存中暂时用不到的信息换出外存。

在操作系统的管理下，在用户看来似乎有一个比实际内存大得多的内存，这就是虚拟内存（**Virtual Memory**）。虚拟性是操作系统虚拟性特征的一个体现：物理内存大小并没有变，只是在逻辑上进行了扩充。

三个特征（必背，注意「多次性」是核心）：

- 多次性：作业运行时无需一次全部装入内存，而是允许分批次调入内存。
- 对换性：作业运行时无需一直常驻内存，而是允许在运行时进行换入和换出。
- 虚拟性：从逻辑上扩充了内存的容量。

虚拟内存容量的两个「受限于」

- 最大容量由计算机的地址结构（**CPU** 寻址范围）决定。
- 实际容量 $=\min($内存容量 $+$ 外存容量, CPU 寻址范围$)$。

例：某计算机地址结构为 32 位、按字节编址，内存 512MB，外存 2GB。则虚拟内存最大容量 $=2^{32}\text{B}=4\text{GB}$；实际容量 $=\min(2^{32}\text{B},\ 512\text{MB}+2\text{GB})=2\text{GB}+512\text{MB}$。

易错推论：「虚拟存储容量只受外存容量限制」错，「只受内存容量限制」错，「虚拟地址的大小由主存和硬盘的大小决定」错——虚拟地址空间大小由地址结构（操作系统）决定。

*3.*虚拟内存的实现方式

虚拟内存的实现必须建立在离散分配的内存管理方式基础上（连续分配无法做到「只装入一部分」还让程序正常跑）。主要有三种：

- 请求分页存储管理
- 请求分段存储管理

- 请求段页式存储管理

与传统非连续分配方式相比，区别只在两点功能上：

- 所访问信息不在内存时，由操作系统负责把信息从外存调入内存——请求调页（请求调段）功能；
- 如果内存空间不够，由操作系统负责把内存中暂时用不到的信息换出外存——页面置换（段置换）功能。

无论哪种方式，都需要以下硬件支持：一定容量的内存和外存；页表机制（或段表机制）作为主要数据结构；中断机构（访问部分尚未调入内存时产生中断）；地址变换机构。

真题考法

【2012-25】关于虚拟存储器，正确的是：**B.** 虚拟存储只能基于非连续分配技术。A 错（连续分配无法实现按需调入）；C、D 错（容量受地址结构与外存共同约束）。【2023-28】错误的是：**D.** 虚拟地址的大小由主存和硬盘的大小决定。【2014-28】能加快虚实地址转换的是：**C.** 仅 Ⅰ、Ⅱ（Ⅰ增大 TLB 容量；Ⅱ让页表常驻内存）。Ⅲ增大交换区只增加外存空间，不加快地址变换。

（二）请求页式管理

请求分页系统 = 基本分页系统 + 请求调页（外存 → 内存）+ 页面置换（内外存页面调换）。与基本分页相比，它在三个方面需要额外的硬件支持。

*1.*请求页表的扩充字段

为了实现「请求调页」，操作系统必须知道每个页面是否已调入内存、若未调入则它在外存中的位置；为了实现「页面置换」，必须能判断哪些页面该被换出、哪些页面被修改过（已修改的必须写回外存）。因此请求页表在基本页表项（页号、页框号）基础上扩充如下字段：

字段	含义	作用
状态位（存在位、有效位）	该页是否在内存中。1 表示在内存、页表项有效；0 表示当前在外存，访问该页表项将导致缺页异常	请求调页的依据
访问字段（访问位）	本页在一段时间内被访问的次数，或最近已有多长时间未被访问（简单实现仅用 1 位表示「是否被访问过」）	供页面置换算法选择换出页面时参考
修改位（脏位）	该页在内存期间是否被修改过	决定换出时是否要写回外存；供置换算法参考
外存地址	该页在外存中的地址	缺页异常处理程序据此去外存取页
保护位	该页的允许访问方式（只读、可读写、可执行等）	访问方式保护

「访问字段」的两种表述

汤小丹教材表述为「访问字段：用于记录本页在一段时间内被访问的次数，或记录本页最近已有多长时间未被访问」。而 CLOCK 类算法只需要一位「是否被访问过」。做题时看题目给的是几位：给 1 位就按 0/1 处理；给计数器就按数值大小处理。统考绝大多数题目是 **1** 位。

*2.*缺页中断处理流程

在请求分页系统中，每当所要访问的页面不在内存时，便产生一个缺页中断，请求操作系统把所缺页调入内存。此时缺页的进程会被阻塞，放入阻塞队列，调页完成后再被唤醒，放回就绪队列。

完整处理步骤：

1. 检查访问合法性：由地址变换机构检查逻辑地址是否越界（页号是否超过页表长度）。越界是非法访问，与缺页是两回事，不进入缺页处理。
2. 确认缺页并取外存地址：确认该页不在内存（状态位为 0），读出其外存地址；产生缺页中断，保存 **CPU** 现场，转入缺页中断处理程序。
3. 判定内存是否有空闲页框：有则直接分配一个空闲页框；无则按页面置换算法选出一个淘汰页——修改位为 **0** 的页可直接覆盖，修改位为 **1** 的页必须先把该页写回外存才能腾出页框。
4. 调入所缺之页：启动磁盘 I/O，把所缺页面读入刚分配的页框。
5. 修改页表：填入页框号，把状态位置 **1**，并置访问位、修改位等；同时更新快表（若题目说明缺页处理会更新 TLB）。
6. 唤醒进程：把进程从阻塞队列移入就绪队列（调页期间其他进程可运行）。
7. 重新执行发生缺页的那一条指令。

缺页中断的两个性质

1. 缺页中断属于内中断，是异常中的「故障（**fault**）」，因为它是由当前执行的指令想要访问的目标页面未调入内存而产生的。
2. 一条指令在执行期间可能发生多次缺页中断。若一条指令的多个操作数分属不同页面且都未调入内存，则每一页都要触发一次缺页中断。

> 注意第 7 步：缺页处理完成后是「重新执行该指令」，不是「直接访存」，这一点在做 EAT 计算题时至关重要。

算例（缺页处理中可能执行的操作）：在缺页处理过程中，操作系统可能执行的操作是（　）。Ⅰ. 修改页表　Ⅱ. 磁盘 I/O　Ⅲ. 分配页框答案Ⅰ、Ⅱ、Ⅲ全选。分配页框（第 3 步）、磁盘 I/O（第 4 步）、修改页表（第 5 步）都必然或可能发生。而【2022-29】问「不一定包含的操作」，答案是淘汰内存中的页——只有当内存无空闲页框时才需要淘汰，若还有空闲页框就无需淘汰。【2013-30】问缺页时可能执行的操作：Ⅰ处理越界错（错，越界与缺页是两回事）、Ⅱ置换页、Ⅲ分配内存 → 答案仅Ⅱ、Ⅲ。

*3.*请求分页的地址变换过程

与基本分页相比，请求分页的地址变换需要额外进行：

- 请求调页：查找到页表项时，要判断状态位，若为 0 则触发缺页；
- 页面置换：需要调入页面但没有空闲内存块时执行；
- 修改请求页表中新增的表项；
- 维护快表：快表中有的页面一定在内存中，若某页被换出外存，快表中的相应表项也要删除。

把 **TLB** 命中 / 缺失与页面在内存 / 缺页组合起来，共有四种情况，这是考纲「含缺页与 TLB 缺失的组合情况」所指：

情况	**TLB**	页面在内存	处理过程	访存次数（不含查 **TLB**）
①	命中	在	直接由 TLB 取出页框号 → 访存	**1**
②	缺失	在	查内存页表得页框号 → 把该页表项装入 **TLB** → 访存	**2**

情况	**TLB**	页面在内存	处理过程	访存次数（不含查 **TLB**）
③	缺失	不在	触发缺页中断 → 调页 → 修改页表（并删除/更新 TLB 中相关项）→ 重新执行该指令	缺页处理 + 重新执行的访存

TLB 中会不会有过期的表项

不会。因为「快表中有的页面一定在内存中」这一约束必须被维护：页面被换出外存时，必须同步删除快表中对应的表项。所以永远不会出现「**TLB** 命中但页面已不在内存」的情况，这也是上表只有三种情况的原因。

真题考法

【2011-28】缺页处理过程中操作系统执行的操作可能是：Ⅰ修改页表 Ⅱ磁盘 I/O Ⅲ分配页框 → 答案 **D**（Ⅰ、Ⅱ、Ⅲ）。

（三）页框分配与回收

*1.*驻留集与最少页框数

驻留集（**Resident Set**）：请求分页存储管理中分配给一个进程的页框的集合。

驻留集大小对系统影响很大，是一个两难：

- 驻留集越小：系统能容纳的并发进程越多、CPU 利用率越高；但每个进程的缺页率越高，缺页频繁会导致抖动，系统把大量时间花在缺页处理上，实际推进进程的时间很少。
- 驻留集越大：每个进程缺页少；但多道程序并发度下降，资源（如 CPU）利用率降低。

最小页框数（**Minimum Number of Frames**）：保证进程能够正常执行所需的最少物理页框数量。若分配数量不足，进程会因无法加载必要页面而无法运行（例如一条指令的取指和取数分属两页，只给一页就永远缺页）。

最小页框数由「最复杂的那条指令」决定

最小页框数由最复杂指令执行时的最大页面需求决定。若某条指令需同时访问 k 个页面，则最小页框数至少为 k。

- 单地址指令 + 直接寻址：至少 **2** 个页框（指令页 + 数据页）。例：`LOAD R1, 0x1000`。
- 单级间接寻址：至少 **3** 个页框（指令页 + 间接地址页 + 数据页）。例：`MOV AX, [BX]`。
- 多级间接寻址：每增加一级间接，页框数逐级增加（`MOV AX, [[BX]]` 更复杂）。
- 指令隐含的多页面访问：如 x86 的 `MOV [A], [B]` 若源、目标跨页，需 **2** 个数据页 + **1** 个指令页 = **3** 个页框。
- 跨页指令：如 `CALL 0x2000` 若指令本身跨越页边界（末尾在 `0x1000FFC`、目标在 `0x2000000`），需 **2** 个代码页。

几个典型架构的经验值：简单单地址指令 2 个页框；x86 复杂指令 3 个以上（代码 + 数据 + 堆栈页）；RISC-V 精简指令 2~3 个页框。

硬件不可绕过：指令格式、寻址方式、对齐规则等硬件特性直接约束最小页框数，软件无法通过优化绕过。注意「最小页框数」不等于「有效工作的页框数」：后者通常大于最小值，以减少缺页率、提升性能。

*2.*页框分配算法

在采用固定分配策略时，把系统可供分配的页框分配给各进程，可采用：平均分配算法（平均分，不考虑进程大小差异）、按比例分配算法（按进程大小按比例分配）、考虑优先级的分配算法（高优

先级进程分得更多页框）。

*3.*分配策略与置换策略的组合

- 按驻留集大小是否可变：
  - 固定分配：操作系统为每个进程分配一组固定数目的页框，进程运行期间不再改变。
  - 可变分配：先为每个进程分配一定数目的页框，在进程运行期间可以根据情况适当增加或减少。
- 按置换范围不同：
  - 局部置换：发生缺页时只能选本进程自己的页框进行置换。
  - 全局置换：可以把操作系统保留的空闲页框分配给进程；也可以把别的进程持有的页框置换到外存，再分配给缺页进程。

搭配起来只有三种合法策略：

组合	局部置换	全局置换
固定分配	可以	不可以
可变分配	可以	可以

为什么没有「固定分配 + 全局置换」

全局置换意味着一个进程拥有的页框数必定会改变——因为置换时可能选到了别的进程的页框，那个进程的页框数就减少了；也可能从系统空闲队列拿到新页框，本进程页框数就增加了。这与「固定分配」的定义直接矛盾，所以这种组合不存在。这是 2015 年真题的直接考点。

三种合法策略：固定分配局部置换（分配一定数量内存块且运行期间不变，缺页时只能从本进程页面中换出一页；缺点是难以事先确定驻留集大小，太少频繁缺页、太多降低并发度）；可变分配全局置换（系统保留一个空闲页框队列，缺页时取一块分配；无空闲页框时选一个未锁定的页面换出；优点是灵活，缺点是盲目给进程加页框会降低并发度，且被调出的页可能是任一进程的）；可变分配局部置换（缺页时只在本进程页框中换出，频繁缺页则多分配几个页框、缺页率很低则适当减少，既保证并发能力又不过多调页，但实现复杂、开销较大）。

*4.*页面调入策略

（1）什么时候调入页面

- 预调页策略：根据局部性原理，一次调入若干相邻页面比一次只调入一页更高效；但若调入的一批页面大多没被访问，效率就很低，因此需要以预测为基础，目前预测成功率平均不超过 **50%**。主要用于进程第一次调入内存时，或按工作集表在调度时把工作集内的页全部调入。
- 请求调页策略：进程运行期间发现缺页时才调入。调入的页面一定会被访问到，但一次只能调入一页，**I/O** 开销较大。

实际系统两种策略结合使用：首次装入用预调页批量调入，运行中主要用请求调页。

（2）从何处调入页面

具有对换功能的系统把磁盘空间划分为文件区（存放文件，追求空间利用率，采用离散分配）和对换区（存放被换出的进程数据，追求换入换出速度，采用连续分配），因此对换区的 **I/O** 速度远快于文件区。据此有三种情况：

- 对换区足够：页面的调入调出都在内存与对换区之间进行；运行前先把相关数据从文件区复制到对换区。
- 对换区不够：不会被修改的数据直接从文件区调入，换出时不必写回；可能被修改的部分换出时写回对换区，下次从对换区调入。
- **UNIX** 方式：数据全部放在文件区，未使用过的页面从文件区调入；被使用过的页面换出时写回对换区，下次直接从对换区调入。

（**3**）如何调入页面

访问的页面未在内存（存在位为 0）→ 触发缺页异常，关中断、保存断点、转入处理程序 → 查页表得外存地址 → 内存能容纳则启动磁盘 I/O 调入并修改页表；内存已满则先按置换算法选出淘汰页（修改位为 **1** 必须先写回磁盘）再调入，并置存在位为 **1** → 重新执行该指令。

*5.*抖动（*Thrashing*）与工作集模型

CPU 利用率与并发进程数的关系是相互促进又相互制约的：进程数少时提高并发度可提高 CPU 利用率；但并发进程增加会降低访存的局部性特征，局部性下降 → 缺页率上升 → **CPU** 利用率下降。

抖动的根本原因：同时在系统中运行的进程太多，导致分配给每个进程的页框数太少，不能满足进程正常运行的基本要求。于是进程频繁缺页，排队等待页面换入换出的进程增多，对磁盘的有效访问时间急剧增加，每个进程的大部分时间都用于页面的换入换出，从而处理机利用率急剧下降而趋于 **0**。

抖动的因果链（背成一条链）

进程太多 → 每进程页框太少 → 频繁缺页 → 页面换入换出排队 → 磁盘有效访问时间剧增 → **CPU** 利用率趋于 **0**

工作集（**Working Set**）模型：由于抖动与分配给进程的页框数（驻留集）有关，于是提出工作集概念。

- 定义：在某段时间间隔里，进程实际访问页面的集合。
- 基于局部性原理，可以用最近访问过的页面来确定工作集。工作集 W 由时间 t 和工作集窗口大小 Δ 确定，记作 $W(t,\Delta)$。
- 工作集大小可能小于窗口尺寸（窗口内可能有重复访问的页面）。
- 计算方法：从当前时刻 t 向前取 Δ 次页面访问序列，去掉重复后得到的集合就是工作集。
- 典型应用：操作系统统计进程的工作集大小，据此分配内存块。例如窗口尺寸为 5、监测发现某进程工作集最大为 3，说明它局部性良好，分配三个以上内存块即可。
- 核心结论：驻留集大小不能小于工作集大小，否则将导致频繁换页（抖动）。

工作集的方向：往「过去」看

工作集窗口是向时间前进的反方向（过去）取的窗口，不包含 t 时刻之后的访问。做真题时容易把箭头之后的页面也算进去。正确做法：只取 t 之前最近的 Δ 次访问，再去重。

抖动的预防方法：

1. 页框分配与置换时采取局部置换策略：进程缺页时只能在分配给它的内存空间内置换，不允许从其他进程内存空间获取新物理块。这样即使该进程抖动，也不会波及其他进程。
2. 作业调度时考虑工作集因素：从外存调入作业前，先检查每个进程在内存中的驻留页面是否足够。若都足够才调入新作业并分配页框；若某些进程页面不够，应优先给它们分配页框，暂时不调入新作业。

3. 主动挂起进程：当并发进程数目已影响 CPU 利用率时，主动调出优先级低（或剩余执行时间最多）的进程，将其换出到磁盘，腾出页框给其他进程。

真题考法

【2011-29】当系统发生抖动（thrashing）时，可以采取的有效措施是（ ）。Ⅰ撤销部分进程 Ⅱ增加磁盘交换区的容量 Ⅲ提高用户进程的优先级答案 **A**（仅Ⅰ）。抖动是页框不够，撤销/挂起进程能降低多道程序度、释放页框；增加交换区只是增加外存空间，不增加主存页框；提高优先级不能减少缺页。【2016-29】访问页序列，工作集窗口大小为 6，求 t 时刻的工作集。答案 **A**（$\{6,0,3,2\}$）：t 之前最近 6 次访问为 `6,0,3,2,3,2`，去重得 $\{6,0,3,2\}$。

*6.*页框回收与页面缓冲算法

页框回收：可变分配中「适当减少分配给某进程的页框」就是页框回收。被回收的页框若内容被修改过，必须先写回磁盘才能回收；一般来说回收的页框被放入空闲页框链尾，内容在下次分配前不会被清空——这正是页面缓冲算法的思想来源。

影响页面换入换出的因素：

1. 页面置换算法：最重要的因素，好的算法使缺页率低，从而减少换入换出次数。
2. 已修改页面写回磁盘的频率：若每换出一页就启动一次磁盘写回，开销很大。若建立已修改换出页面链表，把已修改的换出页面先挂在链表上，只有当链表上页面数目达到一定值（如 **64**）时再一起写回磁盘，就能显著减少磁盘 I/O 次数。
3. 磁盘内容读入内存的频率：有了上述链表后，链表上暂时有一批装有数据的页面。若某进程在这批数据写入磁盘前又要访问这些页面，就无须从磁盘调入，可直接从链表取回，只须花很小的开销就使这些页面重新回到该进程的驻留集中。

页面缓冲算法（**Page Buffering Algorithm**）：在原置换算法基础上增设已修改页面链表，保存已修改且要换出的页面，等数目达到一定值时一起换出磁盘。特点是显著降低页面换入换出的频率、大幅减少磁盘 **I/O** 次数；正因开销大幅减小，采用 **FIFO** 这类简单策略即可、不需要特殊硬件支持。

VAX/VMS 的页面缓冲算法采用可变分配局部置换，并在内存中设置两个链表：

- 空闲页面链表：实为空闲物理页框链表，用于分配给频繁缺页的进程。未被修改的页面要换出时，并不真的写到磁盘，而是把其页框挂在空闲链表末尾。注意：挂在空闲链表中的页面里是有数据的，若以后需要这些数据可直接取下，免除了从磁盘读入的操作。
- 修改页面链表：已修改页面要换出时不立即写回磁盘，而是把页框挂在修改页面链表末尾，以降低写回磁盘的频率，进而降低从磁盘读入的频率。

哪些页框可以被回收：不可回收页（空闲页、内核空间使用的页、临时锁定的页）；可交换页（匿名页或共享页，硬盘上无对应区域，需写入交换区 **swap**）；可同步页（如文件映射到内存的页，硬盘上有备份，只需检查是否被修改，改了才写回）。工程上常分文件页（内核缓存的磁盘数据与文件数据，干净页直接释放内存、脏页先写回磁盘再释放）与匿名页（无实际载体如堆栈，不能直接释放，通过 **Swap** 机制写盘后再释放）。二者的回收都基于 **LRU** 算法。

连续页框的回收——伙伴算法：Linux 用伙伴算法按大小分组记录连续空闲页框。回收时先检查是否存在大小相等的伙伴页框块，若有则合并成大小为原来两倍的新空闲页框块，合并后继续检查能否再合并成更大块。

真题考法

【2022-30】下列选项中，不会影响系统缺页率的是（ ）。A. 页面置换算法 B. 工作集的大小 C. 进程的数量 D. 页缓冲队列的长度答案 **D**。页缓冲队列用于减少已修改页写回磁盘的频率，主要影响缺页

处理的时间与 **I/O** 性能，不直接决定页面访问是否缺页；而置换算法、工作集大小、进程数量都会影响内存中驻留页面的集合，从而影响缺页率。

（四）页置换算法

进程运行时，当所要访问的页面不在内存、需要调入，而内存已无空闲空间时，系统必须从内存中选出一页调出到磁盘对换区。选择换出页面的算法称为页面置换算法。

- 页置换算法的好坏直接影响系统性能。不适当的算法可能导致进程抖动：刚被换出的页很快又要被访问，于是又要调入；刚调入又需调出，如此频繁更换页面，以致进程大部分时间都花在页面置换工作上。
- 一个好的页面置换算法应具有较低的页面置换频率。理论上应把以后不再会访问的页面、或在较长时间内不会再访问的页面换出。

页置换的三条基本认识

1. 置换本质上不改变页框号，只改变页号。因为置换是把同一个页框的内容覆盖掉，而不是另找一个页框。
2. 页面的换入换出需要磁盘 **I/O**，开销很大，因此好的算法要追求更少的缺页率。
3. 缺页时不一定发生页面置换！若还有可用的空闲内存块，直接装入即可，不需要置换。
4. 在 408 范围内，页面置换算法通常指局部置换的算法。

*1.*统一算例的引用串

为了便于横向比较，本章所有置换算法都用同一个引用串：

$$\text{引用串} = 7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4,\ 2,\ 3,\ 0,\ 3,\ 2 \qquad (\text{共}13\ \text{次访问})$$

分别为进程分配 **3** 个页框和 **4** 个页框两种情况。下面逐一给出模拟表格。

*2.*最佳置换算法（*OPT*，*Optimal*）

算法思想：每次淘汰的页面将是以后永不使用的、或者是在最长时间内不再被访问的页面。这样可以保证最低的缺页率。

手工模拟方法：逐列扫描引用串，遇到缺页且无空闲页框时，对当前在内存中的每个页面，往后找它下一次出现的位置；下一次出现位置最靠后（或以后不再出现）的那个页面就是被淘汰者。若某个页面以后永不出现，优先淘汰它。

3 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
页框 1	7	7	7	**2**	2	**2**	2	**2**	2	2	**2**	2	2
页框 2		0	0	0	0	0	0	**4**	4	4	**0**	0	0
页框 3			1	1	1	**3**	3	3	3	3	3	3	3
淘汰页	—	—	—	**7**	—	**1**	—	**0**	—	—	**4**	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	×	×	✓	×	×

淘汰理由（对内存中每页往后找下一次出现位置，最远者被淘汰）：

- 第 4 次访问 2：页 7 不再出现、页 0 下次第 5 位、页 1 下次第 7 位 → 淘汰 **7**。
- 第 6 次访问 3：页 0 下次第 7 位、页 1 不再出现、页 2 下次第 9 位 → 淘汰 **1**。
- 第 8 次访问 4：页 2 下次第 9 位、页 0 下次第 11 位、页 3 下次第 10 位 → 淘汰 **0**（最远）。
- 第 11 次访问 0：页 3、页 4 都不再出现，淘汰 **4**（若淘汰 3，第 12 次访问 3 又要缺页）。

缺页次数 = **7**，缺页率 $= 7/13 \approx 53.8\%$。

4 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
页框 1	7	7	7	7	7	**3**	3	**3**	3	3	3	3	3
页框 2		0	0	0	0	0	0	**4**	4	4	4	4	4
页框 3			1	1	1	1	1	1	**2**	2	2	2	2
页框 4				2	2	2	2	2	2	2	2	2	2
淘汰页	—	—	—	—	—	**7**	—	**1**	—	—	—	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	×	×	×	×	×

第 6 次访问 3 时：页 7 不再出现、页 0 下次第 7 位、页 1 下次第 ∞ 位、页 2 下次第 9 位 → 淘汰 **7**。
第 8 次访问 4 时：页 1 不再出现 → 淘汰 **1**。

缺页次数 = **6**，缺页率 $= 6/13 \approx 46.2\%$。

OPT 是不可实现的

OPT 可以保证最低的缺页率，但实际上只有进程执行过程中才知道接下来会访问哪个页面，操作系统无法提前预判页面访问序列。因此最佳置换算法是无法实现的，它的价值在于作为衡量其他算法优劣的理论基准（下界）。

*3.*先进先出算法（*FIFO*）

算法思想：每次淘汰最早进入内存的页面。

实现方法：把调入内存的页面按调入先后排成一个队列（链表也可），需要换出时选择队头。队列最大长度等于系统为进程分配的内存块数。

3 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
页框（队尾 → 队首）	7	7,0	7,0,1	0,1,2	0,1,2	1,2,3	2,3,0	3,0,4	0,4,2	4,2,3	2,3,0	2,3,0	2,3,0
淘汰页	—	—	—	**7**	—	**0**	**1**	**2**	**3**	**0**	**4**	—	—
是否缺页	✓	✓	✓	✓	×	✓	✓	✓	✓	✓	✓	×	×

缺页次数 = **10**，缺页率 $= 10/13 \approx 76.9\%$。这是三个基础算法里最差的。

4 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
页框	7	7,0	7,0,1	7,0,1,2	7,0,1,2	0,1,2,3	0,1,2,3	1,2,3,4	1,2,3,4	1,2,3,4	2,3,4,0	2,3,4,0	2,3,4,0
淘汰页	—	—	—	—	—	**7**	—	**0**	—	—	**1**	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	×	×	✓	×	×

缺页次数 = **7**。

FIFO 的优点是实现简单；缺点是与进程实际运行的规律不相适应——先进入内存的页面也可能是需要经常访问的（例如循环中最常用的数组首元素），故性能差。

4.Belady 异常

Belady 异常的定义

Belady 异常（**Belady's Anomaly**）：对某些页面置换算法，随着分配给进程的页框数（驻留集）增大，缺页次数不减反增的现象。

会发生 **Belady** 异常的算法：**FIFO**、简单 **CLOCK**、改进型 **CLOCK**。不会发生 **Belady** 异常的算法：**LRU**、**OPT**。（这类算法属于「栈算法 / stack algorithm」，具有包含性质 $S_n(t) \subseteq S_{n+1}(t)$，因此不会出现异常。）

Belady 异常的经典实例：引用串

$$1,\ 2,\ 3,\ 4,\ 1,\ 2,\ 5,\ 1,\ 2,\ 3,\ 4,\ 5 \qquad (\text{共12 次访问})$$

分配 **3** 个页框：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
访问页	1	2	3	4	1	2	5	1	2	3	4	5
页框（新 → 旧）	1	1,2	1,2,3	2,3,4	3,4,1	4,1,2	1,2,5	1,2,5	1,2,5	2,5,3	5,3,4	5,3,4
淘汰页	—	—	—	**1**	**2**	**3**	**4**	—	—	**1**	**2**	—
是否缺页	✓	✓	✓	✓	✓	✓	✓	×	×	✓	✓	×

缺页次数 = **9**。

分配 **4** 个页框：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
访问页	1	2	3	4	1	2	5	1	2	3	4	5

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
页框（新 → 旧）	1	1,2	1,2,3	1,2,3,4	1,2,3,4	1,2,3,4	2,3,4,5	3,4,5,1	4,5,1,2	5,1,2,3	1,2,3,4	2,3,4,5
淘汰页	—	—	—	—	—	—	**1**	**2**	**3**	**4**	**5**	**1**
是否缺页	✓	✓	✓	✓	×	×	✓	✓	✓	✓	✓	✓

缺页次数 = **10**。

Belady 异常的验证

同一个引用串，**3** 个页框缺页 **9** 次，**4** 个页框缺页 **10** 次。页框多了，缺页反而多了 1 次——这就是 Belady 异常。

原因：FIFO 选择替换「最早进入内存」的页面，完全没有考虑这个页面未来是否可能被经常访问。在上述例子中，第 1 页虽然是最早进入的，但在序列后段还要被反复访问；FIFO 却因为「它最老」把它换出，结果很快又要把它调回来，白白多了一次缺页。当页框数从 3 变为 4 时，页面的「入驻/淘汰节奏」整体错位，恰好把这种盲目性放大成了缺页次数的净增。

真题考法

【2014-30】可能出现 Belady 异常现象的是：Ⅰ LRU　Ⅱ FIFO　Ⅲ OPT。答案 **A**（仅Ⅱ）。

*5.*最近最久未使用置换算法（*LRU*，*Least Recently Used*）

算法思想：每次淘汰最近最久未使用的页面。

实现方法：在每个页面的页表项中用访问字段记录该页面自上次被访问以来所经历的时间 t，淘汰时选 t 值最大的页面。软件上内核维护一个双向页面链表，表头是最近刚使用过的页面，表尾是最近最久没有使用的页面：每次访存把被访问页移到表头，缺页时淘汰表尾页面、新页挂到表头；由于每次访存都要修改链表，故开销很大。硬件上可用移位寄存器记录使用情况，或给页面打时间戳（淘汰时选时间戳最大者）。

手工模拟技巧：需要淘汰页面时逆向检查访问序列——若分配了 n 个内存块，则往前找第 n 个不同的、且在内存中的页号，它就是最近最久未使用的页面。

3 个页框的完整模拟（列内按「最近使用 → 最久未使用」排列）：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
最近使用（前）	7	0,7	1,0,7	2,1,0	0,2,1	3,0,2	0,3,2	4,0,3	2,4,0	3,2,4	0,3,2	3,0,2	2,3,0
最久未使用（末）				7	1	2	2	3	0	4	2	2	0
淘汰页	—	—	—	**7**	—	**1**	—	**2**	**0**	**4**	**2**	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	✓	✓	✓	×	×

淘汰理由逐条说明：

- 第 4 次访问 2 缺页，内存中为 $\{7,0,1\}$，按最近使用排序 $1,0,7$，末位 **7** 最久未使用，淘汰 **7**。
- 第 6 次访问 3 缺页，内存状态 $0,2,1$（最近使用序），末位 **1** 最久未使用，淘汰 **1**。
- 第 8 次访问 4 缺页，内存状态 $0,3,2$，末位 **2** 最久未使用，淘汰 **2**。
- 第 9 次访问 2 缺页（刚被淘汰），此时最近使用序为 $4,0,3$，末位 **3** 最久未使用，淘汰 **3**。

缺页次数 = **9**，缺页率 $= 9/13 \approx 69.2\%$。

4 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
最近使用（前）	7	0,7	1,0,7	2,1,0,7	0,2,1,7	3,0,2,1	0,3,2,1	4,0,3,2	2,4,0,3	3,2,4,0	0,3,2,4	3,0,2,4	2,3,0,4
淘汰页	—	—	—	—	—	**7**	—	**1**	**3**	**0**	**4**	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	✓	✓	✓	×	×

缺页次数 = **6**。

LRU 与 OPT 的比较

LRU 用「过去最久未使用」近似「未来最久不使用」（基于时间局部性），因此性能很好，缺页次数单独看与 **OPT** 很接近（本例 3 页框时 LRU 9 次、OPT 7 次；4 页框时两者都是 6 次）。但 **LRU** 实现困难、开销大，实际系统多采用它的近似算法（CLOCK）。

*6.*时钟置换算法（*CLOCK / NRU*）

由于 LRU 实现开销太大，实际系统中大多采用 LRU 的近似算法，**CLOCK** 置换算法就是其中最常用的一种。

简单时钟置换算法（**NRU**，**Not Recently Used**）：

- 算法思想：缺页时从指针处顺序查找未被访问的页面进行置换。它不追求严格的「最近最久未使用」，可理解为淘汰最近较久未访问的页面，实现简单、开销小。
- 实现方法：为每个页面设一个访问位，把内存中的页面链成一个循环队列，指针指向最先调入的页面；页面装入时访问位初始化为 **0**，被访问（读或写）时置 **1**。缺页需要淘汰时，从指针当前位置顺序检查：
  - 访问位为 **0** → 最近没被访问过，选该页换出；
  - 访问位为 **1** → 把它清 **0**，指针移到下一页继续检查。若第一轮扫描中所有页面的访问位都是 1，则已把它们全部清零，第二轮扫描中一定存在访问位为 **0** 的页面。

简单 CLOCK 的扫描轮数

简单 CLOCK 选择一个淘汰页面最多经过两轮扫描。口诀：访问位有 **0** 就选第一个 **0**；全为 **1** 就先全部清 **0**，再选第一个。

3 个页框的完整模拟（记页号（访问位））：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
帧 1	7(1)	7(1)	7(1)	**2(1)**	2(1)	2(1)	2(1)	**4(1)**	4(1)	4(1)	**0(1)**	0(1)	0(1)
帧 2		0(1)	0(1)	0(0)	0(1)	0(0)	0(1)	0(0)	2(1)	2(1)	2(0)	2(0)	2(1)
帧 3			1(1)	1(0)	1(0)	**3(1)**	3(1)	3(0)	3(0)	3(1)	3(1)	**3(1)**	3(1)
淘汰页	—	—	—	**7**	—	**1**	—	**2**	**0**	—	**4**	**3**	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	✓	×	✓	✓	×

关键步骤：

- 第 **4** 次访问 2 缺页：帧 1 的 7 为 1 → 清 0、后移；帧 2 的 0 为 1 → 清 0、后移；帧 3 的 1 为 1 → 清 0、指针回到帧 1（进入第二轮）。帧 1 的 7 已是 0 → 淘汰 **7**，装入 2。共两轮。
- 第 **6** 次访问 3 缺页：第 5 次访问过 0，故 0(1)、1(0)、2(1)。从指针所在的帧 2 起：0 为 1 → 清 0、后移；帧 3 的 1 为 0 → 淘汰 **1**，装入 3。
- 第 **8** 次访问 4 缺页：第 7 次访问过 0（0 变 1），从指针处逐位扫描，淘汰 **2**，装入 4。
- 第 **9**、**11**、**12** 次分别缺页，依次淘汰 **0**、**4**、**3**。

缺页次数 = **9**。**4** 个页框时缺页次数为 **6**（第 6、8 次分别淘汰 7、1，其余命中，与 LRU 的 4 页框结果一致）。

手算 CLOCK 的两条纪律

1. 访问位只在「访问命中」时置 **1**；扫描时访问位为 0 的页面直接淘汰，不置 **1**。常见错误是把「被扫描过的页面」也置 1。
2. 指针位置必须记准：淘汰后指针指向被淘汰页框的下一个页框；不发生缺页的命中时指针不动。每次操作都显式写出指针位置，否则后面全错。

*7.*改进型时钟置换算法

为什么需要改进：简单 CLOCK 只考虑页面是否被访问过。但访问不一定修改，而页面一旦被修改就必须写回外存。因此淘汰时应优先选没有被修改过的页面，从而省下一次磁盘写回。为此再设置一个修改位 M：$M=0$ 表示没被修改过，$M=1$ 表示已被修改过。用二元组 (A, M) 表示「访问位，修改位」，进行至多四轮扫描：

轮次	扫描目标	是否修改标志位
第一轮	找第一个 $(0,0)$ ——最近未被访问、且未被修改	本轮不修改任何标志位
第二轮	若第一轮失败，重新扫描，找第一个 $(0,1)$	本轮把所有扫描过的帧的访问位设为 **0**
第三轮	若第二轮失败，重新扫描，找第一个 $(0,0)$	本轮不修改标志位
第四轮	若第三轮失败，重新扫描，找第一个 $(0,1)$	——

改进型 CLOCK 的三条结论

1. 由于第二轮已把所有帧的访问位清零，第三、四轮扫描一定会有帧被选中，因此至多四轮扫描。
2. 该算法只修改访问位，从不修改修改位。

3. 本质是把页面分成四个优先级，淘汰优先级为 $(0,0) > (0,1) > (1,0) > (1,1)$，即先看是否被访问过，再看是否被修改过。

（**A, M**）四类页面：$(0,0)$ 既不会马上被用、也不用写回，淘汰代价最小，排第 1 位；$(0,1)$ 不会马上被用但要写回，排第 2 位；$(1,0)$ 可能马上要用但不用写回，排第 3 位；$(1,1)$ 既要被用又要写回，排最后。

手工模拟示例：设循环队列 4 帧，指针指向帧 1，各帧状态为 A$(1,0)$、B$(1,1)$、C$(1,0)$、D$(1,1)$。发生缺页后：

- 第一轮：找 $(0,0)$。四帧中没有 $(0,0)$，失败，本轮不改任何标志位。
- 第二轮：重新扫描找 $(0,1)$，同时把扫描过的帧的访问位清零。帧 1 清成 $(0,0)$、后移；帧 2 清成 $(0,1)$，命中，淘汰 **B**。只有帧 1、2 被扫描，帧 3、4 保持不变。共用两轮。

推演所有 $M=0$ 的情形：若 4 帧状态全为 $(1,0)$，第一轮找 $(0,0)$ 失败；第二轮找 $(0,1)$ 时四帧被依次清成 $(0,0)$，无一满足 $(0,1)$，第二轮失败；第三轮重新扫描找 $(0,0)$，帧 1 已是 $(0,0)\rightarrow$ 选中帧 1。共三轮。

真题考法

【2016-26】改进型CLOCK中 A 为访问位、M 为修改位，淘汰次序为(　)。答案 **A.** $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$。算法展开后的等价优先级正是 $(0,0) > (0,1) > (1,0) > (1,1)$。选项 B（$(0,0),(1,0),(0,1),(1,1)$）是「先看修改位」的错解。

*8.*同一引用串下各算法缺页次数对照

引用串 $7,0,1,2,0,3,0,4,2,3,0,3,2$（13 次访问）：

算法	**3** 个页框时缺页次数	缺页率	**4** 个页框时缺页次数	缺页率	是否可能 **Belady** 异常
OPT	**7**	53.8%	**6**	46.2%	否
FIFO	**10**	76.9%	**7**	53.8%	是
LRU	**9**	69.2%	**6**	46.2%	否
CLOCK（简单）	**9**	69.2%	**6**	46.2%	是
改进型 **CLOCK**	依赖 (A,M) 初值	—	依赖 (A,M) 初值	—	是

结论速记：

- 缺页次数排序（3 页框）：OPT(7) < LRU = CLOCK(9) < FIFO(10)。
- **OPT** 一定最优，但不可实现；**LRU** 性能接近 **OPT**，但开销大；**CLOCK** 是 **LRU** 的廉价近似，性能介于 LRU 与 FIFO 之间；**FIFO** 最简单也最差。
- **4** 个页框下 **LRU** 与 **OPT** 都是 **6** 次，说明 LRU 在页框较多时逼近最优。
- **Belady** 异常只发生在 **FIFO / CLOCK** 类算法上；OPT 与 LRU 由于满足栈性质，绝不会有 Belady 异常。

（五）内存映射文件

内存映射文件（**Memory-Mapped Files**，简称 `mmap`）是一种把磁盘上的文件直接映射到进程虚拟地址空间的机制，使得进程可以像访问内存一样读写文件，从而避免频繁的系统调用（如 `read`/`write`）

和数据拷贝。

内存映射文件与虚拟内存的关键区别

	建立映射的两端
虚拟内存管理	物理内存 ↔ 进程虚拟地址空间
内存映射文件 `mmap()`	磁盘上的文件 ↔ 进程虚拟地址空间

用一句话说：虚拟内存把「物理内存」变成「进程眼里的连续地址空间」；内存映射文件把「磁盘文件」变成「进程眼里的连续地址空间」。

工作原理（完整流程）：

1. 用 `open()` 打开文件得到文件描述符 **fd**，再用 `mmap(addr, length, fd, ...)` 请求映射；内核在进程虚拟地址空间中分配一段连续区域用于映射该文件。
2. 此时并没有立即把文件数据加载到内存（这是理解 mmap 高效的关键）。
3. 按需加载：进程首次访问该区域的某个页面时发现存在位为 **0**，触发缺页中断；内核找到该页对应的文件数据，分配一个物理页框装入并更新页表——此时才真正建立虚拟内存到物理内存的映射。
4. 该页框中的数据即文件在内存中的副本（文件缓存页）。后续读写直接作用于内存中的该页；修改会标记脏位，内核在适当时机（内存不足要换出该页、或关闭文件时）把修改写回磁盘。

多个进程可以并发映射同一文件以实现共享：任一进程的写入其他映射了同一部分的进程都能看到，因为各进程的虚拟内存映射指向物理内存的同一页面，而该页面是磁盘块的副本。系统调用还支持写时复制（**Copy-On-Write**），允许进程既按只读共享，也拥有自己修改的副本。

关闭文件 ≠ 解除映射

在多个进程共享某一个磁盘文件时，某个进程关闭文件并不会触发文件同步并且解除映射，因为文件描述符有引用计数，此时只会把引用计数减 1。要解除映射一般需要调用 `munmap(addr, length)`。

内存映射文件的优点：减少数据拷贝（传统 `read/write` 需在内核缓冲区与用户缓冲区之间拷贝，而 `mmap` 直接操作页缓存，实现零拷贝）；方便程序员访问磁盘文件（可像访问内存一样读写，尤其适合频繁随机访问大文件，无需多次 `lseek`）；读入读出全部由内核实现（缺页异常服务程序读入数据、内核负责把脏数据写回，不需用户调用 `read()/write()`）；方便多进程共享同一文件（共享同一份页缓存，也能实现进程间通信）；按需加载、节约物理内存；显著提升 **I/O** 密集型进程的效率。

为什么 **`read/write`** 需要两次拷贝，而 **`mmap`** 可以直接写入用户空间？对 `read/write`，内核先把磁盘块经 DMA 读入内核缓冲区，再把用户需要的指定字长的数据复制到用户 **buf**——必须复制是因为用户没有权限读取一整个磁盘块（整块中可能有敏感数据）；而对缺页处理，与磁盘交换的单位是页面，DMA 预处理时直接把分配给该页的页框物理地址写入 DMA 控制器，由 DMA 直接把页面加载到该页框（用户地址空间），因为读入的整个页面完全属于该进程，不存在读到敏感数据的问题。

页面与磁盘块没有实质关系

操作系统以「页」为单位交换内存与磁盘，而文件系统以「块」为单位管理磁盘存储。若一个页面是磁盘块的 n 倍，一次缺页可能需读多个块；页面与块对齐只是为了方便，二者大小并没有实质关系。

注意区分内存映射 **I/O**（I/O 章节的「统一编址」，把一组内存地址映射到设备寄存器）与内存映射

文件（映射磁盘文件），二者不是一回事。

（六）虚拟存储器性能的影响因素及改进方法

*1.*缺页率及其影响因素

假设一个进程的逻辑空间为 n 页，系统为其分配的内存物理页框数为 m（$m < n$）。在进程运行过程中，访问页面成功（所访问页面在内存中）的次数为 S，访问页面失败（页面不在内存，需要从外存调入）的次数为 F，则总的页面访问次数为

$$A = S + F$$

该进程在其运行过程中的缺页率为

$$f = \frac{F}{A}$$

缺页率（缺页率高即为抖动）是影响虚拟存储器性能的主要因素。它受以下四个因素影响：

1. 页面大小：由局部性原理，页面较大则缺页率较低，页面较小则缺页率较高。页面小时内存碎片少、利用率高，但每个进程要求的页面数多，页表过长、占用大量内存；页面大时页表短，但页内碎片增大。
2. 进程所分配物理页框的数目：页框越多缺页率越低；但超过某个数目后再增加页框对缺页率的改善并不明显，此时只需保证活跃页面在内存中、缺页率维持在很低范围即可。
3. 页面置换算法：算法优劣决定缺页中断次数，缺页率是衡量置换算法的重要指标。
4. 程序固有特性：程序编制的局部化程度越高，缺页程度越低（如按行优先访问行优先存储的二维数组）。

此外，在缺页中断处理时，当由于空间不足而需要置换部分页面到外存时，选择被置换页面还需要考虑置换的代价（例如页面是否被修改过）。没有修改过的页面可以直接放弃，已被修改过的页面则必须先保存，因此两种情况耗时不同。假设被置换页面被修改的概率是 β，被置换页面被修改时的缺页中断处理时间是 t_a，没有被修改时的缺页中断处理时间是 t_b，则缺页中断处理时间为

$$t = \beta \times t_a + (1 - \beta) \times t_b$$

真题考法

【2022-30】不会影响系统缺页率的是：**D.** 页缓冲队列的长度（见前文）。

*2.*有效访问时间（*EAT*）

优先掌握考纲标准公式（不含 **TLB**）。设内存访问时间为 t，缺页处理时间为 $t_{\text{page fault}}$，缺页率为 p，则

$$\boxed{EAT = (1 - p) \times t + p \times t_{\text{page fault}}}$$

每个符号的含义与适用范围：

符号	含义	说明
p	缺页率	$0 \le p \le 1$；一般 p 很小（$10^{-6} \sim 10^{-3}$ 量级）
$1-p$	页面在内存中的概率	此时只需一次访存，时间 $= t$
t	一次内存访问时间	单位通常为 ns
$t_{\text{page fault}}$	缺页处理时间	包括：查页表发现缺失、缺页中断响应与现场保护、选页置换、被修改页写回磁盘、从外存读入所缺页、修改页表、恢复现场并重新执行该指令。主要是磁盘 **I/O** 时间，单位通常为 μs 或 ms

使用前提：题目不考虑快表（**TLB**），即命中时按一次访存计算；且缺页处理时间按题目给出的「缺页中断处理时间 / 缺页服务时间」整体取值。若题目给出的缺页相关时间是磁盘访问时间等分项，需先累加成完整的缺页处理时间。

EAT 公式的两种变形

1. 命中时一次访存（最常见）：

$$EAT = (1-p) \times t + p \times t_{\text{页缺失}}$$

1. 命中时两次访存（无快表的基本分页，每次访问都要先查页表）：

$$EAT = (1-p) \times 2t + p \times t_{\text{页缺失}}$$

1. 有快表时（设查快表时间 λ、命中率 α）：见下文的分情况公式。

做题纪律：先看题目说「访问内存一次」还是「查页表 + 访存」，再决定命中项是 t 还是 $2t$。

算例 **1**（考纲标准形式）：某请求分页系统，内存访问时间 $t = 100$ ns，缺页处理时间 $t_{\text{page fault}} = 8$ ms，缺页率 $p = 10^{-3}$，求有效访问时间 EAT。

统一单位（ns）：$8\text{ ms} = 8 \times 10^6$ ns。

$$EAT = (1-10^{-3}) \times 100 + 10^{-3} \times 8 \times 10^6 = 0.999 \times 100 + 8000 = 99.9 + 8000 = 8099.9\text{ ns} \approx 8.10\ \mu\text{s}$$

解读：p 只有千分之一，但 EAT 仍然被抬高到近 **8.1** μ**s**，是正常访存 100 ns 的约 **81** 倍。这说明缺页处理时间比内存访问时间大好几个数量级，因此缺页率对性能的影响是压倒性的。若把 p 降到 10^{-4}：

$$EAT = 0.9999 \times 100 + 10^{-4} \times 8 \times 10^6 = 99.99 + 800 = 899.99\text{ ns} \approx 0.90\ \mu\text{s}$$

缺页率降低 10 倍，EAT 降低约 9 倍——这就是为什么所有改进手段的着力点都是「降低缺页率」。

算例 **2**（含快表与缺页的完整分情况推导）

设访问内存时间 t，查询 TLB 时间 λ，处理缺页异常时间 ε，TLB 命中率为 α，缺页率为 f。考虑一级页表，且 CPU 处理速度远高于访存速度，因此分析逻辑地址、获取页号与页内偏移量等 CPU 处理时间忽略不计。

情况一：被访问页在内存中，且其对应的页表项在 **TLB** 里（命中）

$$EAT_1 = \lambda + t$$

情况二：被访问页在内存中，但不在 **TLB** 里（**TLB** 缺失但页面存在）

流程：查找 TLB（未命中）→ 访问内存查页表 → 更新 TLB → 用物理地址访存。

$$EAT_2 = \lambda + t + \lambda + t = 2(\lambda + t)$$

（注：查找 TLB 与查页表在实际硬件中可以并行进行，一切以题目要求为准。）

情况三：被访问页不在内存（缺页）

1. 访问 TLB 未命中，用时 λ；
2. 访问页表发现页面缺失，用时 t，触发缺页；
3. 缺页异常处理耗时 ε；
4. 重新执行发生缺页的那条指令，再次访问 TLB 未命中，用时 λ；
5. 页表命中，用时 t；
6. 修改 TLB，用时 λ；
7. 用获得的物理地址访存，用时 t。

$$EAT_3 = 3\lambda + \varepsilon + 3t$$

加权平均：

$$EAT = \alpha\, EAT_1 + (1-\alpha)(1-f)\, EAT_2 + (1-\alpha) f\, EAT_3$$

即

$$EAT = \alpha(\lambda + t) + (1-\alpha)(1-f)\cdot 2(\lambda + t) + (1-\alpha) f \cdot (3\lambda + \varepsilon + 3t)$$

化简得

$$EAT = \big[(2+f) - \alpha(1+f)\big]\lambda + \big[(2-\alpha) + f(1-\alpha)\big]t + f(1-\alpha)\varepsilon$$

验算：取 $\alpha = 1$（TLB 全部命中），则

$$EAT = [(2+f) - (1+f)]\lambda + [(2-1) + 0]t + 0 = \lambda + t = EAT_1$$

公式自洽。再取 $\alpha = 0,\ f = 0$（无 TLB、无缺页）：

$$EAT = (2 + 0 - 0)\lambda + (2-0)t = 2\lambda + 2t = 2(\lambda + t) = EAT_2$$

也自洽。

数值算例：$t = 100$ ns，$\lambda = 10$ ns，$\varepsilon = 8$ ms $= 8 \times 10^6$ ns，$\alpha = 0.9$，$f = 10^{-3}$。

$$EAT = [(2 + 0.001) - 0.9 \times 1.001] \times 10 + [(2 - 0.9) + 0.001 \times 0.1] \times 100 + 0.001 \times 0.1 \times 8 \times 10^6$$

逐项计算：

- TLB 项：$2.001 - 0.9009 = 1.1001$，乘 10 得 11.001 ns；

- 访存项：$1.1 + 0.0001 = 1.1001$，乘 100 得 110.01 ns；
- 缺页项：$0.001 \times 0.1 \times 8 \times 10^6 = 800$ ns。

$$EAT = 11.001 + 110.01 + 800 = 921.011 \text{ ns} \approx 0.92\ \mu\text{s}$$

解读：即使用了 90% 命中率的快表，EAT 仍然主要由缺页项（800 ns）主导。这与算例 1 的结论一致。

做 EAT 题的两个「看题目」

1. 缺页异常处理是只修改页表，还是同时修改页表和 **TLB**？若题目说明同时修改 TLB，则情况三中第 4、6 步的 λ 要相应调整（第 4 步不再需要，或第 6 步并入 ε）。
2. 缺页处理把页面调入内存后，是「重新执行发生缺页的那条指令」，还是「直接拿物理地址去访存」？前者是标准情形（需要再走一遍 TLB 与页表）；若题目说直接访存，则 EAT_3 要减掉第 4、5、6 步的开销。

一切根据题目条件计算，不要死套公式。

*3.*改进方法

虚拟存储器性能的改进围绕降低缺页率与降低缺页处理代价两条主线：

1. 降低缺页率：合理确定驻留集（使驻留集不小于工作集，用可变分配局部置换动态调整）；选用较优的置换算法（优先用 LRU 的近似实现，避免退化为纯 FIFO）；合理选择页面大小（过大加剧页内碎片与单次 I/O 量，过小使页表膨胀、缺页增多）；提高程序的局部性（如按行优先访问行优先存储的数组，减少跨页访问）；控制并发进程数（作业调度时考虑工作集，必要时主动挂起低优先级进程）。
2. 降低缺页处理代价：采用页面缓冲算法（空闲页面链表 + 修改页面链表，批量写回已修改页，被换出页可直接从链表取回）；增大 **TLB** 容量、让页表常驻内存（只加快地址变换，不降低缺页率）；提高磁盘 **I/O** 速度、对换区采用连续分配；预调页策略（首次调入或按工作集批量预取）。
3. 缓解抖动：采用局部置换把抖动限制在进程内部；作业调度时考虑工作集，驻留页面不足则不调入新作业；主动挂起低优先级（或剩余执行时间最多）的进程以腾出页框。

真题考法

【2020-28】影响请求分页系统有效（平均）访存时间的是：Ⅰ缺页率　Ⅱ磁盘读写时间　Ⅲ内存访问时间　Ⅳ执行缺页处理程序的 CPU 时间。答案 **D**（Ⅰ、Ⅱ、Ⅲ、Ⅳ）。因为 $EAT = (1-p)t + p\,t_{\text{page fault}}$ 中，p 对应Ⅰ，t 对应Ⅲ，$t_{\text{page fault}}$ 中包含磁盘读写时间（Ⅱ）与缺页处理程序的 CPU 时间（Ⅳ）。

本章小结表

（七）三种非连续分配方式总对比

对比项	页式管理	段式管理	段页式管理
分配单位	页面（固定大小）	段（可变大小）	先分段、段内再分页
逻辑地址结构	页号 + 页内偏移	段号 + 段内偏移	段号 + 页号 + 页内偏移
地址空间维度	一维	二维	二维
对用户是否可见	不可见	可见	段可见、页不可见

对比项	页式管理	段式管理	段页式管理
数据结构	页表（每进程 1 张）	段表（每进程 1 张）+ 共享段表	段表（每进程 1 张）+ 页表（每段 1 张）
表的表项内容	页号、页框号 + 控制位	段号、段长、段基址 + 控制位	段表项：页表长度、页表块号；页表项：页框号
越界检查次数	1 次（页号 vs 页表长度）	2 次（段号 vs 段表长度；段内偏移 vs 段长）	2 次（段号 vs 段表长度；页号 vs 页表长度）
碎片	页内碎片（内部）	外部碎片	页内碎片（内部）
共享与保护	实现复杂（需 N 个页表项）	实现简单（只需 1 个段表项）	通过共享段指向相同页表实现
无快表访存次数	2 次	2 次	**3** 次
有快表命中访存次数	1 次	1 次	1 次

（八）页面置换算法总对比

算法	淘汰规则	是否可实现	性能	**Belady** 异常	实现开销
OPT	淘汰以后最长时间不再访问的页	不可实现（理论基准）	最优（缺页最少）	否	—
FIFO	淘汰最早进入内存的页	可实现	最差	是	最小
LRU	淘汰最近最久未使用的页	可实现	接近 **OPT**	否	很大
CLOCK（**NRU**）	循环扫描，淘汰第一个访问位为 0 的页（最多两轮）	可实现	介于 LRU 与 FIFO 之间	是	小
改进型 **CLOCK**	按 $(0,0) > (0,1) > (1,0) > (1,1)$ 顺序（最多四轮）	可实现	较好，减少写回开销	是	较小

（九）连续分配三种方式总对比

分配方式	内存划分	多道能力	内部碎片	外部碎片	采用的装入方式	地址变换时机
单一连续分配	系统区 + 一整块用户区	单道	有	无	绝对装入	编译时
固定分区分配	系统区 + 预先划分的若干固定分区	多道（≤ 分区数）	有	无	可重定位装入	装入时
动态分区分配	系统区 + 装入时动态建立的分区	多道	无	有	动态运行时装入	执行时

（十）有效访问时间速查

情形	公式	说明
无 TLB，命中一次访存	$EAT = (1-p)t + p\,t_{\text{pf}}$	考纲标准式
无 TLB，命中需两次访存	$EAT = (1-p)\cdot 2t + p\,t_{\text{pf}}$	命中也要先查页表
有 TLB，命中	$\lambda + t$	只需一次访存
有 TLB，TLB 缺失但页在内存	$2(\lambda + t)$	命中需查页表并更新 TLB
有 TLB，缺页	$3\lambda + \varepsilon + 3t$	含重新执行指令
加权总式	$\big[(2+f)-\alpha(1+f)\big]\lambda + \big[(2-\alpha)+f(1-\alpha)\big]t + f(1-\alpha)\varepsilon$	α 为 TLB 命中率，f 为缺页率

易错清单

- 逻辑地址在「链接」阶段形成（2011-30），物理地址由 **MMU** 在访存时使用。
- 装入方式与地址变换时机配对：绝对装入 → 编译时；可重定位装入 → 装入时；动态运行时装入 → 执行时。只有动态重定位允许程序在内存中移动。
- 静态重定位 ≠ 静态链接；动态重定位 ≠ 动态链接。前者是装入方式，后者是链接方式。
- 内部碎片 **vs** 外部碎片：内部碎片是「分配单元内部」的浪费（过量分配）；外部碎片是「分配单元之间」的浪费（按需分配）。单一连续、固定分区、分页、段页式有内部碎片；动态分区、分段有外部碎片。
- 动态分区四种算法：首次适应综合最好；邻近适应隐含最佳适应的缺点；最佳适应最容易产生外部碎片；最坏适应大分区很快耗尽。别被「最佳」二字误导。
- 回收分区的合并要注意与上块、与下块、与上下块一起合并三种情况，以及合并后分区数目的变化（2017-25 考的是上下同时合并）。
- 分页地址一维、分段地址二维：页号与页内偏移不是独立变量；段号与段内偏移是两个独立变量。
- 分页越界检查 **1** 次，分段 **2** 次。页内偏移量不可能越界；段内偏移量可能越界。
- 页表项存的是「页框号」，不是「页框起始地址」；b 号页框的起始地址为 $b \times L$。
- 页表项常取 **4B** 而非 **3B**，为让一个页面恰好装下整数个页表项。做题以题目给出的页表项大小为准。
- 多级页表优点是「减少页表所占的连续内存空间」，它不加快地址变换、不减少页表项字节数、不直接减少缺页次数。
- **CPU** 页表基址寄存器存的是「一级页表的起始物理地址」（2021-29）；各级页表大小不能超过一个页面。
- 共享段被不同进程引用的段号可以不同（2019-28），但物理内容只有一份。
- 分页共享时页号可以不同、页框号一定相同（2023-30）；段页式共享同理。
- 段页式无快表需 **3** 次访存（查段表 + 查页表 + 访存），快表命中时 1 次。
- 虚拟内存最大容量受「地址结构」限制，实际容量 = min(内存 + 外存, 寻址范围)；绝不能说「由主存和硬盘大小决定」。
- 虚拟内存必须建立在「离散分配」基础上；三特征是多次性、对换性、虚拟性，「多次性」是核

心。

- 缺页中断属于「内中断 / 异常 / 故障」，由当前执行的指令造成；一条指令执行期间可能发生多次缺页中断。
- 缺页处理完成后是「重新执行该指令」，不是「直接访存」——这直接决定 EAT 里要不要重复计算 TLB 与页表访问。
- 缺页 $\neq$ 越界（2013-30）；缺页处理中不一定包含「淘汰内存中的页」（2022-29，还有空闲页框时直接装入）。
- 「固定分配 + 全局置换」不存在（2015-30）：全局置换必然改变进程页框数，与固定分配矛盾。
- 驻留集不能小于工作集，否则抖动；工作集要向「过去」取窗口，不能算入 t 之后的访问（2016-29）。
- **Belady** 异常只发生在 **FIFO**、简单 **CLOCK**、改进型 **CLOCK** 上；**LRU** 与 **OPT** 不会（满足栈性质）。
- **OPT** 不可实现；**FIFO** 最差；**LRU** 性能好但开销大；**CLOCK** 是 **LRU** 的廉价近似。
- 改进型 **CLOCK** 至多四轮，只改访问位、从不改修改位；优先级 $(0,0) > (0,1) > (1,0) > (1,1)$ ——先看访问位，再看修改位（2016-26 的 B 选项即「先看修改位」的错解）。
- 简单 **CLOCK** 至多两轮；手算务必记准指针位置与访问位状态。
- 缺页时不一定发生页面置换；页面置换不改变页框号，只改变哪个页号占用该页框。
- 快表中有的页面一定在内存中，页面换出时必须同步删除快表项，故不会出现「TLB 命中但页面不在内存」。
- `mmap` 映射的是「磁盘文件 $\leftrightarrow$ 进程虚拟地址空间」，与虚拟内存、与「内存映射 I/O」（映射设备寄存器）都不是一回事；且 `mmap` 建立映射时并不立即加载文件数据。
- **EAT** 公式不能用错情形：命中时是「一次访存」还是「查页表 + 访存」；有快表时缺页处理是否更新 TLB、缺页后是重新执行还是直接访存。
- 抖动不能靠「增加交换区容量」或「提高进程优先级」解决，只能靠减少并发进程数 / 增加每进程页框数（2011-29）。

第四章文件管理

本章考纲要求

四、文件管理（一）文件

1. 文件的基本概念
2. 文件元数据和索引节点（inode）
3. 文件的操作：建立，删除，打开，关闭，读，写。
4. 文件的保护
5. 文件的逻辑结构
6. 文件的物理结构

（二）目录

1. 目录的基本概念
2. 树形目录
3. 目录的操作
4. 硬链接和软链接

（三）文件系统

1. 文件系统的全局结构（layout）：文件系统在外存中的结构，文件系统在内存中的结构。
2. 外存空闲空间管理方法
3. 虚拟文件系统
4. 文件系统挂载（mounting）

本章是操作系统部分「重理解、重计算」的典型一章。文件名、目录项、索引节点、打开文件表这几层数据结构之间的层次关系，以及「逻辑块号 → 物理块号」的转换次数，是统考反复出题的两条主线。学习时建议始终带着一个问题：此刻 **CPU** 手里拿的是文件名、文件描述符，还是物理块号?

一、文件的基本概念

(一）数据项、记录与文件

文件的内部是分层组织的，自底向上依次是:

- 数据项（**data item**）：最低级的数据组织形式。基本数据项是数据中不可再分的最小逻辑单位，如「学号」「姓名」；组合数据项由若干基本数据项组成，如「出生日期」由年、月、日组成。
- 记录（**record**）：一组相关数据项的集合，用于描述一个对象在某方面的属性。例如一条学生记录 = 学号 + 姓名 + 出生日期 + 专业。
- 文件（**file**）：由创建者定义、具有文件名的一组相关信息的集合。文件可分为有结构文件（若干记录组成）与无结构文件（字符流）。

文件的定义与三个视角

从用户角度看，文件是具有符号名（文件名）的一组相关信息的有序集合，它可以存储在磁盘等外存上，并且可以被长期保存。

理解文件要抓住三个层次：

- 用户视角：文件是「按名字存取的信息集合」，通过文件名访问，不必知道数据究竟放在哪个盘块。
- 系统视角：文件是外存空间的分配单位，文件系统为每个文件维护一份元数据（FCB/inode）记录它占用了哪些盘块。
- 进程视角：文件是进程可以打开、读写的对象，打开后以「文件描述符」为句柄操作，与文件名脱钩。

（二）文件系统及其功能

文件系统（**File System**）是操作系统中负责管理和存取文件信息的软件机构，也是磁盘上组织文件的方法。它的核心目标只有一个：实现「按名存取」，同时保证高效、安全、可靠。

- 从系统角度看，文件系统负责：文件存储空间的管理、目录管理、逻辑地址到物理地址的映射（文件读写管理）、文件的共享与保护。
- 从用户角度看，文件系统提供了命令接口（终端里敲 `ls`、`cp`）和程序接口（`open`、`read`、`write` 等系统调用）。

文件系统容量不等于磁盘容量

一个文件系统能容纳的文件数量，受索引节点（**inode**）总数限制（搭建文件系统时往往已经固定，格式化时划分了 inode 区）；而单个文件的最大长度受 inode 中地址项的个数与结构限制。所以：

- 「文件系统最多能存多少个文件」由 **inode** 总数决定，与磁盘容量无关；
- 「单个文件最大有多长」由 **inode** 的索引结构决定，与 inode 总数无关。

这两句话是 2013 年真题的标准解法。用 RAID 磁盘阵列时，一个文件还可以分散在多块磁盘上，所以「文件必须放在一块盘上」也是错的。

（三）文件的属性与分类

常见属性包括：名称、标识符、类型、位置（指向设备和设备上文件的指针）、大小、保护（访问权限）、创建者/所有者 ID、创建与最近修改时间、链接计数等。

按不同角度分类：

分类角度	类别
性质和用途	系统文件、用户文件、库文件
数据形式	源文件（`.c`、`.java`）、目标文件（编译后的二进制）、可执行文件
存取控制	只读文件、读写文件、可执行文件
组织形式	普通文件、目录文件、索引文件、特殊文件（设备文件、命名管道等）

文件元数据和索引节点

（四）文件控制块 FCB

为了能对一个文件进行正确的存取，文件系统必须为文件设置一个描述与控制用的数据结构，称为文件控制块（**File Control Block, FCB**）。文件与 **FCB** 一一对应，FCB 的有序集合称为文件目录，其

中的每一个 FCB 就是一个目录项（**directory entry**）。目录本身也被视为一种文件，称为目录文件。FCB 通常包含三类信息：

类别	典型内容
基本信息	文件名、物理位置、逻辑结构、物理结构、文件类型、文件大小
存取控制信息	文件存取权限（文件主、核准用户、一般用户各自的读/写/执行权限）
使用信息	文件数据块指针、文件建立时间、上次修改时间、当前使用计数

FCB 的作用

FCB 建立了文件名与文件物理位置之间的映射，这正是「按名存取」得以实现的根本原因。用户给出文件名，文件系统查目录找到对应的 FCB，从中取出物理地址，再去访问外存。

因此可以记住两个等式：

$$\text{文件目录大小} = FCB\text{大小} \times \text{目录项个数}$$

$$\text{一个}FCB = \text{一个文件目录项}$$

索引节点 **inode** 与目录项的分离

为什么要把 **FCB** 拆开？检索目录时，其实只需要用到「文件名」这一个字段：只有文件名匹配成功了，才需要读出文件的其他信息。若每个目录项都是完整的 FCB，目录文件会占据大量盘块，平均检索时间随之飙升。

于是有了索引节点（**index node**，简称 **i** 节点、**inode**）方案：把 FCB 中除文件名以外的所有信息抽出来，构成一个独立的数据结构 inode；目录项则「瘦身」为文件名 + **inode** 编号（指针）两项。

一个必背的量化例子

设一个 FCB 占 64B，磁盘块大小 1KB，则每块只能放 $1024/64 = 16$ 个 FCB。若目录共有 640 个目录项，则目录文件占 $640/16 = 40$ 个盘块，按文件名检索平均需要比较 $640/2 = 320$ 个目录项，即平均启动磁盘 **20** 次（每次读入一块）。

改用索引节点后，设「文件名 14B + inode 指针 2B」共 16B，则每块可放 $1024/16 = 64$ 个目录项，检索平均只需读入 $320/64 = 5$ 个盘块。

磁盘 **I/O** 次数从 **20** 次降到 **5** 次，这就是索引节点机制的价值。

引入 inode 后，目录项与 inode 的关系是：

- 目录项只记录「文件名 → inode 编号」这一条逻辑映射，它描述的是文件在文件系统中的位置（逻辑位置）。
- **inode** 记录文件的元数据，其中包括文件在存储设备上的物理位置（直接/间接索引），描述的是文件数据实际存放在哪里。

区分「磁盘索引节点」与「内存索引节点」

磁盘索引节点存放在磁盘的 inode 区，是静态的，主要内容有：文件主标识符、文件类型、文件存取权限、文件物理地址（直接与间接索引项）、文件长度、文件链接计数 **count**、文件存取时间。

内存索引节点是文件被打开时从磁盘复制（调入）到内存的，除上述内容外还要增加：

- **inode** 编号：标识对应外存 inode 的编号；

- 状态位：是否被上锁、是否被修改（修改位置位决定关闭时是否需要写回外存）；
- 访问计数：记录当前有多少个进程打开了该文件；
- 逻辑设备号：标识 inode 所在的逻辑设备（分区）；
- 链接指针：分别指向内存空闲链表和散列队列的指针，用于加速查找。

内存 **inode** 可以被多个进程共享，这正是「多个进程打开同一文件时系统打开文件表只有一个表项」的底层原因。

inode 中的多级索引与最大文件大小计算

UNIX/Linux 混合索引的经典结构是 inode 中设 **13** 个地址项 `iaddr[0] ~ iaddr[12]`：

地址项	类型	作用
`iaddr[0] ~ iaddr[9]`	直接地址（10 项）	直接指向数据块
`iaddr[10]`	一级间接地址（1 项）	指向一个索引块，索引块里放数据块号
`iaddr[11]`	二级间接地址（1 项）	指向索引块，其中每项再指向一个索引块
`iaddr[12]`	三级间接地址（1 项）	三级「套娃」索引

通用解题模板

记：磁盘块（数据块与索引块同大小）大小 B，一个地址项占 p 字节，则

$$\text{一个索引块可容纳的地址项数 } n = \left\lfloor \frac{B}{p} \right\rfloor$$

设第 k 级间接地址项个数为 N_k（直接地址视为 0 级，$k=0$），则第 k 级可覆盖的数据量为

$$S_k = N_k \times n^k \times B \qquad (k = 0, 1, 2, 3, \dots)$$

单个文件最大长度（即可寻址的数据总量）为

$$S_{max} = \sum_{k} N_k \cdot n^k \cdot B$$

注意：索引块本身只存放地址，不计入文件的数据长度。这是「文件最大长度」类题目最容易被扣分的地方。

完整算例（**UNIX** 经典参数）。设磁盘块大小 $B = 4\text{KB}$，每个地址项 $p = 4\text{B}$，直接地址 10 个、一级/二级/三级间接各 1 个。

- 一个索引块能放的地址项数：$n = 4096\text{B}/4\text{B} = 1024$ 个。
- 直接地址：$10 \times 4\text{KB} = 40\text{KB}$。
- 一级间接：$1 \times 1024 \times 4\text{KB} = 4\text{MB}$。
- 二级间接：$1 \times 1024^2 \times 4\text{KB} = 4\text{GB}$。
- 三级间接：$1 \times 1024^3 \times 4\text{KB} = 4\text{TB}$。

$$S_{max} = 4\text{TB} + 4\text{GB} + 4\text{MB} + 40\text{KB}$$

第二组算例（更常考的参数）。设 $B = 1\text{KB}$，$p = 4\text{B}$，直接 10 项，一/二/三级间接各 1 项：

- $n = 1024/4 = 256$；
- 直接：$10 \times 1\text{KB} = 10\text{KB}$；一级间接：$256 \times 1\text{KB} = 256\text{KB}$；二级间接：$256^2 \times 1\text{KB} = 64\text{MB}$；三级间接：$256^3 \times 1\text{KB} = 16\text{GB}$。
- 最大文件长度 $= 16\text{GB} + 64\text{MB} + 256\text{KB} + 10\text{KB}$。

逻辑块所在区间与访问磁盘次数的关系（**inode** 已在内存）：以上参数下，逻辑块号（从 0 开始）与访问磁盘块数的对应关系是

逻辑块号范围	所属区域	需要读的磁盘块数
0 ~ 9	直接地址	1（只读目标数据块）
10 ~ 265	一级间接	2（索引块 + 数据块）
266 ~ 65801	二级间接	3（二级索引块 + 一级索引块 + 数据块）
65802 及以上	三级间接	4（三级 + 二级 + 一级索引块 + 数据块）

其中二级间接起始块号 $= 10 + 256 = 266$，二级间接可覆盖 $256^2 = 65536$ 块，故其末块号 $= 266 + 65536 - 1 = 65801$。

第三个算例（对应真题的偏移量定位思路）。设 $B = 1\text{KB}$、$p = 4\text{B}$、直接 10 项、一/二级间接各 1 项，inode 已在内存。求偏移量 1234 与 307400 处所在磁盘块各需访问几个磁盘块。

- 偏移量 1234：$\lfloor 1234/1024 \rfloor = 1$，即第 1 号逻辑块，落在直接地址区，只需读 **1** 个磁盘块。
- 偏移量 307400：$\lfloor 307400/1024 \rfloor = 300$，直接区覆盖 0~9，一级间接覆盖 $10 \sim 10+256-1 = 265$，300 落在二级间接区，需要读二级索引块、一级索引块、目标数据块共 **3** 个磁盘块。

计算时最容易错的三处

1. 块号从 **0** 还是从 **1** 开始。题目说「第 3 个逻辑块（块号从 1 开始）」时，要先减 1 再判断落在哪个区。统考题目一般会明确说明起始编号，没说明时按题目中已出现的例子推断，不要凭习惯。
2. 索引块不计入文件数据长度。求「单个文件最大长度」只累加数据块；求「占用几个索引块」才去数索引块。
3. 把「地址项个数」当成「索引块个数」。一个一级间接地址项对应一个索引块，但这个索引块里装的是 n 个数据块地址。

（五）inode 位图与数据块位图

磁盘上需要记录两种「哪些编号还能用」的信息，UNIX 风格的文件系统用两张位图分开管理：

- **inode** 位图（**inode bitmap**）：每一位对应 inode 区中的一个 inode，标记它是否已被分配。创建一个文件时先查 inode 位图找一个空闲 inode。
- 数据块位图（**block bitmap**）：每一位对应数据区中的一个磁盘块，标记它是否已被分配。为文件分配数据块时查它。

两张位图都存放在磁盘的固定位置，并在使用时按需把相关块读入内存修改，再在适当时机写回。因为一位只表示一个对象，位图占用空间极小、可常驻内存，是「0 表示空闲、1 表示已分配」的常用约定（注意不同教材符号可能相反，做题以题目给的约定为准）。

inode 编号到磁盘块的换算

设磁盘块大小 B，一个 inode 占 s 字节，inode 区起始块号为 b_0，则每个盘块可容纳的 inode 数

$$m = \left\lfloor \frac{B}{s} \right\rfloor$$

inode 编号为 k（从 0 开始编号）的 inode，位于

$$\text{盘块号} = b_0 + \left\lfloor \frac{k}{m} \right\rfloor, \qquad \text{块内偏移} = k \bmod m$$

算例：$B = 4\text{KB}$，$s = 16\text{B}$，inode 区起始块号 6，则 $m = 4096/16 = 256$。编号 1314 的 inode：$\lfloor 1314/256 \rfloor = 5$，$1314 \bmod 256 = 34$，所以它位于 $6 + 5 = 11$ 号磁盘块，是该盘块中的第 34 个 inode。

追问：需要只把这一个 **inode** 读入内存吗？不行。磁盘 I/O 以「块」为最小单位，无法只读块内某一部分，所以必须把整个 **11** 号块读入内存，再从中取用第 34 个 inode。这也是内存 inode 缓存存在的理由——同一块中其他 inode 后续被访问时可直接命中缓存。

二、文件的操作

文件的基本操作可以分成两类：不涉及打开的「目录级操作」（建立、删除）和基于打开表的「句柄级操作」（打开、关闭、读、写）。

（一）建立文件

`create(path)` 在指定路径下建立一个空文件，可以同时指定文件类型与访问权限。步骤如下：

1. 查询磁盘的 inode 位图（或 inode 表），为该文件分配一个空闲 **inode**，在其中登记文件类型、访问权限、创建时间等信息；
2. 打开 `path` 所指示的父目录，在目录中新增一个目录项，目录项的文件名取自 `path` 的最后一部分（分量名），文件名关联到刚分配的 inode 编号上。

（二）删除文件

`delete(path)` 删除指定路径下的目录项；若该文件已无其他链接，还要回收文件实体。步骤如下：

1. 按路径找到目标文件的目录项，进而找到 inode；
2. 根据 inode 中的访问权限判断「删除」操作是否允许（文件保护）；允许则读取其链接计数：
   - 若链接计数为 1，说明这是最后一条链接，释放该文件占用的所有磁盘块（修改数据块位图），同时释放磁盘 **inode**（修改 inode 位图）；
   - 若链接计数大于 1，则只把它减 1，文件数据保留（其他硬链接仍然有效）；
3. 在文件所在目录中删除对应的目录项。

删除文件不等于删除目录

2013 年真题考过：用户在删除某文件的过程中，操作系统不可能执行的操作是「删除此文件所在的目录」。删除文件只删除该文件自己的目录项，以及在没有剩余链接、没有进程打开它时回收其 inode 和数据块。目录还包含其他文件，绝不能被连带删除。

另外，删除文件也不必去删除该文件的「快捷方式」（符号链接）。符号链接是独立的文件对象，系统并不保证它存在，也不能假定它必须同步删除（2021 年真题考点）。

（三）打开文件表与文件描述符

`open(path, mode)` 打开名为 `path` 的文件，成功时返回一个整数——文件描述符（**file descriptor, fd**）。

open 返回的到底是什么

`open` 返回的是「本进程的文件描述符表」中的表项下标，不是文件内容、不是缓冲区指针、也不是文件大小。

打开的本质是：系统检索到指定文件的目录项后，把该文件的元数据（**inode/FCB**）从外存复制到内存，并在系统打开文件表和进程打开文件表中登记表项，然后把这个表项的索引号作为 fd 返回给用户。

此后的 `read`、`write`、`close` 全部以 fd 为参数，不再使用文件名，也就不再做按名查找。所以 2024 年真题问「哪个系统调用的实现中包含文件按名查找功能」，答案是 `open()`。

两级打开文件表的层次结构：

层次	位置	内容	特点
系统级打开文件表	内核，全系统一张	每个已打开文件的 inode 副本、文件状态、访问权限、打开计数	一个文件只对应一个表项；多个进程打开同一文件时共享它
进程级打开文件表（文件描述符表）	内核，每个进程一张，PCB 中记录其起始地址	fd、读写指针（文件偏移量）、访问模式、指向系统表项的指针	每个进程一张，各自独立

`open` 的完整处理过程（进程 P1 首次以只读方式打开 `/root/dir1/c.mp4`）：

1. 路径查找：根据路径逐级查找目录。若目录缓存未命中，就把目录文件所在盘块读入内存，逐层比较分量名，直到找到目标文件的目录项。这一阶段只用文件名。
2. 定位 **inode**：从目录项取出 inode 编号，去系统打开文件表中检索该 inode 是否已在内存：
   - 未命中：从磁盘把该 inode（所在的整个盘块）载入内核缓冲区；检查访问权限，若越权则打开失败；若不越权，在系统级打开文件表中新增一个表项，打开计数置 1，并记录内存 inode 的位置；
   - 命中：同样检查访问权限，通过后打开计数 **+1**，不重复读盘。
3. 登记进程表：在进程 P1 的进程打开文件表中新增一个表项，记录对应的系统打开文件表项索引、访问模式和初始读写指针。
4. 返回 **fd**：把该进程表项的下标作为 fd 返回给用户进程。

三条几乎每年都考的结论

1. 一个文件在系统打开文件表中只有一个表项（无论多少进程打开它）；但每个进程的打开文件表中都会各有一项，且不同进程的 fd 数值可以不同。
2. 读写指针记录在各进程自己的打开文件表中，所以多个进程读同一个文件时互不干扰、各读各的位置（2017 年真题：f1、f2 是硬链接，两个进程分别打开，读写指针位置不一定相同，但它们共享同一个内存索引节点）。
3. 各进程的访问权限可以不同：进程 P1 以只读打开、进程 P2 以读写打开同一个文件是完全合法的，各自受自己申请的模式约束。

open 不一定产生磁盘 I/O

若文件已被其他进程打开、其 inode 已在内存，则本次 `open` 只需在系统打开文件表中把打开计数 +1，并

新建一个进程表项，不需要读磁盘。因此「open 必然引起文件 I/O」是错的。

（四）关闭文件

`close(fd)` 成功返回 0，失败返回 −1。步骤：

1. 根据 fd 找到进程打开文件表中对应的表项并删除；
2. 把系统打开文件表中该表项的打开计数 **−1**：
   - 若减后不为 0，说明还有其他进程在使用，直接返回；
   - 若减后为 0，删除该系统表项；若该内存 inode 被修改过，则按操作系统策略写回外存（若未修改则直接释放内存 inode）。

关闭不等于删除。关闭只做「把内存中的控制信息同步回外存、释放内存中的打开表项」，它不会删除目录项、不会释放外存 inode、也不会修改磁盘 inode 中的链接计数（2023 年真题：文件 F 仅被进程 P 打开，P 关闭 F 时系统需要完成的是「释放 F 的索引节点所占的内存空间」）。

（五）读文件与写文件

`read(fd, buf, count)` 从文件描述符对应的文件中读数据到用户缓冲区，返回实际读到的字节数。处理过程：

1. 用户程序在用户态调用 `read`，通过陷入（trap）进入内核态；
2. 根据 fd 查进程打开文件表，按访问模式判断是否允许读；
3. 允许则经系统打开文件表找到内存 inode，依据读写指针位置 + **count** 确定要读的逻辑块区间，再由 inode 中的索引信息把逻辑块号映射为物理块号，把这些盘块读入内核缓冲区；
4. 从内核缓冲区把所需那部分数据复制到用户空间 `buf`；
5. 返回实际读取的字节数，并相应推进读写指针。

`write(fd, buf, count)` 的处理过程与之对称：

1. 陷入内核；2. 按 fd 检查是否允许写（只读打开时会失败）；3. 由读写指针和 inode 定位目标盘块的物理位置，先把该盘块读入内核缓冲区，用 `buf` 中的数据覆盖缓冲区中的指定区域，再把该盘块写回磁盘；4. 返回实际写入的字节数。

为什么读/写都要经过内核缓冲区

磁盘是块设备，读写最小单位是块；而内存可以按字节随机访问。用户想改一个字节，也必须先把该字节所在的整块读出来、在内存里改、再整块写回。

另外，中断服务程序是在当时占用 **CPU** 的那个进程的地址空间中执行的，它无法直接把数据写进另一个进程的用户空间，所以必须用「映射到每个进程内核空间」的内核缓冲区作为中转。做题时若问「能否直接把数据从磁盘读到用户空间」，答案是不能（除非使用特殊的直接 I/O 机制）。

本知识点的统考考法

- **2012-28**：`read` 系统调用的参数包含文件描述符而不包含文件名；数据不在内存时进程进入睡眠等待；系统调用导致用户态 → 核心态切换。
- **2014-29**：文件首次被打开的过程中，操作系统做的是「将文件控制块读到内存中」，而不是把文件内容读入内存。
- **2020-23 / 2017-31 / 2023-31**：系统打开文件表只有一项、进程表各自一份、关闭只减计数。
- **2024-29**：只有 `open()` 包含按名查找。

- 大题常以「参数已读入内存，访问第 i 个逻辑块需要几次磁盘 I/O」的形式出现，答法见后文物理结构一节。

三、文件的保护

文件保护解决的是谁能以什么方式访问文件的问题。首先要区分两类机制：口令保护与加密保护是为了防止文件被他人存取或窃取；访问控制是为了控制用户对文件的访问方式。

（一）可控制的访问类型

- 读：从文件中读数据；
- 写：向文件中写数据；
- 执行：把文件装入内存并执行；
- 添加（**append**）：在文件末尾追加新信息；
- 删除：删除文件，释放空间；
- 列表清单：列出文件名和文件属性。

重命名、复制、编辑等高层功能可以由系统程序调用底层系统调用来实现，因此保护只需在底层提供。例如复制文件可以由一系列读请求完成，所以具有读权限的用户同时也就具有了复制、打印权限。

写模式与添加模式的区别

以写模式打开文件时会立即把文件截断为 **0** 字节，原有内容永久丢失，即使之后一个字节都没写；以添加模式打开则保留原有内容，读写指针初始位置在文件末尾，新数据自动追加。

（二）访问控制矩阵与访问控制列表

- 访问控制矩阵（**Access Control Matrix**）：行是用户、列是文件，矩阵元素表示该用户对该文件的访问权限。优点是用户与文件权限的对应关系记录得非常清晰；缺点是占用存储空间太多，且它只起记录权限的作用，无法保证文件系统本身不受损。
- 访问控制列表（**Access Control List, ACL**）：在每个文件和目录的 **FCB**（或 **inode**）中增加一张表，记录各个用户可以对该文件执行哪些操作。这是把访问控制矩阵「按列压缩」得到的稀疏表示。
  - 优点：灵活，可实现复杂的访问方式；
  - 缺点：ACL 的长度无法预计，可能导致复杂的空间管理。

为了控制 ACL 的规模，引入了精简的访问列表：以「组」为单位标记各类用户可以执行哪些操作。典型的三类用户是：

用户类别	含义
拥有者（owner）	创建该文件的用户
组（group）	一组需要共享该文件且具有类似访问需求的用户
其他（other）	系统内的其他用户

当某用户请求访问文件时，系统检查该用户所属类别是否有相应权限。系统需要管理分组信息，并把文件主名、所属组名存放在该文件的 FCB 中。

权限位不能用「每类用户 2 位」表示

2017 年真题考过：4 类用户 × 5 种访问权限，若用二进制位串表示权限，至少需要 $4 \times 5 = 20$ 位。

原因是：一个用户对同一个文件可以同时拥有多种权限（既能读又能写又能执行），所以必须为「每一类用户的每一种权限」各留一个二进制位。把权限压缩成 2 位、认为一种用户只能选一种权限，是典型错误。

（三）传统的权限位 rwx

UNIX 风格文件系统用 9 个二进制位表示权限，分成三组，分别对应文件主（**u**）、同组用户（**g**）、其他用户（**o**）：

位	含义（普通文件）	含义（目录）
r 读	可以读取文件内容	可以列出目录中的文件名
w 写	可以修改文件内容	可以在目录中创建、删除、重命名文件
x 执行	可以把文件装入内存并执行	可以进入该目录（作为路径分量被解析）

用 `ls -l` 看到的 `-rwxr-xr--` 就是从高到低依次描述 u、g、o 三组权限。

访问权限 = 用户访问权限 ∩ 文件属性

文件访问控制由用户访问权限和文件属性共同决定，而不是由用户优先级决定。用户打开文件时申请的权限不能超过文件允许的权限；打开成功后，用户对文件的操作不能超过自己打开时申请的权限。这两条构成了访问控制的两道关卡。

（四）口令保护与加密保护

- 口令（**password**）保护：为文件建立 FCB 的同时附上一个口令，并告知允许共享的其他用户；用户请求访问时须提供口令，系统与 FCB 中存储的口令比对。
  - 优点：保存口令的空间开销小，验证口令的时间开销也很小；
  - 缺点：正确的口令就存放在系统内部，不够安全。
- 加密（**cryptography**）保护：用某个「密码」对文件内容本身进行加密，访问文件时须提供正确的密码才能正确解密。
  - 优点：保密性强，不需要在系统中存储密码；
  - 缺点：编码/译码（加密/解密）需要花费一定时间，且对文件的控制不如系统实现的访问控制灵活。

目录也要保护

对多级目录结构来说，不仅要保护单个文件，还要保护子目录内的文件，即需要目录保护机制。目录操作（搜索、创建、删除目录项）与文件操作不同，因此需要不同的保护机制。

四、文件的逻辑结构

文件的逻辑结构是从用户观点出发看到的文件组织形式，即文件内部这些逻辑记录是如何被组织起来构成逻辑文件的，它独立于文件的物理特性，用户看不见物理存放在哪。

对逻辑结构的基本要求：有利于提高检索速度、方便增删改查、降低存储费用（尽量不要求大片连续存储空间）。

（一）无结构文件与有结构文件

- 无结构文件（流式文件）：文件内部是一系列二进制流或字符流，以字节为单位组织，如`.txt`文件。访问时只能通过读写指针穷举搜索。适用于源程序、可执行文件、库函数等。UNIX 系统中绝大多数文件都是流式文件。
- 有结构文件（记录式文件）：由一组相似的记录组成，每条记录由若干数据项组成，通常有一个数据项作为关键字。按记录长度是否相等分为：
  - 定长记录：检索速度快，第 i 条记录的地址可直接算出，便于处理和修改，广泛用于数据处理；
  - 变长记录：检索只能顺序进行，速度慢，也不便于处理修改，但能节省空间。

顺序文件的随机存取条件

$$\text{第}i\text{ 条记录首地址} = i \times L \quad (i\text{ 从0 起}, L\text{ 为定长记录长度})$$

只有当「定长记录 + 物理上顺序存储」同时成立时，顺序文件才支持随机存取；再保证记录按关键字排序（顺序结构），才支持折半查找等快速检索。变长记录无论怎么存，都无法随机存取；物理上是链式存储时，也无法随机存取。

（二）顺序文件

记录一个接一个顺序排列，记录可定长也可变长，物理上可顺序存储或链式存储。按记录顺序与关键字的关系分为：

- 串结构：记录顺序与关键字无关，通常按存入时间决定顺序。检索某记录必须从头顺序查找，较费时。
- 顺序结构：记录按关键字顺序排列。定长时可折半查找，检索效率显著提高。

适用场合：对单条记录的增删改查困难、性能差，因此适合批量操作（例如磁带这类顺序存储设备上每次读写一大批数据时，顺序文件的存取效率最高）。

索引文件

变长记录文件要找到第 i 条记录必须先顺序读前 $i-1$ 条，为了解决这个矛盾，为主文件的每条记录在索引表中设一个表项，表项内容包括指向该记录的指针（逻辑起始地址）和记录长度。

- 索引表本身是定长记录的顺序文件，所以可以快速定位第 i 个索引项；
- 若索引表中的关键字按顺序排列，还支持按关键字折半查找；
- 每增加/删除一条记录都要修改索引表；
- 可以用不同的数据项建立多张索引表（如学生信息表可分别按「学号」「姓名」建索引），这就是数据库索引的思想。

代价：若每条记录平均只有 8B，而每个索引项占 32B，索引表会比文件内容本身还大数倍，存储空间利用率极低。

索引顺序文件

索引顺序文件是索引文件与顺序文件思想的结合：为文件建立一张索引表，但不是每条记录对应一个索引项，而是一组记录对应一个索引项。每个分组内部就是一个顺序文件（组内不必按关键字排序），索引表与各分组顺序文件（主文件）共同构成索引顺序文件。查找时先在索引表中找组，再

在组内顺序找记录。

索引顺序文件的平均查找次数

设共 n 条记录，平均分成 i 组，每组 n/i 条：

- 纯顺序查找：平均 $\frac{n}{2}$ 次；
- 索引顺序查找：组间平均 $\frac{i}{2}$ 次 + 组内平均 $\frac{n/i}{2}$ 次。

求最优分组，令 $f(i)=\frac{i}{2}+\frac{n}{2i}$ 最小，得 $i=\sqrt{n}$ 时总查找次数约为

$$\frac{\sqrt{n}}{2}+\frac{\sqrt{n}}{2}=\sqrt{n}$$

例如 $n=10^4$ 时，顺序查找平均 5000 次，索引顺序查找平均约 100 次。

多级索引顺序文件：$n=10^6$ 条记录时，先建低级索引表（每 100 条一组，共 10000 个表项），再对这 10000 个定长表项按每 100 个一组建顶级索引表（共 100 个表项）。

建 K 级索引的最优分组与平均查找次数

为 N 条记录的文件建 K 级索引，最优分组是每组 $N^{1/(K+1)}$ 条记录，检索一条记录的平均查找次数为

$$\frac{K+1}{2}\cdot N^{\frac{1}{K+1}}$$

例如 $N=10^6$ 建 2 级索引：最优每组 $10^{6/3}=100$ 条，平均查找次数 $=\frac{100}{2}\times 3=150$ 次。

（三）直接文件或散列文件

给定记录的键值，或通过散列（**Hash**）函数转换后的键值，直接决定记录的物理地址。

- 优点：存取速度很高，不需要顺序查找，也不需要维护庞大的索引表；
- 缺点：会引起冲突（不同关键字的散列函数值相同），需要解决冲突；散列函数设计和空间管理相对复杂。

散列思想同样适合磁盘块的分配与回收：基于 Hash 快速定位空闲物理块，分配效率高；回收时只需标记空闲，后续可快速复用，碎片问题相对不突出。

（四）各逻辑结构的查找效率对比

逻辑结构	记录长度	查找方式	平均查找次数	存储额外开销
顺序文件（串结构）	定长/变长	从头顺序扫描	$n/2$	无
顺序文件（顺序结构）	定长	折半查找	$\log_2 n$	无
索引文件	定长/变长	查索引表	索引表内 $O(1)$~$O(\log n)$，变长索引表内需顺序找	每记录一个索引项，开销大
索引顺序文件	定长/变长	组间索引 + 组内顺序	约 $\sqrt{n}$	每组一个索引项，开销较小
直接/散列文件	任意	散列函数直接定位	接近 $O(1)$	需要处理冲突的空间

五、文件的物理结构

文件的物理结构（存储结构）指系统把文件存放在外存上所形成的存储组织形式，用户看不见。它研究两件事：一是文件的分配方式（对磁盘非空闲块的管理），二是文件存储空间管理（对磁盘空闲块的管理，见后文）。

基础约定：

- 磁盘被划分为一个个磁盘块（物理块），内存与磁盘之间的数据交换以「块」为单位；
- 文件的逻辑地址空间也被划分为一个个文件块，因此文件逻辑地址可表示为「逻辑块号 + 块内地址」；
- Windows 中称「簇」，UNIX 中称「磁盘块」；簇是文件的分配单位，扇区是磁盘上最小的物理单元。

两类「随机存取」要分清

- 逻辑结构支持随机存取：能根据记录号直接算出记录对应的逻辑地址（逻辑块号 + 块内地址）。
- 物理结构支持随机存取：能根据逻辑地址直接算出记录对应的物理地址。

「逻辑上支持随机存取」不等于「物理上支持随机存取」——顺序文件是定长记录且物理顺序存放时两者才同时成立。

（一）连续分配

结构：每个文件在磁盘上占有一组连续的块，逻辑上相邻的块物理上也相邻。

地址转换：用户给出逻辑块号，系统找到该文件 FCB 中的起始块号，则

$$\text{物理块号} = \text{起始块号} + \text{逻辑块号}$$

块内地址保持不变。因此 FCB/inode 的物理地址字段只需保存起始块号 + 总块数。

优缺点：

- 优点：支持顺序访问，也支持随机（直接）访问；访问时寻道数与寻道时间最小，顺序存取速度最快；实现简单。
- 缺点：文件长度不宜动态增加——文件末尾的盘块可能已分配给其他文件，一旦需要扩展就要大量移动盘块；反复增删后会产生外部碎片，存储空间利用率低；记录的插入、删除效率低下。

最大文件长度取决于 FCB 中「总块数」字段的位数和盘块大小：文件最大长度 = 最大块数 × 块大小。

连续分配的大题套路（插入/删除需要几次磁盘 I/O）

- 访问第 **3** 个逻辑块（首块已读入内存、FCB 已在内存）：只需计算物理块号后直接读入，**1** 次。
- 修改某块内容：读入 1 次 + 写回 1 次，共 **2** 次。
- 删除第 **2** 块（file-2）：为减少 I/O，选择把前面较少的块整体后移。若前面只有 file-0、file-1 两块，则需读入 file-1 写入 file-2 位置、读入 file-0 写入 file-1 位置，共 **4** 次 I/O；同时要修改 FCB 的起始块号 **+1**、总块数 **−1**。
- 在第 **4** 条记录前插入一条记录（7 条记录、前后都有空闲空间）：把前 3 条记录依次后移（3 次读 + 3 次写 = 6 次），再写入新记录（1 次），共 **7** 次 I/O；FCB 的起始块号要 −1，总块数 +1。

（二）链接分配

链接分配采用离散分配，可以为文件分配离散的磁盘块，分为隐式链接与显式链接。

*1.*隐式链接

结构：每个盘块（除最后一个）的尾部都有一个指向下一个盘块的指针。FCB 中保存起始块号与结束块号。

地址转换：从起始块号读入 0 号逻辑块，从它的尾部指针知道 1 号逻辑块的物理块号，再读入 1 号块……因此访问第 i 号逻辑块（从 **0** 开始）需要 $i+1$ 次磁盘 **I/O**。

优缺点：

- 优点：消除了外部碎片，提高了磁盘利用率；适合文件长度的动态增长（动态分配块）。
- 缺点：只支持顺序访问，不支持随机访问，查找效率低；指向下一块的指针本身要耗费少量存储空间；可靠性差——链中任一指针损坏，其后的数据全部丢失。

最大文件长度取决于：① 一个盘块中用多少位存放链接指针；② 盘块大小。例如盘块 1KB、指针占 2B，则每块可链 $\lceil 1024/2 \rceil$ 个关系，理论上限受块号编码位数约束。

可以引入「簇」：一个簇 = 若干个连续磁盘块，按簇顺序链接，可减少指针占用、加快查找；代价是增加内部碎片，而且分配空间时必须分配整数个簇（一个簇 4 块时，即使再要 1 块也必须分配整个簇）。

隐式链接的磁盘 I/O 计数（FCB 已在内存）

以 c.mp4 为例，7 个文件块 file-0 ~ file-6：

- 访问第 3 个逻辑块（file-2）：需读入 file-0、file-1 才能拿到 file-2 的块号，再读 file-2，共 **3** 次读盘。
- 修改 file-3 的内容：先读 file-0、file-1、file-2、file-3（4 次），再写回 file-3（1 次），共 **5** 次。
- 删除 file-2：读 file-0、file-1、file-2 拿到 file-3 的地址，修改 file-1 的指针后写回 file-1，共 **4** 次（file-2 的数据块标记为空闲不算磁盘 I/O）。
- 在第 4 条记录前插入：读前 3 块（3 次），把新记录写入空闲块并令其指针指向原第 4 块（1 次写），修改第 3 块的指针并写回（1 次写），共 **5** 次。

*2.*显式链接与文件分配表 *FAT*

结构：把所有「指向下一块」的指针从各盘块尾部抽离出来，集中存放在内存的一张链接表——文件分配表（**File Allocation Table, FAT**）中。FAT 的第 i 个表项存放「物理块 i 的下一个物理块号」。文件的第一块（起始块号）作为首地址写入 **FCB** 的物理地址字段，后续块通过查 FAT 得到。

FAT 的三个关键性质

1. 一个文件系统只设置一张 **FAT**，开机时被读入内存并常驻内存。
2. FAT 的各个表项在物理上连续存储，且每个表项长度相同，因此「物理块号」字段在表项中是隐含的（第 i 项就代表块 i）。
3. FAT 用到的数据结构是静态链表——用下标代替指针的链表。约定 `-1` 表示文件最后一块，`-2` 表示该块空闲。

   因此 **FAT** 也可以用于管理空闲磁盘块（2019 年真题：位图、空闲磁盘块链、FAT 都可以用于管理空闲磁盘块，而索引节点不行，索引节点描述的是已存在文件的元数据）。

地址转换：从 FCB 取起始块号，若目标逻辑块号 $i>0$，就在内存中查 FAT 依次得到第 i 块的物理块号。查 **FAT** 不需要读磁盘，因此访问某个逻辑块只需 **1** 次访盘取出记录。

优缺点：

- 优点：支持顺序访问，也支持随机访问（不必依次访问 0 ~ i 号逻辑块）；块号转换在内存中完成，比隐式链接快很多；不产生外部碎片，文件扩展方便。
- 缺点：FAT 需要占用一定的存储空间，而且必须是连续空间。

FAT 表大小的计算（完整算例）：

- 磁盘/分区共 N 个盘块，则 FAT 需要 N 个表项；
- 表项位数 $w = \lceil \log_2 N \rceil$（因为要能表示 N 个块号，以及 `-1`、`-2` 等状态，实际实现常向上取整到一个字节的整数倍）；
- FAT 占用空间 $= N \times w$（bit）。

算例：某分区 1GB，盘块大小 4KB，则盘块数 $N = 1\text{GB}/4\text{KB} = 2^{30}/2^{12} = 2^{18} = 262144$ 个。表项需 $\lceil \log_2 262144 \rceil = 18$ 位，实际取 3B（或按 24 位）存储，FAT 大小约 $262144 \times 3\text{B} = 768\text{KB}$。

算例（换一组参数）：盘块大小 2KB，FAT 每个表项 2B，则表项能表示的块号范围是 $2^{16} = 65536$ 个块，该文件系统支持的最大文件长度约为 $65536 \times 2\textbf{KB} = 128\textbf{MB}$；若 FAT 表项为 4B，则块号范围 2^{32}，最大文件长度约为 $2^{32} \times 2\text{KB} = 8\text{TB}$。这说明 **FAT** 表项长度直接决定了单个文件的最大长度上限（也可能受分区容量限制，取二者中较小者）。

显式链接（**FAT** 常驻内存）的磁盘 **I/O** 计数

同样以 7 个文件块为例，**FAT** 已在内存：

- 访问第 3 个逻辑块：查 FCB 得起始块号，再查内存 FAT 即可定位，**1** 次访盘。
- 修改 file-3：读入 + 写回，**2** 次。
- 删除 file-2：**0** 次——只需修改内存 FAT 中关于该文件的链接关系（把前驱指针指向 file-3），并把 file-2 的表项置为 `-2`（空闲）；由于 FAT 常驻内存，这些修改不需要立即访盘。
- 插入一条记录作为第 4 条：也只需在内存 FAT 中改指针，**0** 次访盘（新数据块本身的写入除外）。

索引分配

实现思想：打开某个文件时，只需把该文件占用的盘块号调入内存即可，完全没必要把整个 FAT 调入内存。于是为每个文件建立一张索引表，第 i 个条目指向文件的第 i 个块，表中记录各逻辑块对应的物理块号。存放索引表的盘块称索引块，存放文件数据的盘块称数据块。

优缺点：

- 优点：支持随机访问（可直接从索引块中读取想要的盘块号）；没有外部碎片问题；文件扩展方便。
- 缺点：由于要为每个文件分配索引块，增加了存储空间开销；索引块太大会浪费空间，太小则无法支持大文件；访问文件可能需要多次磁盘 I/O（要先读索引块）。

按索引层次分为三种方案：

- 单级索引（一级索引）：inode 中放一个索引块地址，指向一个索引块，索引块中依次存放该文件所有数据块号。若 $B = 4\text{KB}$、地址项 4B，一个索引块可放 1024 个块号，对应 $1024 \times 4\text{KB} = 4\text{MB}$ 的文件。
- 多级索引：文件特别大、一个索引块装不下时，用「索引块指向索引块」的方式。一级索引指向的块中，每项再指向一个索引块（二级索引）。$B = 4\text{KB}$、$p = 4\text{B}$ 时，二级索引最多可管理 1024×1024 个数据块，最大文件 $= 1024^2 \times 4\text{KB} = 4\text{GB}$。

  - 优点的另一半：可以支持更大的文件，加快对大型文件的查找速度；
  - 缺点：随着索引级数增加，访问磁盘的次数也在变多。
  - 还有一种链接索引方案：把多个索引块用指针链接起来，每个索引块包含指向下一个索引块的指针。
- 混合索引（**UNIX** 混合索引）：把直接地址、一级间接、二级间接、三级间接混在同一个 **inode** 中，全面照顾各种大小的文件。小文件用直接地址即可直接从 inode 取盘块号；中型文件用一级间接；大型、超大型文件用二级、三级间接。

索引分配的最大文件长度通用公式（再强调一次）

$$S_{max} = \sum_{k} N_k \cdot n^k \cdot B, \qquad n = \left\lfloor \frac{B}{p} \right\rfloor$$

其中 N_k 是第 k 级地址项个数、B 是块大小、p 是地址项大小、$k = 0$ 表示直接地址。

以 2010 年真题为例：inode 有 7 个地址项（4 个直接 + 2 个一级间接 + 1 个二级间接），地址项 4B，索引块与数据块均为 256B。则 $n = 256/4 = 64$，

$$S_{max} = 4 \times 256\text{B} + 2 \times 64 \times 256\text{B} + 1 \times 64^2 \times 256\text{B} = 1\text{KB} + 32\text{KB} + 1024\text{KB} = 1057\text{KB}$$

注意：二级间接里的两个索引层本身占的是索引块，不计入文件数据长度。

（三）三种物理结构的对比

对比项	连续分配	链接分配（隐式）	链接分配（显式/**FAT**）	索引分配
分配单位	一组连续盘块	离散盘块	离散盘块	离散盘块
FCB/inode 需保存	起始块号 + 总块数	起始块号 + 结束块号	起始块号（首块）	索引块地址（多级）
逻辑地址转换	起始 + 逻辑块号	沿链逐块读指针	查内存 FAT	查索引块（可能多级）
随机访问	支持	不支持	支持	支持
顺序访问速度	最快（寻道最少）	慢	快（FAT 在内存）	较快
文件扩展	困难（需大量移动）	容易	容易	容易
外部碎片	有	无	无	无
额外空间开销	无	每块一个指针	一张 FAT（可常驻内存）	每文件至少一个索引块
访问第 i 块需读盘（FCB/inode 已在内存）	1 次	$i+1$ 次（从 0 开始）	1 次	1 + 间接级数次
典型/最佳场景	长度基本不变、要求随机读写速度高	长度常变、只要求顺序访问	长度常变、要求随机读写	长度常变、要求随机读写

访问第 i 个逻辑块需要几次磁盘 I/O ——一份速查表

前提：**FCB / inode** 已在内存（否则还要先读盘装入 inode）。

- 连续分配：**1** 次（直接算出物理块号）。
- 隐式链接：$i+1$ 次（i 从 0 开始计，必须逐块读指针）。

- 显式链接（**FAT** 在内存）：**1** 次。
- 索引分配（单级）：**2** 次（索引块 + 数据块）。
- 索引分配（混合索引）：直接区 **1** 次；一级间接 **2** 次；二级间接 **3** 次；三级间接 **4** 次。

若题目说明「FCB 尚未读入内存」，则在上述次数上再加 1 次读 inode/目录项的 I/O。

物理结构的统考考法

- **2009-28**：适合随机访问且易于文件扩展的是索引结构。连续结构可视为数组——随机访问好但扩展差；链式结构可视为链表——扩展好但随机访问差；索引结构两者兼顾。
- **2013-24**：为支持 CD-ROM 中视频文件的快速随机播放，连续结构性能最好（顺序读取介质、连续块寻道最少）。
- **2010-30 / 2015-29 / 2018-31 / 2022** 大题：混合索引的最大文件长度与逻辑块定位。
- **2016** 大题：FAT 相关计算（表项与最大文件长度、I/O 次数）。
- **2013-26**：与单个文件长度无关的是「索引接点的总数」（只影响文件个数）。

六、目录的基本概念

文件目录是一种数据结构，用于标识系统中的文件及其物理地址，供检索时使用。对目录管理的基本要求是：实现「按名存取」、提高目录检索速度、允许文件重名、支持文件共享。

目录与 FCB/inode 的关系有三种表述要保持一致：

- FCB 的有序集合称为文件目录，一个 **FCB** 就是一个目录项；
- 引入 inode 后，目录项 = 文件名 + **inode** 编号（指针），inode 中存放元数据；目录项描述文件在文件系统中的逻辑位置，inode 描述文件在存储设备上的物理位置；
- 目录本身也是一种文件（目录文件），它也有自己的 inode 和数据块，只不过数据块里存的是目录项而不是普通数据。

因此可以记住：普通文件 = **inode** + 数据块；目录文件 = **inode** + 目录块（目录块中记录了该目录下各文件的名字与 inode 编号）。

常用的目录实现方式：

实现方式	原理	特点
线性列表	用数组或链表存放目录项	实现简单；数组适合静态目录，链表适合频繁增删；检索效率低，需线性扫描
哈希表	用散列函数把文件名映射到索引位置	直接按名访问速度快，适合目录项动态变化；容易浪费空间、冲突处理复杂
键树（Trie）	按文件名字符逐层建树	对大量共享前缀的名字检索极快；实现较复杂，空间需额外优化

七、树形目录

（一）目录结构演进

- 单级目录结构：整个系统只有一张目录表，每个文件占一个目录项。实现了按名存取，但不允许文件重名，查找速度慢，不适用于多用户系统。
- 两级目录结构：分为主文件目录（**MFD**）与用户文件目录（**UFD**）。一个文件系统一张 MFD，一

个 MFD 表项对应一个用户；每个用户一张 UFD，记录该用户所有文件的 FCB。解决了多用户环境下的重名问题，并提高了检索速度，但缺乏灵活性，不能对文件分类。

- 树形（多级）目录结构：最常用。用户可以创建自己的子目录，形成层次结构。
  - 优点：方便对文件分类，层次清晰，可对不同用户赋予不同存取权限，能有效管理和保护文件；
  - 缺点：需要按路径逐级访问中间节点，增加了磁盘访问次数，影响查询速度；不便于实现文件共享（这是引入无环图目录结构与链接的原因）。
- 无环图目录结构：在树形目录基础上增加指向同一节点的有向边，形成有向无环图。可以为不同文件名指向同一文件，甚至指向同一目录。需要为每个共享节点设置共享计数器 **count**，删除时只让计数器减 1，只有减到 **0** 才真正删除节点。
  - 注意：共享文件不同于复制文件——各用户指向的是同一个文件，一个用户修改后其他用户都能看到。

（二）绝对路径、相对路径与当前工作目录

- 文件路径名是由各级目录名与文件名用分隔符（/）连接而成的字符串。
- 绝对路径：从根目录出发的路径，例如 `/照片/2015-08/自拍.jpg`。
- 当前工作目录（当前目录）：由系统为每个进程设置的一个目录，是相对路径的起点。Linux 中用 `.` 表示当前目录，`..` 表示父目录。
- 相对路径：从当前工作目录出发的路径，例如当前目录是 `/照片`时，`./2015-08/自拍.jpg`。

设置当前工作目录的主要作用是加快文件的检索速度（2010-31 真题），它既不节省外存/内存空间，也不直接提高已定位文件的读写传输速度。

（三）路径名解析过程

以绝对路径 `/照片/2015-08/自拍.jpg` 为例：

1. 读入根目录的目录表（目录文件所在盘块）；
2. 在根目录中查找分量名「照片」，得到「照片」目录文件的存放位置（inode 编号），据此读入「照片」目录的目录表——**1** 次读盘；
3. 在「照片」目录中查找「2015-08」，读入该目录的目录表——**1** 次读盘；
4. 在「2015-08」目录中查找「自拍.jpg」，得到该文件的 inode 编号，读出 inode 与数据——总共约 **3** 次读盘。

若把「照片」目录设置为当前工作目录（即把它的目录表留在内存中），用相对路径`./2015-08/自拍.jpg` 访问，就省去了第 1 ~ 2 步的读盘，磁盘 **I/O** 次数减少，访问效率提高。

路径解析中的易错点

1. 每一级中间目录都要读盘（除非命中目录缓存），所以路径越长，开销越大。这正是引入目录项缓存（**dentry cache**）的原因。
2. 从根目录开始的路径一定是绝对路径；但相对路径的写法不一定要带`./`，如 `2015-08/自拍.jpg` 也从当前目录出发。
3. 路径解析的对象是目录项（文件名 → **inode** 编号），不是 **inode** 本身；只有文件名匹配成功后，才需要把 inode 读入内存。

八、目录的操作

统考要求掌握的目录操作：

操作	说明
搜索	根据文件名搜索目录，找到对应目录项
创建文件	在所属目录中增加一个目录项
创建目录	在树形目录结构中创建新的用户文件目录或子目录
删除文件	在目录中删除相应的目录项
删除目录	两种方式：① 不删除非空目录——必须先删除目录中所有文件并递归删除所有子目录；② 可删除非空目录——目录中文件与子目录一并删除
显示目录	列出目录内容（文件名及属性）
修改目录	目录项中保存了某些文件属性，属性变化时须修改相应目录项（如重命名）
移动目录	需要修改相应的路径名
链接目录	通过链接操作让指定文件具有多个父目录，便于共享（树形目录下每个文件/目录只允许有一个父目录，不适合共享）

重名检查与目录项复用

- 创建文件或目录时，必须先检查同一目录内是否已有同名文件/目录，确定不重名后才能建立。不同目录下的文件可以重名，这正是树形目录的优势。
- 删除目录项后，线性列表实现中有三种处理空闲目录项的方法：① 把目录项标记为不再使用（简单，但搜索需跳过）；② 把空闲目录项加入空闲目录项列表（便于复用，需额外结构）；③ 把目录的最后一个目录项复制到该空位并缩短目录长度（目录保持紧凑，但涉及数据移动）。

九、硬链接与软链接

文件共享指系统允许多个用户（进程）共享同一份文件，系统中只需保留该共享文件的一份副本。统考只要求两种实现方式：基于索引节点的共享（硬链接）和基于符号链的共享（软链接）。

（一）硬链接（**Hard Link**）

本质：多个目录项指向同一个索引节点，即多个文件名对应同一份文件数据。UNIX 中用 `ln` 原文件新链接名建立。

- **inode** 与目录项：每个链接都是一个正常的目录项（文件名 + inode 编号），所有目录项的 inode 编号相同，指向同一个 inode；**file** 数据只有一份。
- 链接计数 **count**：inode 中记录该文件被多少个目录项直接指向。创建文件时 count = 1；每建立一个硬链接，count +1；删除一个目录项，count −1；只有 **count** 减到 **0** 时才真正释放文件数据与 **inode**。
- 跨文件系统：不能。inode 编号只在同一个文件系统内唯一，每个文件系统有各自的 inode 表，硬链接无法跨越文件系统。
- 指向性：直接指向 inode，是「同一实体的多个名字」。
- 原文件删除后的行为：删除其中一个链接不影响其他链接对文件的访问；只要 count 仍大于 0，文件数据就仍可正常读写。因此硬链接不会产生悬空指针。
- 访问开销：从目录项直接得到 inode，一次到位，访问速度快。

count 与「删除」的正确理解

在硬链接下，每个拥有链接的用户都可以「删除」文件，但这个删除只是 `count--`；只有当执行 `count--` 之后 **count == 0** 时，系统才真正删除该文件。这就是「硬链接保证原文件被删除后仍能通过其他链接访问数据」的机制。

（二）软链接（符号链接，Symbolic Link）

本质：创建一个 **LINK** 类型的新文件，该文件有自己独立的 **inode** 和数据块，其文件内容就是目标文件的路径名。访问这个符号链时，操作系统截获请求，按其中的路径名去查找目标文件，再对它进行操作。相当于 Windows 的快捷方式。UNIX 中用 `ln -s 原文件新链接名`建立。

- **inode** 与目录项：软链接是独立文件，有独立的 inode 和至少一个数据块（存放路径名），不与目标文件共享 inode。
- 链接计数：建立软链接不改变目标文件的链接计数；软链接自己的计数为 1，删除软链接不影响目标文件。（若再对该软链接建立硬链接，则按普通文件规则各自加 1。）
- 跨文件系统：可以。因为路径名本质上是一串字符，可以指向任意文件系统、甚至网络上的其他机器。
- 指向性：指向路径名，是「指向名字的另一个名字」，属于间接指向。
- 原文件删除后的行为：软链接仍然存在，但已成为无效链接（悬空链接），通过它访问会失败；此时可将符号链删除。若文件主删除原文件的同时有另一用户创建了同名路径的新文件，则 LINK 文件仍然「有效」，但访问到的已不是原来那个文件。
- 访问开销：每次访问都要按路径名逐级查找目录，可能多次读盘，开销大；此外每条符号链本身也要占用一个 inode 和磁盘空间。

硬链接与软链接完整对比

对比项	硬链接	软链接（符号链接）
本质	多个目录项指向同一个 inode	一个独立文件，内容是目标文件的路径名
是否有自己的 inode	没有，与目标文件共用	有，独立 inode 与数据块
目录项内容	文件名 + 目标文件的 inode 编号	文件名 + 自己的 inode 编号（数据块中存路径）
目标文件的链接计数	每建一个硬链接 **+1**	不变
能否跨文件系统	不能	能，还可跨网络
指向性	直接指向 inode（同一实体）	间接，指向路径名
删除原文件后	其他链接照常访问；count 归零才真正删除	变成悬空链接，访问失败
谁能删除「共享文件」	任一拥有硬链接的用户都可使其 count 减 1	只有文件主能删原文件，其他用户只能删自己的 LINK 文件
访问开销	小，直接到 inode	大，需多次读盘逐级解析路径
是否产生悬空指针	不会	会产生（需在访问失败后清理）

链接相关真题

- **2009-31**：F1 计数值为 1，先建软链接 F2、再建硬链接 F3，然后删除 F1。F2 是独立文件，计数恒为 1；F3 与 F1 共享 inode，F1 删除后 count 由 2 减为 1。答案：**F2** 与 **F3** 的引用计数值都是 **1**。
- **2017-31**：f1 的硬链接为 f2，两进程分别打开，得 fd1、fd2。则「f1 和 f2 共享同一个内存索引节点」「fd1 和 fd2 分别指向各自的用户打开文件表中的一项」正确；「读写指针位置保持相同」错误——两个独立的 `open` 建立各自的打开文件表项，读写位置互不影响（只有 `fork`/`dup` 共享同一表项时才联动）。
- **2020** 相关：文件共享、打开文件的相关判断。

十、文件系统的全局结构

（一）文件系统在外存（磁盘）中的结构

磁盘通常被划分为一个或多个分区，每个分区中可以有一个独立的文件系统。整个磁盘与一个分区的典型布局如下：

区域	位置/内容	说明
MBR（主引导记录）	磁盘 0 号扇区	负责引导计算机；其后紧随分区表，记录各分区的起始与结束地址，并标记活动分区
引导块（**boot block**）	每个分区的第一个块	MBR 读取活动分区的第一个扇区即引导块，由其中的程序负责加载并启动该分区中的操作系统。即使该分区没装可启动系统，这一区域也保留（Windows 中称分区引导扇区）
超级块（**super block**）	分区内固定位置	存储文件系统的所有关键信息，系统启动或首次挂载时被读入内存
空闲空间管理信息	紧随超级块	通常以位示图或空闲链表实现，记录未分配的磁盘块
inode 区（**i** 节点表）	空闲空间管理信息之后	每个文件对应一个 inode，完整描述文件的属性与数据块地址
根目录	inode 区之后	整个目录树的起点
数据区（文件和目录）	磁盘剩余空间	存放所有其他文件与目录的实际数据

超级块的典型字段

超级块是文件系统的核心元数据结构，常见字段包括：

- 校验区：区分文件系统类型、检测文件系统合法性；
- **inode** 区起始块号、**inode** 区总块数；
- **bitmap** 起始块号、**bitmap** 总块数（若用两张位图，则分别对应 **inode** 位图与数据块位图）；
- 数据区起始块号、数据区总块数；
- 其他：磁盘块大小、**inode** 大小、根目录的 **inode** 编号、空闲块数目及指针、空闲 inode 数目及指针等。

超级块的内容是在逻辑格式化（对卷的目录区初始化）时写入的。它坏掉，整个文件系统基本就报废了——所以实际系统中超级块常有多份备份。

由超级块参数推导文件系统能力（完整算例）

设磁盘块大小 4KB，一个 inode 占 16B，inode 区起始块号为 6、总块数为 16，bitmap 起始块号为 22、总块数为 16。则：

1. 每个盘块可放的 **inode** 数 $= 4096/16 = 256$ 个；inode 区共 16 块，所以该文件系统最多可容纳 $256 \times 16 = 4096$ 个文件。
2. **inode** 编号为 **1314** 的 **inode** 位置：$\lfloor 1314/256 \rfloor = 5$，$1314 \bmod 256 = 34$，即位于 $6 + 5 = 11$ 号磁盘块，是该块中的第 34 个 inode。

3. 只读一个 **inode** 可以吗：不行，磁盘 I/O 以块为单位，必须把 11 号块整体读入内存，再取其中第 34 个。
4. 数据区最多有多少个磁盘块：由位图决定。若位图共 16 块、每块 4KB，则位图总位数为 $16 \times 4096 \times 8 = 524288$ 位，故数据区最多可管理 524288 个磁盘块。

（二）文件系统在内存中的结构

内存中的结构在挂载时加载、运行期间更新、卸载时释放：

内存结构	内容与作用
内存中的挂载表（**mount table**）	记录每个已挂载文件系统分区的信息：设备标识、挂载点、文件系统类型、挂载选项等
目录项缓存（**dentry cache**）	缓存最近访问过的目录内容（目录名与 inode 号的映射），加快路径解析，减少重复读盘
索引节点缓存（**inode cache**）	缓存最近访问过的 inode，多个进程可共享同一内存 inode
系统级打开文件表	每个当前打开的文件一个 FCB/inode 副本，含文件状态、访问权限、读写指针、打开计数等全局信息
进程级打开文件表	每个进程一张，含文件描述符、读写指针、访问模式，以及指向系统级打开文件表对应表项的指针；其起始地址记录在进程 PCB 中
页面高速缓存（**Page Cache**）	缓存文件的具体数据内容，减少磁盘 I/O；配合延迟写（**write-back**）把多次小写合并

内存结构 ≠ 外存结构

- 外存（磁盘）中有：MBR/引导块、超级块、空闲空间管理信息（位图/空闲链表）、inode 区、根目录、数据区。
- 内存中有：挂载表、目录项缓存、inode 缓存、系统/进程打开文件表、页高速缓存。

考纲条目里的「文件系统在外存中的结构」「文件系统在内存中的结构」，问的就是这两组结构不要混淆。

十一、外存空闲空间管理方法

先明确两个基本概念：

- 卷（**volume**）：文件系统的分区（如 C 卷、D 卷）。它可以是磁盘的一部分，也可以是整个磁盘或 RAID 磁盘阵列。从本质上讲，卷是逻辑磁盘，卷中的扇区在物理设备上不必连续。
- 卷的组成：目录区 + 文件区。目录区存放文件目录信息（FCB）和用于磁盘存储空间管理的信息；文件区存放文件数据。对逻辑卷格式化（初始化）的实质就是对卷内目录区进行初始化——建立空闲空间管理表格、超级块等。

（一）空闲表法

为所有空闲区建立一张空闲表，每个表项记录空闲区的第一个块号和该空闲区的块数，按磁盘块号顺序排列。这属于连续分配的思路。

- 分配：与内存动态分区分配类似，可采用首次适应、最佳适应、最坏适应等算法。系统为新文件分配空间时，顺序检索空闲表各表项，找到第一个大小满足要求的空闲区分配给它，并修改空闲表。

- 回收：同样类似动态分区回收，需分四种情况处理并合并相邻的空闲区：回收区前后都空闲、前后都不空闲、只有前面空闲、只有后面空闲。
- 适用场合：仅当空闲区数量较少时才有效。若存在大量小空闲区，空闲表会变得很大，查询效率很低，因此不适合大型文件系统。适用于建立连续文件。

外存为什么还用连续分配

内存分配很少采用连续分配，但在外存的一些特殊场景中，连续分配具有较高分配速度、能减少磁盘 I/O 频率与寻道时间：

- 磁盘的对换区，为追求换入换出速度采用连续分配；
- 文件系统中，小文件采用连续组织方式分配相邻盘块；
- 多媒体文件（MP3/MP4）为减少寻道时间采用连续组织方式。

（二）空闲链表法

把所有空闲盘区拉成一条空闲链，按链中基本元素的不同分为两种。

空闲盘块链：以单个盘块为单位组成链，每个空闲盘块中存放指向下一个空闲盘块的指针。

- 文件系统中需保存链头、链尾指针；
- 分配：若某文件申请 K 个盘块，从链头开始依次摘下 K 个盘块分配，并修改链头指针；
- 回收：回收的盘块依次挂到链尾，并修改链尾指针（每个回收块都要先读入内存修改指针再写回，磁盘 **I/O** 开销大）；
- 适用：适用于离散分配的物理结构；
- 优点：单个盘块的分配与回收操作极简单；缺点：为一个文件分配多个盘块需多次访问链表，效率低，且管理粒度细导致空闲盘块链通常很长。
- 最难获得连续的空闲磁盘空间。

空闲盘区链：以空闲盘区（一组连续盘块）为单位链接，每个盘区记录本盘区的盘块数和指向下一盘区的指针。

- 分配：若申请 K 个盘块，可采用首次适应等算法从链头开始检索，找到满足要求的空闲盘区分配；也可把不同盘区的盘块同时分配给一个文件，分配后可能要修改链指针与盘区大小；
- 回收：若回收区与某空闲盘区物理相邻则合并，否则作为单独空闲盘区挂到链尾；
- 适用：既适用于离散分配也适用于连续分配，为一个文件分配多个盘块时效率更高；
- 优点：分配回收效率较高，链较短；缺点：需处理盘区的分割与合并，过程较复杂。
- 最容易获得连续的空闲磁盘空间。

（三）位示图法

位示图（**bitmap**）用二进制的一位表示一个盘块的使用状态。统考沿用的约定是：「**0**」表示盘块空闲，「**1**」表示盘块已分配。若磁盘块共有 n 个，则位示图至少要有 n 位。

- 分配：顺序扫描位示图，找到 K 个「0」位（相邻或不相邻皆可），把这些位对应的盘块分配给文件，并把这些位置 **1**。
- 回收：根据回收的盘块号，把位示图中对应的位置 **0** 即可，不需要像链表那样修改指针，因此回收引起的磁盘 I/O 很少。
- 优点：查找高效，能快速找到单个或一组相邻的空闲盘块，便于实现连续分配；结构紧凑，占用空间极小，可常驻内存，避免频繁访问磁盘。占用空间与当前空闲块数量无关（只要磁盘容

量和块大小固定，位图长度就固定）。

- 缺点：位图大小随磁盘容量增大而增大，更适合小型或中等规模系统。

*1.*位示图字号/位号与盘块号的相互换算

设位示图有 m 行、每行 n 位（n 就是「字号」这一行的位数，实际系统里常常就是一个机器字长）。两种编号约定必须分清：

从 1 开始编号（王道教材默认约定）

行号 i、列号（位号）j 均从 **1** 开始，每行 n 位，则：

$$\text{盘块号 } b = n(i-1) + j$$

反向（由盘块号求行列号）：

$$i = (b-1)\ \text{DIV}\ n + 1, \qquad j = (b-1)\ \text{MOD}\ n + 1$$

其中 DIV 是整除，MOD 是取余。

从 0 开始编号

行号 i、位号 j 均从 **0** 开始，每行 n 位，则：

$$\text{盘块号 } b = i \times n + j$$

反向：

$$i = \left\lfloor \frac{b}{n} \right\rfloor, \qquad j = b \bmod n$$

做题时先看题目给的编号起点，再选公式。统考题目一般会明确写「盘块和块内字节均从 0 开始编号」之类的话。

算例 1：由盘块号求位示图位置（从 1 开始）

用 8 个字（字长 32 位）组成的位示图管理内存，共 8 行、32 列，行号与列号均从 1 开始，求块号 100 对应的位置。

$$i = (100-1)\ \text{DIV}\ 32 + 1 = 99\ \text{DIV}\ 32 + 1 = 3 + 1 = 4$$

$$j = (100-1)\ \text{MOD}\ 32 + 1 = 99\ \text{MOD}\ 32 + 1 = 3 + 1 = 4$$

即字号为 **4**、位号为 **4**。

算例 2：位示图所在盘块与块内字节（从 0 开始）

文件系统用位图法表示磁盘空间分配情况，位图存于磁盘的 **32～127** 号块中，每个盘块占 1024 字节，盘块和块内字节均从 **0** 开始编号。要释放的盘块号为 **409612**，求位图中要修改的位所在的盘块号和块内字节序号。

第一步，一个位图块能表示的位数：$1024 \times 8 = 8192$ 位。

第二步，位号即盘块号 409612（0 开始编号时直接对应）：

$$409612 \div 8192 = 50\ \text{余} 12$$

说明该位位于位图的第 **50** 个位图块（从 0 开始）。

第三步，换算为实际盘块号：位图起始于 32 号块，故

$$32 + 50 = 82$$

第四步，块内偏移 12 位，对应字节号：

$$\left\lfloor \frac{12}{8} \right\rfloor = 1$$

所以应修改 **82** 号盘块的第 **1** 个字节（该字节中的第 $12 \bmod 8 = 4$ 位，即从高位起的第 5 位，按位序编号约定略有差异，题目只要求到字节序号）。

算例 3：位示图自身占多大空间

- 容量 10GB 的磁盘分区，簇大小 4KB，用一位标识一个簇是否被分配，则簇数 $= 10 \times 2^{30}/2^{12} = 10 \times 2^{18} = 2621440$ 个，位图大小 $= 2621440/8 = 327680\text{B} = 320\text{KB}$；位图本身也按 4KB 的簇存放，故需 $320\text{KB}/4\text{KB} = \mathbf{80}$ 个簇。
- 页式存储管理中，页大小 4KB、物理内存 16GB，页框数 $= 2^{34}/2^{12} = 2^{22}$，位图大小 $= 2^{22}/8 = 2^{19}\text{B} = \mathbf{512}\text{KB}$。

这两个例子说明：位图大小 = 被管理对象总数 / **8** 字节。

位示图法最常错的两处

1. 位示图中 **0** 和 **1** 的含义。统考约定 0 = 空闲、1 = 已分配；但内存的位示图（页框分配）在某些教材里相反。做题时以题目说明为准。
2. 位图的「位」与「字节」「字」的换算。题目常给「每个盘块 1024 字节」，此时一位图块是 $1024 \times 8 = 8192$ 位，不是 1024 位。

（四）成组链接法

空闲表法和空闲链表法都不适用于大型文件系统——管理结构会随磁盘容量增长而过度膨胀。**UNIX** 系统采用成组链接法，它融合了两者的思想，有效控制了管理结构的规模。

基本思想：把所有空闲盘块划分成若干组（例如每组 100 个盘块）。除最后一组外，每组的第一个盘块用作索引块，记录下一组中所有空闲盘块的块号及其数量；这些索引块依次链接成一条链。第一组的空闲盘块信息（块号列表和数量）在文件系统挂载时被预先加载到内存的一个专用栈结构，称为空闲盘块号栈。

经典例子：空闲区为 201 ~ 7999 号盘块，每组 100 块，则分组为：第一组 201 ~ 300、第二组 301 ~ 400、……、次末组 7801 ~ 7900、最末组 7901 ~ 7999（最末组只有 99 块）。

- 空闲盘块号栈中，超级块里的结构是：空闲盘块数 **N** + 各空闲盘块号；第一组 201 ~ 300 中，**300** 号盘块作为索引块，记录下一组（301 ~ 400）的块号列表。
- 最末组的块号由前一组的最后一个盘块（7900 号）记录，而该盘块中存放的第一个盘块号为「**0**」，用作整条空闲链的结束标志。

成组链接法的分配过程

1. 从空闲盘块号栈顶取出一个盘块号分配给用户，栈指针上移，栈中空闲盘块数减 1；
2. 若栈指针到达栈底（表示当前组已分配完），则需要读取该栈底盘块号所对应的物理盘块内容——其中保存的是下一组的空闲盘块号列表；
3. 先把该内容整体读入空闲盘块号栈以更新它，再把原栈底所指的盘块（其索引作用已完成）分配出去。

以图例说明：系统先依次分配 201 ~ 299 号盘块；当需要分配 300 号盘块时，首先把 300 号盘块的内容（下一组的块号列表）读入空闲盘块号栈，随后把 300 号盘块本身分配给用户。

关键结论：只有当 **0** 号分组（栈中这一组）的空闲盘块分完，才需要进行一次磁盘 **I/O**。

成组链接法的回收过程

1. 系统把释放的盘块号压入空闲盘块号栈顶，栈指针下移，栈中空闲盘块数加 1；
2. 若此时栈已满（达到上限，如 100 项），则需要把当前栈中的全部盘块号写入新回收的这个盘块，使其成为新的索引块；
3. 随后把该盘块号设为新的栈底，并把栈中记录的空闲盘块数重置为 **1**。

关键结论：只有当栈满（**0** 号分组达到上限）时才需要一次磁盘 **I/O**。

两个必背问答：

- 若 0 号分组中仅剩唯一一个索引块，进行一次空闲盘块分配后：该索引块的内容被复制到超级块中，然后该索引块被分配给文件使用。
- 若 0 号分组的盘块数量已达上限，回收一个盘块后：把新回收的块作为新分组的索引块，并把原 **0** 号分组（栈）的信息复制过去。

成组链接法小结

- 描述空闲空间的数据结构（空闲盘块号栈）以及文件系统的关键信息，都存放在磁盘的固定位置；在 UNIX 中这一区域称超级块。
- 挂载时把超级块读入内存，并在运行过程中持续维护内存副本与磁盘副本的一致性。
- 成组链接法通过「分组 + 索引 + 链接」的设计，既避免了长链表的遍历开销，又无须维护完整的空闲表，在可扩展性与性能之间取得了良好平衡。
- 它可以得到连续的空闲磁盘空间，也可得到离散的空闲磁盘空间。

FAT 也算一种空闲空间管理结构

显式链接中的 **FAT** 记录了所有盘块。若某盘块不属于任何文件，就把它的表项标记为 `-2` 表示空闲；分配时遍历查找值为 `-2` 的表项即可，回收时把表项置回 `-2`。所以 2019 年真题中 FAT 被算作可用于管理空闲磁盘块的数据结构。而索引节点不是——它描述的是已存在文件的元数据与数据块地址。

十二、虚拟文件系统

（一）为什么需要 VFS

现代操作系统通常要支持多种文件系统（ext4、NTFS、FAT 等），它们在磁盘组织方式、元数据格式和操作方法上各不相同。若应用程序为每种文件系统分别编写代码，开发困难且系统可维护性差。虚拟文件系统（**Virtual File System, VFS**）位于用户程序与具体文件系统之间，是一个内核软件层、一个抽象层：

- 向上：提供统一的文件访问接口（标准的系统调用 `open`、`read`、`write`、`close` 等），使程序无须关心底层是哪种文件系统、存储在何种设备上；
- 向下：兼容并屏蔽各种不同类型文件系统的差异。

VFS 之所以能衔接各种文件系统，是因为它定义了所有文件系统都支持的、基本的、概念上的接口和数据结构。对于任何特定文件系统，都需要一个映射模块把它自己的特征转换为 VFS 期望的特征。一个新的文件系统要在某操作系统上被使用，就必须满足该操作系统 **VFS** 的规定：必须实现 VFS 规定的「接口函数集」（打开、读写、关闭等操作的函数），且函数的参数格式、返回值规则必须严格符合 VFS 要求。例如 ext4 与 FAT32 的「读文件」实现逻辑完全不同，但输入（文件标识、

缓冲区地址、长度）和输出（实际读取字节数）必须统一。

（二）VFS 的四大对象

VFS 采用面向对象的设计思路，用 C 语言的结构体实现「对象」——每个对象包含数据成员和一组函数指针。四个主要对象是：

对象	代表什么	关键内容	生命周期与存储位置
超级块对象（**superblock**）	一个特定的已挂载文件系统	文件系统类型、大小、所挂载的设备、操作方法（函数）指针、空闲块数目与索引表、空闲 inode 数目与索引表、封锁标记等	内核初始化并注册一个文件系统时在内存分配；**VFS** 超级块只存在于内存中，在文件系统卸载时被自动删除。具体文件系统的超级块（如 ext2 superblock）存放在磁盘上
索引节点对象（**inode / vnode**）	一个特定的文件或目录（物理属性）	文件字节大小、所有者、读/写/执行权限、时间戳、文件类型、数据块位置	具体文件系统的 inode 静态存放于磁盘；使用时调入内存并填写 VFS 的索引节点（**vnode**），因此 vnode 也称动态节点。同一文件系统内 inode 号唯一
目录项对象（**dentry**）	路径中的一个组成部分（目录名或文件名）	文件名、指向父/子目录项的指针、指向对应 vnode 的指针	不对应磁盘上的固定结构，是 VFS 在解析路径时动态生成的内存缓存，组成逻辑目录树
文件对象（**file**）	一个与进程相关的已打开文件	当前读/写位置（文件指针）、访问模式、引用计数，指向对应 vnode 与 dentry 的指针，以及 read/write/seek 等操作接口	进程调用 `open()` 时创建，调用 `close()` 且引用计数归零时销毁。同一文件可同时存在多个文件对象（多个进程各打开一次），但对应的 inode 和 dentry 是唯一的

VFS 四个对象的一句话对照

- 超级块是对一个文件系统的描述；
- 索引节点是对一个文件物理属性的描述；
- 目录项是对一个文件逻辑属性的描述；
- 文件对象是对当前进程打开的文件的描述。

另外注意：普通文件 = **inode** + 数据块；目录文件 = **inode** + 目录块（目录块中记录该目录下各文件的文件名与 inode 编号）。

vnode 不等于 inode

vnode 只存在于主存中，是 VFS 为统一管理各类文件而创建的内存对象；**inode** 既会被调入主存，也在外存中存储（磁盘 inode 是静态的）。两者通过指针关联协同工作：**vnode** 提供统一接口，**inode** 提供实际存储信息。

（三）VFS 的工作过程（以 write 为例）

当进程调用 `write(fd, buf, n)` 时：

1. 内核进入 **VFS** 层，执行 `sys_write()`；
2. 根据文件描述符 fd 定位到对应的文件对象；
3. 通过文件对象获取其关联的 **vnode**；

4. VFS 利用 vnode 中预置的写操作函数指针，调用目标文件系统的具体写方法；
5. 该文件系统把数据写入缓冲区，最终通过设备驱动完成磁盘写入。

因此，对用户程序而言，所有文件操作都通过 VFS 提供的统一接口完成，无须关心底层是 ext4、NTFS 还是其他文件系统。**VFS** 本身并不是一种真正的文件系统——它仅存在于内存之中，不占用磁盘空间，随系统启动而建立，随系统关闭而释放。

VFS 使用的缓存

- 索引节点缓存：每个目录和文件都由一个 vnode 表示，缓存最近访问过的索引节点，加快访问；
- 目录缓存（**dentry cache**）：缓存完整的目录名与其 inode 号之间的映射，加快列出目录与路径解析；
- 页面高速缓存（**Page Cache**）：存放文件的具体数据，需要相同数据时直接从内存读取而不访问磁盘。

十三、文件系统挂载

如同文件在使用之前必须先打开一样，文件系统在被进程访问前也必须先安装，也称挂载（**Mounting**）。

挂载的定义：把一个存储设备（或其分区）上的文件系统「连接」到操作系统目录树中的某个目录上，使其内容可以被访问。用于挂载的那个目录称为挂载点（**mount point**）。

核心要点：

- UNIX/Linux 中所有文件和设备都组织在一个统一的「目录树」中，根目录是 `/`；
- 硬盘、U 盘、网络存储上的数据默认不会自动出现在目录树里；
- 必须通过挂载操作把设备上的文件系统「嫁接」到目录树的某个挂载点（通常是空目录），才能访问其中的文件；
- 未挂载的设备 = 存在但不可见/不可用。

（一）挂载的过程

1. 识别设备：如 `/dev/sdb1`（U 盘第一个分区）、`/dev/nvme0n1p2`（SSD 分区）；
2. 识别文件系统类型：如 ext4、FAT32、NTFS、XFS、Btrfs；
3. 选择挂载点：一个已存在的空目录，如 `/mnt/usb`、`/media/disk`；
4. 执行挂载命令：如 `mount /dev/sdb1 /mnt/usb`；
5. 内核注册：VFS 把该设备的文件系统纳入统一管理，用户即可通过挂载点访问文件。

不同文件系统挂载时内核做的事不同：

文件系统	挂载时内核的动作
FAT	① 读取引导扇区（位于分区开头，偏移 0），获得每扇区字节数、每簇扇区数、保留扇区数、FAT 表个数、根目录项数或根目录起始簇、总扇区数、FAT 表大小等元数据（作用与 UNIX 超级块类似）；② 把第一份 **FAT** 表从磁盘整份读取到内存（因为查 FAT 极其频繁，必须常驻内存）；③ 读取根目录，填充 dentry 缓存；④ 完成挂载
ext4	① 读取超级块；② 读取块组描述符（ext4 把磁盘划分为多个块组，每组有自己的数据块位图、inode 位图和 inode 表）；③ 只读取根目录 `/` 的 **inode**（ext4 不会把所有 inode 都读入内存，其他文件/目录的 inode 在被实际访问到时才按需读入）；④ 完成挂载

挂载完成后的文件访问路径为：

- FAT：路径名 → 目录项 → 获取起始簇号 → 通过内存中的 FAT 表查找后续簇 → 读取数据；
- ext4：路径名 → dentry → inode → 通过 inode 中的块指针（直接/间接/extent）定位数据块 → 读取数据。

挂载点的选择与语义

- 并不是根目录下任何目录都可以作为挂载点。挂载操作会使原有目录中的内容被隐藏，因此根目录以及系统原有目录都不要作为挂载点，否则可能造成系统异常甚至崩溃；挂载点最好是新建的空目录。
- 当一个文件系统被挂载到某个目录上时，在该目录下创建的文件和目录都会被存储在该文件系统中，而不是原来所在的文件系统。
- 挂载后，访问该目录 = 访问设备上的文件系统。
- 挂载后原挂载点目录的内容会被「遮盖」，但不会被删除；卸载（`umount`）后原内容会重新出现。
- 同一个设备可以被挂载到多个不同的挂载点；但同一挂载点在同一时刻只能挂载一个设备。

挂载相关考法

- 2025 年统考考查过虚拟文件系统的原理与特点（VFS 的抽象层作用、四种对象、vnode 与 inode 的区别）。
- 挂载常与「文件系统在内存中的结构」合并出题：挂载时读入什么（超级块、FAT/块组描述符、根目录 inode），什么留在磁盘（其他 inode、数据块）。
- 卸载（umount）时必须把内存中被修改的超级块等元数据写回磁盘（保证一致性），这也是一道常见选择题的选项来源。

本章小结表

知识点	核心结论
数据项/记录/文件	基本数据项 → 组合数据项 → 记录 → 文件，自底向上
FCB	文件与 FCB 一一对应；FCB = 一个目录项；FCB 有序集合 = 文件目录；实现按名存取
inode	FCB 去掉文件名后的部分；目录项 = 文件名 + inode 编号；节省目录检索的磁盘 I/O
磁盘/内存 inode	磁盘 inode 静态；内存 inode 增加编号、状态、访问计数、逻辑设备号、链接指针
inode 最大文件长度	$S_{max} = \sum_k N_k \cdot n^k \cdot B$，$n = \lfloor B/p \rfloor$；索引块不计入数据长度
inode 位图 / 数据块位图	分别标记 inode 与数据块是否已分配；0 空闲、1 已分配
建立文件	分配空闲 inode + 在父目录中新增目录项
删除文件	删除目录项；count = 1 时释放数据块与 inode，否则 count−−；不删除所在目录
打开文件	路径解析 → 载入 inode → 检查权限 → 系统打开文件表（count+1）→ 进程打开文件表 → 返回 fd
关闭文件	删进程表项 → 系统表项打开计数 −1 → 为 0 时删除表项并视情况写回 inode
读/写文件	经内核缓冲区中转；磁盘按块读写；读写指针在进程打开文件表中
文件保护	访问控制矩阵、ACL（可精简为拥有者/组/其他）、口令、加密；权限位 rwx
权限位数计算	用户类别数 × 权限种类数（同类别可同时拥有多种权限）
逻辑结构	顺序文件、索引文件、索引顺序文件（约 $\sqrt{n}$ 次查找）、直接/散列文件
连续分配	起始块号 + 总块数；支持随机访问；外部碎片；扩展困难

知识点	核心结论
隐式链接	指针在盘块尾部；只能顺序访问；第 i 块需 $i+1$ 次读盘
显式链接 FAT	指针集中在 FAT 且常驻内存；支持随机访问；可用 -2 标记空闲块
索引分配	每文件一张索引表；单级/多级/混合（UNIX 13 个地址项）
目录结构	单级（不允许重名）、两级（MFD+UFD）、树形（绝对/相对路径）、无环图（共享计数器）
当前工作目录	作用是加快文件检索速度
硬链接	共用 inode、count+1、不能跨文件系统、原文件删除不影响、访问快
软链接	独立 inode、存路径名、不改 count、可跨文件系统、原文件删除后悬空、访问慢
外存结构	MBR/分区表、引导块、超级块、空闲空间管理信息（位图）、inode 区、根目录、数据区
内存结构	挂载表、目录项缓存、inode 缓存、系统级打开文件表、进程级打开文件表、页高速缓存
空闲表法	记录起始块号与块数；连续分配；适合空闲区少的场合
空闲盘块链	以单块为单位链接；最难获得连续空间
空闲盘区链	以盘区为单位链接；最容易获得连续空间
位示图法	一位一块；0 空闲 1 已分配；1 起编号 $b=n(i-1)+j$；0 起编号 $b=in+j$
成组链接法	UNIX 采用；超级块中的空闲盘块号栈；分组 + 索引 + 链接；只在栈空/栈满时 I/O
VFS	内核抽象层；四对象：超级块、索引节点（vnode）、目录项、文件；vnode 只在内存
挂载	把设备文件系统接到目录树的挂载点；挂载后访问目录即访问该设备；原目录内容被遮盖

易错清单

1. 概念混淆类

- **FCB 与 inode**：FCB 是完整控制块；inode 是 FCB 去掉文件名后的部分。目录项在 FCB 方案中就是 FCB，在 inode 方案中是「文件名 + inode 编号」。
- 目录项与 **inode**：目录项描述文件的逻辑位置（在文件系统中的位置），inode 描述文件的物理位置（在存储设备上的位置）。
- 磁盘 **inode** 与内存 **inode**：内存 inode 多了编号、状态、访问计数、逻辑设备号、链接指针；内存 inode 可被多进程共享。
- **vnode** 与 **inode**：vnode 只在内存中、是 VFS 的统一封装；inode 磁盘上也有。
- 文件系统在外存/内存中的结构：位图、超级块、inode 区在外存；挂载表、各类缓存、打开文件表在内存。

2. 打开文件表类

- 一个文件在系统打开文件表中只有一个表项，无论多少进程打开它；每个进程的打开文件表各有一项。
- 读写指针在进程级打开文件表中（各进程独立）；不同进程的 fd 数值也不相同。
- **open** 不一定读盘：若 inode 已在内存，只需打开计数 +1 并新建进程表项。
- 关闭 $\neq$ 删除：关闭不删除目录项、不释放外存 inode、不修改磁盘 inode 的链接计数。
- 删除文件不删除所在目录，也不必删除该文件的快捷方式/符号链接。
- **open** 返回的是文件描述符（进程打开文件表下标），不是文件内容、不是缓冲区指针。

3. 计算类

- **inode** 最大文件长度：$n = \lfloor B/p \rfloor$，逐级算 $N_k n^k B$ 再求和；索引块不计入文件数据长度；题目可能既有「数据块大小」又有「索引块大小」，一般二者相同，若不同要分别代入。
- 访问第 i 块需要几次读盘：连续 1 次、隐式链接 $i+1$ 次、显式链接 1 次、单级索引 2 次、混合索引按所在区 1/2/3/4 次；前提是 inode 已在内存，否则各加 1。
- 位示图换算：先确认编号从 0 还是从 1 开始；一位图块的位数 = 块字节数 $\times$ 8；位图大小 = 被管理对象数 / 8 字节。
- **FAT** 表大小：表项数 = 盘块数 = 容量/块大小；表项位数 $= \lceil \log_2$ 盘块数$\rceil$；FAT 空间 = 表项数 $\times$ 表项位数。
- 连续分配的插入/删除 **I/O** 次数：先判断往哪边移动代价小，读 + 写各算一次。
- 权限位数：用户类别数 $\times$ 权限种类数，不要压缩成「每类用户 2 位」。
- 索引顺序文件平均查找次数：最优分组 $i = \sqrt{n}$，总次数约 $\sqrt{n}$；多级索引 $(K+1)/2 \cdot N^{1/(K+1)}$。

4. 结构与特性判定类

- 随机访问能力：连续分配支持；隐式链接不支持；显式链接（FAT 常驻内存）与索引分配支持。
- 扩展能力：连续分配差；链接与索引分配好。
- 外部碎片：连续分配有；链接与索引分配没有。
- 最适合随机读写且易扩展：索引结构（2009-28）；**CD-ROM** 视频快速随机播放最优：连续结构（2013-24）。
- 哪个与单个文件长度无关：inode 总数（2013-26，它只决定文件个数）。
- 可用于管理空闲磁盘块的结构：位图、空闲磁盘块链、空闲盘区链、空闲表、FAT、成组链接法；**inode** 不行。
- 占用空间与当前空闲块数量无关的：位图法（2024-26）；空闲表、空闲链表、成组链接法都与当前空闲块数量有关。

5. 目录与链接类

- 当前工作目录的作用是加快文件检索速度，不是省内存/省外存/加快读写传输速度。
- 每一级中间目录都要读盘（除非命中目录缓存），这是路径解析开销的来源。
- 硬链接不能跨文件系统（inode 号只在文件系统内唯一）；软链接可以跨文件系统甚至跨网络。
- 建立硬链接使 **count+1**；建立软链接不改目标文件的 **count**。
- 删除硬链接只是 **count−−**，**count** 归零才真正删除；删除原文件后软链接变悬空。
- 硬链接访问快、软链接访问慢（需重新按路径逐级查找目录）。
- 树形目录不便于文件共享，共享要靠无环图目录结构或链接；无环图需要共享计数器，计数器归零才删除节点。

6. 成组链接法类

- 空闲盘块号栈放在超级块中，挂载时读入内存并常驻，必须保持内存副本与磁盘副本一致。
- 分配时栈空才需要 **I/O**（把栈底盘块中的下一组信息读入栈，再把该盘块分出去）；回收时栈满才需要 **I/O**（把栈中内容写入新回收的盘块，让它成为新索引块）。
- 最末组的结束标志是「**0** 号盘块」，表示链的结束。
- 一个分组的第一块用作索引块，它记录下一组的块号与数量，它本身也是空闲块，最终会被分配出去。

第五章输入/输出管理

本章考纲要求

第三部分操作系统五、输入/输出（I/O）管理（一）I/O 管理基础

1. 设备：设备的基本概念，设备的分类，I/O 接口，I/O 端口。
2. I/O 控制方式：轮询方式，中断方式，DMA 方式。
3. I/O 软件层次结构：中断处理程序，驱动程序，设备独立软件，用户层 I/O 软件。
4. 输入/输出应用程序接口：字符设备接口，块设备接口，网络设备接口，阻塞/非阻塞 I/O。

（二）设备独立软件

1. 缓冲区管理
2. 设备分配与回收
3. 假脱机技术（SPOOLing）
4. 设备驱动程序接口

（三）外存管理

1. 磁盘：磁盘结构，格式化，分区，磁盘调度方法。
2. 固态硬盘：读写性能特性，磨损均衡。

本章是操作系统部分「考点分散但命题直接」的一章，与计组第六章（总线和输入/输出系统）大量重叠。区别在于：计组关心 **I/O** 硬件与控制时序，操作系统关心 **I/O** 软件的分层、设备的分配与调度、外存的存取时间计算。统考在这章的出题形态很固定，主要有四类：

- 层次与流程排序题：I/O 软件四层的顺序、一次 I/O 请求的处理流程、DMA 传输各阶段的先后顺序。
- 对比判断题：中断与 DMA、单缓冲与双缓冲、阻塞 I/O 与非阻塞 I/O、各种磁盘调度算法的优缺点。
- 时间与距离计算题：单/双缓冲处理 n 块数据的总时间、CPU 用于轮询的开销占比、磁盘访问时间、各调度算法的磁头移动总距离。
- 应用题：以键盘输入或打印输出为例，写出从用户程序发起系统调用到数据落地的完整过程（2019、2023 年都出现过）。

复习策略是把「四层结构 + 三种控制方式 + 五个磁盘调度算法 + 两个缓冲公式」这四根主线背死，其余细节挂在主线上理解。

一、I/O 管理基础

（一）设备

*1.*设备的基本概念

I/O 设备是能够向计算机输入数据或接收计算机输出数据的外部设备。一个完整的 I/O 设备由两部分组成：

- 设备本体：执行具体输入/输出操作的物理装置，如磁盘的盘片与磁头、打印机的机械部分。
- 设备控制器（**I/O** 接口）：连接在系统总线上的专用电路，负责把操作系统发出的抽象命令（如「读取第 x 个扇区」）转换为设备可识别的电信号，并协调主机与设备之间的数据传送。

把这两部分分开的意义在于：操作系统只需要面对统一规格的控制器，不必了解成千上万种设备的具体机械细节。设备本体的差异被控制器封装掉了，这正是整个 I/O 软件能分层的前提。

*2.*设备的分类

设备的分类方法有四种，考纲要求的是前三类中的传输速率、信息交换单位和共享属性。

按传输速率分类：

类别	传输速率	典型设备
低速设备	每秒几字节至数百字节	键盘、鼠标
中速设备	每秒数千字节至数十千字节	激光打印机
高速设备	每秒数百千字节至数百兆字节以上	磁盘、光盘、网卡

按信息交换单位分类（最重要，直接决定 I/O 接口的类型）：

- 块设备（**block device**）：以固定大小的数据块为单位进行信息交换，如磁盘、固态硬盘、U 盘、光盘。基本特征是传输速率较高、可寻址（支持随机访问），可以对任意数据块直接读写。块设备的 I/O 通常采用 **DMA** 方式。
- 字符设备（**character device**）：以字符（字节）为单位进行信息交换，如键盘、鼠标、打印机、串口。基本特征是传输速率较低、不可寻址，只能顺序访问，I/O 通常采用中断驱动方式。大多数字符设备属于独占设备，因此字符设备接口必须提供「打开」和「关闭」操作以获取和释放访问权限。

块设备「隐藏了磁盘的二维结构」

磁盘物理上要用「柱面号—盘面号—扇区号」三维地址定位，但块设备接口把磁盘的所有扇区按数据块排成一个线性序列，上层只需给出「第 n 号逻辑块」即可。这个「二维变一维」的抽象是文件系统能在磁盘上建立的前提，也是块设备接口与字符设备接口最本质的区别。

按共享属性分类（决定设备分配策略）：

- 独占设备：任一时刻只允许一个进程使用。一旦分配给某进程，便由该进程独占，直至其主动释放或终止。典型的独占设备多为低速设备，如打印机、磁带机。独占设备采用静态分配方式。
- 共享设备：允许在一段时间内由多个进程宏观上同时访问，微观上交替使用。磁盘是典型的共享设备。共享设备必须是可寻址、可随机访问的设备，并且采用动态分配方式。
- 虚拟设备：通过 **SPOOLing** 技术把独占设备改造为多个逻辑设备，使多个进程能并发使用。其实质是用高速共享设备（如磁盘）作缓冲区，实现独占设备的逻辑共享。

「宏观同时、微观交替」的准确含义

共享设备是「一段时间内可同时访问」，不是「同一时刻」。物理上一个磁盘的磁头在同一瞬间只能在一个位置，它不可能同一时刻既为进程 A 读文件 X、又为进程 B 读文件 Y。操作系统把多个请求排入队列，用磁盘调度算法快速切换服务，因为切换速度远快于人的感知，所以宏观上表现为「同时共享」。「共享设备可以同一时刻被多个进程访问」是错误说法。

3.I/O 接口的功能与组成

I/O 接口（**I/O interface**），又称设备控制器（**device controller**）或 **I/O** 适配器，是 CPU 与设备之间的桥梁。它接收 CPU 发来的命令、控制设备工作，使 CPU 从繁杂的设备控制事务中解脱出来。

主要功能：

1. 接收和识别命令：接收 CPU 发来的控制命令，对命令字进行译码，产生相应的控制信号。
2. 数据交换：实现 CPU 与控制器之间、控制器与设备之间的数据传送。
3. 标识和报告设备状态：保存设备的状态信息（忙、就绪、错误）供 CPU 查询。
4. 地址识别：识别 CPU 要访问的是哪个设备、哪个寄存器。
5. 数据缓冲：用数据缓冲寄存器弥补 CPU 与设备之间的速度差异。
6. 差错控制：对传送的数据进行校验（如 CRC），发现并报告错误。

基本组成：

部件	作用
数据缓冲寄存器（**DBR**）	缓冲数据，匹配 CPU 与外设的速度差异。输入时暂存设备送来的数据，输出时暂存 CPU 送来的数据
状态寄存器	反馈设备状态（忙、就绪、错误）。程序查询方式中 CPU 靠读它判断设备是否就绪
控制寄存器（命令寄存器）	接收 CPU 发来的控制命令（复位、识别、读写控制等）
设备存储器	非必需部件，用于设备自身的运算处理，如显卡的显存
地址译码器与 **I/O** 控制逻辑	对命令字译码，产生控制信号送往外设；收集外设状态写入状态寄存器

状态寄存器与控制寄存器可以合并

状态寄存器和控制寄存器在传送方向上是相反的（状态寄存器是设备 →CPU，控制寄存器是 CPU→ 设备），而且 **CPU** 对它们的访问在时间上一般是错开的（先写命令，再等状态，不会同时进行），因此有的 I/O 接口把它们合并成一个「控制/状态寄存器」，用同一组地址、靠读写方向区分。

与 **I/O** 接口相连的三类总线都是双向的，它们传输的内容要分清：

- 数据总线：真正的 I/O 数据、CPU 写入控制寄存器的命令字（控制字）、CPU 读出的状态字。三者都经数据总线传输。
- 地址总线：I/O 端口地址（供地址译码器选中寄存器）、DMA 时接口作为主设备发出的主存地址。
- 控制总线：读/写信号、时钟与总线控制权协商信号（DMA 请求 HOLD、授权 HLDA）、应答信号、中断响应信号。

数据线上能传什么

统考考过「在 I/O 总线的数据线上传输的信息包括：I/O 接口中的命令字、状态字、中断类型号」。答案是三者都包括。要特别注意中断类型号也走数据线，但它不是设备控制器发出的，而是可编程中断控制器

在中断请求判优之后生成的。另外要区分「通过数据线写入控制寄存器的命令字」与「通过控制总线发送的控制信号」：前者是一个具体的数据值（回答「做什么」），后者是时序与协调信号（回答「怎么传」）。

4.I/O 端口

I/O 端口（**I/O port**）指设备控制器中的数据缓冲寄存器、状态/控制寄存器等的统称。数据缓冲寄存器简称数据端口，状态/控制寄存器简称状态/控制端口。所有 I/O 端口编号组成的空间称为 **I/O** 地址空间。

对数据端口、控制/状态寄存器的访问通过 **I/O** 指令完成。例如 IA-32 提供 `in`、`ins`、`out`、`outs` 四条专门指令，其中 `in`/`ins` 把控制器中寄存器的内容取到 CPU 通用寄存器，`out`/`outs` 把通用寄存器的内容送到控制器寄存器。这些 **I/O** 指令只能在操作系统内核的底层 **I/O** 软件中使用，因此属于特权指令。

I/O 端口的编址方式有两种：

比较项	统一编址（内存映射 **I/O**）	独立编址（**I/O** 映射方式）
概念	把 I/O 端口当成主存单元一起编址	I/O 端口地址与存储器地址无关
访问方式	不需要专门的 I/O 指令，用统一的 `LOAD`/`STORE` 访存指令即可	需要专门的输入输出指令，如 x86 用 `IN`/`OUT` 访问端口、用 `MOV` 访存
优点	端口编址空间大；读写控制逻辑简单；**I/O** 保护可由分段或分页存储管理实现，无需专门的保护机制	用专用 I/O 指令，程序清晰便于编程；端口地址位数少，地址译码速度快；不占用主存地址空间
缺点	占用主存地址空间，使主存地址空间变小；地址位数多，译码速度慢	I/O 指令类型少，一般只能做传送操作，程序设计灵活性差；需要 CPU 提供存储器读写与设备读写两组控制信号，控制逻辑复杂
如何区分 I/O 与主存	靠不同地址码区分（看地址落在哪个范围）	靠不同指令区分；由于二者独立编址，部分地址空间会重叠，因此用不同指令生成不同的总线控制信号来标识
典型处理器	MIPS、ARM 等 RISC 处理器	Intel x86 处理器

统一编址的 I/O 端口不能被 Cache 缓存

I/O 地址虽然映射到了主存空间，但接口中的数据是动态变化的，如果被 Cache 缓存，CPU 读到的将是过时的副本，无法了解设备的实时状态。因此统一编址的系统必须把 I/O 区间标记为「不可缓存」。这是统一编址方式下一个容易被忽略的陷阱。

（二）I/O 控制方式

I/O 控制方式的发展主线只有一条：尽量减少主机对 **I/O** 控制的干预，把主机从繁杂的 **I/O** 控制事务中解脱出来。从程序查询到中断驱动，再到 DMA 与通道，每一次进步都是让 CPU 少管一点、让硬件多管一点。

*1.*轮询方式

轮询方式（程序查询 / 程序直接控制方式）指输入输出完全依靠 CPU 执行程序实现。CPU 要与设备交换数据时，先设置接口的命令寄存器启动设备，然后在设备准备数据的过程中反复读取接口中的状态寄存器查询设备是否就绪，根据查询结果决定是继续等待还是传送数据。

以简单键盘输入为例：按键按下后，键盘控制器把键值放入数据缓冲寄存器 DBR，并把状态寄存器 DSR 的「数据就绪 Ready」位置 1。CPU 需要取键值时，先读 DSR 并判断 Ready 位，为 0 就继续查询，为 1 才读 DBR，读完后 Ready 位自动清 0。

对于复杂设备，流程多一步：CPU 先查询设备状态，就绪后才向 I/O 接口发送命令与参数启动设备，此后设备进入可能较长的设备准备阶段，CPU 则不断查询状态，就绪时才进行实际的数据传输。

轮询有两种策略：

- 独占查询（忙等待）：设备一旦启动，CPU 就一直连续查询，外设与 CPU 完全串行工作，查询期间 CPU 不能执行其他任务。
- 定时查询：CPU 周期性（每隔固定时间）查询一次接口状态，查询之间可以执行其他任务。适用于外设不支持中断的多任务系统。定时查询的难点是定时长度不好确定：太短则数据可能还没准备好、白查一趟；太长则数据可能已就绪而未被及时处理而丢失。

优点：只需软件即可实现，硬件开销少，控制简单。缺点：CPU 被独占，大量时间花在等待与查询上，利用率很低。多见于早期单任务操作系统，现代计算机在操作系统启动引导至多任务系统之前也采用它。

轮询并不一定只适合慢速设备

一般印象里「轮询适合慢速设备、中断适合高速设备」，但这里有反例：对于极高速设备，设备准备数据的时间可能比「两次上下文切换 + 中断服务」的开销还短，此时中断反而成为瓶颈，轮询效率更高。所以程序查询方式在 **NVMe** 这类超高速固态硬盘上也被采用。「轮询只适合慢速设备」是错误说法。

轮询开销占比计算（高频计算题）

这类题只有一个套路：先求「每秒必须查询多少次」，再求「每次查询需要多少个时钟周期」，两者相乘除以 **CPU** 每秒的时钟周期数。

【例】CPU 主频 50MHz，查询程序运行一次至少 500 个时钟周期，设备每秒至少需查询 200 次，求 CPU 用于该设备的 I/O 时间占比。每秒查询需 $200\times500=100000=0.1$ M 个时钟周期，占比 $=0.1\text{M}/50\text{M}=0.2\%$。

【例】主频 500MHz、CPI 为 4，设备 A 数据传输率 2MBps，接口中有一个 32 位数据缓冲寄存器，每次输入输出至少执行 10 条指令。求最多间隔多久查询一次、以及占比。32 位 = 4B，设备准备这 4B 需 $4\text{B}/2\text{MBps}=2\,\mu s$，所以最多每 $2\,\mu s$ 查询一次。CPU 每秒查询 $1s/2\mu s=0.5\text{M}$ 次，每次至少 $4\times10=40$ 个时钟周期，占比 $=40\times0.5\text{M}/500\text{M}=4\%$。

做题要注意：数据传输率按 **10** 为底（$1\text{M}=10^6$），不要按 2^{20} 算。

*2.*中断方式

中断方式（中断驱动 **I/O**）用中断机制彻底避免 CPU 忙等。完整流程如下：

1. 用户系统调用：用户进程 P1 通过 `read()` 等系统调用请求从设备读数据，陷入内核态。
2. 驱动程序启动设备：设备驱动程序把读命令及相关参数通过 I/O 接口发送给设备。
3. 操作系统进程调度：操作系统把 P1 放入 I/O 等待队列（阻塞态），通过上下文切换调度进程 P2 运行。P2 的执行与设备准备数据并行。
4. 设备发出中断请求：设备准备好数据后主动向 CPU 发中断请求，这是中断方式的核心——不需要 CPU 来问，设备自己会报。

5. **CPU** 响应并执行中断服务程序：CPU 暂停当前进程，保存现场，转入设备中断服务程序，在中断服务程序里完成实际的数据传送，并唤醒 I/O 等待队列中的 P1。
6. 恢复运行：中断服务程序最后一条是中断返回指令，返回断点继续执行；P1 已变为就绪态，等待被调度后完成后续 I/O 操作并返回用户态。

优点：设备主动报告，CPU 与外设并行工作，CPU 利用率显著提高；不需要考虑轮询频率问题，实时性更好。缺点：额外开销是两次进程上下文切换的时间加上中断服务程序本身的开销。当设备准备数据的时间远长于这些开销时，中断方式收益很大；反之则可能不如轮询。此外，每次数据传送都要经过 **CPU** 通用寄存器，且以字（字节）为单位，对块设备进行大数据量传送时效率很低。

谁最先拿到键盘输入的数据

统考考过「本地用户通过键盘登录系统时，首先获得键盘输入信息的程序是」——答案是中断处理程序，不是命令解释程序、系统调用服务程序或用户登录程序。键盘产生数据后先发中断，CPU 响应后执行键盘中断处理程序，由它把数据从控制器数据寄存器取走并放入内核缓冲区。中断服务例程执行结束时，输入数据位于内核缓冲区（2024 年考过这一点），既不是控制器数据寄存器，也不是用户缓冲区。

字符打印的经典过程（2023 年大题原型）

以 `printf("darkmoon")` 为例，可以把整条链路串起来：

1. 用户进程 P 调用库函数，最终执行 `write()` 系统调用，指令序列中有陷阱指令，从用户态陷入内核态。
2. 先把字符串复制到内核缓冲区。原因是中断服务程序执行时 CPU 仍运行在其他进程（如 Q）的环境中，地址空间是 Q 的，如果数据留在 P 的用户空间就取不到了；而内核空间被所有进程共享映射。
3. 驱动程序查询打印机状态寄存器直到就绪，把第一个字符从内核缓冲区经通用寄存器写入打印机控制器的数据寄存器（**I/O** 端口），再向控制寄存器写「启动打印」命令。
4. 执行调度程序，把 P 由运行态改为阻塞态并挂入阻塞队列，切换进程 Q 运行。
5. 打印机取走字符打印完毕后，控制器发出中断请求。CPU 在 Q 执行完一条指令后响应，执行中断隐指令（关中断、保存断点、中断服务程序寻址），转入字符打印中断服务程序。
6. 中断服务程序检查是否打印完毕：未完毕就取下一个字符送数据端口并再次启动打印，然后中断返回继续执行 Q；已完毕就把 P 从阻塞队列取出、状态改为就绪态，放入就绪队列。
7. 当调度器选中 P 后，P 执行系统调用的最后阶段，返回用户态继续执行。

因此：第一个字符是在系统调用服务例程控制下打印的，其余字符都由中断服务程序控制打印。这就是 2023 年第 46 题（键盘输入）与 2019 年大题所考的完整链路。

3.DMA 方式

直接内存访问（**Direct Memory Access**，**DMA**）在总线上设置 **DMA** 控制器（**DMAC**），由 DMAC 临时接管总线，代替 CPU 控制外部设备与内存之间的批量数据交换。数据无须经过 **CPU** 寄存器中转，直接通过系统总线在设备与内存之间交换，完全消除了程序查询和中断方式中 CPU 进行实际数据传送的开销。

DMA 方式主要用于高速设备的块数据传输，磁盘、显卡、网卡、声卡都支持 DMA。它也可以用于内存之间的大批量数据搬移。

DMA 控制器的组成：

部件	作用
主存地址寄存器（地址计数器 **AR**）	存放交换数据的内存地址。预处理阶段由 CPU 设置，每次传送后自动加 1 或减 1

部件	作用
字计数器（**WC**）	记录数据块长度。每传送一个字/字节自动减 1，减到 0 时传送完毕并发出 DMA 中断请求
数据缓冲寄存器（**DBR**）	DMAC 作为从设备时接收 CPU 的数据，作为主设备时暂存传输数据。若采用「周期窃取」模式则不经过它，性能更好
命令寄存器 / 状态寄存器	接收 CPU 控制命令 / 向 CPU 反馈 DMAC 状态
DMA 控制逻辑	控制和时序电路，修改地址计数器与字计数器，指定传输方向，协调 DMA 请求与 CPU 响应，并支持中断机制

DMAC 需要总线控制权时通过 **HOLD** 信号向 CPU 发总线请求，CPU 允许时用 **HLDA** 信号授权。支持 DMA 的设备用 **DREQ**（**DMA** 请求）和 **DACK**（**DMA** 应答）两条专用控制线与 DMAC 相连（注意：这两条线通常不属于系统总线）。一个 DMAC 支持多个通路，每条通路有各自独立的内存地址寄存器和字计数器，可同时为多个设备服务。

DMAC 在总线上的两种角色：预处理阶段 CPU 是主设备、DMAC 是从设备；数据传输阶段 DMAC 是主设备、CPU 反而成了旁观者。现代 PC 中 CPU、DMAC、I/O 接口、内存都通过系统总线相连，不存在所谓的「设备和内存之间的专用数据通路」。

DMA 的三种内存争用处理方式：

方式	工作方式	优点	缺点
停止 **CPU** 访问内存	DMAC 一旦获得总线就连续占用多个存储周期，直到整个数据块传完才归还	控制最简单（总线请求/建立/归还各只需一次）；数据传输率最高	DMA 期间 CPU 无法响应任何中断，实时性差；CPU 即使不需要访存也被强制暂停，浪费 CPU 时间
DMA 与 **CPU** 交替访问内存	把每个存储周期固定分为两半，CPU 只用其中一半，DMAC 只用另一半	无需总线请求，CPU 感觉不到 DMA 存在	硬件代价高；双方各自的空闲时间片都被浪费，内存带宽实际减半
周期挪用（周期窃取）	设备每准备好一个字，DMAC 就申请并占用一个存储周期完成传送，随后立即归还总线	按需分配，对 CPU 影响最小；已成为 **DMA** 的主流方法	总线请求、仲裁、归还本身有额外开销，一次挪用实际需 2 ~ 5 个存储周期

周期挪用的三个易错点

1. **CPU** 让出总线的时机是一个存储（总线）周期结束之后，不是「立即暂停当前指令」。一条指令可能包含多个总线周期，CPU 必须做完当前这个总线周期才能让出总线。这与中断请求在一条指令执行结束后响应是两回事。
2. 周期挪用期间 **CPU** 并不是完全不能访问主存。DMA 只在需要传数据时「偷走」一个周期，两次窃取之间的间隙 CPU 完全可以正常访存。「整个数据块传送过程中 CPU 不可以访问主存储器」是错误说法——那是「停止 CPU 访问内存」方式的特点。
3. **DMA** 操作不修改 **CPU** 寄存器，因此不存在现场保护和恢复问题。如果周期挪用发生时 CPU 不需要访存，则对 CPU 执行没有任何性能影响；如果恰好形成访存冲突，则 **DMA** 优先访问内存。

DMA 传输的三个阶段：

阶段	具体操作	结束时状态
预处理阶段（CPU 是主设备，DMAC 是从设备）	① 用 I/O 指令向 DMAC 写入主存起始地址（地址寄存器 AR）、传输长度（字计数器 WC）、传输方向（控制/命令寄存器标志位）；② 向 I/O 接口发启动命令，写入磁盘扇区号、设备地址等；③ 向 DMAC 控制寄存器写入「启动 DMA」命令位	请求进程被阻塞，调度其他进程运行
数据传输阶段（DMAC 是主设备，CPU 袖手旁观）	周期挪用循环：设备备好一个字 → 经 DREQ 通知 DMAC → DMAC 经 HOLD 申请总线 → CPU 在当前机器周期/总线事务结束后用 HLDA 授权并断开总线连接（驱动器进高阻态）→ DMAC 送主存地址到地址总线、发内存写 `MEMW` 与 I/O 读 `IORD` 控制信号、用 DACK 应答设备，设备把数据字送到数据总线，内存控制器写入 → AR 加 1、WC 减 1，撤销 HOLD 立即归还总线	WC 减到 0
后处理阶段（中断服务程序）	DMAC 经 `INTR` 发「DMA 完成」中断 → CPU 响应中断、保存现场，进中断服务程序 → 读 DMAC 状态寄存器判断是否成功 → 成功则把等待进程由阻塞态改为就绪态并放入就绪队列；失败则执行错误处理程序 → 恢复现场、中断返回	进程就绪，等待被调度

DMA 传输顺序记忆

② 初始化 **DMA** 控制器并启动磁盘 → ③ 从磁盘传输一块数据到内存缓冲区 → ①**DMA** 控制器发出中断请求 → ④ 执行「**DMA** 结束」中断服务程序（2017 年原题的四个选项就是这个顺序）。口诀：先配置、再搬运、后中断、最后收尾。

*4.*中断方式与 *DMA* 方式的区别

对比维度	中断方式	**DMA** 方式
核心思想	以 **CPU** 为核心，CPU 亲自处理每一次数据传送请求	以存储器为核心，把数据传送「外包」给专业硬件
数据传送主体	软件（CPU 执行中断服务程序）	硬件（DMA 控制器）
数据是否经过 CPU	每个字/字节都经过 CPU 通用寄存器	不经过 **CPU** 寄存器，设备与内存直接交换
CPU 介入程度	高，每个数据单位传送都要执行一次中断服务程序	低，只在预处理和后处理阶段介入
请求的对象	**CPU** 处理时间（请求中断 CPU）	总线控制权（请求系统总线的临时使用权）
响应时机	一条指令执行结束后	一个总线（机器）周期结束后，响应更快
数据传送单位	字（字节）	数据块
对 CPU 现场的影响	完全中止当前程序，需完整保护现场、恢复现场	不改变 **CPU** 现场，无需保护和恢复
中断次数	每个数据单位一次，频繁	整个数据块仅一次（DMA 完成中断）
CPU 与外设并行度	低（并行性只体现在设备准备数据期间）	高
适用设备	中、低速设备的少量数据传送，如键盘、鼠标	高速设备的批量数据传送，如磁盘、网卡

对比维度	中断方式	**DMA** 方式
功能范围	广泛，除数据传送外还能处理各种随机事件（时钟中断、电源掉电等）	功能单一，只用于成块数据的快速输入输出
优先级	低于 DMA	优先级高，DMA 与 CPU 同时访存时 DMA 优先

关于两者区别的两个常见错误说法

1.「中断方式适用于所有外部设备，DMA 方式仅适用于快速外部设备」——太绝对。中断方式对速率极高的设备效率很低，并不适用；DMA 理论上也能用于低速设备，只是浪费资源。
2.「中断服务程序不直接控制 I/O 硬件」——错误。设备驱动程序和中断服务程序共同直接控制硬件：驱动程序负责发起操作（向命令/数据/地址寄存器写入、启动设备），中断服务程序负责响应完成（从状态寄存器读取结果、向硬件发送确认信号清除中断）。

三种控制方式的综合使用

在实际系统中，这三种方式常常配合使用。以从磁盘读数据为例：

1. 进程 P1 用程序查询方式设置传送参数，初始化 DMAC 的相关寄存器；
2. CPU 向磁盘控制器发「寻道」命令，随后切换到其他进程 P2 执行；寻道完成后磁盘发出「寻道结束」中断；
3. CPU 响应中断，在中断服务程序中发「查找扇区」命令，返回 P2；扇区找到后磁盘发出「查找结束」中断；
4. CPU 响应中断，在中断服务程序中启动 **DMA** 传送；
5. DMAC 完成数据块传送后发出「DMA 结束」中断，CPU 在中断服务程序中做数据校验等后处理。

结论：在用 **DMA** 方式进行硬盘等高速设备的数据传送时，仍然要用到程序查询和中断方式。

（三）I/O 软件层次结构

为使复杂的 I/O 软件具有清晰的结构、良好的可移植性和易适应性，I/O 软件采用层次式结构：每层都利用下层提供的服务完成某些子功能，并屏蔽实现细节，向高层提供服务。整个 I/O 软件可视为四层结构。

*1.*用户层 *I/O* 软件

职责：实现与用户交互的接口。用户直接调用用户层的、与 I/O 操作有关的库函数来操作设备。

关键点：

- 大部分 I/O 软件在操作系统内部，但仍有一小部分在用户层，包括与用户程序链接在一起的库函数。
- 用户层软件必须通过一组系统调用来获取操作系统服务。库函数本身运行在用户态，它不直接操作硬件，而是通过系统调用陷入内核态，把请求交给下一层的设备独立性软件。
- 这一层的目标是隐藏所有底层 **I/O** 操作的复杂性，为用户提供简单、方便、统一的接口。

显式 **I/O** 与隐式 **I/O**：通过库函数和系统调用进行的 I/O 是显式 **I/O**（最常见）。此外还有由虚拟内存系统自动触发的隐式 **I/O**：访问一个不在物理内存的虚拟地址会触发缺页异常、自动发起一次磁盘 I/O；用 `mmap` 把文件映射到虚拟地址空间后，首次访问或修改该地址也会触发磁盘 I/O。

*2.*设备独立性软件

设备独立性（设备无关性）指应用程序独立于具体使用的物理设备。为实现这一点，引入了逻辑设备和物理设备两个概念：应用程序用逻辑设备名请求某类设备，系统执行时再把逻辑设备名映射为物理设备名。

使用逻辑设备名有两大好处：① 增加设备分配的灵活性；② 易于实现 **I/O** 重定向（用于 I/O 操作的设备可以更换，而不必修改应用程序）。

主要功能可分为两方面：

- 执行所有设备的公有操作：设备的分配与回收；将逻辑设备名映射为物理设备名；设备保护（禁止用户直接访问设备）；缓冲管理；差错控制；提供独立于设备、大小统一的逻辑块，屏蔽设备之间信息交换单位大小和传输速率的差异；设备命名（把物理名映射为 `/dev/sda1` 这类符号名）。
- 向用户层（或文件层）提供统一接口：无论何种设备，向用户提供的接口相同，如统一使用 `read`/`write`。

差错控制的两类错误：暂时性错误（如网络丢包）通常通过重试纠正，连续失败才视为持久性错误，主要由驱动程序处理；持久性错误（如磁盘划痕）由操作系统记入坏块表，后续 I/O 自动绕过。

I/O 软件四层的排列顺序（**2011**、**2012**）

自上而下是：用户层 **I/O** 软件 → 设备独立性软件 → 设备驱动程序 → 中断处理程序。中断处理程序最接近硬件，必须在最底层；驱动程序必须在设备独立性软件之下。一次磁盘 I/O 请求的处理流程是：用户程序 → 系统调用处理程序 → 设备驱动程序 → 中断处理程序。

另外，**SPOOLing** 技术的归属有争议：在经典教材（如汤子瀛《计算机操作系统》）中它被划入用户层 I/O 软件，而 **408** 大纲把它划入设备独立性软件。理解上：SPOOLing 的入口和守护进程确实工作在用户态，但它的核心功能（实现设备共享）属于设备独立性软件的职责。统考遇到这类有争议的问题通常会回避，按大纲理解即可。

*3.*设备驱动程序

职责：与硬件直接相关，接收上层软件发来的抽象 **I/O** 请求（如 `read`、`write`），转换为对设备的具体要求后发送给设备控制器，控制 I/O 设备工作；同时把设备控制器发来的信号传送给上层软件，从而为 I/O 内核子系统隐藏设备控制器之间的差异。

具体功能：

1. 抽象请求到具体操作的转换：把逻辑盘块号翻译成硬件能识别的柱面号、磁头号、扇区号等参数。
2. 合法性与权限校验：检查用户请求的操作是否被允许（如只读打开的设备不能写）。
3. 设备状态检查：读控制器状态寄存器，确认设备处于可接收命令的就绪状态。
4. 参数设置与传送：向控制器的命令、数据、方式等寄存器写入本次任务所需参数。
5. 发出 **I/O** 命令并启动设备：设备空闲则立即启动；设备忙则把请求进程的 PCB 挂到该设备的等待队列上。
6. 响应中断：及时响应设备控制器发出的中断请求，并按中断类型调用相应的中断处理程序。

特点：

- 硬件相关性：与特定硬件的物理特性紧密耦合，每类设备配置一个驱动程序，通常由设备制造厂商提供而非 OS 设计者设计。因此在系统中增加一个新设备时需要安装厂商提供的新驱动程

序。

- 与 **I/O** 控制方式紧密相关：程序查询方式的驱动需要循环查询状态；中断方式的驱动需要启动设备后阻塞进程、由中断唤醒；DMA 方式的驱动需要配置 DMA 地址、计数和方向。「驱动程序与 **I/O** 控制方式无关」是错误说法（2022 年考过）。
- 一部分必须用汇编语言书写，很多基本部分已固化在 ROM 中。
- 可重入性：必须在一次调用完成前能被另一个任务（如中断）再次安全调用。典型场景是：驱动代码正在响应 `write` 系统调用执行时，被该设备完成上一次操作产生的硬件中断打断，CPU 转去执行该驱动的另一部分代码（中断服务程序）。如果两部分代码共享全局或静态变量而无保护，就会数据错乱。可重入函数通过不使用这类共享数据或加特殊保护来避免。

设备处理方式（不重要，了解即可）：为每类设备设一个进程；为整个系统设一个 I/O 进程（或分设输入进程和输出进程）；不设专门进程，只为各类设备设驱动程序供用户或系统进程调用（目前用得较多）。

*4.*中断处理程序

职责：保存被中断进程的 CPU 环境，转入相应的中断处理程序进行处理，处理完毕再恢复被中断进程的现场并返回。

主要任务：进程上下文切换、对中断信号源进行测试、读取设备状态、修改进程状态等。

位置：由于中断处理与硬件紧密相关，对用户而言应尽量加以屏蔽，因此放在操作系统的底层，系统的其余部分尽可能少地与它发生联系。

整个中断过程包括中断响应和中断处理两个阶段：中断响应完全由硬件完成，中断处理则由 **CPU** 执行中断服务程序完成。虽然不同中断源对应的服务程序不同，但所有中断服务程序的结构相同，包括准备阶段、处理阶段和恢复阶段。I/O 中断的典型过程是：设备控制器准备好数据后通过中断控制器向 CPU 发中断请求 → 保护被中断进程的 CPU 上下文 → 转入相应的设备中断处理函数 → 进行中断处理 → 恢复被中断进程的上下文。

以一次 **read** 命令总结四层各自的分工

1. 用户要读某设备内容，通过操作系统提供的 `read` 命令接口——经过用户层。
2. 操作系统给用户的接口一般是统一的通用接口，用户发出的 `read` 先经设备独立层解析（确定逻辑块号、检查高速缓存、找到对应驱动程序），然后交给下层。
3. 不同类型的设备对 `read` 的行为不同，需要针对具体设备把 `read` 解析成不同指令——经过设备驱动层。
4. 命令解析完毕后需要中断正在运行的进程、转而执行实际的数据传送，这就需要中断处理程序。

（四）输入/输出应用程序接口

不同类型的设备读写方式差异很大，为便于应用程序访问，操作系统把大量差异隐藏起来，并把设备分成几种常规类型，相应地把 I/O 核心子系统与高层之间的接口也分为若干类。

*1.*字符设备接口

字符设备以字符为单位存取和传输，如键盘、打印机、鼠标。特点是传输速率较低、不可寻址，I/O 通常采用中断驱动方式。具体特征：

- 流式/顺序访问（不可寻址）：数据只能从当前位置顺序读写，没有「块」也没有地址。由于不可寻址，通常为字符设备建立一个字符缓冲区，用户程序通过 `get()` 从缓冲区取字符、通过 `put()` 把字符输出到缓冲区。

- 需要打开和关闭操作：大多数字符设备属于独占设备，接口中必须提供打开和关闭操作来获取和释放访问权限，以实现互斥共享。
- 控制命令丰富：通常支持特殊的 `ioctl()` 系统调用进行设备特定设置（如设置串口波特率）。

*2.*块设备接口

块设备以数据块为单位存取和传输，如磁盘、固态硬盘、U 盘、SD 卡、CD。特点是传输速率较高、可寻址，I/O 常采用 **DMA** 方式。具体特征：

- 可寻址/随机访问：可以直接命令设备「读第 500 号数据块」，隐藏了磁盘的二维结构——把磁盘所有扇区按数据块顺序划分，二维结构变为线性序列，文件系统因此可以在其上构建。
- 固定大小的数据单元：无论读写，基本单位都是一个「块」。即使只想读 1 个字节，操作系统也可能从硬件读整个块再提取所需字节。
- 缓冲机制：操作系统大量使用内存（Buffer Cache）缓存块设备数据。请求的块若已在缓存中就直接从内存返回，极大提高性能；写操作也可能先写缓存，稍后统一刷到设备。
- 将抽象命令映射为低层操作：用户使用的 `open()`、`read()`/`write()`、`lseek()` 无法被设备直接识别，块设备接口把它们映射为设备能识别的具体操作。

*3.*网络设备接口

网络设备接口是专门用于与其他计算机交换数据的接口，不直接与读写相关，而是围绕数据包的收发和连接的管理设计，典型设备是以太网卡。关键特征：

- 不使用常规文件 **API**：应用程序通常不用 `open` 再 `read`/`write` 的方式操作网卡，而是使用一套独立的套接字（**Socket**）**API**。
- 数据包导向：数据以网络数据包形式封装传输，每个包包含目标地址、源地址等信息。
- 连接管理：需要处理建立连接（TCP）、无连接通信（UDP）、监听端口、超时处理等网络概念。
- 异步通信：网络不可靠且有延迟，发送一个数据包后不知道它何时到达、对方何时回复。

主要系统调用：`socket()` 创建通信端点、`bind()` 把地址绑定到套接字、`send()`/`recv()` 收发数据包、`close()` 关闭连接。

*4.*阻塞 *I/O* 与非阻塞 *I/O*

以一次典型的 `read` 操作为例，I/O 包含两个独立的阶段：① 等待数据准备（内核等待数据到达：等键盘输入、等磁盘寻道读出、等网络包到达）；② 数据从内核复制到用户空间。阻塞与非阻塞的核心区别在于它们如何处理第一个阶段。

比较项	阻塞 **I/O**	非阻塞 **I/O**
数据未就绪时的行为	系统调用不立即返回，进程由运行态转为等待（阻塞）态，CPU 去执行其他进程	系统调用立即返回一个错误码，进程不进入等待态
后续处理	数据准备好后内核唤醒该进程，系统调用继续把数据从内核复制到用户空间后返回	进程需反复主动调用该系统调用来检查数据是否就绪，这个过程称为轮询
数据就绪后	由内核主动唤醒并完成复制	仍需进程自己发起调用完成复制，然后才能处理数据
优点	代码逻辑线性、顺序，直观易懂；`read()` 返回后数据一定有效	并发潜力高，一个线程可同时照看多个 I/O 通道，线程利用率高

比较项	阻塞 **I/O**	非阻塞 **I/O**
缺点	并发能力差，一个线程任一时刻只能处理一个 I/O 请求，等待期间整个线程被挂起	编程模型复杂（要写轮询循环）；轮询本身消耗 CPU（忙等待）

为什么非阻塞 I/O 的并发度更高

核心在于线程资源的释放。阻塞模型下一个线程与一个 I/O 通道「绑定等死」，线程生命周期被漫长的 I/O 等待占据；非阻塞模型下线程发起 I/O 调用后立即被「释放」，可以转而为其他成千上万个 I/O 通道服务或执行其他计算。就像一个服务员可以同时照看很多张桌子，而不是站在一张桌子旁死等客人吃完。

阻塞与非阻塞、同步与异步的层次

阻塞/非阻塞描述的是「等待数据准备」这一阶段进程的行为；同步/异步描述的是「数据复制」阶段由谁完成。阻塞 I/O 和非阻塞 I/O 在数据复制阶段都是同步的——都必须由进程自己发起调用、由内核把数据复制到用户空间后才算完成。真正的异步 **I/O** 是发起请求后立即返回，数据准备和数据复制全部由内核完成，内核在整个操作完成后才通知进程，进程在此期间不需要任何轮询或等待。

二、设备独立软件

（一）缓冲区管理

*1.*为什么需要缓冲

缓冲（**buffer**）是一段临时存储区，可以由专门的硬件寄存器组成，但硬件成本高、容量小，一般只用于速度要求极高的场合（如 TLB、设备控制器中的数据缓冲寄存器），更多利用内存作为缓冲区。缓冲区管理的主要功能是组织好这些缓冲区并提供获得和释放的手段。

引入缓冲区的目的有四个：

1. 缓和 **CPU** 与 **I/O** 设备间速度不匹配的矛盾。凡在数据到达速率与离去速率不同的地方都可设置缓冲区。CPU 运算速率远高于 I/O 设备，若在打印机或控制器中设缓冲区，CPU 可快速暂存输出数据后立即去做别的计算，打印机慢慢从缓冲区取数据打印，CPU 效率大幅提高。
2. 减少对 **CPU** 的中断频率，放宽对 **CPU** 中断响应时间的限制。例如速率为 9.6kb/s 的数据通信，若只用一位缓冲接收，则每 100μs 就要中断 CPU 一次，且 CPU 必须在 100μs 内响应，否则数据被冲掉。若设置 8 位缓冲寄存器，中断频率降为原来的 1/8；再设一个 8 位寄存器，响应时间可从 100μs 放宽到 800μs。
3. 解决数据粒度（数据单元大小）不匹配的问题。生产者数据粒度小于消费者时，生产者可连续生产几个单元，凑够消费者要求的单元大小后消费者再取走；反之消费者也可分几次取走一个大数据单元。
4. 提高 **CPU** 与 **I/O** 设备之间的并行性，提高系统吞吐量和设备利用率。

*2.*缓冲与缓存的区别

两者都是一段临时存储区，都能协调高速设备与低速设备的速度差异，但工作原理不同：缓冲是一个「中转站」，缓存是数据源的副本。

特性	缓冲（**Buffer**）	缓存（**Cache**）
核心目的	缓和速度/粒度不匹配	加速对数据的重复访问
工作模式	数据的「中转站」	数据源的「高速副本」「快捷方式」

特性	缓冲（**Buffer**）	缓存（**Cache**）
数据性质	中转数据，可能是唯一的存在（源方发出后不再保留）	一定是源数据的子集（副本）
数据流向	通常是单向的，被消费后即失效	可被反复读取
性能指标	关注吞吐量	关注命中率

磁盘高速缓存就是这个意义上的缓存：操作系统在内存的内核空间开辟高速缓存，把最近从块设备读出或写入的数据保存在 RAM 中。它逻辑上属于磁盘，物理上驻留在内存。读取时先查缓存，命中则直接从内存返回，未命中才发起真正的磁盘 I/O；写入有写通（直写）（同时写缓存和磁盘，安全但性能提升有限）和回写（只写缓存并标记为脏，之后择机刷回磁盘，性能极高但有数据丢失风险，是现代 OS 普遍采用的策略）两种策略。

缓冲区在用户空间还是内核空间

用户进程提出 I/O 请求时指定的缓冲区在用户空间（如 `fread(buf, size, num, fp)` 中的 `buf`）。陷入内核后，内核通常会在内核空间再开辟一个或两个缓冲区，底层 I/O 软件直接使用内核空间的缓冲区存放 I/O 数据。为什么不直接用用户空间的缓冲区？因为外设进行 I/O 期间用户进程被挂起，用户空间的缓冲区所在页面可能被替换出去，那样就无法获得缓冲区中的 I/O 数据；此外还便于解决不同块设备读写单位不一致、便于共享等问题。

*3.*单缓冲

假设用户进程请求块设备读入若干块数据，采用单缓冲时操作系统在主存（内核空间）中为其分配一个缓冲区（若无特别说明，一个缓冲区的大小就是一个块）。

设：从磁盘把一块数据输入到缓冲区的时间为 T，操作系统把缓冲区中的数据传送到用户进程工作区的时间为 M，CPU 对这一块数据处理的时间为 C。

取初始状态为工作区是满的、缓冲区是空的（若无明确说明，通常认为缓冲区与工作区大小相等）：

- 若 $T > C$：工作区数据处理完用时 C 时缓冲区还没充满，需再等到 T 时刻缓冲区才满，然后缓冲区向工作区传送数据，工作区满时缓冲区同时为空，用时 M。到达下一个相同状态，整个过程用时 $T + M$。
- 若 $T < C$：缓冲区早已充满并等待，工作区处理完才能传送，用时 $C + M$。

单缓冲处理每块数据的时间

$$\text{单缓冲每块时间} = \max(T,\ C) + M$$

即「输入」与「处理」可以并行，但「传送」必须串在两者之后。因为只有一个缓冲区，设备向缓冲区写入数据与 CPU 处理工作区数据可以重叠，但缓冲区向工作区传送时必须独占缓冲区。

处理 n 块数据的总时间为：

$$(n-1)\left[\max(T, C) + M\right] + (C + T + M)$$

即前 $n-1$ 块按流水节奏并行，最后一块需要完整的串行收尾（输入、传送、处理各走一遍）。

*4.*双缓冲

由于缓冲区是共享资源，生产者与消费者使用缓冲区时必须互斥。如果消费者尚未取走缓冲区中的数据，即使生产者又生产出新数据也无法送入缓冲区，生产者只能等待。设置两个缓冲区便可解决

这个问题：设备输入时先把数据送入缓冲区 1，装满后转向缓冲区 2 写入；此时操作系统可从已满的缓冲区 1 取数据送用户工作区并由 CPU 处理。

取初始状态为工作区空、其中一个缓冲区满、另一个缓冲区空：

- 若 $T > C + M$：设备输入所需时间大于传送与处理的总时间，设备可连续工作。当缓冲区 2 的数据处理完毕（用时 $C + M$）时缓冲区 1 尚未装满，系统需等待其装满才能把下一块从缓冲区 1 传送到工作区。处理每块数据所需时间为 T。
- 若 $T < C + M$：设备输入较快，CPU 无须等待设备。缓冲区 1 很快装满，但系统仍需等缓冲区 2 的数据完全处理完毕才能切换并传送缓冲区 1 的内容。处理每块数据所需时间为 $C + M$。

双缓冲处理每块数据的时间

$$\text{双缓冲每块时间} = \max(T,\ C + M)$$

即「处理」与「传送」串行（两者都发生在工作区侧，且传送依赖处理的完成），但「传送 + 处理」整体与「输入」并行。

处理 n 块数据的总时间为：

$$(n-1)\,[\max(T,\ C + M)] + (C + T + M)$$

双缓冲在通信中的应用：两台机器通信时每台需配置发送缓冲区和接收缓冲区。若要实现全双工通信（双方同时收发数据），每台机器必须分别设置发送缓冲区和接收缓冲区；若每台只配一个缓冲区，则无法同时进行发送与接收。

*5.*循环缓冲

若输入与输出速度相差较大，双缓冲难以充分发挥作用，此时采用多于两个缓冲区的方案。循环缓冲（环形缓冲）由若干个大小相等的缓冲区构成，每个缓冲区有一个链接指针指向下一个，最后一个的指针指向第一个，构成一个循环队列。系统维护两个指针：`in` 指向下一个待写入的空缓冲区，`out` 指向下一个待读取的满缓冲区，输入进程和计算进程分别推进 `in` 和 `out`。

*6.*缓冲池

单缓冲、双缓冲、循环缓冲都是为特定生产者和消费者进程专门设置的专用缓冲。系统较大时会有许多这样的循环缓冲，既消耗大量内存，利用率又不高。为提高利用率，引入既可用于输入又可用于输出的公用缓冲池。

缓冲池与缓冲区的区别：缓冲区仅仅是一组内存块的链表，而缓冲池是包含了一个管理的数据结构及一组操作函数的管理机制。

缓冲池中具有相同类型的缓冲区链接成一个队列，按使用状况形成三个队列：空白缓冲队列（空缓冲区链成，用于取空缓冲区写入）、输入队列（装满输入数据的缓冲区链成，供用户程序读取）、输出队列（装满输出数据的缓冲区链成，等待被输出设备读取）。

另外按缓冲区在实际运算中扮演的功能不同，设置四种工作缓冲区（`h` 代表 host/用户进程，`s` 代表 system/设备，`in`/`out` 代表方向）：

工作缓冲区	用途
`hin`（收容输入）	缓冲池与 I/O 设备之间，收容输入的数据
`sin`（提取输入）	缓冲池与用户进程空间之间，提取输入的数据

工作缓冲区	用途
hout（收容输出）	缓冲池与用户进程空间之间，收容输出的数据
sout（提取输出）	缓冲池与 I/O 设备之间，提取输出的数据

四种工作方式（规律：取出操作一律从队头取，挂入队列一律挂到队尾）：

工作方式	从哪个队列队首取	装满/取空后挂到哪个队列队尾
收容输入（输入进程）	空缓冲队列	输入队列
提取输入（计算进程）	输入队列	空缓冲队列
收容输出（计算进程）	空缓冲队列	输出队列
提取输出（输出进程）	输出队列	空缓冲队列

缓冲方式的分类归属

单缓冲、双缓冲、循环缓冲属于生产者消费者进程的专用缓冲；缓冲池属于进程共享缓冲。这与内存管理中「请求分页的可变分配全局置换使用系统空页队列」的思想类似，都是把资源集中管理以提高利用率。

单/双缓冲时间计算的两个真题

【2011-31】某文件占 10 个磁盘块，逐个读入主存缓冲区并送用户区分析。一个缓冲区与一个磁盘块大小相同，把一块读入缓冲区的时间 $T = 100\ \mu s$，缓冲区数据传送到用户区的时间 $M = 50\ \mu s$，CPU 分析一块的时间 $C = 50\ \mu s$。求单缓冲和双缓冲下读入并分析完该文件的时间。

- 单缓冲：每块 $\max(100, 50) + 50 = 150\ \mu s$，总时间 $= (10-1) \times 150 + (50+100+50) = 1350 + 200 = 1550\ \mu s$。
- 双缓冲：每块 $\max(100,\ 50 + 50) = \max(100, 100) = 100\ \mu s$，总时间 $= (10 - 1) \times 100 + (50 + 100 + 50) = 900 + 200 = 1100\ \mu s$。

答案选 **B（1550μs、1100μs）**。

【2013-27】系统缓冲区和用户工作区均采用单缓冲，从外设读入一块到系统缓冲区 $T = 100$，从系统缓冲区读入用户工作区 $M = 5$，对用户工作区中一块的分析时间 $C = 90$。求读入并分析 2 个数据块的最短时间。

代入 n 块公式：$(2 - 1) \times [\max(100, 90) + 5] + (90 + 100 + 5) = 1 \times 105 + 195 = 300$。答案选 **C（300）**。

算这类题最容易踩的坑

1. 不要用「$n\times$ 每块时间」。n 块的总时间必须用 $(n - 1)$ 倍的流水时间加上最后一块的完整串行时间 $(C + T + M)$，因为第一块必须先走完「输入 $\rightarrow$ 传送 $\rightarrow$ 处理」才能开始流水。若题目只问「处理每块的平均时间」才用 $\max(\cdot) + M$ 或 $\max(\cdot)$。
2. 单缓冲的 M 在 max 之外，双缓冲的 M 在 max 之内。这是两者公式唯一的差别，也是判断题目该用哪个公式的关键。
3. 注意题目给的是「系统缓冲区 $\rightarrow$ 用户工作区」还是反过来。M 的方向不影响公式形式，但要确认 T 是设备 $\rightarrow$ 缓冲区的时间。
4. 若题目强调「初始状态缓冲区为空、工作区为空」，需按题目给的初始状态重新推一遍，公式给出的是稳定流水后的每块时间。

（二）设备分配与回收

1.设备分配的任务与总原则

设备分配的任务是按照一定策略，为提出 I/O 请求的进程分配合适的设备，并分配相应的控制器和通道，确保 CPU 与 I/O 设备之间能正常通信。

总原则：充分发挥设备的使用效率，尽可能让设备忙碌，又要避免因不合理的分配方法造成进程死锁。

> 通道是一种独立于 CPU、专用于 I/O 控制的处理器，可以控制 I/O 设备与内存直接进行数据交换。通道的启动需要 CPU 执行相应指令，操作结束时向 CPU 发中断信号。数据交换过程中 CPU 无需干预，提高了 CPU 与 I/O 的并行程度。

2.设备分配应考虑的因素

（**1**）设备的固有属性（按共享属性分三类）：

- 独占设备：一个时段只能分配给一个进程（如打印机），利用率低。
- 共享设备：一段时间内可同时分配给多个进程使用（如磁盘），各进程宏观上同时共享、微观上交替使用，因此要注意对访问先后次序进行合理调度，一般采用动态分配方式。
- 虚拟设备：采用 SPOOLing 技术把独占设备改造成虚拟的共享设备，可同时分配给多个进程使用。

（**2**）设备分配算法：通常只采用先来先服务（**FCFS**）或优先级高者优先。FCFS 优先分配给队首进程；最高优先级优先算法把高优先级进程排在队列前方，优先级相同时按 FCFS 排队。

（**3**）设备分配中的安全性：

方式	做法	优点	缺点
安全分配方式	为进程分配一个设备后就将进程阻塞，本次 I/O 完成后才唤醒。一个时段内每个进程只能使用一个设备	破坏了「请求和保持」条件，不会死锁	对一个进程来说 CPU 与 I/O 设备只能串行工作
不安全分配方式	进程发出 I/O 请求后系统分配设备，进程可继续执行并发新的 I/O 请求，只有某个请求得不到满足时才阻塞。一个进程可同时使用多个设备	进程的计算任务与 I/O 任务可并行处理，进程推进迅速	可能具备「请求并保持」条件，可能发生死锁（需配合死锁避免、检测与解除）

（**4**）设备分配方式：

- 静态分配：进程运行前一次性分配其所需的全部设备、控制器和通道，运行结束后归还。破坏了「请求和保持」条件，不会发生死锁，主要用于独占设备。缺点是进程可能长期不用已分配的设备而其他进程又用不上，设备利用率低。
- 动态分配：运行过程中按需申请。利用率高，但有死锁可能。

静态分配与安全分配方式的关系

两者都通过「不让进程同时持有并等待」来破坏死锁条件，但层次不同：静态分配说的是分配时机（一次性分配全部资源），安全分配方式说的是分配后进程的状态（立刻阻塞）。另外注意：静态分配「可以提高设备利用率」是错误说法，恰恰相反，它降低利用率。

3.设备分配管理中的数据结构

设备分配的四个数据结构体现了「通道 → 控制器 → 设备」的从属关系。一个设备控制器可供多个同类设备共享使用，一个通道可供多个设备控制器共享使用。

数据结构	数量	主要字段
设备控制表 **DCT**（Device Control Table）	系统为每个设备配置一张	设备类型、设备标识符（物理设备名，系统内唯一）、设备状态（忙/闲）、指向控制器表的指针、重复执行次数或时间、设备队列的队首指针
控制器控制表 **COCT**（Controller Control Table）	每个设备控制器一张	控制器标识符、控制器状态（忙/闲）、与控制器连接的通道表指针、控制器队列的队首/队尾指针
通道控制表 **CHCT**（Channel Control Table）	每个通道一张	通道标识符、通道状态（忙/闲）、与通道连接的控制器表首址、通道队列的队首/队尾指针
系统设备表 **SDT**（System Device Table）	整个系统只维护一张	每个表目对应一个设备，含设备类型、设备标识符、指向 **DCT** 的指针、驱动程序入口

四张表的记忆要点

SDT 全系统只有一张，是「总目录」，每个表目指向一个 DCT；**DCT** 每个设备一张，其「指向控制器表的指针」连接到 COCT；**COCT** 每个控制器一张，其「与控制器连接的通道表指针」连接到 CHCT；**CHCT** 每个通道一张。请求流程与这套链式结构一致：**SDT** → **DCT** → **COCT** → **CHCT**。

*4.*设备分配的步骤

以独占设备为例：

1. 分配设备：根据 I/O 请求中的物理设备名查找 SDT，找出设备的 DCT，根据 DCT 中的设备状态字段判断状态。若忙，把进程 PCB 挂入设备等待队列；若空闲，把设备分配给该进程。
2. 分配控制器：设备分配后，根据 DCT 找到对应的 COCT，查询控制器状态。若忙则挂入控制器等待队列，空闲则分配给该进程。
3. 分配通道：控制器分配后，根据 COCT 找到对应的 CHCT，查询通道状态。若忙则挂入通道等待队列，空闲则分配给该进程。

只有设备、控制器、通道三者都分配成功，这次设备分配才算成功，之后便可启动 I/O 设备进行数据传送。

用物理设备名分配的三大缺点：① 用户编程时必须使用物理设备名，底层细节对用户不透明；② 换一个物理设备程序就无法运行（即使有同类型设备也不行）；③ 若请求的物理设备正忙，即使系统中还有同类型设备，进程也必须阻塞等待。

改进方法是引入设备独立性：建立逻辑设备名到物理设备名的映射，用户编程时只提供逻辑设备名。改进后用逻辑设备名分配时，操作系统根据逻辑设备名在 SDT 中查找一个用户指定类型且空闲的设备分配给进程，只有所有该类型的设备都忙时才需要阻塞该进程。

逻辑设备表 **LUT**（Logical Unit Table）：每个表项为〈逻辑设备名，物理设备名，设备驱动程序入口地址〉。

- 进程首次请求设备分配时，系统为其分配一台物理设备，同时在 LUT 中新增一个表项；
- 以后该进程再次通过相同的逻辑设备名请求设备时，直接查表即可。

LUT 的两种建立方式：

- 整个系统只有一张 **LUT**：各用户所用的逻辑设备名不允许重复，适用于单用户操作系统。
- 每个用户一张 **LUT**（存入用户进程的 PCB 中）：不同用户的逻辑设备名可重复，适用于多用户操作系统。可类比文件系统中单级目录（不允许重名）与多级目录（不同级可重名）的区别。

▌设备分配相关的三道真题

【2009-32】程序员利用系统调用打开 I/O 设备时，通常使用的设备标识是——逻辑设备。用户程序通过系统调用请求设备时使用与具体硬件无关的逻辑设备名，物理设备名、主设备号、从设备号是内核或驱动程序识别具体硬件时使用的信息，不直接暴露给普通应用程序。

【2020-30】关于具备设备独立性的系统，错误叙述是「更换物理设备后必须修改访问该设备的应用程序」。设备独立性的目标正是让应用程序用逻辑设备名或文件名访问设备，更换设备时只需修改映射或驱动配置，应用程序通常不需要改变。

【2023-32】设备分配需要考虑的因素包括：Ⅰ设备类型、Ⅱ访问权限、Ⅲ占用状态、Ⅳ逻辑设备与物理设备的映射关系，答案是 D（四项全部）。设备类型决定分配规则，权限决定进程能否使用，占用状态决定能否立即分配，映射关系决定从哪个逻辑请求找到哪个实际设备。

补充：在 UNIX/Linux 中，设备名唯一确定一个特殊文件的 **i** 节点，i 节点中含主设备号（确定设备类型，据此定位对应的设备驱动程序）和次设备号（作为参数传给驱动程序，确定进行 I/O 操作的具体设备位置）。这就是「一切皆文件」设计的实现基础。

（三）假脱机技术（**SPOOLing**）

*1.*为什么需要 *SPOOLing*

以打印机为例，未引入 SPOOLing 时 CPU 需要亲自处理每一次数据搬运：从内存读数据 → 写入寄存器 → 从寄存器写到 I/O 端口。这带来四个弊端：

1. **CPU** 资源被独占，效率极低。打印机很慢，CPU 每发一小块数据就可能要停下来查询控制器状态，整个打印过程中 CPU 大部分时间花在等待和搬运上。
2. 用户进程长时间阻塞。发起打印请求的进程在打印任务彻底完成前无法被唤醒，打印几十页文档可能被「冻结」几分钟。
3. 计算与 **I/O** 无法并行。对同一进程而言两者严格串行。
4. 系统整体吞吐量低。慢速 I/O 长时间占用 CPU，其他需要 CPU 的进程也被迫等待。

2.SPOOLing 系统的组成

SPOOLing（**Simultaneous Peripheral Operations On-Line**，外部设备联机并行操作），又称假脱机技术，其思想源于早期的脱机 **I/O** 方式：依赖专用外围控制机，在主机不参与的情况下完成 I/O——输入时外围机把低速输入设备（如纸带机）的数据预先读入高速磁盘；输出时 CPU 先把结果高速写入磁盘，再由外围机送往低速输出设备。由于整个 I/O 过程脱离主机控制，因此称为脱机。

随着多道程序设计技术的发展，操作系统不再需要物理外围机，而通过专门的输入进程与输出进程在主机直接控制下用软件模拟外围机的功能。这种「以磁盘缓冲、用软件模拟脱机 I/O」的机制正是 SPOOLing 的本质：「脱机」承袭自早期脱机 **I/O** 的思想，「假」则体现在用多道程序和磁盘空间替代了物理外围机。

SPOOLing 系统的四个组成部分：

组成部分	位置	作用
输入井和输出井	磁盘上的两个专用存储区域	输入井模拟脱机输入的磁盘，暂存输入设备的数据；输出井模拟脱机输出的磁盘，暂存用户程序的输出数据。每个进程的数据以井文件形式保存，所有文件按顺序链接成输入队列或输出队列
输入缓冲区和输出缓冲区	内存中的两个缓冲区	输入缓冲区暂存由输入设备送来的数据，随后传送到输入井；输出缓冲区暂存从输出井读出的数据，随后传送到输出设备
输入进程和输出进程	内存中的系统进程	输入进程模拟脱机输入的外围控制机，把输入设备的数据经输入缓冲区送到输入井（CPU 需要输入数据时直接从输入井读）；输出进程模拟脱机输出的外围控制机，把用户程序的输出数据写入输出井，并在输出设备空闲时经输出缓冲区送到输出设备

组成部分	位置	作用
井管理程序	内核	控制作业与磁盘井之间的信息交换

SPOOLing 的前提条件

- 需要大容量、高速度的外存（通常为磁盘）作为输入井和输出井；
- 需要多道程序技术的支持；
- SPOOLing 是用软件的方式、建立在通道技术和多道程序技术基础之上的技术。

「SPOOLing 由用户作业控制设备与输入/输出井之间的数据传送」是错误说法（2016 年考过）——设备与井之间的数据传送由系统的输入/输出进程（假脱机服务程序）控制，对用户作业是透明的。

*3.*如何把独占设备改造为共享设备（打印机经典例子）

打印机是典型的独占设备。通过 SPOOLing 技术可把它改造为可供多个用户共享的虚拟设备。

当多个用户进程发出打印请求时，**SPOOLing** 系统接受请求但并不真正立即把物理打印机分配给它们，而是由输出进程为每个进程完成两项操作：

1. 在磁盘输出井中为其分配一个空闲盘块，把待打印数据写入其中；
2. 申请一张用户请求打印表，记录用户的打印要求，把该表挂到假脱机打印队列上。

完成这两步后，用户进程即认为打印任务已完成，可继续执行后续代码。实际上真正的打印尚未开始，但这个过程对用户完全透明。真正的打印由后台的假脱机打印进程负责：当打印机空闲时，该进程检查假脱机打印队列，若队列非空则取出队首请求表，根据表中信息把对应的输出数据从输出井读入内存缓冲区，再送到物理打印机输出；一个任务完成后继续处理下一个；队列为空则进入等待状态，直至有新的打印请求把它唤醒。

结果：由于用户进程只需把数据写入磁盘即可返回，而物理打印由后台进程按队列顺序依次执行，多个用户便可并发提交打印请求。系统为每个请求在输出井中分配独立存储区域，使各进程都以为自己独占了一台打印机。一台物理打印机被虚拟为多台逻辑设备，实现了高效且透明的共享。

4.SPOOLing 的三大特点与优点

SPOOLing 系统的三大特点

1. 提高了 **I/O** 速度：把对低速 I/O 设备的操作转化为对磁盘输出井中数据的存取，如同脱机输入输出一样，有效缓和了 CPU 与低速 I/O 设备之间速度不匹配的矛盾。
2. 将独占设备改造为共享设备：在假脱机打印系统中，实际上并未把物理设备分配给任何进程。
3. 实现了虚拟设备功能：每个进程都认为自己独占了一台设备。

SPOOLing 是一种以空间换时间的技术：牺牲磁盘存储空间（输入井、输出井），换取显著的时间节省。

SPOOLing 引入后的具体优点：

- 提高了用户进程的响应速度。数据一旦拷贝到输出缓冲区并由井管理程序接管，用户进程的任务就算完成，可立即返回。对用户进程而言，它感觉不是在跟慢速打印机打交道，而是跟高速磁盘打交道，只需把数据快速「甩」到输出井。这种把慢速设备「伪装」成高速设备的技术称为设备虚拟化。
- 缓和 **CPU** 与慢速 **I/O** 设备的速度矛盾，极大提高 **CPU** 利用率。CPU 只参与两次高速传送：从用户空间拷贝到内核缓冲区（内存 → 内存，极快）、井管理程序把数据写入磁盘输出井（内存 → 磁盘，很快）。真正的「磁盘井 → 打印机」由独立的输出进程处理，可通过 DMA 进一步减少对 CPU 的打扰。
- 实现计算与 **I/O** 操作的真正并行。CPU 全力执行进程 B 的计算任务，输出进程把磁盘井中进程 A 的打印数据送往打印机，输入进程从读卡器为进程 C 准备输入数据存入输入井——输入、计算、输出宏观上同时进行。

- 提高系统吞吐量。综合以上优点，单位时间内完成的总任务量得到质的飞跃。

关于 SPOOLing 的三个易错判断

1. **SPOOLing** 不能缩短物理设备的传输时间。它并不改变打印机本身的速度，只是让 CPU 不必等待这个慢速过程。认为「SPOOLing 缩短了数据在 I/O 设备与内存之间的物理传输时间」是错误的。
2. 用户进程的大部分等待时间不发生在对磁盘井的读写上。恰恰相反，对磁盘井的读写非常快、等待时间极短，这才是 SPOOLing 的优势所在。
3. 输出进程不是由用户作业创建的，它是一个系统级的后台服务进程（守护进程），由操作系统启动时创建和管理，通常常驻内存而非按需创建，数据来源是输出井而不是任何用户作业的地址空间。

5.SPOOLing 与脱机 *I/O* 的区别与联系

比较项	脱机 **I/O**	**SPOOLing**（假脱机）
是否需要专用外围机	需要物理的外围控制机	不需要，用输入/输出进程模拟外围机的功能
由谁控制	脱离主机控制，由外围机独立完成	在主机直接控制下由系统进程完成
依赖的技术	外围机硬件 + 磁带/磁盘	多道程序设计技术 + 通道技术 + 高速外存
共同点	都用高速外存做缓冲，都实现低速 I/O 与 CPU 计算的并行，都能把一个独占设备虚拟为多个逻辑设备	同左

一句话总结联系与区别：**SPOOLing** 是脱机 **I/O** 思想的「软件化」——用多道程序和磁盘空间替代了物理外围机，从而在主机直接控制下取得与脱机 **I/O** 同样的并行效果。

（四）设备驱动程序接口

*1.*为什么需要统一接口

外设种类繁多、控制接口不一致，不同外设的设备驱动程序千差万别。如果计算机系统中每出现一种新外设，都要为添加新驱动程序而修改操作系统，会给操作系统开发者和系统用户带来很大麻烦。

因此操作系统为所有外设的设备驱动程序规定了统一的接口：新设备驱动程序只要按统一接口规范编制，就可以在不修改操作系统的情况下在系统中添加新驱动程序并使用新外设进行 I/O。

*2.*统一接口的实现方式

对每种设备类型（如磁盘），操作系统定义一组驱动程序必须支持的函数（磁盘至少包含读、写、格式化等）。驱动程序中通常包含一张表格，这张表格具有针对这些函数指向驱动程序自身实现的指针（函数指针表）。装载驱动程序时操作系统记录该表的地址，需要调用某个函数时通过这张表发出间接调用。这个函数指针表定义了驱动程序与操作系统其余部分之间的接口，给定类型的所有设备都必须服从这一要求。

工作流程举例：应用程序需要从某块 SSD 读数据，内核上层软件发出通用的 `read()` 请求，内核根据目标设备查找「该 SSD 驱动的函数指针表」，看到标准 `read()` 接口对应的是 `samsung_ssd_read()` 的地址，于是间接调用该函数完成实际的硬件操作。

统一接口的价值示例：一台采用统一编址（内存映射）的打印机驱动可能用 `memcpy` 把整页数据拷到大块内存缓冲区，用直接读内存地址的方式获取状态；而一台采用独立编址的打印机驱动则用 `outb` 循环逐字节写数据端口，先发命令再从端口读回状态字节。但两者向操作系统提供的接口函数签名完全相同。上层模块调用时完全不知道也不需要关心数据是 `memcpy` 过去的还是一个字节一个字节发出去的。这就

是设备驱动程序接口的强大之处：把上层软件的调用意图与下层硬件的复杂实现彻底解耦。

*3.*设备名到驱动程序的映射

因为采用了统一的设备驱动程序接口，内核中与设备无关的 I/O 软件包含了所有外设统一公共接口中的处理部分。为简化处理，内核高层 **I/O** 软件把所有外设都抽象成一个文件，设备名和文件名在形式上没有任何差别，称为设备文件名。内核中与设备无关的 I/O 软件必须把不同的设备名和文件名映射到对应的设备驱动程序。

在 UNIX/Linux 中，一个设备名唯一确定一个特殊文件的 **i** 节点：i 节点中包含主设备号，主设备号用于定位相应的设备驱动程序；i 节点还包括次设备号，作为参数传给设备驱动程序，用来确定进行 I/O 操作的具体设备位置。除了普通文件和目录文件，还有一类特殊文件包括设备文件、链接文件等，设备特殊文件又分块设备特殊文件和字符设备特殊文件。

关于设备驱动程序的一道辨析题

「对于设备驱动程序的描述，错误的是：不同型号的同类设备提供给相同上层系统的接口可能不同。」

解析：同一个操作系统对同类型的所有设备提出了统一的接口规范要求，因此不同型号的同类设备为同一个系统提供的接口是一样的；而不同系统提出的接口规范可能不一样，所以「同一型号的同类设备提供给不同上层系统的接口可能不同」是正确的。此外，设备驱动程序可以通过 I/O 端口访问设备（正确）；用户对设备的访问依靠驱动程序提供的接口，设备自身的中断处理也需要驱动程序接口支持（正确）。

另一道题:「操作系统为所有块设备定义统一的设备驱动程序接口，最主要的目标是——向上层软件（如文件系统）隐藏不同硬件设备的物理实现差异」。接口的目的不是提高 CPU 访问寄存器的效率，也与编程语言无关，更不只是简化中断流程。

三、外存管理

（一）磁盘

*1.*磁盘结构

磁盘存储器由记录介质（盘片）、磁盘驱动器、磁盘控制器三大部分组成。

- 磁盘控制器：磁盘存储器与主机的 **I/O** 接口，包括控制逻辑、时序电路、「并 → 串」转换和「串 → 并」转换电路。
- 磁盘驱动器：包括读写电路、读/写转换开关、读/写磁头与磁头定位伺服系统。
- 盘片：涂有磁性介质的物理盘片，通过设置磁场方向写入 0 和 1。

五个基本术语：

术语	定义
盘面（**surface**）	盘片的一个面。每个盘片的两个面各有一个磁头，因此盘面号就是磁头号
磁道（**track**）	磁头与盘片相对运动形成的圆。磁头位于不同半径上得到不同磁道。磁道从外向里编址，最外面的为 **0** 号磁道
柱面（**cylinder**）	所有盘面上相同编号的磁道构成的圆柱面。因此磁道号就是柱面号
扇区（**sector**）	每个磁道被分成的若干圆弧，是磁盘可寻址的最小单位，以扇区为单位进行读写
磁头（**head**）	每个盘面对应一个磁头，同一柱面上所有磁头「共同进退」

关键性质：

- 读写磁盘时总是写完一个柱面上所有的磁道后，再移到下一个柱面。这个性质正是「柱面号在地址高位」的物理原因。
- 一个磁道存储的信息量相同，所以内磁道的位密度比外磁道大。
- 硬盘属于机械式部件，读写操作是串行的，不可能在同一时刻既读又写，也不可能在同一时刻读两组数据或写两组数据。

磁盘的分类（了解）：按磁头相对盘片径向是否固定分为固定头磁盘（每个磁道一个磁头）和活动头磁盘（磁头臂可伸缩定位磁道）；按盘片是否可更换分为固定盘磁盘和可换盘磁盘。

*2.*格式化与分区

磁盘从出厂到可用，要经过「低级格式化 → 分区 → 高级格式化 → 安装操作系统」四步。

第一步：低级格式化（物理格式化）

在磁盘能存储数据之前进行，主要工作：

1. 把磁盘划分为扇区，以便磁盘控制器能读写数据。每个扇区的数据结构包括头部、数据区域和尾部，头部和尾部包含扇区号和纠错代码（**ECC**）。控制器写入数据时 ECC 根据数据区域所有字节计算并更新；读取时重新计算并与存储值比较，不一致则表明数据损坏。ECC 能纠正少量错误并报告可恢复的软错误，读写时由控制器自动处理。
2. 检测并标记坏道，避免存储数据时使用这些损坏区域。
3. 初始化逻辑块号到无损磁盘扇区的映射，使制造商能够测试磁盘。

第二步：分区

把磁盘划分为多个独立部分，每个分区可被独立管理和使用。每个分区由一个或多个柱面组成，每个分区的起始扇区和大小都记录在磁盘主引导记录的分区表中。分区类型包括：主分区（每个磁盘最多四个，其中一个可标记为活动分区用于引导操作系统）、扩展分区（为克服主分区数量限制，可包含多个逻辑分区）、逻辑分区（在扩展分区内部创建）。操作系统可把每个分区当作一个单独磁盘，例如一个分区存操作系统可执行代码、另一个存用户数据。分区对于实现多个 **OS** 并存是必需的。

第三步：高级格式化（逻辑格式化）

在分区内创建文件系统，包括：

- 创建文件系统：定义数据在磁盘上的组织方式（FAT32、NTFS、ext4 等）。
- 设置文件系统结构：设置超级块（**superblock**）、**inode** 表、目录结构等。
- 初始化目录和文件表：设置根目录和必要的文件表，以及对保存空闲磁盘块信息的数据结构（如位示图、空闲表）进行初始化。

为提高效率，大多数操作系统把多个相邻扇区组合成更大的块，称为簇（**cluster**）。一簇只能存放一个文件的内容，文件占用空间只能是簇的整数倍，文件大小小于一簇（甚至 0 字节）也要占用一簇。磁盘 **I/O** 按块（扇区）完成，而文件系统 **I/O** 按簇完成，这有效确保了 I/O 有更多的顺序访问和更少的随机访问。

第四步：安装操作系统（在逻辑格式化之后执行）。

格式化各步考什么

【2017-29】磁盘逻辑格式化程序所做的工作是：建立文件系统的根目录（Ⅱ）、对保存空闲磁盘块信息的数据结构进行初始化（Ⅳ）。

- 「对磁盘进行分区（Ⅰ）」属于分区操作；
- 「确定磁盘扇区校验码所占位数（Ⅲ）」属于低级格式化（物理格式化）的工作。

答案选 B（仅Ⅱ、Ⅳ）。

【2017-26】某文件系统的簇和磁盘扇区大小分别为 1KB 和 512B，文件大小 1026B，系统分配给该文件的磁盘空间是：需要簇数 $\lceil 1026/1024 \rceil = 2$，分配空间 $2 \times 1024\text{B} = 2048\text{B}$。按簇分配、不足一簇也占一簇，答案选 D。

坏块的处理：简单磁盘在格式化时扫描并标记坏块，文件系统不再分配它们；正常操作时才变坏的块需人工运行特殊程序搜索锁定，其中数据通常丢失。更复杂的磁盘由控制器维护坏块列表，低级格式化时预留一些备用块（操作系统看不到），用备用块逻辑地替代坏块，称为扇区备用（**sector sparing**）或扇区转寄（**sector forwarding**）。总之对坏块的处理实质上就是用某种机制使系统不去使用坏块。

*3.*磁盘地址结构

磁盘的物理地址结构为：

驱动器号	柱面（磁道）号	盘面（磁头）号	扇区号
一台电脑可能有多个硬盘	移动磁头臂（寻道）	激活某个磁头	通过旋转把特定扇区转到磁头下方

实际计算题中通常只考虑后三段，即（柱面号，盘面号，扇区号），三段都从 0 开始编号。把簇号转换成磁盘物理地址的过程由磁盘驱动程序完成。

为什么柱面号在前

假设磁盘有 8 个柱面、4 个盘面、8 个扇区，则需要各 3、2、3 个二进制位。

- 若地址结构是（盘面号，柱面号，扇区号）：连续读取 $(00,000,000) \sim (00,001,111)$ 时，同一柱面不同盘面的部分可以不移动磁头换盘面磁头读完，但要读到下一个柱面 $(00,001,000)$ 起就必须启动磁头臂把磁头移到下一个磁道。
- 若地址结构是（柱面号，盘面号，扇区号）：连续读取 $(000,00,000) \sim (000,01,111)$ 时，柱面号相同、只是盘面号不同，不需要移动磁头臂，只需激活相邻盘面的磁头即可，节省大量时间。

结论：把柱面号放在高位，使逻辑上相邻的块在物理上尽量落在同一柱面内，从而把「寻道」这个最耗时的操作推迟到万不得已时才做。这是「柱面号在地址高位」的根本原因。

*4.*磁盘的性能指标与访问时间

（**1**）记录密度：沿磁道分布方向上单位长度内的磁道数目叫道密度；沿磁道方向上单位长度内存放的二进制信息数目叫位密度。

- 低密度存储方式：所有磁道上的扇区数相同，因此内道的位密度比外道高。
- 高密度存储方式：每个磁道上的位密度相同，因此外道的扇区数比内道多，整个磁盘容量比低密度盘高得多。

（**2**）存储容量：未格式化容量按道密度和位密度计算，包括头空、ID 域、CRC 码等信息，是可用磁化单元的总数；格式化后的实际容量只包含数据区，一般比未格式化容量小 10%～20%。

（**3**）数据传输速率：完成磁头定位和旋转等待后单位时间内读出或写入的二进制信息量。由于磁盘同一时刻只有一个磁头读写，内部数据传输速率等于单位时间内磁头划过的磁道弧长乘以位密度：

$$\text{内部数据传输速率} = \frac{\text{每分钟转速}}{60} \times \text{内圆周长} \times \text{最内道位密度}$$

若转速为 r（转/秒）、每条磁道容量为 N 字节，则 $D = rN$。

（**4**）平均存取时间：

$$\text{响应时间} = \text{排队延迟} + \text{控制器时间} + \text{寻道时间} + \text{旋转等待时间} + \text{数据传输时间}$$

后三项之和称为存取时间：

$$\boxed{\text{存取时间} = \text{寻道时间} + \text{旋转等待时间} + \text{数据传输时间}}$$

时间项	公式	说明
寻道时间 T_s	$T_s = s + m \cdot n$	s 为启动磁头臂的时间（约 2ms），m 为磁头每跨越一条磁道的耗时（约 0.2ms），n 为需要跨越的磁道数
旋转延迟（延迟时间）T_r	$T_r = \frac{1}{2} \cdot \frac{1}{r} = \frac{1}{2r}$	r 为转速（转/秒）。定位过程中最多旋转 1 圈、最少不用旋转，所以平均需要旋转半圈，故除以 2。若转速为 r 转/分，则旋转一圈的时间为 $60/r$ 秒
传输时间（读数据）T_t	$T_t = \frac{b}{N} \cdot \frac{1}{r} = \frac{b}{rN}$	转速 r（转/秒），本次读/写字节数为 b，每条磁道字节数为 N。b 字节需 b/N 条磁道，读写一条磁道刚好是转一圈的时间 $1/r$
传输时间（访问一个扇区）	$\frac{\text{旋转一圈的时间}}{\text{一条磁道的扇区总数}} = \frac{\text{一个扇区的容量}}{\text{数据传输率}}$	用于求一个扇区扫过磁头的时间

转速单位的陷阱

转速可以用 r 转/秒或 r 转/分表示，有些题目会给出「r 秒/转」或「r 分/转」，此时公式需要相应取倒数。看到「7200rpm」要立刻意识到这是转/分，旋转一圈的时间是 $60/7200$ 秒 $= 8.33$ ms，平均旋转延迟为 4.17 ms。单位判断错误是这类题失分的第一大原因。

另外注意：磁盘的传输速率中 **K**、**M**、**G**、**T** 一般以 **10** 为底（$1\text{K} = 10^3$、$1\text{M} = 10^6$、$1\text{G} = 10^9$、$1\text{T} = 10^{12}$），不是以 2 为底。

磁盘访问时间完整算例（2022-44 大题原型）

某磁盘驱动器有 4 个双面盘片，每个盘面 20000 个磁道，每个磁道 500 个扇区，每个扇区 512 字节，转速 7200rpm，平均寻道时间 5ms。

（**1**）地址信息的三个字段名称及各占多少位

三个字段是柱面号、磁头号（盘面号）、扇区号（CHS）。

- 柱面号：磁道数 20000，需 $\lceil \log_2 20000 \rceil = 15$ 位；
- 磁头号：4 个双面盘片共 $4 \times 2 = 8$ 个盘面（8 个磁头），需 $\lceil \log_2 8 \rceil = 3$ 位；
- 扇区号：每道 500 个扇区，需 $\lceil \log_2 500 \rceil = 9$ 位。

（**2**）一个扇区的平均访问时间

转速 7200rpm $= 120$ 转/秒，旋转一圈 $1/120$ s $= 8.33$ ms。

- 平均旋转延迟 $= \dfrac{1}{2} \times 8.33$ ms $= 4.17$ ms；
- 传输时间（一个扇区）$= \dfrac{8.33 \text{ ms}}{500} = 0.017$ ms（也等于 512B/ 数据传输率）；
- 平均寻道时间 $= 5$ ms。

$$T_a = 5 + 4.17 + 0.017 \approx 9.19 \text{ ms}$$

（**3**）周期挪用 **DMA** 下总线请求次数

磁盘控制器数据缓冲区 64 位 $= 8$ 字节，一个扇区 512 字节需 $512/8 = 64$ 次传送，故一个扇区读写过程中 **DMA** 控制器向 **CPU** 发送 **64** 次总线请求。若 CPU 检测到 DMA 总线请求时也需要访问主存，**DMA** 控制器仍可获得总线使用权，因为 **DMA** 的总线请求优先级高于 **CPU**，以防止高速设备的数据来不及传送而被覆盖丢失。

（**4**）磁盘容量

$$\text{容量} = \text{盘面数} \times \text{每面磁道数} \times \text{每道扇区数} \times \text{每扇区字节数} = 8 \times 20000 \times 500 \times 512 \text{ B} \approx 40.96 \text{ GB}$$

5.磁盘调度方法

磁盘调度的主要目标是减少平均寻道时间：寻道时间在存取时间中占主导且直接受调度算法影响；旋转延迟与传输时间都与转速成反比，属于硬件参数，OS 层面难以优化。

下面统一使用一组经典请求序列贯穿演示所有算法：磁头初始位于 **100** 号磁道，请求队列（按到达顺序）为

$$55,\ 58,\ 39,\ 18,\ 90,\ 160,\ 150,\ 38,\ 184$$

若算法需要方向，约定磁头先沿磁道号增大的方向移动；磁盘磁道号为 $0 \sim 199$。

（1）先来先服务（FCFS）

按进程请求访问磁盘的先后顺序进行调度，是最简单的调度算法。

访问顺序：$100 \to 55 \to 58 \to 39 \to 18 \to 90 \to 160 \to 150 \to 38 \to 184$

移动距离：$45 + 3 + 19 + 21 + 72 + 70 + 10 + 112 + 146 = 498$ 个磁道，平均寻道长度 $= 498/9 \approx 55.3$。

- 优点：公平、简单。请求较少且访问的磁道比较集中时，性能尚可。
- 缺点：大量进程竞争磁盘、请求磁道很分散时，磁头频繁长距离移动，寻道时间长。

（2）最短寻道时间优先（SSTF）

每次优先处理与当前磁头距离最近的磁道。可以保证每次寻道时间最短，但不能保证总的寻道时间最短（本质是贪心）。

访问顺序：$100 \to 90 \to 58 \to 55 \to 39 \to 38 \to 18 \to 150 \to 160 \to 184$

移动距离：$10 + 32 + 3 + 16 + 1 + 20 + 132 + 10 + 24 = 248$ 个磁道，平均 $= 248/9 \approx 27.5$。

> 另一种等价的移动距离算法：从 100 先一路减到 18 再跳到 184，即 $(100 - 18) + (184 - 18) = 82 + 166 = 248$。

- 优点：性能优于 FCFS，平均寻道时间短。
- 缺点：可能产生「饥饿」现象——磁头在一个小区域内来回移动，较远的磁道（如 184 号）长期得不到服务；每次最短不一定全局最短。

（3）扫描算法（SCAN，电梯调度算法）

为避免 SSTF 的饥饿问题，规定磁头只有在到达最外侧磁道后才转向内侧移动，到达最内侧磁道后才转向外侧移动，在 SSTF 基础上引入了磁头移动方向的约束。因为行为像电梯，所以也叫电梯调度算法。

访问顺序（先向磁道号增大方向，抵达 199 后折返）：$100 \to 150 \to 160 \to 184 \to 199 \to 90 \to 58 \to 55 \to 39 \to 38 \to 18$

移动距离：$(199 - 100) + (199 - 18) = 99 + 181 = 280$ 个磁道（若按「移动到端点」计）。若按王道教材把端点也算作经过的磁道（即含 200 号），则为 $(50 + 10 + 24 + 16) + (110 + 32 + 3 + 16 + 1 + 20) = 282$ 个磁道，平均 $282/9 \approx 31.3$。

- 优点：寻道性能较好，可避免「饥饿」现象。
- 缺点：① 只有到达最边上的磁道才能改变方向，而处理完 184 号请求后其实已不需要继续外移；② **SCAN** 算法对各个位置磁道的响应频率不平均，不利于远离磁头当前方向的那一端——刚处理过 90 号的位置下次要等磁头走很长一段距离，而 184 号附近刚被服务过很快又会轮到。

（4）循环扫描算法（C-SCAN）

针对 SCAN 响应频率不平均的问题：规定磁头只朝某个特定方向移动时才处理磁道访问请求，返回时直接快速移动到起始端而不处理任何请求。

访问顺序（向磁道号增大方向服务，到 199 后快速回到 0，再向增大方向服务）：$100 \to 150 \to 160 \to 184 \to 199 \to 0 \to 18 \to 38 \to 39 \to 55 \to 58 \to 90$

移动距离（按王道含端点计法）：$50+10+24+16+200+18+20+1+16+3+32=390$ 个磁道，平均 $390/9\approx 43.3$。

- 优点：比起 SCAN，对各个位置磁道的响应频率很平均，消除了对两端磁道请求的不公平。
- 缺点：① 仍要求到达最边上磁道才改变方向；② 磁头返回时其实只需返回到 18 号磁道即可，不必回到最边缘；③ 平均寻道时间比 SCAN 更长。

（5）LOOK 与 C-LOOK

SCAN 和 C-SCAN 都要求磁头严格从磁盘一端移动到另一端。实际上可以改进：磁头只需移动到当前方向上最远的请求位置即可返回，无须抵达物理端点。改进后的算法分别称为 **LOOK** 和 **C-LOOK**（因为朝某方向移动前会「查看」该方向是否还有待处理请求）。

LOOK：访问顺序 $100\to 150\to 160\to 184\to 90\to 58\to 55\to 39\to 38\to 18$，移动距离 $(184-100)+(184-18)=84+166=250$ 个磁道，平均 $250/9\approx 27.5$。

C-LOOK：访问顺序 $100\to 150\to 160\to 184\to 18\to 38\to 39\to 55\to 58\to 90$，移动距离 $(184-100)+(184-18)+(90-18)=84+166+72=322$ 个磁道，平均 $322/9\approx 35.8$。

各算法的移动距离对照表（同一请求序列）

磁头初始 100，请求序列 $55, 58, 39, 18, 90, 160, 150, 38, 184$（9 个请求），磁道范围 $0\sim 199$，初始方向为磁道号增大：

算法	访问顺序	移动总距离	平均寻道长度
FCFS	55, 58, 39, 18, 90, 160, 150, 38, 184	**498**	55.3
SSTF	90, 58, 55, 39, 38, 18, 150, 160, 184	**248**	27.5
SCAN（到端点）	150, 160, 184, 199, 90, 58, 55, 39, 38, 18	**280**（含端点写法 282）	31.3
LOOK	150, 160, 184, 90, 58, 55, 39, 38, 18	**250**	27.5
C-SCAN（到端点）	150, 160, 184, 199, 0, 18, 38, 39, 55, 58, 90	**390**	43.3
C-LOOK	150, 160, 184, 18, 38, 39, 55, 58, 90	**322**	35.8

读表要点：① **SSTF** 和 **LOOK** 在这组数据上移动距离相同（**248 vs 250**），说明「贪心最近」和「带方向的最近」在请求分布均匀时效果接近；② **C-SCAN** 因强制回到端点而最差，C-LOOK 是它的改进；③ 若题目没有特别说明，**SCAN** 就按 **LOOK** 理解、**C-SCAN** 就按 **C-LOOK** 理解（见下条提示）。

做磁盘调度计算题的四个关键约定

1. 若题目中无特别说明，**SCAN** 就是 **LOOK**，**C-SCAN** 就是 **C-LOOK**。反过来，**SCAN** 与 **C-SCAN** 必须到达端点，**LOOK** 与 **C-LOOK** 可以跳过端点。判断依据是题干的措辞：「扫描算法」「电梯算法」通常按 LOOK 处理；明确说「移动到最外侧/最内侧」「到达端点」才按 SCAN 处理。
2. 必须先确定磁头的初始移动方向。方向不同，访问序列和移动距离都不同。若题目只说「从外侧向内侧移动」，由于磁道号由外向内递增，即向磁道号增大的方向移动。
3. 注意「移动距离」是否包含回头到端点的部分。C-SCAN 必须计入从一端快速回到另一端的距离（如 199→0 距离 199）。
4. 注意请求序列中可能包含与当前磁头位置相同的磁道（如 2021 年题中当前在 184 且请求序列首项就是 184）。此时访问该磁道移动距离为 0，但必须计入「已完成的请求」，且它会被算法安排在某个位置处理（SSTF 会立刻处理它）。

▌磁盘调度真题四个算例

【2009-29】磁头在 105 道、向磁道号增大方向移动，请求序列 $35, 45, 12, 68, 110, 180, 170, 195$，采用 **SCAN**（电梯）算法得到的访问序列是：

小于 105 的请求 $12, 35, 45, 68$；大于 105 的请求 $110, 170, 180, 195$。SCAN 先沿当前方向依次处理：$110 \to 170 \to 180 \to 195$（到端点后折返），再按递减顺序访问剩余：$68 \to 45 \to 35 \to 12$。

答案 **A**：**110, 170, 180, 195, 68, 45, 35, 12**。

【2015-32】硬盘 200 个磁道（$0 \sim 199$），请求序列 $130, 42, 180, 15, 199$，磁头在 58 号磁道从外侧向内侧移动（即磁道号增大方向），按 **SCAN** 处理完后磁头移过的磁道数：

先处理当前方向上的请求：$58 \to 130 \to 180 \to 199$，距离 $199 - 58 = 141$；到达 199 后折返，处理剩余请求 $42 \to 15$，距离 $199 - 15 = 184$。总距离 $= 141 + 184 = 325$。

答案 **C**（**325**）。

【2018-30】磁臂黏着（系统总是访问磁盘的某个磁道而不响应对其他磁道的访问请求）不会出现在哪种算法中？

答案 **A**：**FCFS**。FCFS 严格按请求到达顺序服务，不会因为某个磁道离当前磁头近而反复优先服务它，已进入队列的请求只需等待有限个更早到达的请求。**SSTF** 尤其容易因不断优先处理近距离请求而使远距离请求长期等待；SCAN 与 C-SCAN 在特定热点请求下也可能黏着（这正是 NStepSCAN 和 FSCAN 要解决的问题）。

【2024-32】磁盘 400 个磁道（$0 \sim 399$），采用 **C-SCAN** 调度，完成对 200 号磁道的请求后磁头向磁道号减小的方向移动，还有 7 个请求：$300, 120, 110, 0, 160, 210, 399$。求磁头移动总距离。

C-SCAN 只沿一个方向扫描，到端点后直接回到另一端再沿原方向继续扫描。

第一步递减扫描：先处理小于 200 的请求，按递减访问 $200 \to 160 \to 120 \to 110 \to 0$，距离 $200-0 = 200$。第二步循环回卷：到达 0 后磁头直接移动到 399，距离 $399 - 0 = 399$，399 号本身有请求，到达后立即处理。第三步继续递减扫描：$399 \to 300 \to 210$，距离 $399 - 210 = 189$。

完整访问顺序：$160 \to 120 \to 110 \to 0 \to 399 \to 300 \to 210$，总距离 $200 + 399 + 189 = 788$。

答案 **C**（**788**）。

*6.*减少旋转延迟的优化：交替编号与错位编号

磁盘是连续自转设备，磁头读/写一个物理块后需要经过短暂的处理时间才能开始读/写下一块。在这个处理时间里，磁盘仍在继续旋转，如果逻辑上相邻的两个扇区在物理上也相邻，那么等 CPU 处理完当前块时，下一个扇区已经转过去了，就必须再等磁盘转一整圈才能读到它，造成严重的延迟。

解决办法是在低级格式化时做两种编号优化：

1. 交替编号（对一个盘面内的扇区）：把逻辑上连续的扇区在物理上隔开存放，即物理编号按「1, 3, 5, ⋯, 2, 4, 6, ⋯」这类方式排布。这样在读完一个扇区、经过处理时间后，下一个逻辑扇区刚好转到磁头下方，避免多转一圈。
2. 错位编号（对不同盘面的相同位置）：同柱面不同盘面的扇区错位编号，连续读/写相邻两个盘面的逻辑记录时也能减少磁头延迟时间。

一句话记忆：交替编号解决「同一盘面内相邻扇区」的延迟，错位编号解决「同一柱面内不同盘面」的延迟。两者都在低级格式化时完成。

*7.*提高磁盘 *I/O* 速度的其他方法

方法	做法
磁盘高速缓存	在内存中开辟内核空间缓存最近读写的盘块。逻辑上属于磁盘，物理上属于内存
调整磁盘请求顺序	用磁盘调度算法减少寻道时间

方法	做法
提前读（预读）	顺序访问文件时，读当前块的同时把下一个盘块也读入缓冲区
延迟写	缓冲区数据本应立即写回磁盘，但考虑到可能不久后再被访问，把它挂在空闲缓冲区队列末尾。随着空闲缓冲区使用逐渐前移，到达队首并被申请时才真正写回磁盘。只要它仍在队列中，任何访问该数据的进程都可直接读出而不必访问磁盘
优化物理块分布	用位示图时找一片相邻空闲盘块；用空闲盘块链表时把同一磁道上的若干盘块组成一簇，以簇为单位分配。这样访问这几个盘块时不必移动或较少移动磁头
虚拟盘（**RAM** 盘）	用内存空间仿真磁盘，对用户透明。主要问题是内存易失，掉电或故障数据会丢失，因此常存放临时文件。与磁盘高速缓存的区别是虚拟盘的内容完全由用户控制，而磁盘高速缓存的内容由 **OS** 控制
磁盘阵列 **RAID**	用多个独立物理磁盘组成逻辑盘，数据分割交叉存储、并行访问

「不能改善磁盘 **I/O** 性能」的选项

【2012-32】下列不能改善磁盘设备 I/O 性能的是「在一个磁盘上设置多个分区」。

设置多个分区的主要目的是逻辑隔离、安装多个文件系统或便于管理，不能直接减少寻道、旋转延迟或传输次数。而重排 I/O 请求次序（磁盘调度）、预读与滞后写（延迟写）、优化文件物理块分布都能直接改善性能。

【2015-28】在系统内存中设置磁盘缓冲区的主要目的是「减少磁盘 **I/O** 次数」。缓冲区不能直接缩短平均寻道时间（那是磁盘调度的事），也不是可靠性或设备独立性的主要实现机制。

（二）固态硬盘

*1.*固态硬盘的结构

固态硬盘（**Solid State Disk**，**SSD**）基于闪存（**Flash Memory**）技术，属于电可擦除 **ROM**（**EEPROM**）。其存储介质与 U 盘类似，但容量更大、存取性能更好。

组成：

- 闪存翻译层（**Flash Translation Layer**）：负责翻译逻辑块号、找到对应的页（**Page**），相当于磁盘控制器。
- 存储介质（闪存颗粒）：多个闪存芯片 → 每个芯片包含多个块（**block**） → 每个块包含多个页（**page**）。

层次关系：闪存芯片（chip）→ 块（block）→ 页（page）。读写的单位是页，擦除的单位是块，块大于页。

*2.*读写性能特性

固态硬盘的五条读写特性

1. 以页为单位读/写（相当于磁盘的「扇区」）。注意：需要写某页信息时，必须先对该页所在的块进行擦除操作。
2. 以块为单位擦除。擦干净的块，其中的每一页都可以写一次、读无限次。
3. 支持随机访问。系统给定一个逻辑地址，闪存翻译层可以通过电路迅速定位到对应的物理地址，没有寻道时间和旋转延迟。
4. 读快、写慢。要写的页如果已有数据，则不能直接写入，必须先把块内其他页全部复制到一个新的（已擦除的）块中，再写入新的页。这就是写放大的根源：一次看似很小的写操作，实际可能触发整个块的搬迁与重写。
5. 容易磨损。SSD 的一个「块」被擦除次数过多（反复写同一个块）可能会坏掉，而机械硬盘的扇区不会因为写的次数太多而坏掉。

与机械硬盘的对比：

比较项	机械硬盘（**HDD**）	固态硬盘（**SSD**）
访问方式	移动磁臂 + 旋转盘片定位，有寻道时间和旋转延迟	用电路控制访问位置，随机访问性能高
随机读性能	差（受机械延迟限制）	远优于机械硬盘
写性能	与读性能接近	写比读慢（需先擦除，有写放大）
噪音与抗震	有噪音、怕震动	安静无噪音、耐摔抗震
能耗与价格	能耗高、单位容量便宜	能耗低、单位容量更贵
寿命机制	扇区不会因写次数多而损坏	块被擦除次数过多会损坏

三组容易混淆的单位与操作

1. 闪存的「擦除」单位是块（**block**），「读/写」单位是页（**page**）。题目常问「写一页需要先擦除什么」，答案是该页所在的整块，不是该页。
2. 闪存翻译层不是设备控制器——它功能上相当于磁盘的磁盘控制器（负责逻辑地址到物理地址的翻译），但它是 SSD 内部的一个软件/固件层。
3. **SSD**「读快写慢」不是说它比机械硬盘慢。SSD 的写速度仍然远快于机械硬盘，只是相对于它自己的读速度而言较慢。

*3.*磨损均衡

为什么需要磨损均衡：SSD 的块有擦除次数上限，如果反复写同一部分块，这些块会先损坏，而其余块还远未达到寿命，整盘就提前报废。磨损均衡（**Wear Leveling**）的思想就是把「擦除」平均分布到各个块上，以提升整体使用寿命。

动态磨损均衡与静态磨损均衡

- 动态磨损均衡：写入数据时，优先选择累计擦除次数少的空闲（新）闪存块。实现简单，但只对「有写入发生的块」起作用——那些长期只被读、从不被写的数据所占据的块，其擦除次数一直很低，无法参与均衡。
- 静态磨损均衡：SSD 控制器监测并自动进行数据分配与迁移，让老旧的闪存块承担以读为主的存储任务，让较新的闪存块承担更多的写任务。即使没有新数据写入，控制器也会定期扫描并搬移冷数据，主动制造写入机会来均衡磨损。静态磨损均衡比动态磨损均衡更优秀、更彻底，因为它是唯一能解决「冷数据长期占据低磨损块」问题的策略。

实际效果：得益于磨损均衡算法，SSD 的实际使用寿命显著提升。例如一块 256GB 的 SSD，若其闪存块的擦除次数上限为 3000 次左右，通过磨损均衡可以使整个盘的总写入量接近「块数 × 单块擦除上限」，而不是由最先损坏的那一小部分块决定。

SSD 的常考判断

统考考过「下列关于固态硬盘的说法中，错误的是」。常见错误选项有：

- 「固态硬盘的写速度比机械硬盘慢、性能甚至弱于常规硬盘」——错误。SSD 的写速度确实比它自己的读速度慢，但仍远快于机械硬盘。
- 「固态硬盘不需要磨损均衡」——错误。恰恰相反，磨损均衡是 SSD 必不可少的机制。
- 「固态硬盘的擦除单位是页」——错误。读/写单位是页，擦除单位是块。

本章小结表

考点	核心结论
设备分类（按信息交换单位）	块设备（磁盘，可寻址，DMA）；字符设备（键盘打印机，不可寻址，中断驱动）
设备分类（按共享属性）	独占（打印机，静态分配）；共享（磁盘，动态分配，可寻址可随机访问）；虚拟（SPOOLing 改造）
I/O 接口组成	数据缓冲寄存器、状态寄存器、控制寄存器、设备存储器、地址译码与 I/O 控制逻辑
统一编址 vs 独立编址	统一：访存指令访问、占主存空间、译码慢、端口不能 **Cache**；独立：专用 I/O 指令、不占主存空间、译码快、程序灵活性差
轮询方式	CPU 忙等、设备与 CPU 串行、不用中断；分独占查询与定时查询
中断方式	设备主动请求、CPU 与外设并行、每个字/字节中断一次、需保护恢复现场
DMA 方式	请求总线控制权、块传送、不经过 CPU 寄存器、不改变 CPU 现场、整块只中断一次
中断 vs DMA 响应时机	中断：一条指令执行结束后；DMA：一个总线周期结束后
I/O 软件四层（自上而下）	用户层 I/O 软件 → 设备独立性软件 → 设备驱动程序 → 中断处理程序
设备独立性软件职责	公有操作（分配回收、逻辑 → 物理映射、保护、缓冲、差错控制、统一逻辑块）+ 统一接口
字符/块/网络设备接口	字符：get/put、需 open/close、不可寻址；块：read/write/seek、线性块序列；网络：socket API、数据包导向
阻塞 vs 非阻塞 I/O	区别在等待数据准备阶段：阻塞则进程挂起等待；非阻塞则立即返回错误码并轮询
缓冲的四个目的	缓和速度矛盾、减少中断频率、解决粒度不匹配、提高并行性
单缓冲每块时间	$\max(T, C) + M$；n 块总时间 $(n-1)[\max(T, C) + M] + (C + T + M)$
双缓冲每块时间	$\max(T, C + M)$；n 块总时间 $(n-1)\max(T, C + M) + (C + T + M)$
循环缓冲	多个等大缓冲区连成循环队列，维护 `in`/`out` 指针
缓冲池	三个队列（空白、输入、输出）+ 四种工作缓冲区（`hin`/`sin`/`hout`/`sout`）；规律：队头取、队尾挂
缓冲 vs 缓存	缓冲是「中转站」（数据可能是唯一的）；缓存是数据源副本（求命中率）
DCT/COCT/CHCT/SDT	SDT 全系统一张；DCT 每设备一张；COCT 每控制器一张；CHCT 每通道一张。流程 SDT→DCT→COCT→CHCT
安全 vs 不安全分配	安全：分配后立即阻塞，破坏「请求和保持」，不死锁但串行；不安全：可继续请求，可并行但可能死锁
静态 vs 动态分配	静态：运行前全部分配，不死锁，利用率低，用于独占设备；动态：按需申请，利用率高，可能死锁
LUT	⟨ 逻辑设备名，物理设备名，驱动程序入口地址 ⟩；首次分配建表，再次请求直接查表
SPOOLing 组成	输入井/输出井（磁盘）、输入缓冲区/输出缓冲区（内存）、输入进程/输出进程、井管理程序
SPOOLing 三大特点	提高 I/O 速度；把独占设备改造为共享设备；实现虚拟设备（以空间换时间）
SPOOLing 条件	大容量高速度外存 + 多道程序技术 + 通道技术（软件实现）
脱机 vs 假脱机	脱机需物理外围机、脱离主机控制；假脱机用系统进程模拟外围机、在主机控制下完成

考点	核心结论
设备驱动程序统一接口	用函数指针表定义驱动与 OS 其余部分的接口，新增设备无需修改 OS
磁盘地址结构	（柱面号，盘面号，扇区号）或（驱动器号，柱面号，盘面号，扇区号）；柱面号在前，因为同柱面不同盘面换磁头不需寻道
磁盘管理四步	低级格式化（物理格式化）→ 分区 → 高级格式化（逻辑格式化）→ 安装操作系统
存取时间公式	$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$，其中寻道时间 $T_s = s + mn$ 占主导
FCFS / SSTF	FCFS 公平但寻道长；SSTF 贪心最短，可能饥饿（磁臂黏着）
SCAN / LOOK	SCAN 到端点才折返，方向约束避免饥饿，对两端不公平；LOOK 只到最远请求处折返
C-SCAN / C-LOOK	单向服务、回程不服务，响应频率均匀；C-LOOK 只需回到最远的请求处
交替编号 / 错位编号	同盘面相邻扇区交替编号；同柱面不同盘面错位编号；均在低级格式化时完成
SSD 读写单位	读/写以页为单位，擦除以块为单位；写前需擦除整块
SSD 写放大	写有数据的页要先把它所在块的其他页全部复制到新块再写入
磨损均衡	动态：写时优先选擦除次数少的块；静态：控制器主动迁移，让老块承担读、新块承担写（更优秀）

易错清单

本章 30 个最容易失分的点

1. 「共享设备可以同一时刻被多个进程访问」是错的。共享设备是「一段时间内」可同时访问，微观上交替使用，且必须是可寻址、可随机访问的设备。块设备接口把磁盘的二维结构隐藏成线性块序列，这是文件系统能建立在磁盘上的前提。
2. 统一编址的 **I/O** 端口不能被 **Cache** 缓存，因为接口中的数据是动态变化的，缓存会导致 CPU 读到过时状态。
3. 统一编址不需要专门的 **I/O** 指令，用 `LOAD/STORE` 即可；独立编址需要 `IN/OUT` 这类专用指令，且 I/O 指令是特权指令。
4. 状态寄存器和控制寄存器可以合二为一，因为传送方向相反且 CPU 访问时间错开。
5. 中断类型号在数据线上传输，但它不是设备控制器发出的，而是可编程中断控制器判优后生成的。
6. 中断响应发生在一条指令执行结束后；**DMA** 响应发生在一个总线周期结束后，DMA 响应更快。中断方式请求的是 CPU 处理时间，DMA 方式请求的是总线控制权。
7. **DMA** 数据传送不经过 **CPU** 寄存器，不改变 **CPU** 现场，所以无需保护和恢复现场。
8. 周期挪用方式下，**CPU** 在整个数据块传送过程中并非不能访问主存，只在 DMA 占用那一个周期时让出总线。「CPU 不可以访问主存储器」是「停止 CPU 访问内存」方式的特点。
9. 周期挪用一次实际需 **2～5** 个存储周期，因为总线请求、仲裁、归还本身有开销。
10. **DMA** 与 **CPU** 同时访存时 **DMA** 优先，这是为了防止高速设备数据丢失。
11. 「中断方式适用于所有外部设备」是错的；极高速设备用中断反而效率更低，轮询（如 NVMe SSD 的同步 I/O）反而更优。
12. **DMA** 方式的数据通路位于「设备接口和主存之间」，不是 CPU 和主存之间，也不是设备接口和 DMA 控制器之间（2024 年考过）。
13. **I/O** 软件四层自上而下是：用户层 **I/O** 软件 → 设备独立性软件 → 设备驱动程序 → 中断处理程序。驱动程序必须在设备独立性软件之下，中断处理程序最接近硬件。一次磁盘 I/O 请求的处理流程是：用户程序 → 系统调用处理程序 → 设备驱动程序 → 中断处理程序。

14. 计算柱面号、磁头号、扇区号（**CHS**）的是设备驱动程序，不是用户程序、系统调用程序或中断处理程序（2013 年考过）。
15.「驱动程序与 **I/O** 控制方式无关」是错的。程序查询、中断、DMA 三种控制方式直接决定驱动程序的编码方式。
16. 程序员的系统调用传的是逻辑设备名，物理设备名、主设备号/次设备号是内核与驱动程序使用的（2009、2020 年考过）。主设备号确定设备类型并据此定位驱动程序，次设备号被驱动程序用来确定具体设备。
17. 单缓冲每块时间 $\max(T, C) + M$，M 在 max 之外；双缓冲 $\max(T, C + M)$，M 在 max 之内。这是两者唯一差别，也是判断用哪个公式的关键。
18. n 块数据的总时间不能简单用 $n\times$ 每块时间，必须用 $(n-1)$ 倍流水时间加最后一块的完整串行时间 $(C + T + M)$。
19. 单缓冲、双缓冲、循环缓冲是专用缓冲；缓冲池是共享缓冲。
20. 缓冲池四个工作缓冲区 `hin`**(**收容输入**)**、`sin`**(**提取输入**)**、`hout`**(**收容输出**)**、`sout`**(**提取输出**)**，规律是「取出从队头、挂入到队尾」。
21. **SDT** 全系统只有一张；**DCT**、**COCT**、**CHCT** 分别是每设备、每控制器、每通道一张。分配请求流程是 SDT→DCT→COCT→CHCT，且只有设备、控制器、通道三者都分配成功，设备分配才算成功。
22. 静态分配降低设备利用率，不是提高。它的优点是破坏「请求和保持」从而避免死锁。
23. **SPOOLing** 不能缩短物理设备的传输时间，也不改变设备本身的速度，它只让 CPU 不必等待。
24.「**SPOOLing** 由用户作业控制设备与输入/输出井之间的数据传送」是错的，由系统的输入/输出进程控制，对用户透明。
25. **SCAN/C-SCAN** 必须到端点，**LOOK/C-LOOK** 可跳过端点；若题目无特别说明，**SCAN** 即 **LOOK**、**C-SCAN** 即 **C-LOOK**。
26. 转速单位要看清：7200rpm 是「转/分」，旋转一圈是 $60/7200\text{s} = 8.33\text{ms}$，平均旋转延迟为 4.17ms。若题目给「秒/转」则公式要取倒数。
27. 磁盘传输速率中 **K/M/G/T** 以 **10** 为底（$1\text{M} = 10^6$），不要按 2^{20} 算。
28.「柱面号在前」的原因是同柱面不同盘面换磁头不需要寻道，这是把最耗时的寻道操作尽量推迟。
29. 逻辑格式化做的是「建根目录 + 初始化空闲块管理结构」；分区、确定校验码位数分别属于分区和低级格式化（2017 年考过）。
30. **SSD** 的读/写单位是页，擦除单位是块；写有数据的页必须先擦除它所在的整块。「SSD 的性能弱于机械硬盘」「SSD 擦除单位是页」都是错的。